

From Graphical Models to Structured Generative AI

*Inference, Learning, Optimization, Sampling, and Scientific
Applications*

Working manuscript evolved from the INFERLO living-book project

Michael (Misha) Chertkov

Program in Applied Mathematics and Department of Mathematics,
University of Arizona, Tucson, AZ 85721, USA

chertkov@arizona.edu

Working manuscript – August 2026

This working manuscript is the restructured continuation of the INFERLO living-book project.
Corrections and comments are welcome at chertkov@arizona.edu.

Preface: From the Physics of Algorithms to Structured Generative AI

I began as a theoretical physicist. During my Master’s studies I worked on quantum spin systems; during my PhD and later postdoctoral work I moved to fluid mechanics and turbulence. These subjects appeared very different, but both taught me to think in terms of many interacting degrees of freedom, probability distributions, collective behavior, rare fluctuations, and the tension between exact equations and useful reduced descriptions.

A later project on the stochastic propagation of optical-fiber pulses led me toward error correction. The practical success of low-density parity-check codes made graphical models impossible to ignore: a graph was not merely a picture of interactions, but an algorithmic representation of a probability distribution. It exposed the local structure of a global inference problem and suggested distributed computations—most prominently belief propagation—that could be exact on trees and remarkably effective on sparse graphs with loops.

Around 2005 I began to describe this direction to myself as the *Physics of Algorithms*. The phrase reflected a two-way exchange. Statistical physics supplied intuition, variational principles, gauge transformations, perturbative corrections, and an insistence on understanding collective effects. Algorithms supplied concrete finite-dimensional objects on which these ideas could be tested, proved, and implemented. Loop calculus and loop series emerged from this program as a way to characterize, and sometimes correct, the gap between tree-based belief propagation and exact inference on graphs with cycles.

The INFERLO living-book project grew from an attempt to systematize this body of ideas—inference, learning, and optimization with graphical models—in a form broader and more connected than the individual research papers. In parallel, graphs led me to energy systems: power and gas networks, flows over graphs, uncertainty, rare events, and control. Other graphical-model projects followed, including matching and permanents, linear-programming decoding and inference, and graphical-model learning beyond the simplest sufficient-statistics setting.

When neural networks returned to the center of machine learning through deep learning, they also brought me back to fluid mechanics. I began to explore what I called Physics-Informed Machine Learning and now more often call physics-informed AI: not replacing equations and structure by data, but using learned approximations inside models that retain physical meaning. More recently, diffusion models, path-integral control, bridge formulations, and Sampling Decisions have made the stochastic-process side of this program central. They suggest that graphical models should no longer be viewed only as static probability distributions on a fixed

set of variables. They should also organize probability laws on trajectories, controlled changes of measure, and structured generative dynamics.

Meanwhile, the original INFERLO notes remained a living book but spent periods on the back burner. Two related book projects moved ahead: *Principles and Methods of Applied Mathematics*, published by World Scientific in 2025, and *Mathematics of Generative AI*, developed from a University of Arizona course and in production with World Scientific. The time has now come to return to INFERLO—not simply to finish the old notes, but to transform them into *From Graphical Models to Structured Generative AI*.

This transformation is personal, but the book is not intended as an autobiography. The trajectory matters because it motivates the architecture. Quantum spin systems, turbulence, communication and error correction, energy networks, graphical-model algorithms, neural approximation, and generative stochastic processes are connected by the same recurring problem: how to represent and compute with high-dimensional probability while respecting the structure of the underlying system.

The new title reflects that trajectory. Graphical models remain the connecting theoretical and algorithmic language. Generative AI provides new representations, approximators, and sampling mechanisms. Science and engineering provide the setting in which structure, uncertainty, constraints, dynamics, and verification are indispensable rather than optional. The aim is therefore not to append a survey of modern AI to an older graphical-model text. It is to develop a structure-first account of inference, learning, optimization, sampling, and generation that can move between classical algorithms, stochastic processes, neural approximations, and scientific applications.

This book has grown through collaborations and discussions with many colleagues and students. Their influence appears throughout the mathematical developments, examples, questions, and applications. I remain responsible for the particular synthesis, emphasis, and any errors that remain.

Contents

Preface: From the Physics of Algorithms to Structured Generative AI	ii
1 Structured Probability, Inference, Learning, and Generation	1
1.1 Why graphical structure matters — and why it matters now	1
1.2 The central probability objects	2
1.3 Five recurring tasks	3
1.4 How modern generative models fit	3
1.5 Where neural networks enter	4
1.6 Three signature mathematical spines	5
1.7 A Rosetta Stone for structured generative modeling	6
1.8 The book’s contract with the reader	8
I Structured Probability: General Methodology	11
2 Graphical Models: Representations, Partition Functions, and First Examples	12
2.1 From local factors to a global probability distribution	14
2.1.1 Factor-graph representation	16
2.1.2 Gibbs energies and temperature	17
2.1.3 Hypergraph representation	17
2.2 Normal factor graphs: variables on edges	17
2.2.1 From a factor graph to a normal factor graph	18
2.2.2 From a normal factor graph to a factor graph	20
2.3 Conditional independence and Markov random fields	20
2.4 Pairwise graphical models	22
2.4.1 Exact reduction of higher-order factors to pairwise form	22
2.4.2 Ising models: fields, interactions, loops, and frustration	22
2.4.3 Gaussian graphical models	24
2.5 Directed graphical models and Bayesian networks	26
2.5.1 A Bayesian network becomes a factor graph directly	26
2.5.2 From factor scopes to the moral/primal graph	27
2.5.3 d-separation	28
2.6 What graphical structure buys us	32
2.7 Further reading and synthesis problems	33

3	Exact Elimination, Dynamic Programming, and Message Passing	34
3.1	Exact elimination on an arbitrary graph	36
3.1.1	From global summation to local elimination	36
3.1.2	Bayesian networks: from family factors to the elimination graph	37
3.1.3	Fill-in, induced width, and treewidth	38
3.2	Exact sum-product message passing on trees	40
3.2.1	Tree distributions from consistent marginals	40
3.2.2	Messages as partial partition functions	41
3.3	Junction trees: exact inference on loopy graphs	44
3.3.1	Clusters, separators, and the running-intersection property	44
3.3.2	Clique-to-clique messages	45
3.4	What exact dynamic programming teaches us	46
4	Other Inference, Optimization and Learning Challenges	48
4.1	One model, several computational questions	49
4.2	MAP optimization and its probabilistic relaxations	50
4.2.1	MAP is state optimization	50
4.2.2	An exact linear program on the full probability simplex	51
4.2.3	Temperature connects hard optimization to probability	51
4.3	Marginals, evidence, and conditioned inference	52
4.4	Soft inference as variational optimization	54
4.4.1	The Gibbs variational principle	54
4.4.2	From a global distribution to local beliefs	55
4.4.3	A forward pointer: region posets and entropy counting	55
4.5	Exact generation from conditioned inference	56
4.6	Graphical-model learning as repeated inference	58
4.6.1	Exponential families and sufficient statistics	58
4.6.2	Maximum likelihood and moment matching	58
4.7	Relations among the tasks	60
4.8	Further reading and synthesis problems	60
5	Transformations of Graphical Models	62
5.1	Gauge and reparametrization transformations	64
5.1.1	Gauge transformations in the normal factor graph representation	64
5.1.2	Reparametrization in the normal factor graph representation	68
5.1.3	Reparametrization in the factor graph representation	69
5.1.4	Reparametrization in the hypergraph representation	71
5.2	Graph transformations and conditioning	74
5.3	Tree transformations	75
5.3.1	Computation tree	75
5.3.2	Self-avoiding tree for the hard-core model	76
5.4	What the transformations accomplish	78

6	Variational Inference: Free Energies, Bounds, and Message Passing	80
6.1	Mean-field approximation	82
6.1.1	Product-family restriction in a normal factor graph	82
6.1.2	Mean field and MAP	83
6.1.3	Mean-field free energy and the lower bound on the partition function	83
6.1.4	Factor-graph and pairwise forms	84
6.2	Bethe approximation in a normal factor graph	85
6.2.1	Tree reconstruction and the Bethe ansatz	86
6.2.2	Bethe energy and entropy	86
6.2.3	Stationarity gives belief propagation	88
6.2.4	Belief propagation as gauge fixing	88
6.2.5	Belief propagation as fixing reparametrizations	90
6.2.6	Zero temperature and the NFG local LP	90
6.3	Bethe approximation in a factor graph	90
6.3.1	Bethe free energy and the local polytope	91
6.3.2	BP equations for beliefs and messages	91
6.3.3	BP as a gauge fixing	92
6.3.4	Primal-domain LP relaxation of MAP	93
6.3.5	Dual-domain LP relaxation and reparametrization	93
6.3.6	Message-domain representation of the Bethe free energy	94
6.4	Tree-reweighted variational inference	94
6.5	Message-passing algorithms	97
6.5.1	Loopy sum-product	97
6.5.2	Min-sum and the zero-temperature limit	97
6.5.3	Min-sum diffusion	98
6.5.4	Distributed and large-scale implementations	99
6.6	Generalized belief propagation and Kikuchi free energies	99
6.6.1	Cycle-free region diagrams	99
6.6.2	Kikuchi free energy	100
6.6.3	Generalized BP equations and message passing	101
6.6.4	Generalized BP as reparametrization	102
6.7	What the variational hierarchy buys us	102
7	Approximate Elimination: Mini-Buckets, Gauges, and Renormalization	104
7.1	Bucket Elimination Algorithm	105
7.2	Mini-Bucket Elimination Algorithm	108
7.3	Weighted Mini-Bucket Elimination Algorithms	110
7.4	Gauge Transformed Weighted Mini-Bucket Elimination Algorithm	112
7.5	Mini-Bucket Renormalization Algorithm	115
7.5.1	Efficiency, Complexity and Global Perspectives	118
7.6	What approximate elimination buys us	120
8	Markov Chain Monte Carlo: Invariance, Mixing, and Exact Sampling	122
8.1	Finite-state Markov chains	123
8.2	Detailed balance, global balance, and probability currents	124
8.3	Gibbs and heat-bath sampling	125
8.4	Metropolis–Hastings sampling	125

8.5	From trajectories to estimators	127
8.6	Coupling from the past and exact sampling	128
8.7	Annealing, partition functions, and randomized approximation	129
8.8	Connections forward	130
9	Loop Calculus: Exact Corrections Beyond Belief Propagation	131
9.1	Loop Series derived via Reparametrizations in NFG Representation	132
9.2	Loop Series derived via Gauges in NFG Representation	135
9.2.1	Binary Alphabet	137
9.2.2	q -ary Alphabet	137
9.3	Loop Series in the Factor Graph Representations	138
9.3.1	Loop Series derived via BP Reparametrization in FG Representation	138
9.3.2	Loop Series derived via Gauges in FG Representation	140
9.4	Loop Series based on Generalized Belief Propagation	141
9.5	Cluster Cumulant Expansion	143
9.5.1	CCE based on Belief Propagation	143
9.5.2	CCE based on Generalized Belief Propagation	147
9.6	Tree-Re-Weighting as Re-Parametrization(s)	150
9.6.1	Fractional interpolation between tree reweighting and belief propagation	151
9.7	Re-summing Loop Series	152
9.7.1	Loop series as a measure to sample, and as a device for proofs	152
9.8	Beyond BP Gauges	153
9.9	From loop calculus to tensor networks and quantum graphical models	153
9.10	What loop calculus adds to the inference hierarchy	154
10	Graphical Model Learning	156
10.1	Learning a Spanning Tree: the Chow–Liu Algorithm	157
10.2	Efficient Local Learning of Graphical Models	159
10.2.1	Regularized Pseudo-Likelihood Estimator	161
10.2.2	Learning via Interaction Screening	165
10.2.3	Implementation Details and Comparative Tests	167
10.3	Infer-Learning	168
10.4	Learning with Hidden Variables	170
10.4.1	Expectation–Maximization	170
10.4.2	Energy-based gradients and contrastive divergence	171
10.4.3	Restricted Boltzmann Machines, deep belief networks, and neuralization	172
II	Scientific Applications and Special Cases	174
11	Applications: Structure as the Common Language	175
11.1	Physics: from equilibrium factors to stochastic dynamics	177
11.1.1	Glauber dynamics	177
11.1.2	Particle tracking as a perfect-matching model	178
11.2	Information theory: graphical error-correcting codes	180
11.3	Computer science: satisfiability as a hard-factor model	183
11.4	Robotics: planning as inference on trajectories	183

11.5	Infrastructure networks: topology, physics, and uncertainty	185
11.6	Social networks: the independent-cascade model	186
11.7	Biological networks: several kinds of structure	187
11.8	Structured language before foundation models: conditional random fields	188
11.9	Generative AI: from graphs to graphical structure	189
11.10	Application Rosetta Stone	190
12	MAP: Exact LP Classes, Message Passing, and Tightening	192
12.1	Fractional Polymorphism Models	193
12.1.1	Submodular models	194
12.2	Total Unimodularity and Integral Linear Programs	195
12.3	Provably Convergent Distributed Algorithms	196
12.4	Successive LP Relaxations and Cutting Planes	197
12.5	Network-Flow Models and Research Challenges	197
12.5.1	Submodular-cost multi-commodity flow	198
12.5.2	Dissipative and physics-constrained flows	199
12.5.3	Piecewise-linear objectives with TUM constraints	199
13	Soft Inference Problems: Canonical Computational Laboratories	200
13.1	Perfect Matchings and the Permanent	201
13.1.1	Bethe and fractional free energies	201
13.2	Gaussian Graphical Models	203
13.2.1	Bethe free energy and the primal mean equations	203
13.2.2	Gaussian belief propagation in information form	203
13.3	Planar and Other Determinant-Easy Models	205
13.4	Two-Level Optimization Involving a Partition Function	206
14	Advanced Structure Learning: Orders, DAGs, and Sparse Graphical Models	208
14.1	Learning Bayesian-network structure	209
14.1.1	Learning through variable orders	210
14.1.2	Continuous optimization and exact smooth acyclicity	211
14.2	Learning sparse undirected graphical models	213
14.3	Three routes around combinatorial structure search	214
III	Graphical Models Neuralization	216
15	Neuralizing Graphical Models: Learned Factors, Messages, Operators, and Corrections	217
15.1	Four modes of neuralization	218
15.1.1	Five tests for a neural surrogate	218
15.2	When does neuralization actually buy something?	219
15.3	Neural factors and energies	219
15.4	Unfolding belief propagation	220
15.5	From learned messages to graph neural networks	221
15.6	Amortized inference as learning an operator	222
15.7	Neural elimination, coarse graining, and tensor truncation	222

15.8 Learning the reference rather than the answer	223
15.9 Learning a residual correction	224
15.10 From static correction to learned proposal design	225
15.11 Recurring computational laboratories	225
16 Hidden Variables, Variational Learning, and Amortized Inference	226
16.1 Latent variables create an inference problem inside learning	227
16.2 EM: alternate exact inference and parameter fitting	227
16.3 Variational inference and the ELBO	227
16.4 Why amortization?	228
16.5 Variational autoencoders as neuralized graphical models	228
16.6 When is neuralization useful?	228
16.7 Hybrid amortization: predict, then correct	229
16.8 Transition to structured generation	229
IV Generative AI Structuralization	230
17 Structured Generative Processes: From Static Factors to Space–Time Graphs	231
17.1 From a structured target to a generative mechanism	232
17.2 Five insertion points for structure	232
17.3 The space–time factor graph	233
17.4 A small algebra of compositions	234
17.5 Product objects: PoE, PoIE, and PoPE	236
17.6 Why naive score addition targets PoIE rather than the noised PoE	238
17.7 Known structure plus an unknown residual	239
17.8 A research challenge: one component per graphical factor	240
17.9 What should be verified?	241
18 Structured Transport, Scores, and Diffusion	242
18.1 Transport maps with sparse dependency structure	243
18.2 Scores inherit factorization when the factors are known	243
18.3 But diffusion smoothing changes graphical locality	243
18.4 Hybrid known-plus-learned scores	243
18.5 Guidance as factor insertion	244
18.6 Constraint-aware diffusion	244
18.7 Connection to flows and flow matching	244
18.8 Forward link	245
19 Structured Bridges, Doob Transforms, and Path-Integral Generation	246
19.1 Conditioning a path measure	247
19.2 Schrödinger bridges and endpoint matching	248
19.3 Path-integral and linearly solvable control	248
19.4 A hierarchy of neuralization choices	248
19.5 Graphical structure in space and time	248
19.6 Importance sampling and exact correction	249
19.7 Scientific constraints as control information	249

20 Structured Autoregressive, Discrete, and Language Generation	250
20.1 The chain rule is already a generative model	251
20.2 Explicit sequence structure: CRFs versus autoregressive models	252
20.3 Hard constraints: LDPC codes and discrete generation	252
20.4 Discrete diffusion and the parallel-decoding problem	253
20.5 Inference-time graphical modeling: Mean-Field Parallel Decoding	253
20.5.1 Binary commit variables and the temporary MRF	254
20.5.2 Mean-field relaxation	255
20.6 Beyond mean field: bringing the rest of the GM toolbox to decoding	257
20.6.1 Loopy BP on the commit graph	257
20.6.2 Fractional BP, loop information, and learned approximation choice	257
20.6.3 From pairwise token interactions to token factor graphs	258
20.7 Sampling Decisions and Decision Flow	259
20.8 GFlowNets and reward-proportional generation	260
20.9 Local Boltzmann guidance and unresolved factors	260
20.10 Synthesis: graphical models inside neural decoding	260
21 Structured Generative Architectures: Synthesis	262
21.1 One target, five routes to generation	263
21.2 A neuralization–structuralization matrix	263
21.3 Metrics beyond sample appearance	263
21.4 Two principal benchmark spines	264
21.5 Known structure plus learned residual	264
21.6 Exact correction as a design principle	264
21.7 Open synthesis questions	265
V Appendixes	266
22 Notations and Abbreviations	267
22.1 Notations	268
22.2 Abbreviations	269
23 Graphs, Variables, Functions, and Operations	270
23.1 Graphs	270
23.1.1 General undirected graphs	270
23.1.2 Specialized undirected graphs	271
23.1.3 Directed graphs	272
23.1.4 Hypergraphs	273
23.2 Variables	273
23.3 Factor functions	274
23.4 Operations	275
23.4.1 Generic summation	275
23.4.2 Multilinear and gauge transformations	275
23.4.3 Optimization	275
23.4.4 Differentiation	275

24 Historical Software Resources**276**

Chapter 1

Structured Probability, Inference, Learning, and Generation

This book is a structure-first account of high-dimensional probability. Its primary subject is the representation, inference, optimization, learning, and generation of parameterized probability distributions whose dependencies, constraints, geometry, or dynamics admit explicit structure. Graphical models provide the central language; statistical physics, variational analysis, algorithms, stochastic processes, and tensor methods provide the mathematical machinery. Neural networks and modern generative models enter as powerful new parameterizations, principal and computational mechanisms/constructs. The ultimate purpose is to build reliable, interpretable, and verifiable generative methods for science and engineering.

1.1 Why graphical structure matters — and why it matters now

High-dimensional probability is difficult because a generic joint distribution grows exponentially with dimension and because the tasks posed to it — normalization, marginalization, constrained optimization, learning, and sampling — are generally computationally hard. Structure is the mechanism that makes progress possible.

The word *structure* is used broadly but concretely in this book. It may mean a sparse factor graph, conditional independence, a low-treewidth decomposition, a tensor factorization, a conservation law, a symmetry, a geometric constraint, a local interaction, a hierarchy of scales, or a Markovian evolution. Such structure can provide:

- **Representation:** a compact specification of an otherwise exponentially large probability law;
- **Decomposition:** local computations that can be assembled into global inference or optimization;
- **Algorithmic tractability:** exact solutions on trees and other special structures, and controlled approximations away from them;

- **Physical and scientific consistency:** explicit encoding of constraints, symmetries, balances, and interactions;
- **Interpretability and verification:** identifiable locations where approximations, learned components, and failures enter;
- **Transfer and data efficiency:** separation of what is known structurally from what must be learned from data.

The AI revolution makes this program timely in two opposite and complementary ways. First, neural networks can approximate functions and operators at scales that were previously inaccessible. Second, their empirical power makes the absence of structure more consequential: black-box models can violate constraints, extrapolate unreliably, hide approximation error, and require data volumes unavailable in science. The challenge is therefore not to choose between explicit and neural models, but to combine them without losing the mathematical meaning of either.

There is also a useful sense in which neural networks are themselves graphical objects. A feed-forward network is a directed computational graph whose nodes carry intermediate variables and whose edges encode transformations; recurrent networks become directed graphs after unrolling in time, and attention architectures similarly define data-dependent interaction graphs [122, 195]. A deterministic computational graph is not, by itself, a probabilistic graphical model: its internal variables are deterministic functions of their parents rather than random variables governed by conditional probability kernels. But the distinction becomes narrow once stochastic latent variables, conditional kernels, or probabilistic outputs are attached to the graph. This overlap is one reason graphical-model language remains useful for understanding what a neural architecture computes and where explicit probabilistic or scientific structure can be inserted. We return to this viewpoint explicitly in the neuralization part of the book; see also the classical probabilistic-graphical-model perspective of [108, 202].

1.2 The central probability objects

The central static object is a parameterized probability distribution

$$p_\theta(x) = \frac{1}{Z_\theta} \prod_{a \in \mathcal{F}} \psi_{a,\theta}(x_{\partial a}) = \frac{1}{Z_\theta} \exp(-E_\theta(x)), \quad (1.1)$$

where the variables $x = (x_i)_{i \in \mathcal{V}}$ may be discrete, continuous, or mixed; the factors $\psi_{a,\theta}$ express local interactions or constraints; and Z_θ normalizes the law when normalization is finite and explicitly required.

The dynamic counterpart is a probability law on trajectories. In discrete time it may be written as

$$\mathbb{P}_\theta(dx_{0:T}) = \mu_{0,\theta}(dx_0) \prod_{t=0}^{T-1} K_{t,\theta}(dx_{t+1} \mid x_t), \quad (1.2)$$

with additional spatial or graphical factorization inside $\mu_{0,\theta}$ and $K_{t,\theta}$.

Here $\mu_{0,\theta}(dx_0)$ and $K_{t,\theta}(dx_{t+1} \mid x_t)$ denote probability measures, not ordinary differentials that are being multiplied. Informally, $\mu_{0,\theta}(dx_0)$ assigns probability to an infinitesimal neighborhood of the initial state, while $K_{t,\theta}(dx_{t+1} \mid x_t)$ assigns conditional probability to the next

state given the current one. Their product means: draw x_0 from $\mu_{0,\theta}$, then draw x_1 from $K_{0,\theta}(\cdot | x_0)$, and continue sequentially; the result is the joint probability measure on the entire path $(x_0, \dots, x_T)$. When densities exist with respect to a reference measure, one may equivalently read Eq. (1.2) as the familiar product of an initial density and transition densities, accompanied by $dx_0 \cdots dx_T$.

In continuous time the corresponding objects are generators, stochastic differential equations, Fokker–Planck equations, path measures, and controlled changes of measure. Eqs. (1.1) and (1.2) are the static and dynamic anchors of the book.

1.3 Five recurring tasks

The reader is repeatedly asked five questions.

Representation	How is the probability distribution or path measure structured? Which factors, graphs, geometries, symmetries, constraints, latent variables, or dynamics are explicit?
Inference	How do we compute or approximate marginals, conditionals, expectations, likelihoods, posterior distributions, scores, or other observables?
Optimization	How do we find probable, optimal, or constrained configurations, trajectories, controls, or decisions?
Learning	How do we infer factors, parameters, graph structure, latent representations, scores, drifts, or controls from data?
Generation	How do we efficiently produce new samples or trajectories with the intended statistical, structural, and physical properties?

These tasks are coupled. Learning usually requires inference; optimization may be a zero-temperature limit of inference; generation may be implemented by a stochastic process whose drift is learned through an inference objective; and an apparently modern generative architecture may be another representation of a classical probabilistic construction.

1.4 How modern generative models fit

Modern generative models do not sit outside graphical models. They reorganize the same central questions and make previously inaccessible representations and algorithms practical.

- **Energy-based models** retain an explicit or implicit energy but often avoid direct normalization, shifting the difficulty to learning and sampling. The modern lineage includes neural energy constructions descending from Hopfield and Boltzmann-machine ideas and their deep-learning extensions [94, 88, 122].
- **Latent-variable models and variational autoencoders** combine a directed graphical model with variational and amortized inference. The variational autoencoder makes this connection especially explicit by learning both a generative conditional model and an amortized approximation to the posterior [105].

- **Normalizing flows** represent generation as an invertible deterministic transport from a simple reference law. The normalizing-flow viewpoint was developed for flexible variational families in [166]; a broad modern review is [158].
- **Score-based and diffusion models** learn local information about a family of noisy marginals and generate through reverse-time stochastic dynamics. Canonical developments include the nonequilibrium-thermodynamics construction of [185], denoising diffusion probabilistic models [91], and the score-SDE synthesis of [186].
- **Schrödinger bridges, Doob transforms, and path-integral control** formulate generation as a controlled change of path measure subject to endpoint, likelihood, or reward information. For the bridge viewpoint see the survey [135]; for the linearly-solvable/path-integral control connection see [99, 65].
- **Autoregressive models and transformers** use a directed factorization and neural conditional kernels to generate one component or block at a time; discrete diffusion and parallel decoding challenge the strict sequential order. The Transformer architecture is introduced in [195]; here we emphasize its probabilistic interpretation as a highly parameterized family of conditional kernels rather than the architecture alone.
- **Tensor networks** reinterpret structured sums and contractions, connecting graphical inference to classical simulation and quantum computation. A useful entry point to matrix-product and projected-entangled-pair-state tensor networks is the review [157].

The book emphasizes both directions of influence. Modern AI can improve classical graphical-model tasks through learned factors, messages, proposals, and elimination rules. Conversely, graphical models can bring locality, constraints, modularity, and verifiability to modern generative models.

1.5 Where neural networks enter

A neural network is introduced only after identifying the mathematical object it approximates. Typical correspondences are

$$\begin{array}{llll}
 \psi_a(x_{\partial a}) & \rightsquigarrow & \psi_{a,\theta}(x_{\partial a}), & E(x) \rightsquigarrow E_\theta(x), \\
 m_{a \rightarrow i}(x_i) & \rightsquigarrow & m_{a \rightarrow i,\theta}(x_i), & q(z \mid x) \rightsquigarrow q_\phi(z \mid x), \\
 T(x) & \rightsquigarrow & T_\theta(x), & s_t(x) \rightsquigarrow s_\theta(t, x), \\
 u_t(x) & \rightsquigarrow & u_\theta(t, x), & K(x' \mid x) \rightsquigarrow K_\theta(x' \mid x).
 \end{array} \tag{1.3}$$

The relevant questions are therefore not only whether a network is expressive, but also:

- Which probabilistic, variational, dynamical, or algorithmic object does it represent? This question is the bridge between classical graphical/variational descriptions [202] and modern neural parameterizations [122].
- What locality, symmetry, normalization, positivity, causality, or conservation properties must be preserved? Graph convolutions and group-equivariant networks provide canonical examples in which locality or symmetry is built into the architecture rather than learned from scratch [106, 54].

- How is it trained, and which inference problem is hidden inside the training objective? Variational learning makes the issue explicit: optimization of model parameters is coupled to approximate posterior inference, and amortization replaces repeated instance-wise inference by a learned map [98, 105].
- How does approximation error propagate into inference, optimization, or generation? Classical variational inference already teaches that local approximations can bias global quantities [202]; in score-based generation, errors in an estimated score enter the reverse-time dynamics themselves [186].
- Which parts of the resulting model can be verified independently of the training data? The answer depends on the retained structure: one may certify a mathematical property of the learned map or sampler, or verify physical/engineering consistency through an explicit model layer rather than through empirical accuracy alone; examples of these two philosophies include [37, 69].

1.6 Three signature mathematical spines

Three developments organize the book. The first is static and algebraic:

$$\begin{array}{l} \text{trees} \longrightarrow \text{exact BP} \longrightarrow \text{loopy BP} \longrightarrow \text{gauges and reparametrizations} \\ \qquad \qquad \longrightarrow \text{loop series and cumulants} \longrightarrow \text{tensor-network contraction.} \end{array} \quad (1.4)$$

The second is dynamic and path based:

$$\begin{array}{l} \text{Markov processes} \longrightarrow \text{Monte Carlo and Langevin sampling} \longrightarrow \text{time reversal} \\ \qquad \qquad \longrightarrow \text{Doob transforms and bridges} \longrightarrow \text{controlled and guided generation.} \end{array} \quad (1.5)$$

The first spine links the original *Physics of Algorithms* perspective to current tensor-network and quantum-classical computation. Its classical graphical-model foundations include exact message passing on trees and variational/message-passing formulations [160, 202]; loop calculus makes the beyond-tree correction explicit [46, 48], while tensor-network language exposes the same contraction structure from a complementary direction [157]. The second links equilibrium and non-equilibrium statistical mechanics to diffusion models, path-integral control, bridge formulations, and Sampling Decisions. Its lineage runs from Markov-chain Monte Carlo and annealed importance sampling [139, 151], through reverse-time diffusion [8, 186], to Schrödinger bridges and linearly-solvable/path-integral control [135, 99, 65]. Graphical structure should be carried through both spines rather than added afterward as an application-specific correction.

A third development runs transversely through both. It is not a separate model class but a recurring design principle: choose a tractable structured reference, keep an exact correction, and use any remaining freedom to make the correction easy to evaluate. The cleanest probabilistic statement is a change of measure. If P is the desired law and Q_ϑ is a tractable reference law with $P \ll Q_\vartheta$, then

$$\frac{dP}{dQ_\vartheta}(x) = \frac{W_\vartheta(x)}{\mathbb{E}_{Q_\vartheta}[W_\vartheta]}, \quad \mathbb{E}_P[f] = \frac{\mathbb{E}_{Q_\vartheta}[fW_\vartheta]}{\mathbb{E}_{Q_\vartheta}[W_\vartheta]}. \quad (1.6)$$

For partition functions the same logic appears as

$$Z = Z_{\vartheta}^{\text{ref}} \tilde{Z}_{\vartheta},$$

where the residual $\tilde{Z}_{\vartheta}$ is exact for every admissible choice of the design parameter ϑ . The parameter may describe a gauge, counting numbers, a fractional interpolation, a proposal distribution, an annealing schedule, or a path-space drift. Exactness and usefulness are therefore distinct questions.

This *reference-correction principle* appears repeatedly:

- belief propagation supplies a local reference and loop calculus supplies the exact generalized-loop correction in Chapter 9;
- tree reweighting and fractional belief propagation provide alternative variational references, again with exact multiplicative residuals in Chapter 9;
- Metropolis–Hastings and annealed importance sampling use tractable proposals followed by exact accept/reject or importance corrections in Chapter 8;
- Doob transforms, importance sampling, and Feynman–Kac particle systems implement the same idea on path space in Chapter 19.

The recurring algorithmic question is not merely “is the reference accurate?” but

which reference makes the exact residual concentrate?

This distinction is central to the neuralization and structuralization parts of the book. A neural network need not learn the final answer: it may instead learn a better gauge, warm start, fractional parameter, proposal, coarse-graining rule, or residual. Conversely, a generative model may remain approximate internally while an exact or asymptotically exact correction layer restores the intended probability law. The same design logic can be seen in exact loop corrections to belief propagation [46], in annealed-importance corrections [151], and, in contemporary diffusion composition and guidance, in Feynman–Kac correction schemes [184].

1.7 A Rosetta Stone for structured generative modeling

Table 1.2 compares classical graphical models and modern generative frameworks through a common vocabulary rather than through architecture-specific terminology. It will be revisited and refined throughout the book.

Table 1.2: A Rosetta Stone for classical graphical models and modern generative models.

Framework	Probability representation	Principal learned or approximated object	Inference / normalization issue	Generation mechanism and role of structure
Graphical model	Product of local factors or conditional kernels	Factors, parameters, graph, messages, beliefs	Partition functions, marginals, maximum a posteriori configurations, posteriors	Elimination, belief propagation, and Markov chain Monte Carlo; the graph exposes locality and conditional independence
Tensor network	Multilinear contraction of local tensors	Tensors, gauges, bond truncations	Contraction complexity and approximation error	Sampling or observable evaluation through contractions; structure controls bond and separator dimensions
Energy-based model	$p_\theta(x) \propto e^{-E_\theta(x)}$	Energy or unnormalized density	Unknown Z_θ ; negative phase and sampling	Markov chain Monte Carlo, Langevin, or learned samplers; graphical energy can enforce local interactions
Latent model / variational autoencoder	$p_\theta(x, z) = p(z)p_\theta(x z)$	Decoder and approximate posterior $q_\phi(z x)$	Posterior inference and evidence bound	Sample z , then decode; structured latent variables encode hierarchy or known mechanisms
Normalizing flow	Pushforward $p_\theta = T_{\theta\#}p_0$	Invertible map or velocity field	Exact likelihood requires tractable Jacobian or divergence	Deterministic transport; structure enters the map, coupling pattern, geometry, and constraints
Score / diffusion model	Family of noisy marginals and reverse-time dynamics	Score, noise predictor, drift, or velocity	Scores replace explicit normalization; the reverse process must be approximated	Reverse stochastic or ordinary differential equations, or discrete denoising; graph structure can localize scores and guidance
Bridge / controlled process	Controlled change of a reference path measure	Desirability, potential, drift, policy, endpoint scaling factors	Endpoint matching, Feynman–Kac weights, or Sinkhorn-type consistency	Doob-transformed or controlled dynamics; structure enters passive dynamics, costs, and terminal factors
Compositional product model	Product, ratio, mixture, or path-space composition of component laws	Component scores, energies, drifts, exponents, or correction potentials	Products of terminal laws do not in general commute with diffusion or marginalization; path normalization and weight degeneracy become central	Compose pretrained components through intermediate targets, path measures, or Feynman–Kac correction; structure specifies which components interact and on which scopes
Autoregressive transformer	$p(x) = \prod_i p_\theta(x_i x_{<i})$	Conditional kernels and attention maps	Exact likelihood is available; ordering and long-range dependence remain central	Sequential or block generation; factorization, masks, and interaction graphs encode structure
Structured scientific generative AI	Any of the above plus explicit scientific constraints	Factors, symmetries, operators, scores, drifts, controls, or surrogates	Verification, sparse data, out-of-distribution behavior, and multiscale closure	Constrained or guided generation consistent with physical, chemical, biological, or engineering structure

1.8 The book’s contract with the reader

The book will:

- derive exact algorithms where structure permits exactness and explain the separator, treewidth, contraction, or dynamical mechanisms responsible;
- develop approximate methods as explicit mathematical approximations rather than unexplained heuristics, and whenever an exact or certified benchmark is available, push examples from an easy regime through a crossover into a regime where the approximation fails, becomes loose, or loses its guarantee;
- retain loop calculus and tensor-network connections as a central beyond-tree theory;
- treat Markov and stochastic processes as a foundational route to sampling and generation, not as an isolated appendix;
- introduce neural networks as parameterizations of identified probabilistic, variational, or dynamical objects;
- use the reference–correction principle to separate what is computed structurally from what remains to be sampled, learned, or corrected;
- connect modern generative models to graphical-model inference, learning, optimization, and sampling in both directions;
- use scientific examples to test structure, uncertainty, constraints, and verification rather than merely to decorate the theory;
- distinguish what is derived, what is approximated, what is learned, and what remains unverified.

Who this book is for. The primary audience is graduate students, researchers, and mathematically oriented practitioners in applied mathematics, probability and statistics, computer science, physics, engineering, control, information theory, and scientific machine learning. The book is written as an advanced textbook with a monograph-style synthesis: it develops a common mathematical language rather than surveying a list of AI architectures. Selected foundational chapters and examples can also be used by mathematically mature advanced undergraduates, but the book is not intended to be a first course in probability, graph theory, statistical mechanics, or machine learning.

Companion and preparatory references. No single prerequisite text follows the path taken here, and the references below should be viewed as companions rather than as a reading list that must be completed first. Readers coming primarily from machine learning may use Murphy’s probabilistic treatment [148] for a broad probabilistic-ML foundation and Russell–Norvig [171] for the wider algorithmic AI context. MacKay [137] is particularly useful for the triangle connecting information theory, inference, coding, and learning, with a computational style close to several examples in this book.

For a systematic development of probabilistic graphical models, Koller–Friedman [108] is the comprehensive classical reference, while Wainwright–Jordan [202] develops graphical models

through exponential families, convexity, and variational inference. Readers who want a stronger modern statistical foundation for high-dimensional estimation and concentration arguments may consult Wainwright [204].

A parallel intellectual line comes from statistical physics and the theory of computation. Mézard–Montanari [149] is the closest companion for the statistical-physics view of inference, coding, random constraints, and message passing; Moore–Mertens [144] gives a broad view of computation in which probability, combinatorics, algorithms, and physics meet. For readers who want the deeper spin-glass background behind replicas, broken symmetry, and some of the intuition later inherited by inference on disordered systems, Mézard–Parisi–Virasoro [150] is the classic advanced source. These books predate, or largely develop independently of, the current generative-AI wave; part of the purpose of the present book is to make their structural ideas converse directly with neural and generative methods.

What is assumed — and what is not. The working prerequisites are mathematical maturity and a first exposure to probability, linear algebra, multivariable calculus, and optimization. A reader should be comfortable reading equations, manipulating finite sums and integrals, and following an argument in which a mathematical object is defined and then reused. Prior courses in graphical models, statistical mechanics, information theory, stochastic control, stochastic calculus, or deep learning are *not* assumed. When these subjects enter, the relevant concepts will be introduced locally, with pointers to the compact mathematical appendices and to standard references for readers who want a more systematic treatment. Acronyms will be expanded at first use; the notation-and-abbreviations appendix is meant as a lookup table, not as a substitute for definitions in the text.

The book nevertheless becomes more demanding as it proceeds. In particular, later chapters on variational approximations, loop corrections, stochastic processes, bridges, and structured generation use ideas developed earlier without re-deriving them from scratch. A reader who meets unfamiliar notation should therefore first ask whether it is a new concept or only a compact notation for an earlier one. The appendices are designed to help with this translation.

How to read the book. The chapters are connected, but the book does not have to be read linearly from beginning to end. Three natural routes are:

- **Graphical-model foundations:** Chapters 2–10, followed by selected applications and special cases in Chapters 11–14. This route develops representation, exact and approximate inference, sampling, loop corrections, and learning before entering neuralization.
- **From graphical models to structured generative AI:** Chapters 1–4 and 6–10 provide the probabilistic and algorithmic core; Chapters 15–21 then develop neuralization, amortized inference, structured diffusion and transport, path-space generation, autoregressive/discrete generation, and synthesis. Chapters 11–14 can be inserted as application laboratories when useful.
- **Research or reading-seminar route:** Chapter 1 provides the map; readers already familiar with graphical models may sample Chapters 5–10 according to need and then concentrate on Chapters 15–21, returning to earlier chapters whenever a classical reference construction is invoked.

These routes are suggestions rather than prerequisites. The recurring examples — binary spin models, coding and constraint models, Gaussian models, matching, and stochastic/path-space models — are deliberately reused so that the reader can compare different algorithms on the same mathematical object.

How to use the companion notebooks. The computational material is organized by chapter under notebooks/chXX/. The notebooks are intentionally small and transparent. They serve four purposes: (i) reproduce computational figures and numerical examples in the text; (ii) provide exact or high-accuracy baselines for small models; (iii) let the reader perturb parameters, constraints, graph structure, or algorithms; and (iv) expose crossover and failure regimes of approximations. They are not intended as production-scale AI implementations, distributed-training examples, or software-engineering templates.

A useful workflow is:

1. read the mathematical example and identify the probability law, objective, or stochastic process being studied;
2. run the companion notebook unchanged and match its main quantities to the corresponding equations and figures;
3. change one ingredient—for example a coupling, temperature, graph edge, constraint, proposal, or noise level—and predict the qualitative effect before rerunning;
4. compare the numerical result with an exact calculation, theorem, bound, or known limiting regime whenever one is available;
5. only then modify the algorithm itself or replace the supplied implementation by your own.

The mathematical problem is stated in the book independently of the code. Readers are encouraged to write their own implementations; the supplied notebooks are reference implementations and experimental laboratories, not required dependencies for following the prose.

How instructors can use the book. A one-semester graduate course need not cover every chapter. A course centered on probabilistic graphical models can use Chapters 2–10 as its spine and choose applications from Chapters 11–14. A course centered on structured generative AI can move more quickly through the classical material and devote the second half to Chapters 15–21. Exercises are intended for local comprehension and computation; the larger *Problems* are designed for homework projects, reading assignments, or open-ended computational investigations. The companion notebooks can be used as demonstrations, starting points for assignments, or benchmarks against which students test their own code.

What the reader should expect from the exposition. The book will sometimes move between languages — probability, statistical mechanics, optimization, graphical models, stochastic processes, and modern AI — because the same mathematical object is often simpler in one language than in another. The goal is not to require fluency in all of these subjects in advance. The goal is to make the translations explicit enough that, by the end, the reader can recognize when two apparently different algorithms are manipulating the same structured probability law in different representations.

Part I

Structured Probability: General Methodology

Chapter 2

Graphical Models: Representations, Partition Functions, and First Examples

Reader's Guide.

Why this chapter? Before discussing belief propagation, variational approximations, Monte Carlo, learning, or generative models, we need a precise language for the probability distribution on which those algorithms operate. A graphical model is that language. The graph does not replace the distribution and it is not merely a visualization. It records which variables interact locally, which constraints involve which variables, or which conditional distributions are needed to build the global law.

The central idea can be stated without any graph terminology: *a complicated global probability distribution is assembled from simpler local rules*. Once those local rules have been identified, a graph records who depends on whom. Later algorithms exploit this bookkeeping to avoid, whenever possible, treating the entire high-dimensional distribution as one unstructured table or function.

What you should know before starting. For the first pass through this chapter, elementary probability and basic linear algebra are sufficient. You do not need prior knowledge of factor graphs, Markov random fields, Bayesian networks, or statistical mechanics. The Ising and Gibbs terminology introduced below is used as a convenient language for weighted probability distributions; the physics interpretation is helpful but not required. The graph-theoretic notions needed here are elementary and are also summarized in the appendices.

What you should be able to do after reading. You should be able to take a verbal description of local interactions or constraints and turn it into a factorized probability model; distinguish a factor graph from a normal factor graph, a Markov random field, and a Bayesian network; convert a Bayesian-network DAG directly into a factor graph; distinguish that

direct construction from the later projection to a moral/primal graph; use d-separation as the conditional-independence calculus of a DAG; identify the partition function, marginals, a maximum a posteriori configuration, and a sampling task; and explain what computational information is exposed by each representation. The aim at this stage is representation, not yet efficiency: all of the small examples can be solved by direct enumeration, which gives us an exact baseline for later chapters.

A first factor in plain language: a seating example. Suppose three guests—Alice, Bob, and Carol—must occupy three labeled seats. Let $x_A, x_B, x_C \in \{1, 2, 3\}$ denote their seat numbers. A local rule saying that Alice and Bob cannot occupy the same seat can be represented by the function

$$f_{AB}^{\text{distinct}}(x_A, x_B) = \begin{cases} 1, & x_A \neq x_B, \\ 0, & x_A = x_B. \end{cases}$$

This function is a *factor*: it looks only at the two variables in its scope and assigns a weight to each local configuration. Similar factors enforce $x_A \neq x_C$ and $x_B \neq x_C$. A softer preference—say, that Alice would rather not sit next to Bob—could assign weight $e^{-\lambda}$ when $|x_A - x_B| = 1$ and weight 1 otherwise. Multiplying all such local factors assigns a single weight to every complete seating arrangement. Dividing by the sum of these weights turns them into probabilities.

Nothing essentially changes when “guest” becomes a spin, code bit, molecule, network state, latent variable, or token, and when “seating rule” becomes an interaction energy, parity check, conservation law, likelihood term, or learned compatibility score. The notation introduced below is simply a scalable way to say which local rule sees which variables.

How to use the companion notebooks. The six notebooks in `notebooks/ch02/` follow the conceptual order of the chapter: factorization and exact enumeration; conversion between factor-graph representations; conditional independence; Ising chains and frustrated loops; Gaussian graphical models; and a time-unrolled dynamic graphical model. On a first reading, the code may be ignored. On a computational reading, start each notebook only after reading the corresponding example, run it unchanged, and then alter one interaction, constraint, or graph edge. The text states the mathematical problem completely, so writing an independent implementation is an equally valid exercise.

Five questions to keep asking. Every representation in this chapter will eventually be used for the same five tasks:

1. **representation**: how is a high-dimensional distribution assembled from local objects?
2. **inference**: how do we compute marginals, conditionals, and expectations?
3. **optimization**: how do we find probable or constrained configurations?
4. **learning**: how do we infer factors or parameters from data?
5. **generation**: how do we produce new samples efficiently?

In this chapter these questions are answered mostly by exact enumeration. The rest of the book asks how graphical structure changes the answer when enumeration becomes impossible.

2.1 From local factors to a global probability distribution

Before introducing the general notation, keep the seating example in mind. A *variable* records an unknown or random quantity; a *factor* is a nonnegative function that examines only a small subset of variables and scores or constrains their joint local state; the *product of factors* gives an unnormalized weight to a complete configuration; and the *partition function* is the total weight required to convert those weights into probabilities. The formulas below say exactly this, but for an arbitrary collection of variables and local rules.

Let

$$\mathbf{x} = (x_1, \dots, x_N), \quad x_i \in \mathcal{X}_i, \quad \mathcal{X} = \mathcal{X}_1 \times \dots \times \mathcal{X}_N.$$

The set $\mathcal{X}_i$ is the *alphabet* or state space of variable x_i . It may be finite, such as $\mathcal{X}_i = \{-1, +1\}$, or continuous, such as $\mathcal{X}_i = \mathbb{R}$.

Suppose that a family of nonnegative factors $\{f_a\}_{a \in \mathcal{F}}$ is given. Each factor depends only on a subset of variables, called its *scope*,

$$\partial a \subseteq \{1, \dots, N\}, \quad \mathbf{x}_a = (x_i \mid i \in \partial a).$$

The local factors define an unnormalized weight, a partition function, and a probability distribution¹:

$$w(\mathbf{x}) := \prod_{a \in \mathcal{F}} f_a(\mathbf{x}_a), \tag{2.1}$$

$$Z := \int_{\mathbf{x} \in \mathcal{X}} \prod_{a \in \mathcal{F}} f_a(\mathbf{x}_a), \tag{2.2}$$

$$p(\mathbf{x}) := \frac{1}{Z} \prod_{a \in \mathcal{F}} f_a(\mathbf{x}_a). \tag{2.3}$$

The compact integral notation in Eq. (2.2) covers three cases. If all variables are discrete, it means the finite or countable sum

$$\int_{\mathbf{x} \in \mathcal{X}} h(\mathbf{x}) := \sum_{x_1 \in \mathcal{X}_1} \dots \sum_{x_N \in \mathcal{X}_N} h(x_1, \dots, x_N).$$

If the variables are continuous, $p(\mathbf{x})$ is understood as a density with respect to a specified product reference measure $d\mu(\mathbf{x})$ —Lebesgue measure unless stated otherwise— and the same notation abbreviates $\int_{\mathcal{X}} h(\mathbf{x}) d\mu(\mathbf{x})$. Mixed discrete – continuous models combine counting and continuous measures in the corresponding coordinates. We use the compact symbol because most graphical-model manipulations are identical in the three cases; Appendix 23.4.1 states the convention systematically. The notation for variables, vector variables, factors, and their domains is collected in Appendices 23.2 and 23.3. The partition function Z is simply the total unnormalized mass and converts the local weights into a normalized global probability law.

¹Eq. (2.2) is the first formula in the book which serves as a foundation for — thus pointing at — an algorithm. Here and below we attract attention of the reader to such facts by placing a pointer at the right margin.

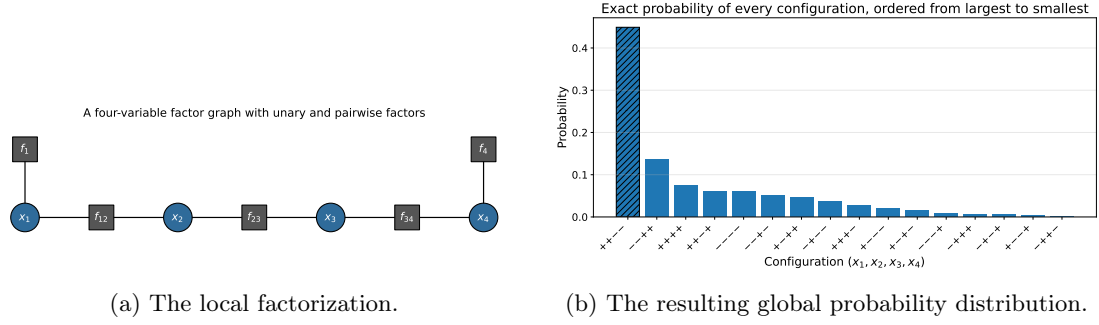


Figure 2.1: Local structure and global probability for Example 2.1.1. The graph stores which factors depend on which variables; the distribution assigns a probability to every complete configuration.

Example 2.1.1 (A four-variable binary chain). Let $x_i \in \{-1, +1\}$ for $i = 1, \dots, 4$, and consider

$$w(\mathbf{x}) = f_1(x_1)f_{12}(x_1, x_2)f_{23}(x_2, x_3)f_{34}(x_3, x_4)f_4(x_4),$$

$$f_1(x_1) = \exp(h_1x_1), \quad f_4(x_4) = \exp(h_4x_4), \quad f_{ij}(x_i, x_j) = \exp(J_{ij}x_ix_j).$$

Only $2^4 = 16$ configurations exist, so Z , all marginals, the maximum a posteriori (MAP) configuration — the complete state with largest posterior probability — and exact samples can be computed by direct enumeration. This small model introduces all five recurring tasks without approximation and will serve as our first exact computational laboratory.

Companion notebook: Local factors, exact enumeration, and generation

The notebook `ch02/ch02_01_factorization_and_enumeration.ipynb` constructs the model in Example 2.1.1, enumerates all configurations, computes Z , marginals, expectations, connected correlations, and the MAP state, and generates exact independent samples. It also verifies a response identity linking a derivative of $\log Z$ to an expectation. The results are illustrated in Fig. (2.1).

The notebook is deliberately downstream of the mathematical statement: nothing in Example 2.1.1 requires the supplied code. A reader may instead enumerate the 16 configurations by hand, write an independent implementation, or use the notebook as a reference. The main computational habit we will cultivate throughout the book is to keep an exact small-model baseline whenever possible and only then replace it by a scalable approximation.

The example clarifies the distinction among the basic computational tasks. For a subset $S \subseteq \{1, \dots, N\}$, a marginal is

$$p_S(\mathbf{x}_S) = \int_{\mathbf{x}_{\bar{S}}} p(\mathbf{x}),$$

where $\bar{S}$ is the complementary set of variables. For an observable $g(\mathbf{x})$,

$$\mathbb{E}[g(\mathbf{x})] = \int_{\mathbf{x} \in \mathcal{X}} g(\mathbf{x})p(\mathbf{x}).$$

A MAP configuration solves

$$\mathbf{x}^{\text{MAP}} \in \arg \max_{\mathbf{x} \in \mathcal{X}} p(\mathbf{x}) = \arg \max_{\mathbf{x} \in \mathcal{X}} w(\mathbf{x}),$$

and generation means producing random configurations distributed according to p .

Algorithmic lens: exact enumeration

For finite alphabets, Eq. (2.2) is already an exact algorithm: enumerate every $\mathbf{x} \in \mathcal{X}$, evaluate the product of local factors, and sum. If $|\mathcal{X}_i| = D$ for every i , the cost is $O(D^N |\mathcal{F}|)$. The red margin pointer marks formulas or constructions that can be read directly as algorithms. Later chapters replace this exponential procedure by algorithms that exploit graph structure.

Exercise 2.1.1. In `ch02/ch02_01_factorization_and_enumeration.ipynb`, reverse the sign of one coupling. Identify which configurations gain probability, and explain the change using only the affected local factor.

Exercise 2.1.2. Modify the same notebook by adding the hard factor $\mathbf{1}(x_1 x_4 = 1)$. Recompute Z , the MAP configurations, and the one-variable marginals. Explain why the factor changes the support rather than merely reweighting all configurations.

Checkpoint before adding graph terminology. At this point, make sure the five objects in the first example are distinct: the variables x_i ; the local factors f_a ; the unnormalized global weight $w(\mathbf{x})$; the normalization Z ; and the normalized probability $p(\mathbf{x})$. A factor graph adds no new probability rule. It is a diagrammatic encoding of which of the variables each factor is allowed to inspect. Keeping this distinction in mind prevents a common source of confusion later, when the same distribution is represented by several different graphs or transformed without changing Z .

2.1.1 Factor-graph representation

The factorization in Eq. (2.3) is represented by an undirected bipartite graph

$$\mathcal{G}_{\text{FG}} = (\mathcal{V}, \mathcal{F}, \mathcal{E}).$$

The set $\mathcal{V} = \{1, \dots, N\}$ contains variable vertices, $\mathcal{F}$ contains factor vertices, and

$$(i, a) \in \mathcal{E} \iff i \in \partial a.$$

Thus an edge records that factor f_a depends on variable x_i . The graph displays the *scope structure*; it does not by itself specify the numerical values of the factors. Basic graph and bipartite-graph terminology is collected in Appendix 23.1.

A factor may be soft, hard, or mixed. A soft factor assigns different positive weights to local configurations. A hard factor takes values in $\{0, 1\}$ and removes configurations that violate a constraint. Zero factors are allowed in the general factor-graph representation, a point that matters when we discuss the Hammersley–Clifford theorem in Section 2.3.

Historically, factor graphs were developed as a common language for coding, signal processing, statistics, and statistical physics; see, for example, [132]. The same language now provides a natural interface between explicit scientific structure and learned local functions.

2.1.2 Gibbs energies and temperature

When the factors are positive, it is often convenient to write

$$E_a(\mathbf{x}_a) := -\beta^{-1} \log f_a(\mathbf{x}_a), \quad (2.4)$$

where $\beta > 0$ is an inverse-temperature parameter. Then Eq. (2.3) becomes

$$p(\mathbf{x}) = \frac{1}{Z} \exp \left(-\beta \sum_{a \in \mathcal{F}} E_a(\mathbf{x}_a) \right), \quad Z = \int_{\mathbf{x} \in \mathcal{X}} \exp \left(-\beta \sum_{a \in \mathcal{F}} E_a(\mathbf{x}_a) \right). \quad (2.5)$$

At large β , probability concentrates near configurations of minimal total energy. At small β , energy differences are suppressed and the distribution becomes flatter. In statistical physics β has a physical interpretation; elsewhere it is often a useful continuation parameter connecting inference and optimization.

For a parameter θ appearing linearly in the exponent, derivatives of $\log Z$ generate moments. For example, if

$$p_\theta(\mathbf{x}) = Z(\theta)^{-1} \exp(\theta T(\mathbf{x}) + R(\mathbf{x})),$$

then

$$\frac{\partial \log Z}{\partial \theta} = \mathbb{E}_{p_\theta}[T(\mathbf{x})].$$

This identity foreshadows the role of the partition function in learning exponential-family models.

2.1.3 Hypergraph representation

A factor graph can also be described by a hypergraph. Let $\mathcal{V}^h$ be the set of variable indices and let $\mathcal{E}^h \subseteq 2^{\mathcal{V}^h}$ be a family of hyperedges. Associate a factor $f_g(\mathbf{x}_g)$ with each $g \in \mathcal{E}^h$. Then

$$p(\mathbf{x}) = \frac{1}{Z} \prod_{g \in \mathcal{E}^h} f_g(\mathbf{x}_g), \quad (2.6)$$

$$Z = \int_{\mathbf{x} \in \mathcal{X}} \prod_{g \in \mathcal{E}^h} f_g(\mathbf{x}_g). \quad (2.7)$$

The factor-graph and hypergraph descriptions contain the same scope information: a factor vertex is equivalent to a hyperedge. Further graph and hypergraph terminology is summarized in Appendix 23.1.4. Hypergraphs become especially useful when regions of variables and factors are treated as composite objects, as in generalized belief propagation and cluster variational methods developed in Sections 6.6 and 6.6.1.

2.2 Normal factor graphs: variables on edges

In a normal factor graph, variables are associated with edges and factors with vertices. The graph, variable, and factor conventions used below are summarized in Appendices 23.1.1, 23.2, and 23.3. Let

$$\mathcal{G}_{\text{NFG}} = (\mathcal{U}, \mathcal{E})$$

be an undirected multigraph; parallel edges and self-loops may be allowed when the incidence structure requires them. Each edge $e \in \mathcal{E}$ carries a variable $x_e \in \mathcal{X}_e$, and each factor vertex $u \in \mathcal{U}$ carries a nonnegative function

$$g_u(\mathbf{x}_{\delta u}), \quad \mathbf{x}_{\delta u} = (x_e \mid e \in \delta u),$$

where δu is the multiset of edges incident to u . The distribution and partition function are

$$p(\mathbf{x}) = \frac{1}{Z} \prod_{u \in \mathcal{U}} g_u(\mathbf{x}_{\delta u}), \quad Z = \int_{\mathbf{x} \in \mathcal{X}} \prod_{u \in \mathcal{U}} g_u(\mathbf{x}_{\delta u}). \quad (2.8) \quad \text{👉}$$

For positive factors, define

$$E_u(\mathbf{x}_{\delta u}) := -\beta^{-1} \log g_u(\mathbf{x}_{\delta u}), \quad (2.9)$$

and obtain

$$p(\mathbf{x}) = \frac{1}{Z} \exp \left(-\beta \sum_{u \in \mathcal{U}} E_u(\mathbf{x}_{\delta u}) \right). \quad (2.10)$$

The edge-variable representation was central to the development of loop calculus [46, 47]. The terminology *normal factor graph* was subsequently standardized and linked to holographic transformations and tensor contractions [5, 70, 145]. The adjective *normal* refers to the graph representation, not to probabilistic normalization by Z .

2.2.1 From a factor graph to a normal factor graph

The conversion duplicates every variable once for each incident factor and then enforces consistency among the duplicates. For every incidence $(i, a) \in \mathcal{E}$, introduce an edge variable

$$y_{ia} \in \mathcal{X}_i.$$

For each original factor $a \in \mathcal{F}$, define an NFG factor

$$g_a((y_{ia})_{i \in \partial a}) := f_a((y_{ia})_{i \in \partial a}).$$

For each original variable $i \in \mathcal{V}$, introduce an equality factor

$$\Delta_i((y_{ia})_{a \in \partial i}) := \begin{cases} 1, & y_{ia} = y_{ib} \text{ for all } a, b \in \partial i, \\ 0, & \text{otherwise.} \end{cases}$$

The NFG partition function is therefore

$$\begin{aligned} Z_{\text{NFG}} &= \int_{\{y_{ia}\}} \left(\prod_{a \in \mathcal{F}} g_a((y_{ia})_{i \in \partial a}) \right) \left(\prod_{i \in \mathcal{V}} \Delta_i((y_{ia})_{a \in \partial i}) \right) \\ &= \int_{\mathbf{x} \in \mathcal{X}} \prod_{a \in \mathcal{F}} f_a(\mathbf{x}_a) = Z_{\text{FG}}. \end{aligned} \quad (2.11)$$

The second equality follows because the equality factors eliminate all inconsistent edge assignments and leave one common value x_i for every original variable.

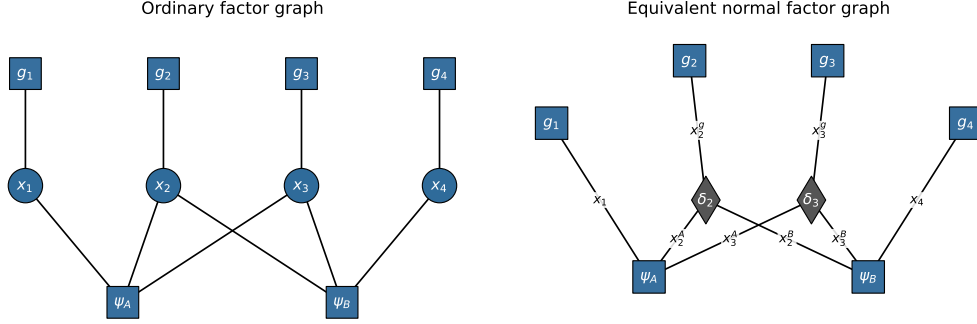


Figure 2.2: Equivalent factor-graph and normal-factor-graph representations of two parity checks with local evidence. In the NFG, each original variable is copied onto incident edges and an equality factor enforces agreement among the copies.

Algorithm 1 Mapping a factor graph to a normal factor graph

```

1: function FG-TO-NFG( $\mathcal{V}, \mathcal{F}, \mathcal{E}, \{\mathcal{X}_i\}, \{f_a\}$ )
2:    $\mathcal{U} \leftarrow \mathcal{V} \cup \mathcal{F}$ 
3:   for each incidence  $(i, a) \in \mathcal{E}$  do
4:     create an edge  $e_{ia}$  carrying  $y_{ia} \in \mathcal{X}_i$ 
5:   for each  $a \in \mathcal{F}$  do
6:      $g_a((y_{ia})_{i \in \partial a}) \leftarrow f_a((y_{ia})_{i \in \partial a})$ 
7:   for each  $i \in \mathcal{V}$  do
8:      $g_i((y_{ia})_{a \in \partial i}) \leftarrow \Delta_i((y_{ia})_{a \in \partial i})$ 
9:   return  $(\mathcal{U}, \{e_{ia}\}, \{g_u\})$ 

```

Example 2.2.1 (Parity constraints and equality factors). *Let $x_1, x_2, x_3, x_4 \in \{0, 1\}$ and impose*

$$x_1 \oplus x_2 \oplus x_3 = 0, \quad x_2 \oplus x_3 \oplus x_4 = 0,$$

Here $\oplus$ denotes addition modulo two, so each equation requires an even number of ones. Together with unary evidence factors, these constraints define a factor graph with two higher-order hard factors. The normal representation replaces each shared variable by edge copies and equality factors. The two representations have identical partition functions and identical physical marginals; the two constructions are shown side by side in Fig. 2.2.

Companion notebook: Factor graph to normal factor graph

The notebook `ch02/ch02_02_factor_graph_to_normal_factor_graph.ipynb` implements Example 2.2.1, computes both partition functions by enumeration, verifies equality of the physical marginals, and demonstrates what fails when an equality factor is omitted.

Algorithm 1 summarizes the construction above: it creates one edge variable per FG incidence, copies each original factor unchanged, and places one equality factor at every original variable vertex.

Algorithm 2 Mapping a normal factor graph to a factor graph

```

1: function NFG-TO-FG( $\mathcal{U}, \mathcal{E}, \{\mathcal{X}_e\}, \{g_u\}$ )
2:    $\mathcal{V} \leftarrow \{v_e : e \in \mathcal{E}\}$ 
3:    $\mathcal{F} \leftarrow \{a_u : u \in \mathcal{U}\}$ 
4:   for each incidence of edge  $e$  at vertex  $u$  do
5:     add the FG incidence  $(v_e, a_u)$ 
6:   for each  $u \in \mathcal{U}$  do
7:      $f_{a_u}((x_e)_{e \in \delta u}) \leftarrow g_u((x_e)_{e \in \delta u})$ 
8:   return  $(\mathcal{V}, \mathcal{F}, \mathcal{E}_{\text{FG}}, \{f_{a_u}\})$ 

```

Exercise 2.2.1. Use `ch02/ch02_02_factor_graph_to_normal_factor_graph.ipynb` to replace the even-parity factors by odd-parity factors. Verify preservation of Z and all physical marginals under the FG-to-NFG transformation.

Exercise 2.2.2. Extend the parity model from binary to ternary variables with constraints modulo three. Specify the equality factors and compare the number of FG variable states with the number of unconstrained NFG edge states.

2.2.2 From a normal factor graph to a factor graph

The inverse conversion is direct. Every NFG edge becomes a variable vertex, every NFG factor vertex remains a factor vertex, and incidence in the NFG becomes an edge in the bipartite factor graph. If e joins vertices u and v , the corresponding FG variable x_e is adjacent to factors g_u and g_v . For a self-loop, the two incidences are retained explicitly.

Thus

$$Z_{\text{FG}} = \int_{\{x_e\}_{e \in \mathcal{E}}} \prod_{u \in \mathcal{U}} g_u((x_e)_{e \in \delta u}) = Z_{\text{NFG}}. \quad (2.12)$$

Algorithm 2 makes the inverse incidence construction explicit. Together, Algorithms 1 and 2 show that the two representations are equivalent at the level of the global weight and partition function.

The two conversions are representation changes, not approximations. Their main importance is conceptual and algorithmic: some transformations, loop expansions, and tensor contractions are most transparent in NFG form, while conditional independence and variable-centered modeling are often more transparent in FG or Markov-random-field form.

2.3 Conditional independence and Markov random fields

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph with variables on its vertices. In an undirected model an edge has no parent-child semantics: it records that two variables may participate in the same local compatibility structure. Conditional independence is therefore read geometrically, by ordinary graph separation.

For disjoint sets $A, B, C \subseteq \mathcal{V}$, the set B *separates* A from C if every path from a vertex in A to a vertex in C intersects B . A distribution is globally Markov with respect to $\mathcal{G}$ if

$$\mathbf{x}_A \perp \mathbf{x}_C \mid \mathbf{x}_B$$

which reads – "whenever B separates A from C ". Equivalently, delete the vertices in B : if no path remains from A to C , the graph asserts the corresponding conditional independence. The symbol $\perp$ denotes statistical independence, while the vertical bar $|$ indicates conditioning.

A clique is a set of mutually adjacent vertices, and a maximal clique is a clique not contained in a larger clique. Let $\mathfrak{C}$ be the family of maximal cliques. A clique-factorized distribution has the form

$$p(\mathbf{x}) = \frac{1}{Z} \prod_{C \in \mathfrak{C}} f_C(\mathbf{x}_C), \quad f_C \geq 0. \quad (2.13)$$

The potential f_C is a compatibility weight on a local configuration; it need not be normalized and it does not, by itself, define a direction of influence. The maximal cliques are generally *overlapping*, not independent pieces of the model, and the potentials are not unique: smaller-scope factors can always be absorbed into any maximal clique that contains their scope.

The Hammersley–Clifford theorem [53, 110] states that, for a strictly positive distribution, the global Markov property with respect to $\mathcal{G}$ is equivalent to the existence of a factorization over the cliques of $\mathcal{G}$. It is therefore a theorem about the equivalence of *undirected conditional-independence semantics* and *clique factorization*. It is not needed merely to draw a factor graph or its primal graph from already-given factors. Strict positivity is essential for the converse implication: hard zero-support constraints are perfectly legitimate factor-graph objects, but the simple equivalence between graph separation and positive clique factorization then requires care.

Undirected representation in one sentence. The graph tells us *which variables must be considered together*; separation tells us *which variables become independent after conditioning*; and clique potentials tell us *how compatible each local joint state is*. Directional generation and directional conditional-independence semantics enter later, with Bayesian networks. Section 2.5.2 then shows how the two languages are connected.

Example 2.3.1 (Separation in an undirected chain). *For an undirected chain*

$$X_1 - X_2 - X_3,$$

the vertex X_2 separates X_1 from X_3 . A distribution that is globally Markov with respect to this graph therefore satisfies

$$X_1 \perp X_3 \mid X_2.$$

At this stage the rule is entirely spatial: delete the conditioned vertex and ask whether a path remains. Directed graphs have a different separation calculus, introduced only after their factorization semantics are established in Section 2.5.

Exercise 2.3.1. *Add a fourth variable to the undirected chain. List the conditioning sets that graphically separate X_1 from X_4 , and verify the corresponding conditional independences in any strictly positive pairwise model on the chain.*

2.4 Pairwise graphical models

A pairwise graphical model associates unary factors with vertices and pairwise factors with edges of an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$:

$$\begin{aligned} p(\mathbf{x}) &= \frac{1}{Z} \left(\prod_{i \in \mathcal{V}} f_i^{(\text{pw})}(x_i) \right) \left(\prod_{(i,j) \in \mathcal{E}} f_{ij}^{(\text{pw})}(x_i, x_j) \right) \\ &= \frac{1}{Z} \exp \left(\beta \sum_{i \in \mathcal{V}} h_i^{(\text{pw})}(x_i) + \beta \sum_{(i,j) \in \mathcal{E}} J_{ij}^{(\text{pw})}(x_i, x_j) \right), \end{aligned} \quad (2.14)$$

where $f_i^{(\text{pw})} = \exp(\beta h_i^{(\text{pw})})$ and $f_{ij}^{(\text{pw})} = \exp(\beta J_{ij}^{(\text{pw})})$. Pairwise models are expressive enough for many applications and form the basic setting for belief propagation, Gaussian inference, Ising models, graph-based optimization, and many neural message-passing architectures.

2.4.1 Exact reduction of higher-order factors to pairwise form

Any finite factor graph can be converted exactly to a pairwise model by introducing one auxiliary variable for each higher-order factor. For factor a with scope ∂a , define

$$z_a \in \mathcal{Z}_a := \prod_{i \in \partial a} \mathcal{X}_i.$$

Write $(z_a)_i$ for the component associated with x_i , and let $d_a = |\partial a|$. Introduce pairwise factors

$$\psi_{ia}(x_i, z_a) := f_a(z_a)^{1/d_a} \mathbb{1}(x_i = (z_a)_i), \quad i \in \partial a.$$

Then

$$\int_{z_a \in \mathcal{Z}_a} \prod_{i \in \partial a} \psi_{ia}(x_i, z_a) = f_a(\mathbf{x}_a). \quad (2.15)$$

Indeed, the consistency factors leave only the unique auxiliary state $z_a = \mathbf{x}_a$, while the product of the d_a roots reconstructs f_a . Applying the construction independently to every higher-order factor preserves the partition function and all marginals of the original variables.

Algorithm 3 implements this auxiliary-variable construction factor by factor. It is exact, but the enlarged alphabet of each z_a records the potential computational price of enforcing pairwise structure.

The reduction is exact but may be computationally unattractive. If every original variable has alphabet size D and a has degree d_a , then the auxiliary alphabet has size D^{d_a} . Pairwise structure alone therefore does not guarantee tractability; alphabet sizes and separator dimensions matter as much as graph degree.

2.4.2 Ising models: fields, interactions, loops, and frustration

For binary variables $x_i \in \{-1, +1\}$, every strictly positive pairwise model can be written as an Ising model,

$$p(\mathbf{x}) = \frac{1}{Z} \exp \left(\beta \sum_{i \in \mathcal{V}} h_i^{(\text{Ising})} x_i + \beta \sum_{(i,j) \in \mathcal{E}} J_{ij}^{(\text{Ising})} x_i x_j \right). \quad (2.16)$$

Algorithm 3 Mapping a finite factor graph to a pairwise factor graph

```

1: function FG-TO-PAIRWISE( $\mathcal{V}, \mathcal{F}, \mathcal{E}, \{\mathcal{X}_i\}, \{f_a\}$ )
2:   initialize the pairwise graph with the original variables
3:   for each factor  $a \in \mathcal{F}$  do
4:     if  $|\partial a| = 1$  then
5:       retain  $f_a$  as a unary factor
6:     else
7:       introduce  $z_a \in \prod_{i \in \partial a} \mathcal{X}_i$ 
8:       for each  $i \in \partial a$  do
9:         add edge  $(i, a)$  with factor  $\psi_{ia}(x_i, z_a) = f_a(z_a)^{1/|\partial a|} \mathbf{1}(x_i = (z_a)_i)$ 
10:  return the pairwise model over  $(\mathbf{x}, \{z_a\})$ 

```

This follows because any function on two binary variables has a unique Walsh expansion

$$\theta_{ij}(x_i, x_j) = c_{ij} + u_{i \leftarrow ij} x_i + u_{j \leftarrow ij} x_j + J_{ij}^{(\text{Ising})} x_i x_j.$$

The coefficients are obtained by averaging θ_{ij} against 1, x_i , x_j , and $x_i x_j$. In particular,

$$u_{i \leftarrow ij} = \frac{1}{4} \sum_{x_i, x_j \in \{-1, +1\}} x_i \theta_{ij}(x_i, x_j), \quad (2.17)$$

$$J_{ij}^{(\text{Ising})} = \frac{1}{4} \sum_{x_i, x_j \in \{-1, +1\}} x_i x_j \theta_{ij}(x_i, x_j). \quad (2.18)$$

The edge-induced fields $u_{i \leftarrow ij}$ are added to any unary field at node i .

Singleton and pairwise marginals may be parameterized by moments:

$$p_i(x_i) = \frac{1}{2} (1 + \mu_i x_i), \quad (2.19)$$

$$p_{ij}(x_i, x_j) = \frac{1}{4} (1 + \mu_i x_i + \mu_j x_j + \mu_{ij} x_i x_j), \quad (2.20)$$

where

$$\mu_i = \mathbb{E}[x_i], \quad (2.21)$$

$$\mu_{ij} = \mathbb{E}[x_i x_j]. \quad (2.22)$$

When $J_{ij}^{(\text{Ising})} \geq 0$, an edge favors alignment and is called ferromagnetic or attractive. Negative coupling favors anti-alignment. A cycle with an odd number of negative couplings cannot satisfy every preferred relation simultaneously and is frustrated. Frustration is the simplest indication that loops can have irreducibly global consequences.

Example 2.4.1 (A chain and a frustrated triangle). *A ferromagnetic chain provides a tree-structured baseline. An antiferromagnetic triangle is the smallest frustrated loop: every low-energy configuration must leave at least one edge unsatisfied. Comparing the two models while varying β separates the effects of temperature, topology, and competing interactions.*

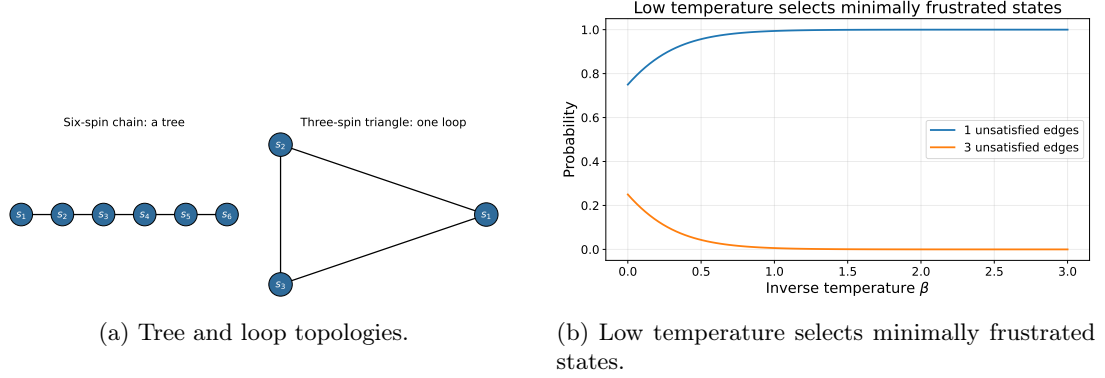


Figure 2.3: The Ising examples distinguish local interaction preferences from global compatibility. On the triangle, no assignment can satisfy all antiferromagnetic edges.

Companion notebook: Ising chains, loops, and frustration

The notebook `ch02/ch02_04_ising_chain_and_triangle.ipynb` computes exact thermodynamic and probabilistic quantities for the chain and triangle models in Fig. 2.3, visualizes correlation patterns, and counts unsatisfied edges as temperature varies.

Exercise 2.4.1. Use the companion notebook to compare antiferromagnetic cycles of odd and even length. Explain why the parity of the cycle changes the minimum number of unsatisfied edges.

Exercise 2.4.2. Verify numerically that $\partial \log Z / \partial J_{ij} = \beta \mathbb{E}[x_i x_j]$. Compare a finite-difference estimate with exact enumeration over a range of temperatures.

The Ising model connects graphical inference to statistical physics, coding, combinatorial optimization, and modern tensor-network methods. Classical exact solutions on special lattices and the theory of spin glasses provide important historical context [21, 161, 156, 13, 150].

2.4.3 Gaussian graphical models

Let $\mathbf{x} \in \mathbb{R}^N$, let $\mathbf{J}$ be symmetric positive definite, and let $\mathbf{h} \in \mathbb{R}^N$. The information-form Gaussian density is

$$p(\mathbf{x}) = \frac{1}{Z(\mathbf{J}, \mathbf{h})} \exp \left(-\frac{1}{2} \mathbf{x}^T \mathbf{J} \mathbf{x} + \mathbf{h}^T \mathbf{x} \right). \quad (2.23)$$

Completing the square gives

$$Z(\mathbf{J}, \mathbf{h}) = (2\pi)^{N/2} \det(\mathbf{J})^{-1/2} \exp \left(\frac{1}{2} \mathbf{h}^T \mathbf{J}^{-1} \mathbf{h} \right), \quad (2.24)$$

with mean and covariance

$$\bar{\mathbf{x}} = \mathbb{E}[\mathbf{x}] = \mathbf{J}^{-1} \mathbf{h}, \quad \Sigma = \mathbb{E}[(\mathbf{x} - \bar{\mathbf{x}})(\mathbf{x} - \bar{\mathbf{x}})^T] = \mathbf{J}^{-1}. \quad (2.25)$$

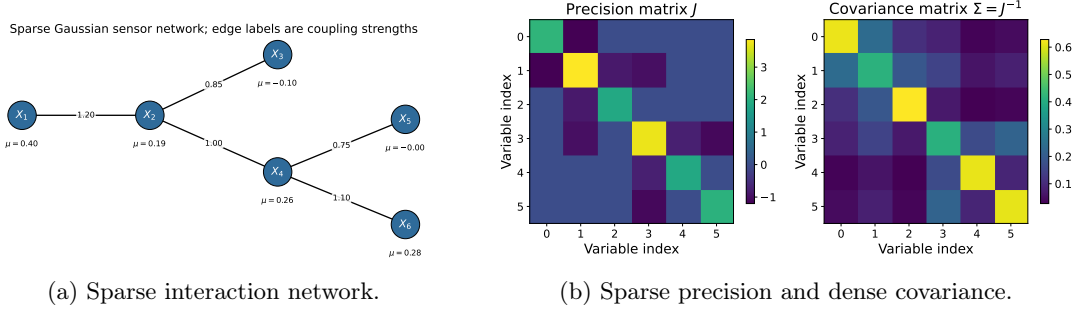


Figure 2.4: Structure in a Gaussian graphical model is encoded by the precision matrix, not generally by the covariance matrix.

The sparse matrix $\mathbf{J}$ is the precision or information matrix. For a nondegenerate Gaussian distribution,

$$J_{ij} = 0 \iff X_i \perp X_j \mid \mathbf{x}_{\mathcal{V} \setminus \{i,j\}}.$$

Thus conditional independence is sparse in the precision matrix even when the covariance matrix is dense.

The pairwise factorization is

$$p(\mathbf{x}) = \frac{1}{Z(\mathbf{J}, \mathbf{h})} \left(\prod_{i \in \mathcal{V}} f_i^{(\text{Gauss})}(x_i) \right) \left(\prod_{(i,j) \in \mathcal{E}} f_{ij}^{(\text{Gauss})}(x_i, x_j) \right), \quad (2.26)$$

$$f_i^{(\text{Gauss})}(x_i) := \exp \left(-\frac{1}{2} J_{ii} x_i^2 + h_i x_i \right), \quad (2.27)$$

$$f_{ij}^{(\text{Gauss})}(x_i, x_j) := \exp(-J_{ij} x_i x_j). \quad (2.28)$$

This representation underlies Gaussian belief propagation and sparse state estimation; see [138]. Belief propagation and exact message passing are introduced in Chapter 3, while Gaussian belief propagation is developed further in Section 13.2.

Example 2.4.2 (A sparse Gaussian sensor network). *Consider five latent scalar states coupled along a sparse network and subject to local linear information terms. The precision matrix follows the graph, while inversion produces a dense covariance. Exact Gaussian samples verify the analytic mean, covariance, marginal densities, and conditional-independence relations.*

Companion notebook: Sparse Gaussian graphical model

The notebook `ch02/ch02_05_gaussian_graphical_model.ipynb` evaluates the correct partition function for the model in Fig. 2.4, compares precision and covariance, samples the model, verifies moments, and tests conditional independence by residualization.

Exercise 2.4.3. *In the companion notebook, remove one edge by setting the corresponding off-diagonal precision entry to zero while preserving positive definiteness. Compare the graphical separation, partial correlation, and ordinary covariance before and after the modification.*

Exercise 2.4.4. *Add a noisy observation of one state as a unary Gaussian factor. Derive and verify the updated precision matrix, information vector, mean, and covariance.*

Gaussian graphical models are distinguished by closure: every marginal of a jointly Gaussian vector is again Gaussian, and every conditional distribution is Gaussian as well. Consequently, exact elimination changes only means, information vectors, and covariance or precision matrices; it never leaves the Gaussian family. Dense linear algebra computes $\det(\mathbf{J})$, $\mathbf{J}^{-1}\mathbf{h}$, and selected covariance entries in polynomial time, but typically at cubic cost in N . Sparsity can reduce this cost substantially; Sections 3.1 and 13.2 develop graph-based alternatives.

2.5 Directed graphical models and Bayesian networks

A directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ distinguishes an edge $i \rightarrow j$ from $j \rightarrow i$. Let $\pi(i)$ denote the parent set of node i ; the directed-graph notation is summarized in Appendix 23.1.3. Directed arrows can be useful bookkeeping even outside Bayesian networks, but the probabilistic semantics require care. If a directed graph contains cycles, an arbitrary collection of local normalized conditionals need not define a consistent joint law through their product. In this book the term *Bayesian network* is therefore reserved for a directed *acyclic* graph (DAG) with normalized local conditionals.

For a DAG,

$$p(\mathbf{x}) = \prod_{i \in \mathcal{V}} p_i(x_i \mid \mathbf{x}_{\pi(i)}). \quad (2.29)$$

The product is normalized, so $Z = 1$. A topological ordering gives an exact generation algorithm: sample each variable after its parents have been sampled. This is ancestral sampling, the finite-dimensional prototype of autoregressive generation. The arrows organize normalized conditionals and a sampling order; they are not, by themselves, a causal claim. Causal semantics require additional assumptions about interventions, hidden confounding, and the data-generating mechanism.

2.5.1 A Bayesian network becomes a factor graph directly

Constructing factor graph representation from a Bayesian network is straightforward as each conditional probability in Eq. (2.29) is already a factor. Introduce one factor vertex a_i with scope equal to the complete Bayesian-network family,

$$f_{a_i}(\mathbf{x}_{a_i}) := p_i(x_i \mid \mathbf{x}_{\pi(i)}), \quad \partial a_i = F_i := \{i\} \cup \pi(i). \quad (2.30)$$

Then

$$p(\mathbf{x}) = \prod_i f_{a_i}(\mathbf{x}_{a_i})$$

is exactly the factor-graph representation defined earlier in this chapter. The factor graph keeps the conditional tables and their scopes but does not need arrowheads on its incidence edges.

Example 2.5.1 (One DAG, its factor graph, and its moral graph). *Consider*

$$X_1 \longrightarrow X_3 \longleftarrow X_2, \quad X_3 \longrightarrow X_4,$$

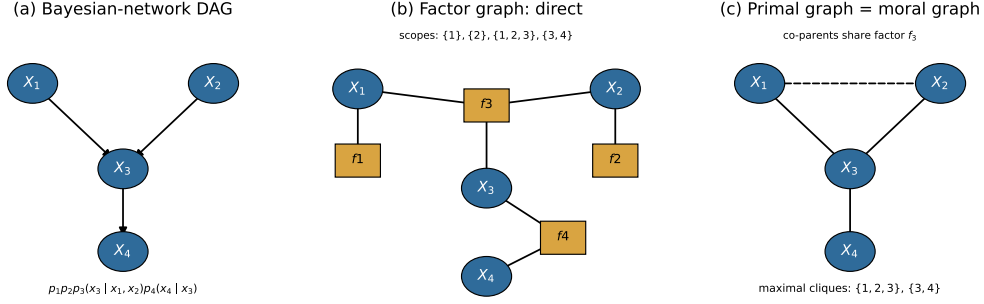


Figure 2.5: Three representations of the same Bayesian-network law in Example 2.5.1. The factor graph in the middle is constructed directly from the conditional scopes; so-called moralization – illustrated in step (c) and discussed below in Section 2.5.2 – is not needed for this step. Projecting those scopes onto variables produces the primal graph on the right, which is exactly the moral graph of the DAG. The dashed edge between the co-parents X_1 and X_2 appears because both variables occur in the family factor $p_3(x_3 | x_1, x_2)$.

with Bayesian-network factorization

$$p(x_1, x_2, x_3, x_4) = p_1(x_1)p_2(x_2)p_3(x_3 | x_1, x_2)p_4(x_4 | x_3). \quad (2.31)$$

The factor graph is obtained immediately, with scopes

$$\{1\}, \quad \{2\}, \quad \{1, 2, 3\}, \quad \{3, 4\}.$$

In particular, X_1 and X_2 both connect to the factor p_3 even though there is no directed edge between them in the DAG.

2.5.2 From factor scopes to the moral/primal graph

If we now want a *variable-only* undirected graph for elimination, clique reasoning, or junction-tree construction, we project each factor scope onto its variables: two variables are adjacent whenever they occur in a common factor. This is the *primal graph* of the factor graph. For Bayesian-network family factors, the primal graph is exactly the *moral graph* of the DAG.

The usual graphical recipe for moralization is therefore just a compact way of recovering the primal graph from the conditional scopes:

1. **Marry the parents.** (This is moral.) For every node with two or more parents, connect every pair of co-parents by an undirected edge.
2. **Drop the arrows.** Replace every remaining directed edge by an undirected edge.

The algebraic reason is Eq. (2.30): every family $F_i = \{i\} \cup \pi(i)$ occurs jointly in one factor and hence must be a clique in the primal graph.

In Example 2.5.1, the moral/primal graph has maximal cliques

$$C_1 = \{1, 2, 3\}, \quad C_2 = \{3, 4\}.$$

They overlap at X_3 ; they are not statistically independent components. The original factors can be grouped into clique potentials, for example

$$\psi_{C_1}(x_1, x_2, x_3) = p_1(x_1)p_2(x_2)p_3(x_3 \mid x_1, x_2), \quad \psi_{C_2}(x_3, x_4) = p_4(x_4 \mid x_3),$$

so that

$$p(x_1, x_2, x_3, x_4) = \psi_{C_1}(x_1, x_2, x_3)\psi_{C_2}(x_3, x_4).$$

More generally, every Bayesian-network family factor can be assigned to any maximal clique that contains its scope. Hammersley–Clifford is not required to perform this grouping: the factors were already given. For a strictly positive law, the theorem adds the converse semantic statement that the resulting clique-factorized distribution is globally Markov with respect to the undirected graph, and that every positive distribution globally Markov with respect to that graph admits such a clique factorization.

Three objects, three jobs. The DAG exposes normalized local conditionals and an ancestral sampling order. The factor graph exposes the local factor scopes directly. The moral/primal graph is a variable-only projection of those scopes and exposes which variables must coexist during generic undirected inference. Moving from the DAG to the factor graph is direct; moralization is the later DAG-to-primal-graph shortcut.

2.5.3 d-separation

The previous construction answered a representation question: *which local factors define the joint law, and which variables share those factors?* Conditional independence in the original DAG is a different question. Its calculus is *d-separation*. A non-collider on a path blocks that path when conditioned upon; a collider blocks the path when it is unobserved but can open it when the collider or one of its descendants is conditioned upon. Thus one cannot simply erase arrowheads in a DAG and use ordinary undirected separation.

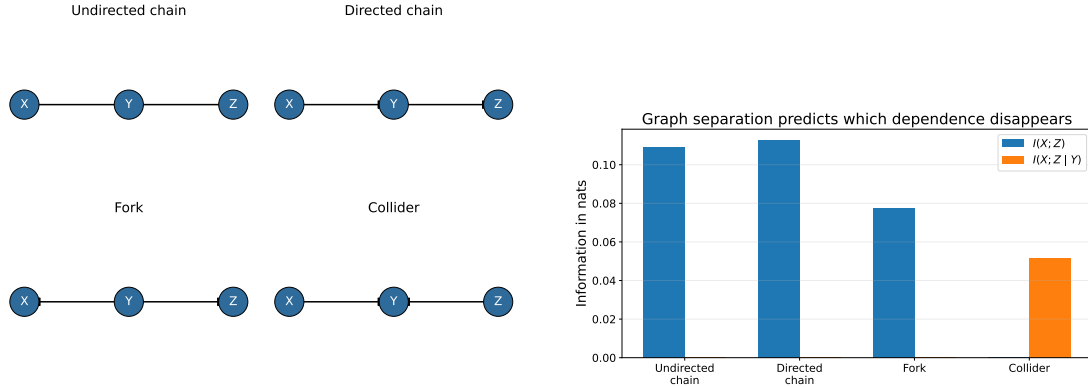
The chain, fork, and collider are the elementary motifs. A directed chain $X_1 \rightarrow X_2 \rightarrow X_3$ and a fork $X_1 \leftarrow X_2 \rightarrow X_3$ imply $X_1 \perp X_3 \mid X_2$. A collider $X_1 \rightarrow X_2 \leftarrow X_3$ has the opposite qualitative behavior: the parents may be independent marginally but become dependent after conditioning on the common child, a phenomenon known as *explaining away*. Standard references include [160, 159, 110, 24, 148].

There is a useful theorem that converts a *specific* d-separation query into ordinary graph separation. To test

$$A \perp B \mid C,$$

first restrict the DAG to the ancestors of $A \cup B \cup C$, moralize that ancestral subgraph, delete the conditioned vertices C , and test whether A and B are separated. This ancestral-moralization criterion is equivalent to d-separation [110, 120]. It should not be confused with constructing the factor graph, which was already done directly in Eq. (2.30).

For Example 2.5.1, the marginal query $X_1 \perp X_2$ keeps only the two root ancestors and therefore leaves them disconnected. For $X_1 \perp X_2 \mid X_3$, the ancestral graph contains X_1, X_2, X_3 ; moralization marries the parents, and deleting X_3 leaves the edge $X_1 - X_2$. Conditioning on the common effect therefore creates dependence between the causes.



(a) Undirected and directed separation motifs. (b) Mutual and conditional mutual information.

Figure 2.6: The collider distinguishes d-separation from ordinary undirected separation: conditioning can create dependence rather than remove it.

Companion notebook: From DAG scopes to factor graphs, moral graphs, and d-separation

The notebook `ch02/ch02_03_conditional_independence.ipynb` separates the two constructions computationally. It first builds the factor scopes of Example 2.5.1 and produces Fig. 2.5. It then implements ancestral moralization as a d-separation test, reproduces Fig. 2.7, and verifies the corresponding dependence and conditional dependence numerically through mutual information.

Exercise 2.5.1. For the directed chain $X_1 \rightarrow X_2 \rightarrow X_3$, the fork $X_1 \leftarrow X_2 \rightarrow X_3$, and the collider $X_1 \rightarrow X_2 \leftarrow X_3$, first write the Bayesian-network factorization and construct the factor graph without moralization. Then construct the primal graph of the factor scopes and verify that it equals the moral graph. Which motif creates a new co-parent edge?

Exercise 2.5.2. For the DAG in Example 2.5.1, identify the maximal cliques of the moral graph and assign each original conditional factor to a maximal clique containing its scope. Give two different valid assignments when possible and explain why clique potentials are not unique.

Exercise 2.5.3. For the same DAG, apply ancestral moralization to the queries $X_1 \perp X_2$, $X_1 \perp X_2 \mid X_3$, and $X_1 \perp X_2 \mid X_4$. Draw the graph used for each query and verify the answers numerically with a strictly positive binary Bayesian network.

Exercise 2.5.4. Let one child have k parents and no other incoming edges. Show that the factor graph contains one $(k+1)$ -variable family factor and that the moral/primal graph contains a $(k+1)$ -clique. Explain why bounded in-degree controls individual conditional-table size but does not, by itself, guarantee small global treewidth when many such family scopes overlap.

Problem 2.1 (What is lost when arrows are removed? ²). Construct two distinct DAGs on the same small vertex set that have the same full moral graph. Compare their d-separation

²Throughout the book, a problem develops a larger than exercise synthesis task. Problems are intended to

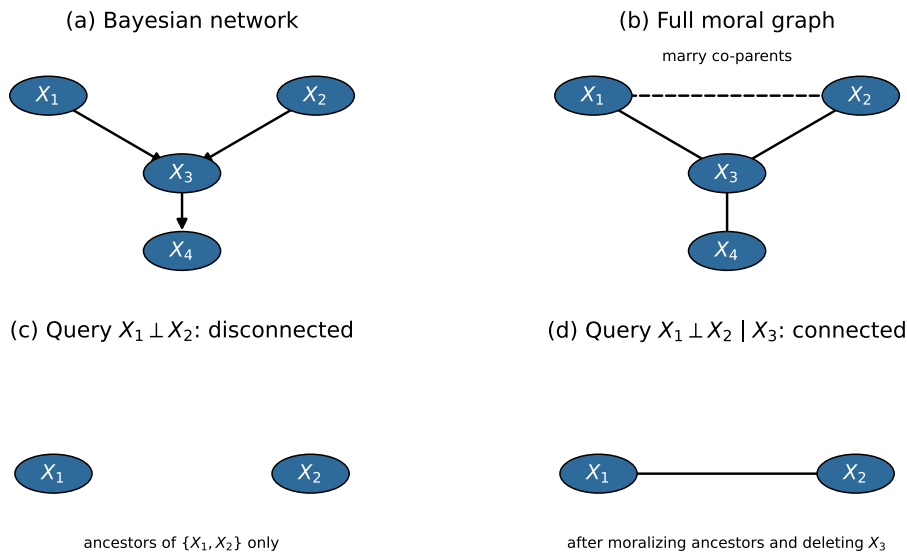


Figure 2.7: Query-specific ancestral moralization for Example 2.5.1. The full moral graph is useful as a variable-only inference representation, while the bottom panels answer two particular d-separation queries after first restricting to the appropriate ancestral subgraphs.

statements and their ancestral-sampling factorizations. Identify which distinctions survive in the factor graph and moral graph and which are lost. Then describe what additional information would be required to reconstruct a directed generative model from the undirected representation alone.

Advanced connection: from factor scopes toward a junction tree

A factor graph can be used directly by many message-passing algorithms. For exact inference on a loopy model, one common next step is to project the factor scopes to a primal graph, add triangulation/fill-in edges, collect the resulting maximal cliques, and connect those cliques into a junction tree satisfying the running-intersection property. For a Bayesian network this gives the schematic path

DAG $\longrightarrow$ family factor graph $\longrightarrow$ moral/primal graph $\longrightarrow$ triangulation $\longrightarrow$ junction tree.

The d-separation test above is not a required step in this construction. Chapter 3 develops elimination and junction trees in detail.

be more challenging than the exercises and are often more open ended. They may require combining several constructions, making modeling choices, or proposing a computational investigation rather than carrying out a single prescribed calculation.

Algorithmic lens: ancestral sampling

Choose a topological ordering $i_1, \dots, i_N$. For $k = 1, \dots, N$, sample $x_{i_k} \sim p_{i_k}(\cdot \mid \mathbf{x}_{\pi(i_k)})$. If each conditional can be evaluated and sampled efficiently and the in-degree is controlled, one complete sample is generated in time proportional to the number of local conditional evaluations.

Advanced connection: dynamic graphical models and path-space generation

A Markov process becomes a DAG after time is unfolded. For a network-valued state $\mathbf{x}_t$, a locally factorized path distribution has the form

$$p(\mathbf{x}_{0:T}) = p_0(\mathbf{x}_0) \prod_{t=1}^T \prod_{i \in \mathcal{V}} p_{t,i}(x_{t,i} \mid \mathbf{x}_{t-1, \pi(i)}). \quad (2.32)$$

The spatial graph determines which components of the previous state influence each local transition. The time-unrolled graph is acyclic even when the spatial graph contains loops. This construction is the bridge from static graphical models to stochastic processes, Monte Carlo dynamics, discrete diffusion, bridge sampling, and controlled generation.

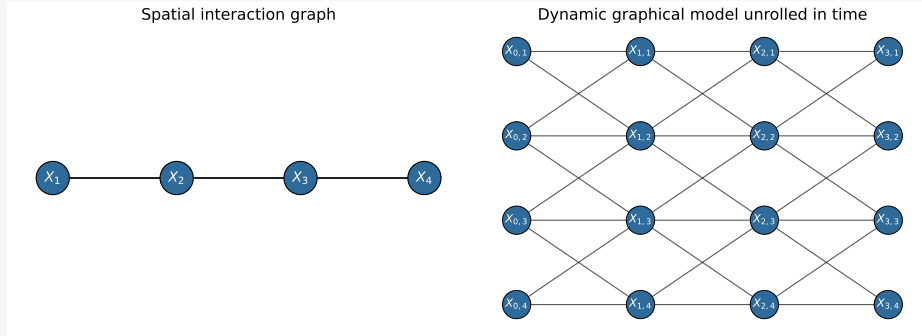


Figure 2.8: A spatial interaction graph and its time-unrolled directed representation.

Figure 2.8 makes explicit how spatial loops become an acyclic graph after time is unfolded. The notebook `ch02/ch02_06_dynamic_graphical_model.ipynb` constructs a small binary network process, forms its exact transition matrix, propagates the state distribution, generates trajectories, and compares exact inference with Monte Carlo. The more systematic theory is deferred to the stochastic-process and structured-generation part of the book.

Exercise 2.5.5. *In the companion notebook, introduce a time-dependent intervention that suppresses activation at one node. Compare exact final-time marginals, Monte Carlo estimates, and the probability of a rare high-activity event.*

Advanced connection: where neural networks enter

The graphical model specifies *which* local objects are needed; a neural network may param-

eterize one of those objects. Typical replacements are

$$f_a(\mathbf{x}_a) \longrightarrow f_{a,\theta}(\mathbf{x}_a), \quad p_i(x_i \mid \mathbf{x}_{\pi(i)}) \longrightarrow p_{i,\theta}(x_i \mid \mathbf{x}_{\pi(i)}),$$

with analogous neural parameterizations of energies, messages, approximate posteriors, transport maps, scores, drifts, and controls. The graph contributes locality, conditional structure, symmetries, or constraints; the neural network contributes flexible approximation. Boltzmann machines, restricted Boltzmann machines, neural message passing, variational autoencoders, autoregressive networks, and diffusion models instantiate this principle in different ways. Their detailed treatment belongs later in the book, after the underlying inference and stochastic-process objects have been developed.

2.6 What graphical structure buys us

Graphical representation is useful only insofar as it exposes computational or modeling structure. The same local factorization may support several different algorithms, and the best representation may depend on the task.

Visible structure	Immediate consequence	Later development
Tree factor graph	exact elimination and message passing	belief propagation and dynamic programming
Sparse separators	complexity controlled by separator size	junction trees and approximate elimination
Normal factor graph	local edge-variable contractions	gauges, loop series, and tensor networks
Pairwise binary interactions	fields, correlations, frustration	Ising inference, optimization, and learning
Sparse Gaussian precision	conditional independence and sparse algebra	Gaussian BP, filtering, and state estimation
DAG factorization	normalized local conditionals and exact ancestral generation	autoregressive models and causal inference
Bayesian-network factor graph	conditional-family scopes made explicit	generic factor-graph algorithms and local computation
Moral/primal graph	variable-only projection of factor scopes	elimination complexity, triangulation, and junction trees
Time-unrolled Markov structure	local path probabilities	diffusion, bridges, and stochastic control
Neural local objects	flexible learned functions with explicit structure	structured generative AI for science

The central challenge is now visible. A local description can be compact, but global computations may still require summing or optimizing over exponentially many configurations. The next chapters ask when graph structure makes these tasks exact and efficient, how approximation enters when it does not, and how the same representations support learning and generation.

2.7 Further reading and synthesis problems

General references on graphical models include [202, 110, 24, 148]. Normal factor graphs and their transformation theory are discussed in [132, 5, 70]. Connections to loop calculus and holographic or tensor-network transformations appear in [46, 47, 145]. Gaussian graphical models and walk-sum interpretations are developed in [138].

Problem 2.2 (Three structured models). *For each of the following models, construct a small explicit instance, write its natural graphical representation, and identify the representation, inference, optimization, learning, and generation tasks.*

1. *A weighted matching model relating two finite sets, with local compatibility weights and degree constraints.*
2. *A ternary parity-check model in which each check requires a subset sum to vanish modulo three.*
3. *A flow model on a directed network with real-valued edge flows, local conservation constraints, and soft penalties for nodal imbalance.*

For each instance:

1. *write the factor-graph or normal-factor-graph representation;*
2. *convert it to the other representation using Algorithms 1 or 2;*
3. *determine whether a useful pairwise reduction exists and quantify the auxiliary alphabet sizes;*
4. *identify transformations of local factors that preserve the global distribution or partition function;*
5. *propose one exact-enumeration notebook experiment and one later-chapter algorithm that could replace enumeration.*

Chapter 3

Exact Elimination, Dynamic Programming, and Message Passing

Reader's Guide.

Why this chapter? Chapter 2 taught us how to write one large probability law as a product of small local factors. The first computational question is then almost embarrassingly simple: *can we use ordinary distributivity to avoid summing over every complete configuration at once?* Exact elimination is the systematic answer. It is the graphical-model version of doing the inside of a nested sum first, remembering only the result needed by the variables that remain.

What you should know before starting. You need only the factorized weight and partition function from Chapter 2, together with the idea that summation or integration can be reordered. No prior knowledge of dynamic programming, treewidth, junction trees, or message passing is assumed. The chapter introduces these notions as different descriptions of the same elimination mechanism.

A two-step calculation before the general algorithm. Consider three discrete variables with

$$w(x_1, x_2, x_3) = f_{12}(x_1, x_2)f_{23}(x_2, x_3).$$

A direct calculation of $Z = \sum_{x_1, x_2, x_3} w$ enumerates every triple. Instead, first summarize everything on the left of x_2 by

$$m_{1 \rightarrow 2}(x_2) := \sum_{x_1} f_{12}(x_1, x_2),$$

and then compute

$$Z = \sum_{x_2, x_3} m_{1 \rightarrow 2}(x_2)f_{23}(x_2, x_3).$$

The function $m_{1 \rightarrow 2}$ is already a *message*: a cached partial sum carrying exactly the information that the eliminated variable x_1 can transmit to the rest of the problem. Variable elimination, dynamic programming, and sum-product message passing differ mainly in how systematically we organize such partial sums.

What you should be able to do after reading. You should be able to eliminate variables exactly; read fill-in and separator size from an elimination order; explain why trees are easy; derive sum-product messages as partial partition functions; and understand a junction tree as an exact way of grouping variables until the remaining interaction graph is a tree. The main lesson is not a particular formula but a reusable algorithmic pattern: *combine locally, eliminate, cache the separator function, and continue*.

How to use the companion notebooks. The notebooks in notebooks/ch03/ should be read as increasingly structured versions of the same computation. First compare elimination orders and their induced widths; then run exact sum-product on a tree and check every marginal against enumeration; finally use the junction-tree notebook to see how clustering restores tree-like message passing on a loopy graph. On a first pass, small examples by hand are more valuable than optimizing the code.

Chapter 2 introduced a high-dimensional probability distribution as a product of local factors. This chapter asks when that local representation can be converted into an exact computation. The common mechanism is *elimination*: select a variable, combine all factors that depend on it, and sum or integrate it out. The resulting factor summarizes everything the eliminated variable contributed to the remaining problem.

The same mechanism appears under several names. In probability and graphical models it is called *variable elimination* or *bucket elimination* (if a bucket, including a number of variables is eliminated); in control and optimization it is a form of *dynamic programming*; in sparse linear algebra it parallels elimination and fill-in; and in tensor networks it is a contraction order. The terminology and emphases differ, but the computational object is the same: an intermediate function defined on a separator between eliminated and surviving variables. General accounts of these connections can be found in [16, 57, 108].

The chapter develops three progressively more structured situations:

1. **exact elimination on an arbitrary factor graph**, where intermediate factors may grow exponentially;
2. **exact sum-product message passing on a tree**, where no fill-in is created and the cost is linear in the number of graph elements for fixed local alphabet and degree;
3. **exact inference on a loopy graph through a junction tree**, where the exponential cost is isolated in the largest cluster or separator.

The examples are implemented in the companion notebooks under ch03/. They will be reused later when exact intermediate factors are replaced by approximations, low-rank tensors, variational messages, or learned surrogates.

3.1 Exact elimination on an arbitrary graph

3.1.1 From global summation to local elimination

Consider a factor graph with variables $\mathbf{x} = (x_1, \dots, x_N)$, factors $\{f_a\}_{a \in \mathcal{F}}$, and partition function

$$Z = \int_{\mathbf{x} \in \mathcal{X}} \prod_{a \in \mathcal{F}} f_a(\mathbf{x}_a). \quad (3.1)$$

Direct enumeration treats the product as one dense function of all variables. Elimination instead exploits the distributive law. Let x_i be the next variable to eliminate, and let

$$\mathcal{F}_i := \{a \in \mathcal{F} \mid i \in \partial a\}$$

be the set of factors whose scopes contain x_i . Their product is a temporary local function. Summing over x_i creates the new factor

$$g_i(\mathbf{x}_{S_i}) := \int_{x_i} \prod_{a \in \mathcal{F}_i} f_a(\mathbf{x}_a), \quad S_i := \left(\bigcup_{a \in \mathcal{F}_i} \partial a \right) \setminus \{i\}. \quad (3.2) \quad \text{👉}$$

The set S_i is the *separator* left behind by eliminating x_i . All factors in $\mathcal{F}_i$ are replaced by g_i ; factors not involving x_i remain unchanged. Eq. (3.2) is an exact algebraic identity, not an approximation.

Read Eq. (3.2) as “summarize and forget.” All factors that mention x_i are temporarily multiplied together. We then sum over x_i , so x_i disappears, but its influence does not: it is stored in g_i , whose arguments are precisely the surviving variables that had interacted with x_i . The computational danger is now visible before any complexity theorem is stated—if many surviving variables meet in g_i , the table for g_i becomes large. Separator size is therefore the natural measure of the cost created by one elimination step.

The operation is summarized in Algorithm 4. The algorithm makes explicit that the input and output have the same mathematical type: a list of factors. Only their scopes and numerical tables change.

Algorithm 4 Eliminate one variable from a factorized model

```

function ELIMINATEVARIABLE( $\mathcal{F}, i$ )
   $\mathcal{F}_i \leftarrow \{a \in \mathcal{F} \mid i \in \partial a\}$ 
   $S_i \leftarrow (\bigcup_{a \in \mathcal{F}_i} \partial a) \setminus \{i\}$ 
   $g_i(\mathbf{x}_{S_i}) \leftarrow \int_{x_i} \prod_{a \in \mathcal{F}_i} f_a(\mathbf{x}_a)$ 
   $\mathcal{F} \leftarrow (\mathcal{F} \setminus \mathcal{F}_i) \cup \{g_i\}$ 
  return  $\mathcal{F}, S_i$ 

```

Applying Algorithm 4 repeatedly yields Algorithm 5. An *elimination order* $\pi = (\pi_1, \dots, \pi_N)$ is a permutation of the variables. Different orders produce the same exact Z , but generally produce different intermediate factors.

Algorithm 5 Exact variable elimination for the partition function

```

function EXACTELIMINATION( $\mathcal{F}, \pi$ )
  for  $t \leftarrow 1$  to  $N$  do
    ( $\mathcal{F}, S_{\pi_t}$ )  $\leftarrow$  ELIMINATEVARIABLE( $\mathcal{F}, \pi_t$ )
    record the separator size  $|S_{\pi_t}|$ 
  multiply the remaining constant factors
  return  $Z$ 

```

Algorithm 5 is the exact dynamic-programming template used throughout the book. To compute a marginal rather than Z , one leaves the requested variables uneliminated. To compute a maximum-a-posteriori value, one replaces summation by maximization. To contract a tensor network, one interprets variables as indices and factors as tensors. The algebra changes only through the local elimination operator.

Algorithmic lens: Elimination as a reusable computational pattern

The basic pattern is *collect–combine–eliminate–replace*. First collect all local objects containing the selected variable. Then combine them, eliminate that variable, and replace the original objects by the resulting separator function. Later chapters will alter only the representation of the separator function: exact tables here, bounded-width approximations in Chapter 7, and tensor or neural representations in later parts of the book.

3.1.2 Bayesian networks: from family factors to the elimination graph

Variable elimination does not require a separate algorithm for Bayesian networks, and it does not require d-separation. First convert the DAG directly to a factor graph by treating each conditional probability $p_i(x_i \mid x_{\text{pa}(i)})$ as a factor on its family $\{i\} \cup \text{pa}(i)$, exactly as in Section 2.5.1. Evidence can then be absorbed into the relevant factors and Algorithm 4 applied without any further probabilistic translation.

If we want a variable-only graph for analyzing elimination complexity, we form the primal graph of those family scopes. For a Bayesian network this primal graph is precisely the moral graph of Section 2.5.2: co-parents are adjacent because they occur together in one conditional factor. Moralization is therefore best understood here as a shortcut from the DAG directly to the same primal graph that the factor scopes already determine.

This observation has an important complexity consequence. The directed skeleton and maximum in-degree do not by themselves determine the cost of exact posterior inference. Exact elimination is controlled by induced width after the relevant factor scopes have been projected to the primal/moral graph and fill-in has been added by the chosen elimination order. One may first prune variables irrelevant to a particular inference task, for example by keeping the ancestral submodel needed by the queried variables and evidence, but this preprocessing should not be confused with running a d-separation test. A node with k parents already creates a $(k + 1)$ -variable family factor; overlapping family scopes can then create large cliques and further fill-in.

Factor adjacency, moral edges, and fill-in are different layers. In the factor graph, co-parents meet at the child conditional factor. In the primal/moral graph, that shared factor is represented by a co-parent edge. During elimination, still more edges may be added as fill-in. None of these edges is a new causal assertion: they record increasingly coarse summaries of which variables must coexist in exact computations.

Exercise 3.1.1. Start from the Bayesian network in Example 2.5.1. Write its four conditional factors and draw the factor graph directly. Then construct the primal graph of those scopes and verify that it is the moral graph in Fig. 2.5. Eliminate X_4 , then X_3 , and record the separators. Compare with an order that eliminates X_3 first and explain every resulting adjacency as factor-scope adjacency, a moral co-parent edge in the variable-only projection, or elimination fill-in.

Exercise 3.1.2. For the same example, note that the moral graph is already chordal. Construct a junction tree with clusters $\{1, 2, 3\}$ and $\{3, 4\}$, identify the separator, and assign the original Bayesian-network factors to clusters. Explain why no d -separation calculation is required anywhere in this construction.

3.1.3 Fill-in, induced width, and treewidth

The difficulty of exact elimination lies in the size of g_i . Suppose every variable has an alphabet of size D . A separator containing k variables requires a dense table with D^k entries. Thus the cost is controlled by the largest separator encountered along the order, not simply by the number of original factors.

The same growth can be accessed graphically. In the primal graph, two variables are adjacent whenever they occur in a common factor. Eliminating x_i connects all surviving neighbors of i , because they now appear together in g_i . New connections created in this way are called *fill-in edges*. For an order π , its *induced width* is the largest number of surviving neighbors at an elimination step. The minimum induced width over all orders is the graph's *treewidth*. Equivalent definitions use tree decompositions and junction trees; see [59, 108, 120].

Finding an optimal elimination order is itself computationally difficult. Practical implementations therefore use heuristics such as minimum degree, minimum fill, nested dissection, or problem-specific contraction strategies. These heuristics depend primarily on graph structure; numerical magnitudes or approximate low-rank structure may motivate different choices. The latter possibility becomes important in approximate elimination and tensor-network contraction.

Example 3.1.1 (A 3×3 binary grid). Consider a pairwise Ising model on a 3×3 grid. The graph has nine binary variables and twelve pairwise factors. All $2^9 = 512$ configurations can still be enumerated, so exact enumeration provides a check on elimination. Row-major, center-first, and minimum-fill orders all return the same partition function, but their largest intermediate tables differ.

Companion notebook: Elimination order and treewidth

The notebook `ch03/ch03_01_elimination_order_and_treewidth.ipynb` implements exact table-based elimination, verifies the partition function by enumeration, constructs fill-in edges, and compares several elimination orders. Fig. 3.1 reports results of and is generated by the notebook. The implementation is deliberately generic: changing the local factors

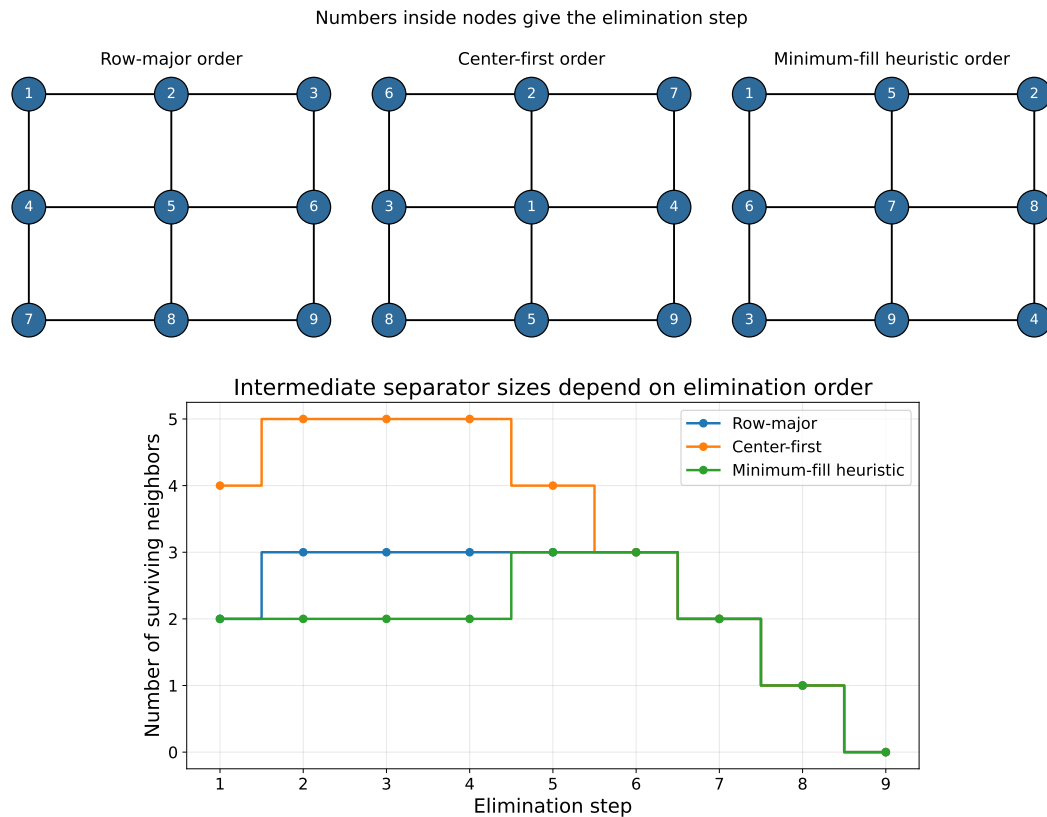


Figure 3.1: Exact elimination on the grid of Example 3.1.1. Top: three elimination orders, with the elimination step shown inside each node. Bottom: separator size (as a measure of the remaining graph complexity) along each order. The result is order independent, but the intermediate complexity is not.

does not change the elimination engine.

Exercise 3.1.3. In `ch03/ch03_01_elimination_order_and_treewidth.ipynb`, replace the grid by a chain of nine variables. Compare a leaf-to-root order with an order that eliminates the middle variable first. Explain the difference through fill-in.

Exercise 3.1.4. Implement the minimum-degree heuristic and compare it with the minimum-fill heuristic. Construct a graph on which the two heuristics choose different first variables.

Exercise 3.1.5. Replace summation by maximization in the elimination routine. Verify the resulting MAP value and configuration by enumeration. Identify which steps require storing back-pointers if the maximizing configuration itself is desired.

Advanced connection: to sparse linear algebra and tensor contraction

For a Gaussian graphical model, elimination is sparse Gaussian elimination and fill-in appears in a Cholesky factor. For a tensor network, factor multiplication and marginalization become tensor contraction and index summation. In all three settings, the contraction or elimination order controls intermediate dimension. Modern tensor-network counting algorithms make this connection explicit [113].

3.2 Exact sum-product message passing on trees

Why a tree turns elimination into message passing. Return to the three-variable chain in the reader’s guide. Eliminating the leaf x_1 creates a function only of x_2 ; no new interaction between previously unrelated variables can appear. The same is true at every leaf of every tree. Thus an elimination can be sent inward as a one-edge function and later sent back outward to recover all marginals. Calling these separator functions “messages” emphasizes that the same local summary can be reused by neighboring computations rather than recomputed from scratch.

A tree is the exceptional case in which elimination can proceed from leaves inward without creating fill-in. Every eliminated leaf leaves behind a function of only the variable on the edge connecting it to the remaining tree. These one-edge summaries are the messages of the sum-product algorithm. The algorithm is exact because removing any edge separates the tree into two disjoint components.

3.2.1 Tree distributions from consistent marginals

Before deriving the algorithm, it is useful to state a structural property of tree distributions: locally consistent node and edge marginals determine the complete joint distribution.

3.2.1.1 Pairwise models

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a tree, let $p_i(x_i)$ denote a singleton marginal, and let $p_{ij}(x_i, x_j)$ denote the marginal on edge (i, j) . Assume the marginals are normalized and locally consistent,

$$\int_{x_j} p_{ij}(x_i, x_j) = p_i(x_i), \quad \int_{x_i} p_{ij}(x_i, x_j) = p_j(x_j).$$

Then the joint distribution is

$$p(\mathbf{x}) = \frac{\prod_{(i,j) \in \mathcal{E}} p_{ij}(x_i, x_j)}{\prod_{i \in \mathcal{V}} p_i(x_i)^{d_i - 1}}, \quad (3.3)$$

where d_i is the degree of node i . A proof follows by removing a leaf and applying the consistency relation, then continuing inductively. Eq. (3.3) is exact only because the graph is a tree. On a loopy graph the same expression becomes the Bethe reconstruction, studied later as a variational approximation.

3.2.1.2 Factor graphs

Suppose the bipartite factor graph is a tree. Let $b_a(\mathbf{x}_a)$ be the marginal on factor scope ∂a , and let $b_i(x_i)$ be the singleton marginal. If

$$\int_{\mathbf{x}_a \setminus x_i} b_a(\mathbf{x}_a) = b_i(x_i) \quad \text{for every } i \in \partial a,$$

then

$$p(\mathbf{x}) = \frac{\prod_{a \in \mathcal{F}} b_a(\mathbf{x}_a)}{\prod_{i \in \mathcal{V}} b_i(x_i)^{d_i-1}}. \quad (3.4)$$

This is the factor-graph analogue of Eq. (3.3). It makes clear why exact inference on a tree requires only local marginals and their overlap consistency.

3.2.1.3 Normal factor graphs

For a normal factor graph that is a tree, let $p_a(\mathbf{x}_a)$ be the marginal associated with factor node a , and let $p_{ab}(x_{ab})$ be the marginal of the edge variable shared by neighboring factors a and b . Then

$$p(\mathbf{x}) = \frac{\prod_{a \in \mathcal{V}} p_a(\mathbf{x}_a)}{\prod_{(a,b) \in \mathcal{E}} p_{ab}(x_{ab})}. \quad (3.5)$$

Again, a leaf-removal proof uses only normalization and marginal consistency. This NFG form will be especially useful for reparametrizations, gauges, loop series, and tensor-network interpretations.

3.2.2 Messages as partial partition functions

Choose an arbitrary root. For every directed edge $i \rightarrow j$ in a pairwise tree, removing edge (i, j) defines a component containing i . The message $m_{i \rightarrow j}(x_j)$ is the partition function of that entire component with x_j held fixed. It satisfies

$$m_{i \rightarrow j}(x_j) = \int_{x_i} \psi_i(x_i) \psi_{ij}(x_i, x_j) \prod_{k \in \partial i \setminus \{j\}} m_{k \rightarrow i}(x_i). \quad (3.6)$$



The term **message** refers to this local summary being transmitted across an edge; **sum-product** refers to the alternating multiplication and summation in Eq. (3.6); and **belief propagation** refers to the local marginal estimates obtained from the messages. Pearl introduced and popularized this terminology in probabilistic reasoning [160].

For a general factor graph, it is convenient to distinguish variable-to-factor and factor-to-variable messages:

$$m_{i \rightarrow a}(x_i) := \prod_{b \in \partial i \setminus \{a\}} m_{b \rightarrow i}(x_i), \quad (3.7) \quad \text{👉}$$

$$m_{a \rightarrow i}(x_i) := \int_{\mathbf{x}_a \setminus x_i} f_a(\mathbf{x}_a) \prod_{j \in \partial a \setminus \{i\}} m_{j \rightarrow a}(x_j). \quad (3.8) \quad \text{👉}$$

On a tree, a leaf-to-root pass computes all messages directed toward the root, and a root-to-leaf pass computes the reverse messages. No iterative convergence test is needed.

The corresponding singleton and factor beliefs are

$$b_i(x_i) := \frac{1}{Z_i} \prod_{a \in \partial i} m_{a \rightarrow i}(x_i), \quad (3.9)$$

$$b_a(\mathbf{x}_a) := \frac{1}{Z_a} f_a(\mathbf{x}_a) \prod_{i \in \partial a} m_{i \rightarrow a}(x_i), \quad (3.10)$$

where Z_i and Z_a normalize the respective beliefs. On a tree these beliefs are the exact marginals and satisfy the consistency conditions used in Eq. (3.4).

Algorithm 6 Exact two-pass sum-product on a tree factor graph

```

function TREESUMPRODUCT( $\mathcal{G}, \{f_a\}, r$ )
  choose an arbitrary root  $r$ 
  orient every edge toward  $r$ 
  for nodes in leaf-to-root order do
    compute the unique outgoing message using Eqs. (3.7)–(3.8)
  reverse all edge orientations
  for nodes in root-to-leaf order do
    compute the remaining reverse message
  compute beliefs using Eqs. (3.9)–(3.10)
  return exact marginals and  $Z$ 

```

Algorithm 6 requires one message in each direction along every edge. For bounded degree and fixed alphabets, its cost is linear in the number of nodes and factors. Message normalization may be introduced for numerical stability; multiplying a message by a positive constant does not change normalized beliefs.

For later use, the same update in NFG notation is

$$M_{a \rightarrow b}(x_{ab}) = \int_{\mathbf{x}_a \setminus x_{ab}} f_a(\mathbf{x}_a) \prod_{c \in \partial a \setminus \{b\}} M_{c \rightarrow a}(x_{ac}). \quad (3.11) \quad \text{👉}$$

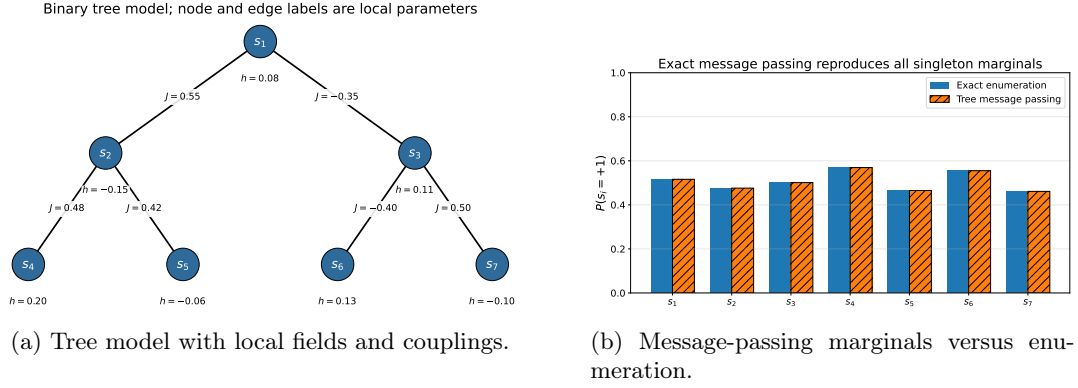


Figure 3.2: Exact sum-product inference for Example 3.2.1. Each message summarizes one component created by cutting an edge.

The edge and node beliefs are

$$B_{ab}(x_{ab}) := \frac{M_{a \rightarrow b}(x_{ab})M_{b \rightarrow a}(x_{ab})}{\int_{x_{ab}} M_{a \rightarrow b}(x_{ab})M_{b \rightarrow a}(x_{ab})}, \quad (3.12)$$

$$B_a(\mathbf{x}_a) := \frac{f_a(\mathbf{x}_a) \prod_{c \in \partial a} M_{c \rightarrow a}(x_{ac})}{\int_{\mathbf{x}_a} f_a(\mathbf{x}_a) \prod_{c \in \partial a} M_{c \rightarrow a}(x_{ac})}. \quad (3.13)$$

They satisfy

$$B_{ab}(x_{ab}) = \int_{\mathbf{x}_a \setminus x_{ab}} B_a(\mathbf{x}_a) = \int_{\mathbf{x}_b \setminus x_{ab}} B_b(\mathbf{x}_b). \quad (3.14)$$

Eq. (3.14) is the local-consistency statement that underlies the exact tree factorization in Eq. (3.5).

Example 3.2.1 (Exact messages on a seven-node binary tree). *A binary pairwise model is placed on a balanced seven-node tree. Every directed edge carries a two-component message. Exact enumeration over $2^7 = 128$ states verifies the partition function and all singleton marginals returned by Algorithm 6.*

Companion notebook: Exact sum-product on a tree

The notebook `ch03/ch03_02_sum_product_on_trees.ipynb` constructs the tree in Fig. 3.2, evaluates all directed messages, computes exact marginals and Z , and checks them against enumeration. It also displays the complete collection of normalized messages, making the partial-partition-function interpretation explicit.

Exercise 3.2.1. *Change the root in `ch03/ch03_02_sum_product_on_trees.ipynb`. Verify that the message schedule changes but the final marginals and partition function do not.*

Exercise 3.2.2. Derive an expression for the pairwise belief $b_{ij}(x_i, x_j)$ in terms of the two directed messages on edge (i, j) and the incoming messages from the remaining neighbors. Implement it and compare with enumeration.

Exercise 3.2.3. Add one edge to the tree and iterate the same local updates. Compare the resulting beliefs with exact enumeration. Explain precisely which partial-partition-function interpretation is lost.

Advanced connection: Other semirings and dynamic-programming objectives

The sum-product equations use ordinary addition and multiplication. Replacing the sum by a maximum gives max-product or max-sum message passing for MAP optimization. Replacing the local operations by another distributive pair leads to the generalized distributive law, which unifies many dynamic-programming algorithms. The tree topology controls exactness; the chosen algebra controls the task being solved.

3.3 Junction trees: exact inference on loopy graphs

Junction trees in one sentence. A loopy graph may be hard because eliminating one variable makes several surviving variables interact. A junction tree accepts this price explicitly: put variables that must interact together into a *cluster*, connect the clusters so that they form a tree, and run the same exact message-passing logic on cluster states. Exactness is recovered because the cluster graph is a tree; the remaining cost is exponential in cluster size rather than in the total number of original variables.

A loopy graph can be transformed into a tree of overlapping clusters. The transformation does not make a hard problem easy; rather, it moves the exponential complexity into the cluster state spaces and makes the required separators explicit.

3.3.1 Clusters, separators, and the running-intersection property

A **junction tree** consists of clusters $C \subseteq \mathcal{V}$ connected as a tree. Every original factor scope must be contained in at least one cluster. Moreover, for every variable i , the set of clusters containing i must form a connected subtree. This condition is called the *running-intersection property*. If neighboring clusters C and D are connected, their intersection

$$S_{CD} := C \cap D$$

acts as a separator.

Junction trees can be obtained from an elimination order or from a triangulated, also called chordal, graph. Eliminating a variable makes its current neighborhood a clique. The maximal cliques created by the elimination define candidate clusters, and a maximum-weight spanning tree over clique intersections can be used to construct the cluster tree.

For a generic factor graph, the structural pipeline is

$$\begin{aligned} \text{factor scopes} &\longrightarrow \text{primal graph} \longrightarrow \text{triangulated graph} \\ &\longrightarrow \text{maximal cliques} \longrightarrow \text{junction tree.} \end{aligned}$$

For a Bayesian network, the first two stages can be displayed more explicitly as

$$\begin{aligned} \text{DAG} &\longrightarrow \text{family factor graph} \longrightarrow \text{moral/primal graph} \\ &\longrightarrow \text{triangulation} \longrightarrow \text{junction tree.} \end{aligned}$$

The factor graph is obtained directly from the conditional tables; moralization is the variable-only projection of those family scopes. Triangulation then adds only the extra fill-in needed for a chordal cluster representation. Ancestral pruning may reduce the model before this pipeline for a particular query, but d-separation is a conditional-independence test, not a mandatory junction-tree construction step. Standard treatments are given in [121, 120, 108].

If $b_C(\mathbf{x}_C)$ are normalized cluster marginals and $b_{S_{CD}}(\mathbf{x}_{S_{CD}})$ are their consistent separator marginals, then the exact joint distribution factors as

$$p(\mathbf{x}) = \frac{\prod_{C \in \mathcal{C}} b_C(\mathbf{x}_C)}{\prod_{(C,D) \in \mathcal{E}_{JT}} b_{S_{CD}}(\mathbf{x}_{S_{CD}})}. \quad (3.15)$$

This is the cluster analogue of Eqs. (3.3), (3.4), and (3.5).

3.3.2 Clique-to-clique messages

Assign every original factor to one cluster containing its scope, and let $\Phi_C(\mathbf{x}_C)$ be the product of factors assigned to cluster C . For neighboring clusters C and D , the message from C to D is

$$m_{C \rightarrow D}(\mathbf{x}_{S_{CD}}) = \int_{\mathbf{x}_C \setminus \mathbf{x}_{S_{CD}}} \Phi_C(\mathbf{x}_C) \prod_{B \in \partial C \setminus \{D\}} m_{B \rightarrow C}(\mathbf{x}_{S_{BC}}). \quad (3.16) \quad \text{👉}$$

Eq. (3.16) is simply tree sum-product with cluster-valued nodes. The calibrated cluster belief is

$$b_C(\mathbf{x}_C) \propto \Phi_C(\mathbf{x}_C) \prod_{B \in \partial C} m_{B \rightarrow C}(\mathbf{x}_{S_{BC}}). \quad (3.17)$$

After a two-pass schedule, neighboring cluster beliefs agree on their separator marginal. Any requested singleton or low-order marginal can then be obtained by marginalizing a cluster containing the relevant variables.

Algorithm 7 separates structural preprocessing from numerical inference. The first three steps depend on scopes and graph topology. The last three steps operate on factor values. If the model parameters change but the graph does not, the same junction-tree structure can be reused.

Example 3.3.1 (A five-cycle converted into a clique tree). *A binary pairwise model on a five-cycle is triangulated by two fill-in edges. The resulting maximal cliques are $\{1, 2, 5\}$, $\{2, 3, 5\}$, and $\{3, 4, 5\}$. They form a three-node junction tree with separators $\{2, 5\}$ and $\{3, 5\}$. Exact cluster messages recover the singleton marginals of the original loopy model.*

Algorithm 7 Exact junction-tree inference

```

function JUNCTIONTREEINFERENCE( $\mathcal{F}, \pi$ )
    eliminate variables according to  $\pi$  and record the induced cliques
    remove nonmaximal cliques
    connect the maximal cliques into a tree satisfying running intersection
    assign each original factor to a clique containing its scope
    run two-pass cluster message passing using Eq. (3.16)
    compute calibrated clique beliefs using Eq. (3.17)
    return exact marginals and  $Z$ 

```

Companion notebook: Exact inference through a junction tree

The notebook `ch03/ch03_03_junction_tree.ipynb` constructs Fig. 3.3, assigns the original factors to cliques, computes separator messages and calibrated clique beliefs, and verifies all singleton marginals by enumeration.

Exercise 3.3.1. Use a different elimination order in `ch03/ch03_03_junction_tree.ipynb`. Construct the resulting fill-in edges, maximal cliques, and separators. Compare the largest clique size.

Exercise 3.3.2. Verify separator consistency numerically by marginalizing the two adjacent clique beliefs to their common variables. Explain why this consistency guarantees that Eq. (3.15) is normalized.

Exercise 3.3.3. Add a chord to the original five-cycle. Determine whether the graph is already chordal and whether the junction-tree width changes.

Problem 3.1. Develop an elimination-order heuristic that combines graph fill-in with a numerical compressibility score for the intermediate factors. Test it on a family of binary grids or tensor networks. Compare exact table size, numerical rank after matrix reshaping, and total runtime. The goal is not necessarily to beat minimum fill uniformly, but to identify regimes in which numerical structure changes the preferred order.

3.4 What exact dynamic programming teaches us

The chapter has exposed a single algorithmic spine:

factorization $\longrightarrow$ elimination $\longrightarrow$ separator functions $\longrightarrow$ tree messages $\longrightarrow$ junction trees.

Three conclusions will recur throughout the book.

First, exact computational complexity is controlled by separator dimension. Treewidth is therefore not merely a graph-theoretic descriptor; it is the dimension of the exact summaries that must be propagated.

Second, message passing is dynamic programming written in a distributed form. On trees, messages are exact partial partition functions. On loopy graphs, the same local equations can be used approximately, but their exact subtree interpretation is lost.

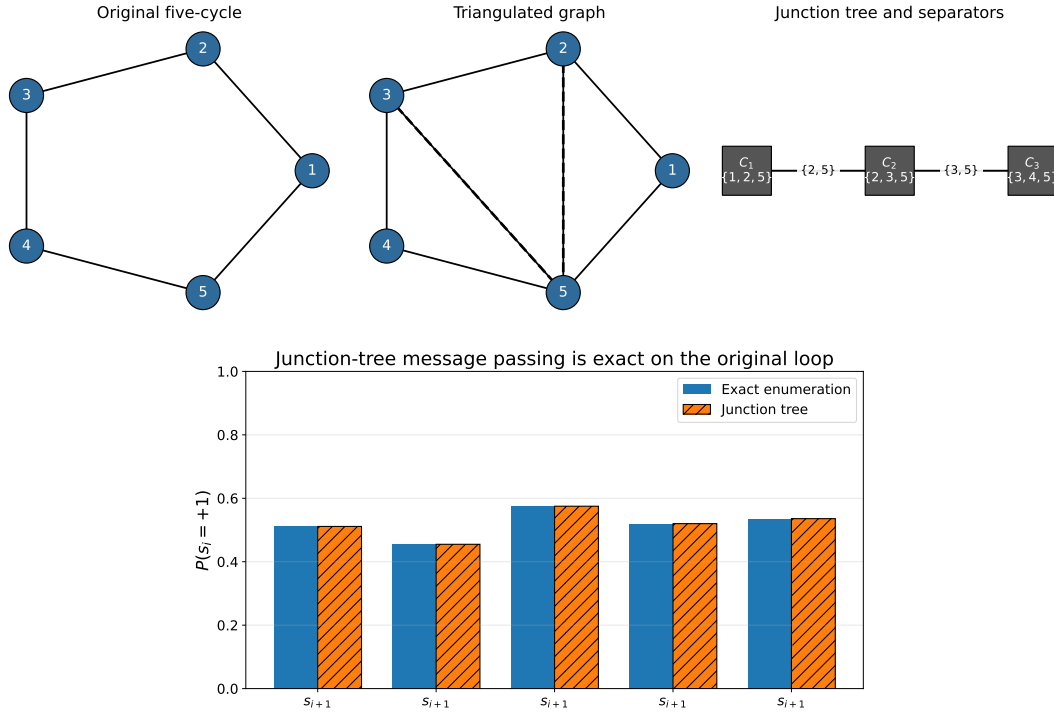


Figure 3.3: Junction-tree inference for Example 3.3.1. Top: original loop, triangulated graph, and junction tree. Dashed edges are computational fill-in edges, not new physical interactions. Bottom: exact marginals from enumeration and junction-tree message passing.

Third, junction trees reveal rather than remove exponential complexity. This observation motivates the next major family of methods: approximate elimination, variational message passing, loop corrections, tensor compression, and learned representations of intermediate objects.

Chapter 4

Other Inference, Optimization and Learning Challenges

Reader's Guide.

Why this chapter? The same factors can be used to ask very different questions. One reader may want the most likely configuration; another a marginal probability; another an exact random sample; another the parameters that best explain data. It is useful to separate these tasks before introducing specialized algorithms, because a method that is natural for one task may be unnecessary or even meaningless for another.

What you should know before starting. You should be comfortable with the normalized law $p(\mathbf{x}) = w(\mathbf{x})/Z$ from Chapter 2 and with exact elimination from Chapter 3. Basic conditional probability and elementary optimization are sufficient. Linear-programming, variational, and exponential-family language is introduced when it first becomes useful.

One tiny distribution, five different questions. Suppose two binary variables have unnormalized weights

	$x_2 = 0$	$x_2 = 1$	
$x_1 = 0$	6	2	$\implies \quad Z = 12.$
$x_1 = 1$	1	3	

From this same table we can ask: (i) which state is most probable?— $(0, 0)$; (ii) what is $p(x_2 = 1)$?— $5/12$; (iii) after observing $x_1 = 1$, what is $p(x_2 = 1 \mid x_1 = 1)$?— $3/4$; (iv) how do we generate an exact sample?—first draw one variable from its marginal and then the other from the corresponding conditional; and (v) if the four entries depend on unknown parameters, how should data change those parameters? The chapter develops scalable versions of these elementary questions.

What you should be able to do after reading. You should be able to distinguish maximum a posteriori (MAP) optimization, marginal/evidence calculations, conditioned

inference, variational “soft” inference, exact generation, and parameter learning; translate between several of them; and recognize which repeated inference calculation sits inside a learning or generation procedure.

How to use the companion notebooks. The four notebooks in notebooks/ch04/ deliberately reuse small models for different tasks. Use the first to compare MAP, marginals, evidence, and temperature; the second to visualize the Gibbs variational principle; the third to verify exact chain-rule generation empirically; and the fourth to see maximum-likelihood learning as repeated moment matching. The useful comparison is across notebooks: the probabilistic object stays almost the same while the computational question changes.

The preceding chapters introduced structured probability distributions and exact algorithms for partition functions and marginals. A graphical model, however, is rarely constructed only to compute its normalization. The same local factors support several distinct computational tasks:

1. **optimization:** find one or more highly probable configurations;
2. **conditioned inference:** update marginals after observations or constraints are imposed;
3. **soft inference:** characterize uncertainty through probabilities, expectations, and entropy;
4. **generation:** produce independent samples or stochastic trajectories;
5. **learning:** infer unknown factors or parameters from data.

This chapter organizes these tasks around a common mathematical object, the parameterized probability distribution. Its purpose is to expose the reductions and distinctions that will guide the rest of the book. Exact elimination from Chapter 3 can solve each task on sufficiently structured graphs, but the required intermediate objects are generally exponential. Later chapters develop specialized optimization, variational, Monte Carlo, loop-correction, and learning algorithms.

The numerical examples deliberately reuse the four-spin chain, frustrated triangle, Gaussian network, and tree models introduced in Chapters 2 and 3. The companion notebooks are collected under ch04/. General references for the computational tasks discussed here include [202, 110, 24, 148].

4.1 One model, several computational questions

Let $\mathbf{x} = (x_1, \dots, x_N)$ take values in $\mathcal{X}$, and write the unnormalized weight and probability distribution as

$$w(\mathbf{x}) = \prod_{a \in \mathcal{F}} f_a(\mathbf{x}_a) = \exp(-E(\mathbf{x})), \quad p(\mathbf{x}) = \frac{w(\mathbf{x})}{Z}, \quad Z = \int_{\mathbf{x} \in \mathcal{X}} w(\mathbf{x}). \quad (4.1)$$

The notation $\int_{\mathbf{x} \in \mathcal{X}}$ denotes summation for finite alphabets and integration for continuous state spaces. The global energy E is the sum of the local energies introduced in Eq. (2.9). The five tasks above ask different questions about the same weight w .

Task	Mathematical output	Typical use
MAP optimization	$\mathbf{x}^* \in \arg \max_{\mathbf{x}} p(\mathbf{x})$	decoding, reconstruction, constrained design
Marginal inference	$p(\mathbf{x}_A), p(\mathbf{x}_B \mid \mathbf{x}_A)$, or $\mathbb{E}[g(\mathbf{x})]$	uncertainty quantification, prediction, state estimation
Partition function	Z or $\log Z$	normalization, evidence, free energy, learning
Generation	$\mathbf{x}^{(s)} \stackrel{\text{i.i.d.}}{\sim} p$	simulation, synthetic data, uncertainty propagation
Learning	estimated parameters or factors	inverse problems, system identification, data-driven modeling

These outputs are related but not interchangeable. A MAP state does not describe uncertainty. Marginals do not automatically provide independent samples. Learning parameters usually requires repeated inference at many candidate parameter values. The rest of the chapter makes these relationships precise.

4.2 MAP optimization and its probabilistic relaxations

4.2.1 MAP is state optimization

A *maximum-a-posteriori* configuration is any state

$$\mathbf{x}^* \in \arg \max_{\mathbf{x} \in \mathcal{X}} p(\mathbf{x}) = \arg \max_{\mathbf{x} \in \mathcal{X}} w(\mathbf{x}) = \arg \min_{\mathbf{x} \in \mathcal{X}} E(\mathbf{x}). \quad (4.2)$$

In the factor-graph notation of Eq. (2.3), the same state optimization is

$$\mathbf{x}^* \in \arg \max_{\mathbf{x} \in \mathcal{X}} \prod_{a \in \mathcal{F}} f_a(\mathbf{x}_a). \quad (4.3)$$

For finite $\mathcal{X}$, exact enumeration is exponential in the number of variables. Exact elimination replaces summation by maximization, producing the *max-product* or *min-sum* form of dynamic programming. On trees this yields exact max-product message passing; on general graphs its complexity is controlled by induced width, just as in Section 3.1. The sum-product and max-product algorithms are instances of the same distributive-algebra viewpoint, often expressed through semirings [116, 110].

The term MAP should be distinguished from *maximum likelihood*. MAP in Eq. (4.2) optimizes over a state $\mathbf{x}$ for a fixed model. Maximum likelihood, discussed in Section 4.6, optimizes model parameters using observed data. The two optimizations may coexist in latent-variable models, but they are not synonyms.

Algorithmic lens: max-product elimination

Algorithm 5 becomes an exact MAP algorithm after replacing every local summation or integration by maximization or minimization. To reconstruct a maximizing configuration, one stores at each elimination step which value of the eliminated variable attained the

optimum and then performs a backward pass. This is the graphical-model form of the Viterbi or Bellman backtracking procedure [16, 116].

4.2.2 An exact linear program on the full probability simplex

Why introduce a huge linear program if it is not yet computationally useful? Think of $q(\mathbf{x})$ as allowing us to randomize over complete configurations. Minimizing the average energy over all such randomizations cannot beat simply putting all mass on the best configuration, so the formulation is exactly equivalent to MAP. Its value is architectural: later we will replace the exponentially large set of globally valid distributions by much smaller sets of locally consistent beliefs. The exact full-simplex formulation is therefore the reference problem from which useful relaxations are derived.

For a finite state space, define the probability simplex

$$\Delta(\mathcal{X}) := \left\{ q : \mathcal{X} \rightarrow [0, 1] \mid \sum_{\mathbf{x} \in \mathcal{X}} q(\mathbf{x}) = 1 \right\}. \quad (4.4)$$

A distribution $q \in \Delta(\mathcal{X})$ is a trial or proxy distribution over complete configurations. The expected energy is linear in q :

$$\mathcal{E}(q) := \mathbb{E}_q[E(\mathbf{x})] = \sum_{\mathbf{x} \in \mathcal{X}} q(\mathbf{x}) E(\mathbf{x}).$$

Therefore MAP can be written exactly as

$$\min_{q \in \Delta(\mathcal{X})} \mathcal{E}(q) = \min_{\mathbf{x} \in \mathcal{X}} E(\mathbf{x}). \quad (4.5)$$

Every extreme point of $\Delta(\mathcal{X})$ is a delta distribution supported on one configuration, so a minimizer of the linear objective can be chosen at an extreme point. Eq. (4.5) is exact but not yet useful computationally: q has $|\mathcal{X}|$ components, which is exponential for a product state space. Later LP relaxations replace this global simplex by collections of locally consistent beliefs; see Chapters 6 and 12 and the convex-analytic treatment in [202].

4.2.3 Temperature connects hard optimization to probability

For $T > 0$, introduce the temperature-dependent Gibbs distribution

$$p_T(\mathbf{x}) = \frac{\exp(-E(\mathbf{x})/T)}{Z_T}, \quad Z_T = \int_{\mathbf{x} \in \mathcal{X}} \exp(-E(\mathbf{x})/T). \quad (4.6)$$

At positive temperature, low-energy states are preferred but uncertainty remains. As $T \downarrow 0$, the distribution concentrates on the MAP set. For a finite model with several exactly degenerate minima, the limiting distribution is uniform over those minima. This zero-temperature connection between Gibbs probabilities and optimization is standard in statistical mechanics and underlies simulated annealing and many entropy-regularized optimization methods [75, 107].

Fig. 4.1 illustrates the transition numerically. The energy landscape is unchanged as T varies; only the balance between energetic preference and entropy changes.

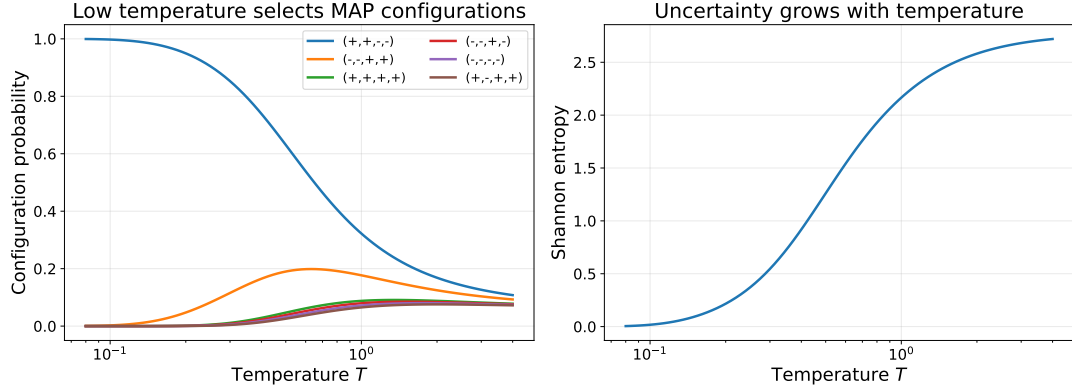


Figure 4.1: Temperature converts hard optimization into a probability distribution. The left panel tracks the most probable configurations of the four-spin chain introduced in Chapter 2; the right panel shows the corresponding entropy. At low temperature, probability concentrates on the MAP configuration.

Companion notebook: MAP, temperature, and evidence

The notebook `ch04/ch04_01_map_temperature_and_conditioning.ipynb` reuses the four-variable Ising chain from Chapter 2. It enumerates the energy landscape, follows the low-temperature concentration on MAP states, and studies how clamping one variable changes distant marginals.

Exercise 4.2.1. *Modify the notebook so that two configurations have exactly the same minimum energy. Verify the limiting probabilities as $T \downarrow 0$, and explain how the answer changes if the degeneracy is broken by a small local field.*

Exercise 4.2.2. *Implement max-product elimination on the four-node chain and compare its forward tables and backward reconstruction with exact enumeration. Then apply the same implementation to the frustrated triangle.*

4.3 Marginals, evidence, and conditioned inference

Let $A \subseteq \{1, \dots, N\}$, and suppose $\mathbf{x}_A = \mathbf{a}$ is observed. The *restricted partition function*

$$Z(\mathbf{a}) := \int_{\mathbf{x} \setminus \mathbf{x}_A} w(\mathbf{x})|_{\mathbf{x}_A = \mathbf{a}} \quad (4.7)$$

sums the weight over all configurations consistent with the evidence. The evidence probability is

$$p(\mathbf{x}_A = \mathbf{a}) = \frac{Z(\mathbf{a})}{Z}. \quad (4.8)$$

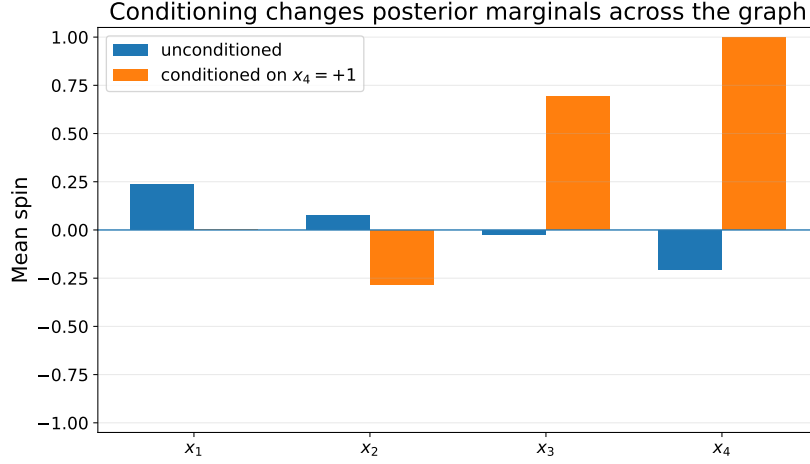


Figure 4.2: Exact one-variable means before and after conditioning on $x_4 = +1$ in the four-spin chain. Evidence changes the posterior distribution throughout the connected graph, not only at the observed variable.

For disjoint subsets A and B , a conditional probability is a ratio of two restricted partition functions:

$$p(\mathbf{x}_B = \mathbf{b} \mid \mathbf{x}_A = \mathbf{a}) = \frac{Z(\mathbf{a}, \mathbf{b})}{Z(\mathbf{a})}. \quad (4.9)$$

Thus marginal and conditioned inference can be implemented by the same elimination machinery used for Z , leaving requested variables uneliminated or inserting evidence factors. In a factor graph, hard evidence $x_i = a_i$ is represented by the unary factor $\mathbb{1}(x_i = a_i)$. Soft evidence is represented by a nonnegative likelihood factor.

The propagation of evidence in Fig. 4.2 is the basic mechanism behind filtering, smoothing, decoding, diagnosis, and probabilistic state estimation. The graph determines which paths can transmit information; the factor values determine the strength and sign of the response.

Expectations can also be obtained from derivatives of a log partition function. Suppose a scalar parameter λ couples to an observable $g(\mathbf{x})$:

$$Z(\lambda) = \int_{\mathbf{x} \in \mathcal{X}} w_0(\mathbf{x}) \exp(\lambda g(\mathbf{x})).$$

Then

$$\begin{aligned} \frac{\partial \log Z}{\partial \lambda} &= \mathbb{E}_\lambda [g(\mathbf{x})], \\ \frac{\partial^2 \log Z}{\partial \lambda^2} &= \text{Var}_\lambda (g(\mathbf{x})). \end{aligned} \quad (4.10)$$

This response identity appeared numerically in the first Chapter 2 notebook and becomes central to exponential-family learning in Section 4.6.

Exercise 4.3.1. Use the Chapter 2 Gaussian notebook to add a noisy observation factor at one node. Compute the conditioned mean and covariance both by Gaussian algebra and by treating the observation as a local factor.

4.4 Soft inference as variational optimization

Optimization and probability are connected more strongly than through the zero-temperature limit. The partition function itself is the optimum of an entropy-regularized problem.

4.4.1 The Gibbs variational principle

Energy versus entropy: the simplest variational picture. At zero temperature, only the best state matters. At positive temperature, a distribution is rewarded both for placing mass on low-energy states and for spreading mass over multiple plausible states. The Gibbs free energy combines these two pressures. The exact Gibbs variational principle says that if we were free to optimize over *every* global distribution, the optimizer would be the Gibbs law itself. Variational inference becomes nontrivial only when later chapters restrict or relax that enormous feasible set.

For $q \in \Delta(\mathcal{X})$, define Shannon entropy

$$\mathcal{H}(q) := - \sum_{\mathbf{x} \in \mathcal{X}} q(\mathbf{x}) \log q(\mathbf{x})$$

and the temperature-dependent free-energy functional

$$\mathcal{F}_T(q) := \mathbb{E}_q[E(\mathbf{x})] - T\mathcal{H}(q). \quad (4.11)$$

For the Gibbs distribution in Eq. (4.6), direct substitution gives

$$\mathcal{F}_T(q) = -T \log Z_T + T D_{\text{KL}}(q \| p_T), \quad (4.12)$$

where

$$D_{\text{KL}}(q \| p) := \sum_{\mathbf{x} \in \mathcal{X}} q(\mathbf{x}) \log \frac{q(\mathbf{x})}{p(\mathbf{x})}$$

is the Kullback–Leibler divergence. Since the divergence is nonnegative and vanishes only when $q = p_T$, Eq. (4.12) yields the exact Gibbs variational principle

$$-T \log Z_T = \min_{q \in \Delta(\mathcal{X})} \mathcal{F}_T(q), \quad q^* = p_T. \quad (4.13)$$



The free-energy principle is classical in statistical mechanics, information theory, and variational inference [75, 118, 98, 202].

Fig. 4.3 makes two points explicit. First, the exact probability law is an optimizer. Second, the objective is a balance: energy rewards probable configurations, while entropy prevents premature collapse onto a single state.

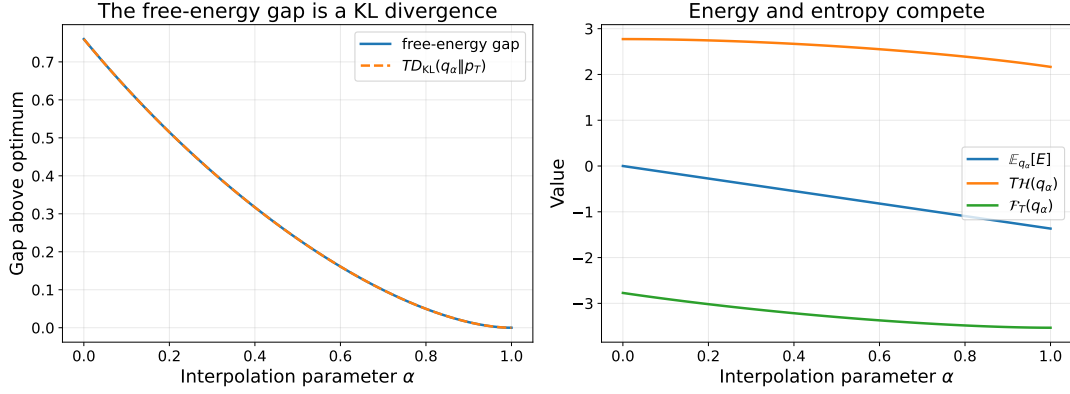


Figure 4.3: The exact Gibbs variational identity for the four-spin chain. The left panel verifies that the free-energy gap equals $TD_{\text{KL}}(q||p_T)$. The right panel separates the energy and entropy contributions along a path from the uniform distribution to the exact Gibbs law.

Companion notebook: The Gibbs variational principle

The notebook `ch04/ch04_02_gibbs_variational_principle.ipynb` verifies Eq. (4.12), minimizes the exact free energy through a softmax parameterization of the global distribution, and shows why the resulting exact optimization still has exponentially many variables.

4.4.2 From a global distribution to local beliefs

The exact variable $q(\mathbf{x})$ in Eq. (4.13) is exponentially large. Variational graphical-model methods replace it by a tractable family or by local beliefs attached to variables, factors, or larger regions. The main questions are then:

1. which local marginals are retained;
2. which consistency constraints are imposed;
3. how the global entropy is represented or approximated.

For tree-structured models, the exact factorization in Chapter 3 yields an exact decomposition of entropy into node and edge contributions. On loopy graphs, Bethe, Kikuchi, cluster-variational, and generalized belief-propagation constructions approximate this decomposition using selected regions and counting numbers [21, 103, 221]. Their systematic development belongs in Chapter 6; the present chapter establishes the global variational problem they approximate.

4.4.3 A forward pointer: region posets and entropy counting

Aforementioned Kikuchi and cluster-variational constructions organize overlapping regions as a finite partially ordered set under inclusion. If r and s are regions, define the incidence or *zeta*

function

$$\zeta(r, s) := \begin{cases} 1, & r \subseteq s, \\ 0, & \text{otherwise.} \end{cases} \quad (4.14)$$

Its inverse in the incidence algebra is the *Möbius function* μ , characterized recursively by

$$\mu(r, r) = 1, \quad \mu(r, s) = - \sum_{r \subseteq u \subsetneq s} \mu(r, u) \quad \text{for } r \subsetneq s. \quad (4.15)$$

These coefficients prevent overcounting when local entropies or cumulants are assembled from overlapping regions. The full entropy decomposition and its truncations are developed in Chapter 6; the same incidence-algebra language reappears in the loop and cumulant constructions of Chapter 9. See [103, 221] for the graphical-model setting.

Exercise 4.4.1. *In the Gibbs-variational notebook, restrict q to a product distribution $q(\mathbf{x}) = \prod_i q_i(x_i)$. Derive the mean-field objective and compare its optimum with the exact law for the chain and the frustrated triangle.*

4.5 Exact generation from conditioned inference

Exact generation is just a probabilistic decision tree. For two variables, sample x_1 from $p(x_1)$ and then sample x_2 from $p(x_2 | x_1)$. For N variables, repeat the same step. Nothing approximate has entered: the product of the conditionals is exactly the joint distribution. The hard part is not the random draw but obtaining each conditional efficiently. Thus exact inference becomes a subroutine inside exact generation.

A sample $\mathbf{x}^{(s)}$ is distributed according to p if $\Pr(\mathbf{x}^{(s)} = \mathbf{x}) = p(\mathbf{x})$. Samples $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(S)}$ are independent and identically distributed, abbreviated i.i.d., if every sample has law p and the joint sample law factorizes across s .

Choose any ordering $\pi = (\pi_1, \dots, \pi_N)$. The probability chain rule gives

$$p(\mathbf{x}) = \prod_{t=1}^N p(x_{\pi_t} | x_{\pi_1}, \dots, x_{\pi_{t-1}}). \quad (4.16) \quad \text{👉}$$

The chain rule follows from repeated use of conditional probability and is independent of the chosen ordering. It immediately gives Algorithm 8.

Algorithm 8 is exact because the probability of producing any complete assignment is the product in Eq. (4.16). Repeating the algorithm with fresh random numbers produces i.i.d. samples. The challenge is computational: each conditional may require a new partition-function or marginal calculation. On a tree, messages can be cached and reused; on a junction tree, calibrated clique beliefs can answer many conditionals without restarting elimination. This reuse is illustrated in Chapter 3.

The tree in Fig. 4.4 should not be confused with the original graphical model. It is a tree of partial assignments induced by a sampling order. Modern autoregressive generators implement the same chain-rule idea, replacing exact conditional inference by learned conditional models. Discrete diffusion and Sampling Decisions will later revisit the possibility of choosing or updating several variables in parallel.

Algorithm 8 Exact conditional generation by decimation

```

function EXACTCONDITIONALSAMPLE( $p, \pi$ )
  initialize an empty assignment  $\mathbf{a}$ 
  for  $t \leftarrow 1$  to  $N$  do
    compute  $p(x_{\pi_t} \mid \mathbf{a})$  by conditioned inference
    draw  $x_{\pi_t} \sim p(x_{\pi_t} \mid \mathbf{a})$ 
    append the sampled value to  $\mathbf{a}$ 
  return the complete assignment  $\mathbf{x}$ 

```

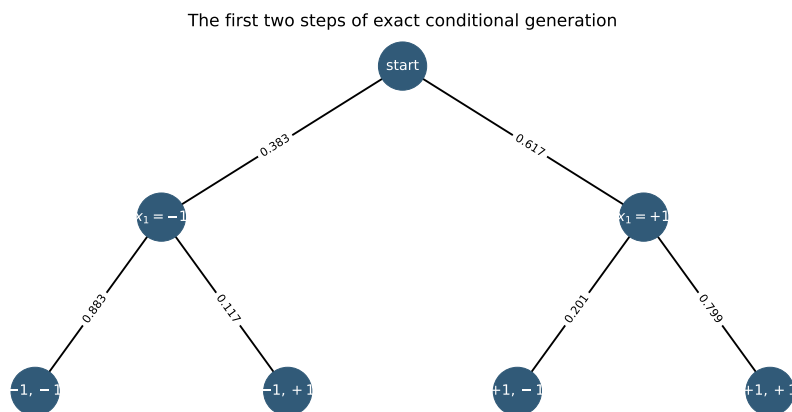


Figure 4.4: The first two levels of exact conditional generation for the four-spin chain. Each node is a partial assignment, and each edge is labeled by the exact conditional probability of the next choice. The full decision tree represents the chain-rule factorization of the joint law.

Companion notebook: Exact independent generation

The notebook `ch04/ch04_03_exact_decimation_sampling.ipynb` implements Algorithm 8, caches repeated conditional queries, compares several variable orders, and verifies convergence of empirical frequencies to the exact distribution.

Exact conditional generation differs fundamentally from MCMC. MCMC constructs a dependent Markov chain whose stationary law is p ; consecutive states are generally correlated, and effective sampling quality depends on mixing and autocorrelation. Chapter 8 develops this alternative when exact conditionals or exact global inference are unavailable.

Exercise 4.5.1. Replace the enumeration oracle in the decimation notebook by sum-product messages on the chain. Count the number of message updates required to generate one sample and to generate many samples with cached messages.

Exercise 4.5.2. Condition on $x_4 = +1$ and generate i.i.d. posterior samples. Verify both the conditioned marginals and the probability of the posterior MAP state.

4.6 Graphical-model learning as repeated inference

4.6.1 Exponential families and sufficient statistics

An exponential-family model has the form

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \exp(\boldsymbol{\theta}^T \boldsymbol{\phi}(\mathbf{x}) - \Phi(\boldsymbol{\theta})), \quad \Phi(\boldsymbol{\theta}) := \log Z(\boldsymbol{\theta}). \quad (4.17)$$

The vector $\boldsymbol{\theta} \in \boldsymbol{\Theta} \subseteq \mathbb{R}^d$ contains model parameters. The vector $\boldsymbol{\phi}(\mathbf{x}) \in \mathbb{R}^d$ contains *sufficient statistics*: all data dependence of the likelihood enters through these functions. The log-partition function Φ normalizes the distribution. Exponential families provide a common language for discrete graphical models, Gaussian models, generalized linear models, and many energy-based models [11, 202].

For an Ising model, components of $\boldsymbol{\phi}$ may be node spins x_i and edge products $x_i x_j$; their parameters are local fields and couplings. Differentiating Φ gives

$$\nabla_{\boldsymbol{\theta}} \Phi(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{\theta}}[\boldsymbol{\phi}(\mathbf{x})] := \boldsymbol{\mu}(\boldsymbol{\theta}), \quad (4.18)$$

$$\nabla_{\boldsymbol{\theta}}^2 \Phi(\boldsymbol{\theta}) = \text{Cov}_{\boldsymbol{\theta}}(\boldsymbol{\phi}(\mathbf{x})). \quad (4.19)$$

Thus Φ is convex, and strictly convex when the sufficient statistics are nonredundant under the model. Eq. (4.18) generalizes the response identity in Eq. (4.10).

For finite $\mathcal{X}$, the *moment polytope* is

$$\mathcal{M} := \text{conv}\{\boldsymbol{\phi}(\mathbf{x}) \mid \mathbf{x} \in \mathcal{X}\}. \quad (4.20)$$

It contains every moment vector realizable as $\mathbb{E}_q[\boldsymbol{\phi}(\mathbf{x})]$ for some distribution q . The geometry of $\mathcal{M}$ connects probabilistic inference to convex optimization and LP relaxations [202].

4.6.2 Maximum likelihood and moment matching

Learning as “data moments minus model moments.” In an exponential family, the data tell us empirical averages of sufficient statistics. The current model predicts its own averages of the same statistics. The likelihood gradient is their difference. Consequently one learning step has a simple interpretation: change the parameters in the direction that makes model statistics look more like data statistics. The computational difficulty is that evaluating the model statistics is itself an inference problem and must generally be repeated as the parameters change.

Suppose S i.i.d. observations $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(S)}$ are drawn from an unknown member of the exponential family. The maximum-likelihood estimate is

$$\hat{\boldsymbol{\theta}} \in \arg \max_{\boldsymbol{\theta} \in \boldsymbol{\Theta}} \ell(\boldsymbol{\theta}). \quad (4.21)$$

Define empirical sufficient statistics

$$\hat{\boldsymbol{\mu}} := \frac{1}{S} \sum_{s=1}^S \boldsymbol{\phi}(\mathbf{x}^{(s)}). \quad (4.22)$$

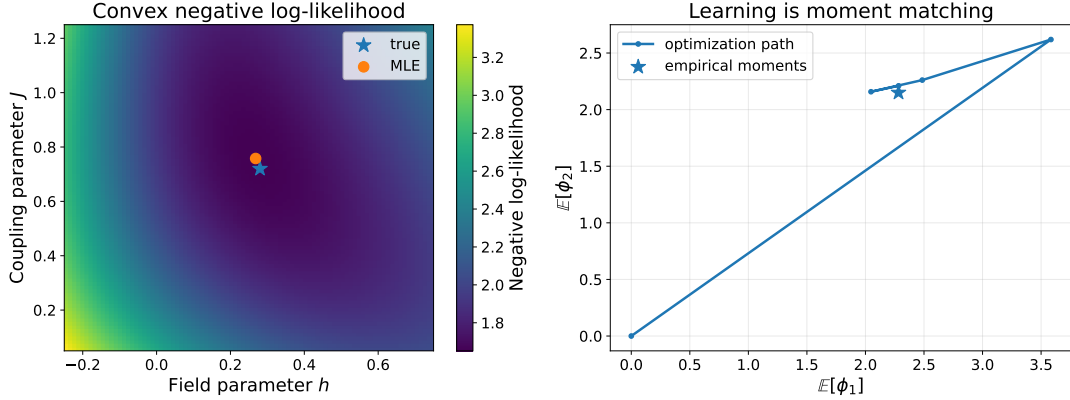


Figure 4.5: Maximum-likelihood learning for a two-parameter Ising chain. The left panel shows the convex negative log-likelihood surface and the locations of the true and fitted parameters. The right panel shows the optimization path in moment space; the optimum matches empirical sufficient statistics.

The average log-likelihood is

$$\ell(\theta) = \theta^T \hat{\mu} - \Phi(\theta). \quad (4.23)$$

Its gradient is

$$\nabla_{\theta} \ell(\theta) = \hat{\mu} - \mathbb{E}_{\theta} [\phi(x)]. \quad (4.24)$$

Therefore, whenever an interior unregularized maximum exists, it satisfies the moment-matching condition

$$\mathbb{E}_{\hat{\theta}} [\phi(x)] = \hat{\mu}. \quad (4.25)$$

The negative log-likelihood is convex because its Hessian is the covariance matrix in Eq. (4.18). A *minimal* exponential family has no nonzero coefficient vector whose linear combination of sufficient statistics is constant over $\mathcal{X}$; minimality removes parameter redundancy and yields identifiability in the interior [11, 202].

Fig. 4.5 illustrates why learning is coupled to inference. Every likelihood and gradient evaluation requires $\Phi(\theta)$ and model expectations at the current parameter value. Exact learning is easy in the small notebook because all states can be enumerated. In a large loopy model, this inner inference problem is often the dominant computational cost.

Companion notebook: Exponential-family learning

The notebook `ch04/ch04_04_exponential_family_learning.ipynb` learns the field and coupling of a homogeneous four-spin Ising chain. It verifies moment matching, visualizes likelihood geometry, and studies finite-sample parameter recovery.

Practical strategies include exact inference on tractable graphs, variational approximations to Φ , MCMC estimates of model moments, conditional or pseudo-likelihood objectives, score matching, and estimators that exploit additional correlations or structural assumptions. Chapter 10 develops these alternatives in detail.

Exercise 4.6.1. *Extend the learning notebook to separate fields h_i and couplings J_{ij} . Determine which empirical moments are matched and study the effect of adding an ℓ_2 penalty.*

Exercise 4.6.2. *Repeat the learning experiment for the Gaussian graphical model from Chapter 2. Show explicitly how maximum likelihood matches empirical first and second moments when the covariance estimate is nonsingular.*

4.7 Relations among the tasks

The chapter can be summarized as a network of exact reductions:

Starting object	Reduction	Result
Partition-function oracle	clamp variables and take ratios	marginals and conditionals
Conditional-inference oracle	apply Algorithm 8	exact i.i.d. generation
Log partition function	differentiate parameters	moments and covariances
Positive-temperature Gibbs law	take $T \downarrow 0$	MAP set
Exponential-family likelihood	repeatedly infer model moments	maximum-likelihood learning
Global variational principle	restrict distributions or localize beliefs	tractable approximations

The reductions are conceptually powerful but do not by themselves guarantee efficiency. Exact conditioned inference may be exponential. Taking T very small can create numerical stiffness and optimization barriers. Exact generation may call inference repeatedly. Exact learning may call it at every optimization step. The remainder of the book asks which structures, approximations, stochastic processes, and learned representations make these tasks practical.

4.8 Further reading and synthesis problems

The graphical-model view of MAP, marginal inference, and learning is developed systematically in [202, 110, 148]. The Gibbs variational principle and its role in approximate inference are discussed in [75, 98]. Bethe and Kikuchi approximations originate in [21, 103]; their connection to belief propagation and region graphs is developed in [221]. Exponential-family duality and moment geometry are treated in [11, 202].

As in Chapter 2, the Problems below are deliberately more open ended than the exercises. They require combining several constructions, making modeling choices, or designing a computational investigation.

Problem 4.1 (From an inference oracle to a structured generator). *Choose one of the Chapter 2 models and assume that an oracle returns any requested conditional marginal exactly.*

1. *Design an exact conditional generator for at least three variable orders.*
2. *Identify which oracle queries can be cached and reused.*
3. *Replace the oracle by exact tree or junction-tree messages whenever the graph permits.*
4. *Compare the number and size of intermediate objects across orders.*
5. *Propose a parallel update rule and determine precisely why it is or is not exact.*

The final deliverable should include a derivation, an executable extension of a Chapter 4 notebook, and a discussion of how the construction relates to autoregressive generation or Sampling Decisions.

Problem 4.2 (Graph-aware regularization of MAP). *Let $q_0 \in \Delta(\mathcal{X})$ be a prior distribution and consider replacing hard MAP by an optimization over distributions,*

$$\min_{q \in \Delta(\mathcal{X})} (\mathbb{E}_q [E(\mathbf{x})] + \lambda \mathcal{R}(q, q_0)). \quad (4.26)$$

Compare at least two choices of $\mathcal{R}$: relative entropy and an optimal-transport cost. For the transport version, design a ground cost that respects the graph, such as weighted Hamming distance or the cost of changing a connected set of variables. Study limiting behavior as λ varies and determine whether the regularized solution can be computed from local structure. Distributionally robust and transport-regularized formulations provide useful context [25].

Problem 4.3 (Learning when exact inference is unavailable). *Generate data from a small loopy Ising model and compare three parameter estimators:*

1. *exact maximum likelihood by enumeration;*
2. *an estimator using approximate model moments;*
3. *a conditional or local objective that avoids the partition function.*

Measure parameter error, moment error, held-out log-likelihood, and computational effort as sample size and coupling strength vary. Explain which errors originate from finite data and which originate from approximate inference. The problem should produce a notebook, a reproducibility record, and a recommendation about when each estimator is appropriate.

Chapter 5

Transformations of Graphical Models

Reader's Guide.

Why this chapter? An algorithm does not act on an abstract probability distribution; it acts on a *representation* of that distribution. Two representations can encode the same global object while exposing very different local computations. This chapter studies deliberate changes of representation that preserve exactly the quantity we care about while rearranging the local pieces into a more useful form.

What you should know before starting. You need the factor-graph and normal-factor-graph representations of Chapter 2 and the message/elimination viewpoint of Chapter 3. Matrix inverses and elementary graph notions are sufficient. No prior familiarity with gauge theory is assumed: here a gauge is first and foremost an algebraic change of local coordinates with a precisely stated invariant.

The easiest reparametrization: multiply here, divide there. Take a chain weight

$$w(x_1, x_2, x_3) = f_{12}(x_1, x_2)f_{23}(x_2, x_3).$$

For any positive function $r(x_2)$, define

$$\tilde{f}_{12}(x_1, x_2) = f_{12}(x_1, x_2)r(x_2), \quad \tilde{f}_{23}(x_2, x_3) = f_{23}(x_2, x_3)/r(x_2) \quad .$$

Then $\tilde{f}_{12}\tilde{f}_{23} = f_{12}f_{23}$ for every complete configuration. The global distribution has not changed at all; only the local accounting has. Belief-propagation reparametrizations are a systematic many-edge version of this cancellation.

The easiest gauge: change a local basis and cancel it on the neighboring factor. For the scalar contraction $Z = a^\top b$, choose invertible matrices G, H with $G^\top H = I$. Then $(Ga)^\top(Hb) = a^\top b$. A normal-factor-graph gauge performs this cancellation on every edge.



Unlike a positive scalar reparametrization, a generic matrix gauge need not preserve each configuration weight or even positivity of transformed local factors; it preserves the total contraction. Keeping these two invariants distinct prevents much of the apparent abstraction below.

What you should be able to do after reading. You should be able to distinguish gauges from reparametrizations; verify their invariants; understand conditioning and graph conversion as computational transformations; and read computation/self-avoiding trees as ways of replacing a loopy neighborhood by a tree-shaped computation.

How to use the companion notebooks. The notebooks in `notebooks/ch05/` are especially useful because transformation formulas can look more abstract than they are. Start with gauge invariance and numerically check that local entries move while Z does not. Then compare normal-factor-graph (NFG) and factor-graph (FG) reparametrizations and verify that complete-configuration probabilities remain unchanged. Finish with conditioning and tree transformations. In each notebook, ask first: *what is deliberately changed, and what is guaranteed not to change?*

The same global probabilistic object can often be represented by many different collections of local factors and, in some cases, by different graphs. This freedom is not merely cosmetic. A suitable representation may expose exact cancellations, produce locally normalized beliefs, reveal a tractable tree structure, or prepare the model for a controlled approximation. The present chapter develops three families of transformations:

1. **gauge transformations**, which preserve the partition function while changing the local factors and generally changing the weight assigned to an individual configuration;
2. **reparametrizations**, which redistribute local contributions without changing the probability distribution;
3. **graph and tree transformations**, which change the graphical representation while preserving, or allowing recovery of, a selected inference quantity.

These constructions connect the representations of Chapter 2, the exact elimination and tree algorithms of Chapter 3, and the broader tasks summarized in Chapter 4. They will be used repeatedly in the variational formulations of Chapter 6, approximate elimination in Chapter 7, and the loop-series and tensor-network developments of Chapter 9. Background on normal factor graphs, hypergraphs, partially ordered sets, and multilinear transformations is collected in the Appendixes; see in particular Sections 2.2, 23.1.4, and 23.4.2. General references include [202, 110, 70, 5].

Transformation	Preserved object	What changes	Later role
Gauge	Partition function	Local tensors and configuration weights	Holographic transformations, loop calculus, tensor contractions
Reparametrization	Full probability distribution	Allocation of local factors, beliefs, local normalizers	BP, variational inference, dual algorithms
Graph transformation	Selected partition function, marginal, or ratio	Graph, state space, or conditioned variables	Cutset conditioning, elimination, exact reductions
Tree transformation	Root computation or selected marginal	Loopy graph becomes an unrolled or self-avoiding tree	BP analysis, correlation decay, exact hard-core marginals

5.1 Gauge and reparametrization transformations

A useful distinction before the formulas. Gauge and reparametrization equations can look nearly identical because both insert local objects that cancel across an edge. The practical test is the invariant. If every complete configuration retains its weight up to one overall constant, we have a reparametrization and can continue to interpret the transformed factors probabilistically. If only the final contraction Z is guaranteed to survive, we have the more general gauge freedom. Later algorithms select particular representatives from these equivalence classes—for example, representatives whose local beliefs satisfy consistency conditions.

Gauge and reparametrization transformations share the idea that local factors are not unique. They differ in the invariant they preserve. A gauge transformation inserts mutually cancelling linear maps on the two ends of every normal-factor-graph edge. A reparametrization inserts mutually cancelling positive scalar functions. The second operation is therefore more restrictive: it leaves the weight of every complete configuration unchanged up to one global constant. Recall from Section 2.2.1 that an ordinary factor graph can be converted into a normal factor graph by introducing variable copies and equality factors. A generic gauge applied afterward preserves the normal-factor-graph contraction but generally destroys this special equality-factor structure; consequently, the transformed NFG need not convert back to an FG over the same graph. Reparametrizations were developed in graphical-model optimization and tree-reweighted inference [176, 177, 205]; the gauge formulation used here was introduced in loop calculus [46, 47] and is closely related to holographic transformations and Holant problems [193, 35, 5, 36].

5.1.1 Gauge transformations in the normal factor graph representation

Consider a normal factor graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with a finite alphabet $\mathcal{X}_\alpha$ on each undirected edge α . Every undirected edge has two orientations, denoted α_d and $\bar{\alpha}_d$. If α joins vertices a and b , the original edge variable x_α is shared by both factors, whereas the primed variables x'_{α_d} and $x'_{\bar{\alpha}_d}$ introduced below are independent integration variables at the two endpoints.

For every directed edge choose a square matrix G_{α_d} indexed by $\mathcal{X}_\alpha \times \mathcal{X}_\alpha$. Transform each local factor according to

$$\tilde{f}_a(\mathbf{x}_a) = \int_{\mathbf{x}'_a} f_a(\mathbf{x}'_a) \prod_{\alpha_d \in e(a)} G_{\alpha_d}(x_\alpha, x'_{\alpha_d}). \quad (5.1)$$

Here and below, the integral symbol denotes a sum when the alphabet is discrete. The pair of matrices on every edge is constrained by

$$\forall \alpha \in \mathcal{E} : \quad \int_{x_\alpha \in \mathcal{X}_\alpha} G_{\alpha_d}(x_\alpha, x') G_{\bar{\alpha}_d}(x_\alpha, x'') = \delta(x', x''). \quad (5.2)$$

In matrix notation, Eq. (5.2) is $G_{\alpha_d}^\top G_{\bar{\alpha}_d} = I$; thus the paired gauge is the inverse transpose of the first. If the matrices are nonsingular, the equivalent relation

$$\forall \alpha \in \mathcal{E} : \quad \int_{x' \in \mathcal{X}_\alpha} G_{\alpha_d}(x_1, x') G_{\bar{\alpha}_d}(x_2, x') = \delta(x_1, x_2) \quad (5.3)$$

expresses $G_{\alpha_d} G_{\bar{\alpha}_d}^\top = I$.

The partition function of the transformed model is

$$\tilde{Z} = \int_{\mathbf{x}} \prod_{a \in \mathcal{V}} \tilde{f}_a(\mathbf{x}_a). \quad (5.4)$$

Proposition 5.1.1 (Gauge invariance in a normal factor graph). *If the paired edge matrices satisfy Eq. (5.2), the transformation in Eq. (5.1) preserves the partition function: $\tilde{Z} = Z$.*

Proof. Substitute Eq. (5.1) into Eq. (5.4) and exchange the order of integration. For every undirected edge α , the integration over x_α produces the contraction in Eq. (5.2), restoring a delta function between the two original copies of the edge variable. Performing this cancellation independently on all edges recovers the original normal-factor-graph contraction and therefore $\tilde{Z} = Z$. $\square$

Equivalently,

$$Z = \int_{\mathbf{x}} \prod_{a \in \mathcal{V}} f_a(\mathbf{x}_a) = \int_{\mathbf{x}} Z^{(G)}(\mathbf{x}), \quad (5.5)$$

$$Z^{(G)}(\mathbf{x}) = \prod_{a \in \mathcal{V}} \left(\int_{\mathbf{x}'_a} f_a(\mathbf{x}'_a) \prod_{\alpha_d \in e(a)} G_{\alpha_d}(x_\alpha, x'_{\alpha_d}) \right). \quad (5.6)$$

The cancellation is edge-local, but the transformed local factors need not be nonnegative. Therefore a general gauge transformation preserves a tensor contraction or partition function, not necessarily a probabilistic interpretation of each transformed factor. This distinction is central to holographic algorithms and to the loop-series construction of Chapter 9.

A useful probabilistic specialization selects one distinguished symbol $0_\alpha \in \mathcal{X}_\alpha$ on every edge and uses the corresponding rows of the gauge matrices. Define node and edge gauge beliefs by

$$\mathfrak{B}_a^{(G)}(\mathbf{x}_a) := \frac{f_a(\mathbf{x}_a) \prod_{\alpha_d \in e(a)} G_{\alpha_d}(0_\alpha, x_\alpha)}{\int_{\mathbf{x}_a} f_a(\mathbf{x}_a) \prod_{\alpha_d \in e(a)} G_{\alpha_d}(0_\alpha, x_\alpha)}, \quad (5.7)$$

$$\mathfrak{B}_\alpha^{(G)}(x_\alpha) := G_{\alpha_d}(0_\alpha, x_\alpha) G_{\bar{\alpha}_d}(0_\alpha, x_\alpha). \quad (5.8)$$

Paired gauge matrices act on the two ends of every NFG edge

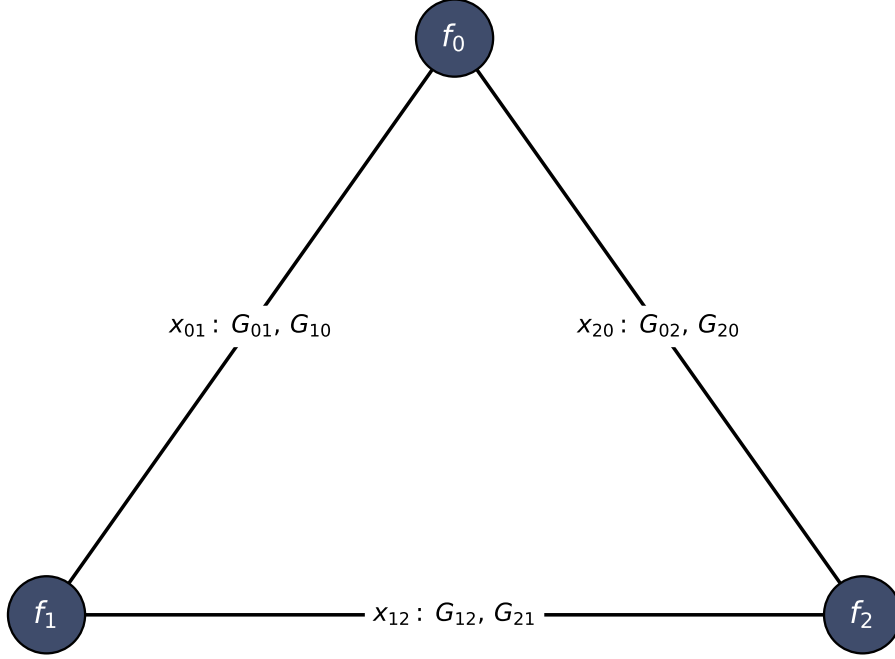


Figure 5.1: Gauge matrices are attached to the two orientations of every normal-factor-graph edge. The paired matrices satisfy Eq. (5.2), so their contraction restores the original equality constraint. Dark nodes use white labels throughout the companion figures.

They are normalized probability distributions provided that

$$G_{\alpha_d}(0_\alpha, x_\alpha) \geq 0, \quad \forall \alpha_d \in \mathcal{E}_d, \quad x_\alpha \in \mathcal{X}_\alpha, \quad (5.9)$$

$$\int_{x_\alpha \in \mathcal{X}_\alpha} G_{\alpha_d}(0_\alpha, x_\alpha) G_{\bar{\alpha}_d}(0_\alpha, x_\alpha) = 1. \quad (5.10)$$

The second condition is the $(0_\alpha, 0_\alpha)$ component of Eq. (5.3). These beliefs need not be marginally consistent:

$$\int_{\mathbf{x}_a \setminus x_\alpha} \mathfrak{B}_a^{(G)}(\mathbf{x}_a) \neq \mathfrak{B}_\alpha^{(G)}(x_\alpha)$$

for a generic gauge. Consistency will become a distinguished gauge-fixing condition in the variational and loop-calculus chapters.

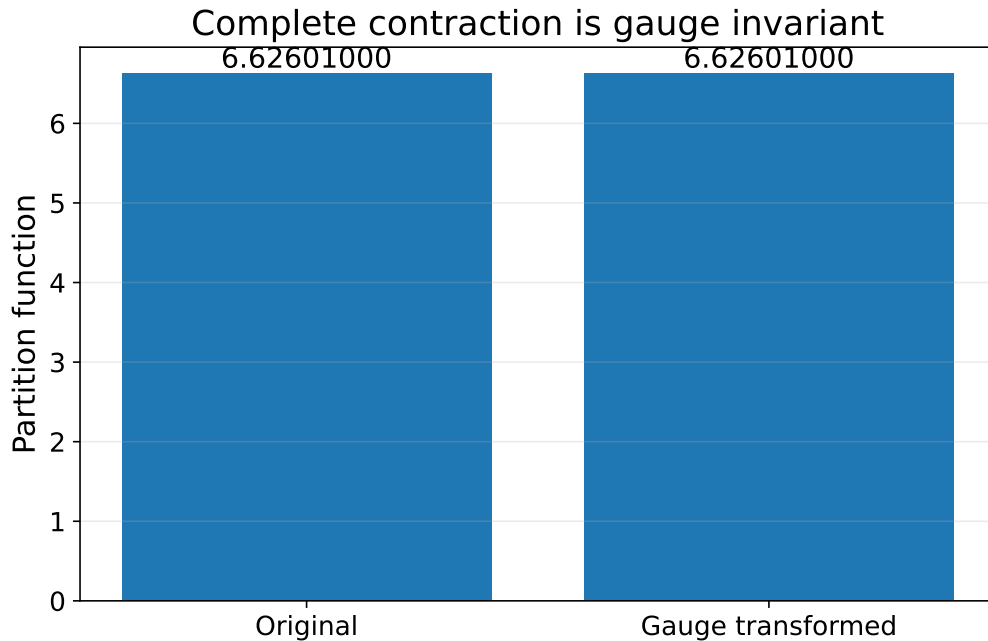


Figure 5.2: Numerical verification of gauge invariance for a binary normal factor graph. Individual transformed factors and configuration-level contributions change, while the total partition function remains invariant up to floating-point error.

Companion notebook: Gauge invariance as local basis change

The notebook `ch05/ch05_01_gauge_invariance.ipynb` applies nontrivial paired matrices to a small binary normal factor graph. It verifies Eqs. (5.2)–(5.6), visualizes the transformed local factors, and separates partition-function invariance from configuration-level invariance.

Exercise 5.1.1. *Modify one gauge matrix in the notebook without changing its partner. Measure the violation of Eq. (5.2) and the resulting partition-function error. Then reconstruct the correct partner as an inverse transpose.*

Exercise 5.1.2. *Construct a paired gauge for which some transformed factor entries are negative. Verify that the total partition function is still correct and explain why the transformed representation no longer defines local nonnegative probability factors.*

Exercise 5.1.3 (Gauge transformations in an ordinary factor graph). *Start from an ordinary factor graph and convert it to the normal-factor-graph representation of Section 2.2.1, in which every variable node becomes an equality factor. Attach paired gauge matrices to all incidence edges and write the transformed ordinary factors and transformed equality factors explicitly. Prove that the resulting contraction has the same partition function. Then determine which restrictions on the incidence gauges are needed for every transformed equality factor to remain an equality factor, so that the transformed model can still be interpreted as an ordinary factor graph on the same bipartite graph.*

5.1.2 Reparametrization in the normal factor graph representation

A reparametrization redistributes positive functions along edges without changing the weight of any complete configuration. Associate a positive function $R_{\alpha_d} : \mathcal{X}_\alpha \rightarrow \mathbb{R}_{>0}$ with each directed edge. Multiplying and dividing by these functions gives the telescoping identity

$$\begin{aligned} P(\mathbf{x}) &\propto \prod_{a \in \mathcal{V}} f_a(\mathbf{x}_a) \\ &\propto \frac{\prod_{a \in \mathcal{V}} \left(f_a(\mathbf{x}_a) \prod_{\alpha_d \in e(a)} R_{\alpha_d}(\mathbf{x}_\alpha) \right)}{\prod_{\alpha \in \mathcal{E}} R_{\alpha_d}(\mathbf{x}_\alpha) R_{\bar{\alpha}_d}(\mathbf{x}_\alpha)}. \end{aligned} \quad (5.11)$$

Define the normalized node and edge beliefs

$$\mathfrak{B}_a^{(R)}(\mathbf{x}_a) := \frac{f_a(\mathbf{x}_a) \prod_{\alpha_d \in e(a)} R_{\alpha_d}(\mathbf{x}_\alpha)}{C_a^{(R)}}, \quad C_a^{(R)} := \int_{\mathbf{x}_a} f_a(\mathbf{x}_a) \prod_{\alpha_d \in e(a)} R_{\alpha_d}(\mathbf{x}_\alpha), \quad (5.12)$$

$$\mathfrak{B}_\alpha^{(R)}(\mathbf{x}_\alpha) := \frac{R_{\alpha_d}(\mathbf{x}_\alpha) R_{\bar{\alpha}_d}(\mathbf{x}_\alpha)}{C_\alpha^{(R)}}, \quad C_\alpha^{(R)} := \int_{\mathbf{x}_\alpha} R_{\alpha_d}(\mathbf{x}_\alpha) R_{\bar{\alpha}_d}(\mathbf{x}_\alpha). \quad (5.13)$$

The exact distribution can then be written as

$$P(\mathbf{x}) = \frac{1}{\mathcal{Z}^{(R)}} \frac{\prod_{a \in \mathcal{V}} \mathfrak{B}_a^{(R)}(\mathbf{x}_a)}{\prod_{\alpha \in \mathcal{E}} \mathfrak{B}_\alpha^{(R)}(\mathbf{x}_\alpha)}, \quad (5.14)$$

and the residual global normalization is

$$\mathcal{Z}^{(R)} := \int_{\mathbf{x}} \frac{\prod_{a \in \mathcal{V}} \mathfrak{B}_a^{(R)}(\mathbf{x}_a)}{\prod_{\alpha \in \mathcal{E}} \mathfrak{B}_\alpha^{(R)}(\mathbf{x}_\alpha)}. \quad (5.15)$$

We deliberately use two visually different symbols. The calligraphic $\mathcal{Z}^{(R)}$ is the global normalization in the belief representation. The ordinary local factor

$$Z = \mathcal{Z}^{(R)} \mathcal{Z}^{(R)}, \quad \mathcal{Z}^{(R)} := \frac{\prod_{a \in \mathcal{V}} C_a^{(R)}}{\prod_{\alpha \in \mathcal{E}} C_\alpha^{(R)}} \quad (5.16)$$

is easy to compute once the functions R are fixed. The global object $\mathcal{Z}^{(R)}$ remains as difficult as the original partition function in a generic loopy model.

Proposition 5.1.2 (Exact reparametrization of a normal factor graph). *For any collection of positive directed-edge functions R_{α_d} , the beliefs in Eqs. (5.12) and (5.13) satisfy the exact distributional identity in Eq. (5.14); the original partition function is related to the local and global normalizers by Eq. (5.16).*

Proof. Insert the normalized beliefs from Eqs. (5.12) and (5.13) into Eq. (5.14). Every function R_{α_d} appears once in a node belief and once in the corresponding edge belief, so all directed-edge functions cancel configuration by configuration. What remains is the original product of factors multiplied by the ratio of local normalizers. Normalizing the resulting expression gives Eq. (5.15), and collecting the local constants gives Eq. (5.16). $\square$

Eq. (5.15) can also be expressed as an expectation with respect to the independent-edge distribution

$$\mathfrak{B}^{(\text{R};\text{MF})}(\mathbf{x}) := \prod_{\alpha \in \mathcal{E}} \mathfrak{B}_{\alpha}^{(\text{R})}(x_{\alpha}). \quad (5.17)$$

Specifically,

$$\mathcal{Z}^{(\text{R})} = \mathbb{E}_{\mathfrak{B}^{(\text{R};\text{MF})}} \left[\prod_{a \in \mathcal{V}} \frac{\mathfrak{B}_a^{(\text{R})}(\mathbf{x}_a)}{\prod_{\alpha_d \in e(a)} \mathfrak{B}_{\alpha}^{(\text{R})}(x_{\alpha})} \right]. \quad (5.18)$$

The notation “MF” anticipates the mean-field product ansatz of Section 6.1: edge variables are independent under the proposal distribution, while the likelihood ratio in Eq. (5.18) restores their graphical dependence.

A full gauge contains an entire invertible matrix on each directed edge, whereas a reparametrization uses only one positive scalar function per directed edge. For binary alphabets, the distinguished ground rows of a gauge can be matched closely to reparametrization functions. For larger alphabets, the remaining gauge rows provide additional degrees of freedom. This difference is one reason gauge-based and reparametrization-based approximation schemes diverge beyond the binary setting.

A generic reparametrization does not make the beliefs consistent. The relations

$$\mathfrak{B}_{\alpha}^{(\text{R})}(x_{\alpha}) = \int_{\mathbf{x}_a \setminus x_{\alpha}} \mathfrak{B}_a^{(\text{R})}(\mathbf{x}_a), \quad \alpha_d \in e(a),$$

select a special reparametrization. On a tree, these are exact marginal-consistency equations from Chapter 3; on a loopy graph, their fixed-point form is belief propagation. Chapter 6 derives the same conditions from stationary points of the Bethe free energy, and Chapter 9 uses the corresponding gauge to suppress all dangling-edge corrections.

Companion notebook: Reparametrization and BP consistency

The notebook `ch05/ch05_02_reparametrization_and_bp.ipynb` verifies distribution invariance on the chain introduced in Chapter 2. It then chooses the reparametrization functions from exact tree messages and checks node–edge marginal consistency.

Exercise 5.1.4. Rescale every directed-edge function as $R_{\alpha_d} \leftarrow c_{\alpha_d} R_{\alpha_d}$ with positive constants. Determine how the local normalizers, $Z^{(\text{R})}$, and $\mathcal{Z}^{(\text{R})}$ change, and verify that Z and $P(\mathbf{x})$ remain invariant.

5.1.3 Reparametrization in the factor graph representation

Let $\mathcal{G} = (\mathcal{V}_1, \mathcal{V}_2, \mathcal{E})$ be a factor graph, with variable nodes $a \in \mathcal{V}_1$, factor nodes $A \in \mathcal{V}_2$, and neighborhood $\mathcal{N}(a) = \{A \in \mathcal{V}_2 : (a, A) \in \mathcal{E}\}$. Write $d_a = |\mathcal{N}(a)|$. Associate a positive function $R_{A \rightarrow a}(x_a)$ with every directed factor-to-variable incidence. A useful telescoping identity is

$$\prod_{A \in \mathcal{V}_2} f_A(\mathbf{x}_A) = \frac{\prod_{A \in \mathcal{V}_2} \left(f_A(\mathbf{x}_A) \prod_{a \in A} \prod_{B \in \mathcal{N}(a) \setminus \{A\}} R_{B \rightarrow a}(x_a) \right)}{\prod_{a \in \mathcal{V}_1} \left(\prod_{A \in \mathcal{N}(a)} R_{A \rightarrow a}(x_a) \right)^{d_a - 1}}. \quad (5.19)$$

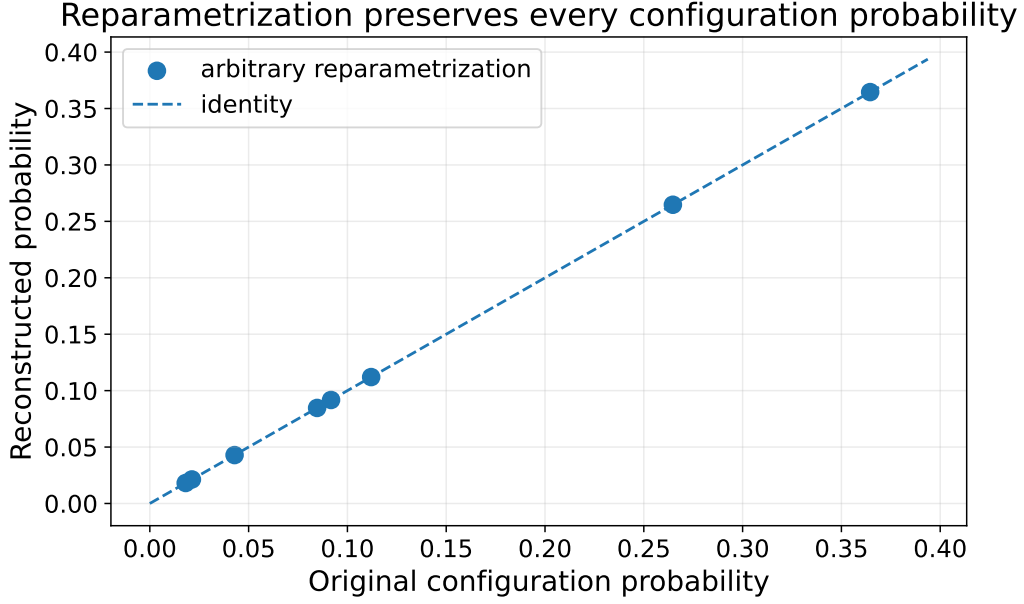


Figure 5.3: Reparametrization of a three-variable chain. Local factor tables change, but the normalized global distribution is unchanged for every configuration. This is stronger than gauge invariance, which only guarantees equality of partition functions.

Indeed, each function $R_{A \rightarrow a}$ appears exactly $d_a - 1$ times in the numerator and the same number of times in the denominator. This explicit counting is important: multiplying factor A by its own outgoing function would not cancel and would not define an exact reparametrization.

Define factor and variable beliefs by

$$\mathfrak{B}_A^{(R)}(\mathbf{x}_A) := \frac{f_A(\mathbf{x}_A) \prod_{a \in A} \prod_{B \in \mathcal{N}(a) \setminus \{A\}} R_{B \rightarrow a}(x_a)}{C_A^{(R)}}, \quad (5.20)$$

$$\mathfrak{B}_a^{(R)}(x_a) := \frac{\prod_{A \in \mathcal{N}(a)} R_{A \rightarrow a}(x_a)}{C_a^{(R)}}, \quad (5.21)$$

where each $C_A^{(R)}$ and $C_a^{(R)}$ normalizes the corresponding belief. The probability distribution takes the exact tree-like form

$$P(\mathbf{x}) = \frac{1}{\mathcal{Z}^{(R)}} \frac{\prod_{A \in \mathcal{V}_2} \mathfrak{B}_A^{(R)}(\mathbf{x}_A)}{\prod_{a \in \mathcal{V}_1} \left(\mathfrak{B}_a^{(R)}(x_a) \right)^{d_a - 1}}, \quad (5.22)$$

with

$$\mathcal{Z}^{(R)} := \int_{\mathbf{x}} \frac{\prod_{A \in \mathcal{V}_2} \mathfrak{B}_A^{(R)}(\mathbf{x}_A)}{\prod_{a \in \mathcal{V}_1} \left(\mathfrak{B}_a^{(R)}(x_a) \right)^{d_a - 1}}. \quad (5.23)$$

The original partition function factorizes as

$$Z = Z^{(\mathbf{R})} \mathcal{Z}^{(\mathbf{R})}, \quad Z^{(\mathbf{R})} := \frac{\prod_{A \in \mathcal{V}_2} C_A^{(\mathbf{R})}}{\prod_{a \in \mathcal{V}_1} \left(C_a^{(\mathbf{R})}\right)^{d_a-1}}. \quad (5.24)$$

Proposition 5.1.3 (Exact reparametrization of an ordinary factor graph). *For any positive incidence functions $R_{A \rightarrow a}$, Eqs. (5.20) and (5.21) produce the exact representation in Eq. (5.22), and the partition function factorizes according to Eq. (5.24).*

Proof. The telescoping identity in Eq. (5.19) shows that every $R_{A \rightarrow a}$ occurs exactly $d_a - 1$ times in both numerator and denominator. Substituting the normalized factor and variable beliefs preserves this cancellation and leaves the original factor product times the stated ratio of local normalizers. Global normalization yields Eqs. (5.22)–(5.24). $\square$

On a tree, Eq. (5.22) reduces to the exact reconstruction formula of Chapter 3 once the beliefs are true marginals. On a loopy graph, imposing

$$\mathfrak{B}_a^{(\mathbf{R})}(x_a) = \int_{\mathbf{x}_A \setminus x_a} \mathfrak{B}_A^{(\mathbf{R})}(\mathbf{x}_A), \quad A \in \mathcal{N}(a),$$

selects the belief-propagation reparametrization discussed in Sections 6.2 and 6.3.2.

5.1.4 Reparametrization in the hypergraph representation

The factor-graph construction extends to a hierarchy of overlapping regions. Let $\mathcal{E}^h$ be a finite family of hyperedges ordered by set inclusion. Every original factor is assigned to its corresponding hyperedge; hyperedges introduced only to complete the hierarchy carry the unity factor. For a hyperedge g , define its strict descendants and ancestors and their inclusive versions by

$$\begin{aligned} \text{Des}(g) &:= \{h \in \mathcal{E}^h : h \subset g\}, & \text{Des}^+(g) &:= \text{Des}(g) \cup \{g\}, \\ \text{Anc}(g) &:= \{p \in \mathcal{E}^h : g \subset p\}, & \text{Anc}^+(g) &:= \text{Anc}(g) \cup \{g\}. \end{aligned} \quad (5.25)$$

Associate a positive parent-to-child function $R_{p \rightarrow h}(\mathbf{x}_h)$ with every selected inclusion relation $h \subset p$; relations not included in a chosen region graph may be understood to carry the unity function. Define the unnormalized regional quantity

$$\Phi_g^{(\mathbf{R})}(\mathbf{x}_g) := \prod_{h \in \text{Des}^+(g)} \left(f_h(\mathbf{x}_h) \prod_{p \in \text{Anc}(h) \setminus \text{Des}^+(g)} R_{p \rightarrow h}(\mathbf{x}_h) \right).$$

The regional belief is

$$\mathfrak{B}_g^{(\mathbf{R})}(\mathbf{x}_g) := \frac{\Phi_g^{(\mathbf{R})}(\mathbf{x}_g)}{\int_{\mathbf{x}_g} \Phi_g^{(\mathbf{R})}(\mathbf{x}_g)}. \quad (5.26)$$

The product in $\Phi_g^{(\mathbf{R})}$ has a simple boundary interpretation: it contains all factors supported inside g and all reparametrization functions entering descendants of g from regions lying outside g .

Theorem 5.1.4 (Hypergraph reparametrization). *Let*

$$P(\mathbf{x}) = \frac{1}{Z} \prod_{g \in \mathcal{E}^h} f_g(\mathbf{x}_g).$$

Suppose the counting numbers $c(g)$ satisfy

$$\forall g \in \mathcal{E}^h : \sum_{g' \supseteq g} c(g') = 1. \quad (5.27)$$

Then the distribution admits the exact representation

$$P(\mathbf{x}) = \frac{1}{Z^{(R)}} \prod_{g \in \mathcal{E}^h} \left(\mathfrak{B}_g^{(R)}(\mathbf{x}_g) \right)^{c(g)}, \quad (5.28)$$

where

$$Z^{(R)} := \int_{\mathbf{x}} \prod_{g \in \mathcal{E}^h} \left(\mathfrak{B}_g^{(R)}(\mathbf{x}_g) \right)^{c(g)}. \quad (5.29)$$

Proof. Consider first an original factor f_h . It enters $\mathfrak{B}_g^{(R)}$ precisely when $g \supseteq h$, so its total exponent in Eq. (5.28) is $\sum_{g \supseteq h} c(g) = 1$. Now consider $R_{p \rightarrow h}$. It enters exactly those beliefs with $h \subseteq g$ but $p \not\subseteq g$. Its total exponent is therefore

$$\sum_{g \supseteq h} c(g) - \sum_{g \supseteq p} c(g) = 1 - 1 = 0.$$

All reparametrization functions cancel, while every original factor remains once. Local belief normalizers produce one global multiplicative constant, absorbed into $Z^{(R)}$. $\square$

The distinction between the preceding propositions and Theorem 5.1.4 is intentional. Propositions 5.1.1, 5.1.2, and 5.1.3 formalize established gauge and reparametrization identities. The theorem isolates their extension to an arbitrary inclusion-ordered region hierarchy. To the author's knowledge, the precise statement and cancellation proof in Theorem 5.1.4 have not appeared in this form in the published literature; the result is therefore highlighted separately rather than treated only as an algebraic remark.

The counting numbers can be computed recursively from maximal regions downward. If $\mu(h, g)$ denotes the Möbius function of the inclusion poset introduced in Appendix 23.1.4, then

$$c(h) = \sum_{g \supseteq h} \mu(h, g). \quad (5.30)$$

Consequently, the same distribution can be written as

$$P(\mathbf{x}) \propto \prod_{g \in \mathcal{E}^h} \prod_{h \subseteq g} \left(\mathfrak{B}_h^{(R)}(\mathbf{x}_h) \right)^{\mu(h, g)}. \quad (5.31)$$

Region inclusion hierarchy and Möbius counting numbers

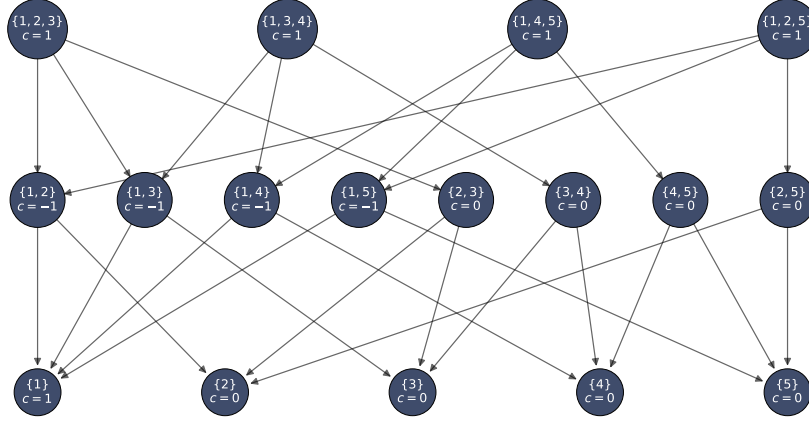


Figure 5.4: Three-level region hierarchy for Example 5.1.5. Maximal triple regions are followed by pair and singleton regions. Each enlarged node uses a two-line white label: the region appears on the first line and its counting number from Eq. (5.27) on the second.

If $C_g^{(R)} = \int_{\mathbf{x}_g} \Phi_g^{(R)}(\mathbf{x}_g)$, the exact partition function satisfies

$$Z = Z^{(R)} \mathcal{Z}^{(R)}, \quad Z^{(R)} := \prod_{g \in \mathcal{E}^h} \left(C_g^{(R)} \right)^{c(g)}. \quad (5.32)$$

This is the algebraic foundation of region-graph and generalized-belief-propagation constructions [222, 221, 211]. Chapter 6 uses the same counting numbers to build Kikuchi free energies; Chapter 9 uses Möbius-type organization to control higher-order corrections.

Example 5.1.5. Build the reparametrization and counting numbers for the three-level hierarchy in Fig. 5.4.

Solution. The model contains the maximal triple regions 123, 134, 145, and 125, all pair regions shown in the figure, and singleton regions. To retain the full region hierarchy explicitly, write

$$P(x_1, \dots, x_5) \propto f_{123} f_{134} f_{145} f_{125} f_{12} f_{13} f_{14} f_{15} f_{23} f_{34} f_{45} f_{25} \prod_{i=1}^5 f_i, \quad (5.33)$$

where every factor is evaluated on the variables indexed by its subscript. Factors not present in a particular application may be set to unity.

Starting from the maximal regions, Eq. (5.27) gives

$$c(123) = c(134) = c(145) = c(125) = 1.$$

The central pairs 12, 13, 14, and 15 belong to two maximal regions and therefore have counting number -1 . The outer pairs 23, 34, 45, and 25 belong to one maximal region and have counting

number 0. At the singleton level, $c(1) = 1$ and $c(2) = c(3) = c(4) = c(5) = 0$. Hence only nine beliefs appear with nonzero powers:

$$P(x_1, \dots, x_5) \propto \frac{\mathfrak{B}_{123}^{(R)} \mathfrak{B}_{134}^{(R)} \mathfrak{B}_{145}^{(R)} \mathfrak{B}_{125}^{(R)} \mathfrak{B}_1^{(R)}}{\mathfrak{B}_{12}^{(R)} \mathfrak{B}_{13}^{(R)} \mathfrak{B}_{14}^{(R)} \mathfrak{B}_{15}^{(R)}}.$$

For completeness, the nonzero-power beliefs are listed explicitly below. The symbol $\propto$ suppresses only the local normalization of each belief:

$$\begin{aligned} \mathfrak{B}_{123}^{(R)} &\propto f_{123} f_{12} f_{13} f_{23} f_1 f_2 f_3 R_{134 \rightarrow 13} R_{125 \rightarrow 12} R_{14 \rightarrow 1} R_{15 \rightarrow 1} R_{25 \rightarrow 2} R_{34 \rightarrow 3}, \\ \mathfrak{B}_{134}^{(R)} &\propto f_{134} f_{13} f_{14} f_{34} f_1 f_3 f_4 R_{123 \rightarrow 13} R_{145 \rightarrow 14} R_{12 \rightarrow 1} R_{15 \rightarrow 1} R_{23 \rightarrow 3} R_{45 \rightarrow 4}, \\ \mathfrak{B}_{145}^{(R)} &\propto f_{145} f_{14} f_{15} f_{45} f_1 f_4 f_5 R_{134 \rightarrow 14} R_{125 \rightarrow 15} R_{12 \rightarrow 1} R_{13 \rightarrow 1} R_{34 \rightarrow 4} R_{25 \rightarrow 5}, \\ \mathfrak{B}_{125}^{(R)} &\propto f_{125} f_{12} f_{15} f_{25} f_1 f_2 f_5 R_{123 \rightarrow 12} R_{145 \rightarrow 15} R_{13 \rightarrow 1} R_{14 \rightarrow 1} R_{23 \rightarrow 2} R_{45 \rightarrow 5}, \\ \mathfrak{B}_{12}^{(R)} &\propto f_{12} f_1 f_2 R_{123 \rightarrow 12} R_{125 \rightarrow 12} R_{13 \rightarrow 1} R_{14 \rightarrow 1} R_{15 \rightarrow 1} R_{23 \rightarrow 2} R_{25 \rightarrow 2}, \\ \mathfrak{B}_{13}^{(R)} &\propto f_{13} f_1 f_3 R_{123 \rightarrow 13} R_{134 \rightarrow 13} R_{12 \rightarrow 1} R_{14 \rightarrow 1} R_{15 \rightarrow 1} R_{23 \rightarrow 3} R_{34 \rightarrow 3}, \\ \mathfrak{B}_{14}^{(R)} &\propto f_{14} f_1 f_4 R_{134 \rightarrow 14} R_{145 \rightarrow 14} R_{12 \rightarrow 1} R_{13 \rightarrow 1} R_{15 \rightarrow 1} R_{34 \rightarrow 4} R_{45 \rightarrow 4}, \\ \mathfrak{B}_{15}^{(R)} &\propto f_{15} f_1 f_5 R_{125 \rightarrow 15} R_{145 \rightarrow 15} R_{12 \rightarrow 1} R_{13 \rightarrow 1} R_{14 \rightarrow 1} R_{25 \rightarrow 5} R_{45 \rightarrow 5}, \\ \mathfrak{B}_1^{(R)} &\propto f_1 R_{12 \rightarrow 1} R_{13 \rightarrow 1} R_{14 \rightarrow 1} R_{15 \rightarrow 1}. \end{aligned}$$

Every R factor cancels after the beliefs are raised to their counting numbers, while each original factor in Eq. (5.33) remains with exponent one. Regions with counting number zero are still useful because their beliefs participate in consistency constraints in generalized belief propagation.

Companion notebook: Möbius counting on a region hierarchy

The notebook `ch05/ch05_03_hypergraph_mobius.ipynb` constructs the hierarchy in Fig. 5.4, solves the triangular system Eq. (5.27), and verifies factor and reparametrization exponents numerically.

Exercise 5.1.5. Add the pair region 24 to the hierarchy, together with any maximal region needed to make it nontrivial. Recompute all counting numbers and verify the two cancellation identities used in the proof of Theorem 5.1.4.

5.2 Graph transformations and conditioning

Gauge and reparametrization transformations leave the graph fixed. A graph transformation instead changes the representation or conditions on part of the state space. Variable elimination in Chapter 3 is the first example: removing a variable creates a new factor on its neighbors. Chapter 7 will replace an expensive exact factor by a controlled approximation.

A second exact construction is *cutset conditioning*. In an NFG, split the edge variables into disjoint sets,

$$\mathcal{E} = \mathcal{E}_1 \sqcup \mathcal{E}_2, \quad \mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2).$$

For a fixed value of $\mathbf{x}_2$, define the conditional partition function

$$Z_{1|2}(\mathbf{x}_2) := \int_{\mathbf{x}_1} \prod_{a \in \mathcal{V}} f_a(\mathbf{x}_a).$$

Then

$$p(\mathbf{x}_1, \mathbf{x}_2) = p(\mathbf{x}_1 | \mathbf{x}_2) p(\mathbf{x}_2), \quad p(\mathbf{x}_1 | \mathbf{x}_2) = \frac{\prod_{a \in \mathcal{V}} f_a(\mathbf{x}_a)}{Z_{1|2}(\mathbf{x}_2)},$$

and

$$Z = \int_{\mathbf{x}_2} Z_{1|2}(\mathbf{x}_2).$$

If fixing $\mathbf{x}_2$ breaks every cycle, the remaining model in $\mathbf{x}_1$ is a tree and $Z_{1|2}(\mathbf{x}_2)$ is computed exactly by Chapter 3. The outer sum over the cutset remains exponential in $|\mathcal{E}_2|$, so the method is effective when a small cycle cutset exists. This classical conditioning idea underlies hybrid elimination schemes and is closely related to the decimation-based exact generation of Chapter 4; see [160, 110].

5.3 Tree transformations

Why transform a loopy graph into a tree? Chapter 3 showed that trees support exact local message passing. A natural response to a loop is therefore to *unroll* it: duplicate variables as necessary so that a finite neighborhood of the computation becomes tree shaped. The duplicated tree is not, in general, the same probability model. Its role is algorithmic: it reproduces a selected sequence of message updates or, in special models such as the hard-core model, reproduces a selected root marginal exactly after suitable boundary conditions are imposed.

Trees are important because exact message passing is linear in the number of tree edges when local alphabet sizes are fixed. There are several ways to exploit this fact. Tree-reweighted methods average over spanning trees and lead to a convex variational upper bound on the log partition function; these are developed in Section 6.4 and in [205, 202]. Here we retain two different tree constructions: a computation tree that represents a finite number of loopy-BP iterations, and a self-avoiding tree that gives an exact root marginal for the hard-core model.

5.3.1 Computation tree

Fix a root r in a pairwise graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. The depth- n *computation tree* contains all nonbacktracking walks $(r, i_1, \dots, i_k)$ of length $k \leq n$. Two tree nodes may carry the same original-graph label; they are distinct because they represent different walks. Local factors are copied from the original graph, and the leaves receive boundary messages determined by the initialization of the loopy message-passing algorithm.

After n synchronous sum-product iterations, the belief at r in the loopy graph is exactly the root belief obtained by exact inference on the depth- n computation tree. This equivalence is a statement about the computation performed by the algorithm, not about equality between the loopy model and an infinite tree. It is used to analyze convergence, uniqueness, and decay of boundary influence [209, 189, 95, 140].

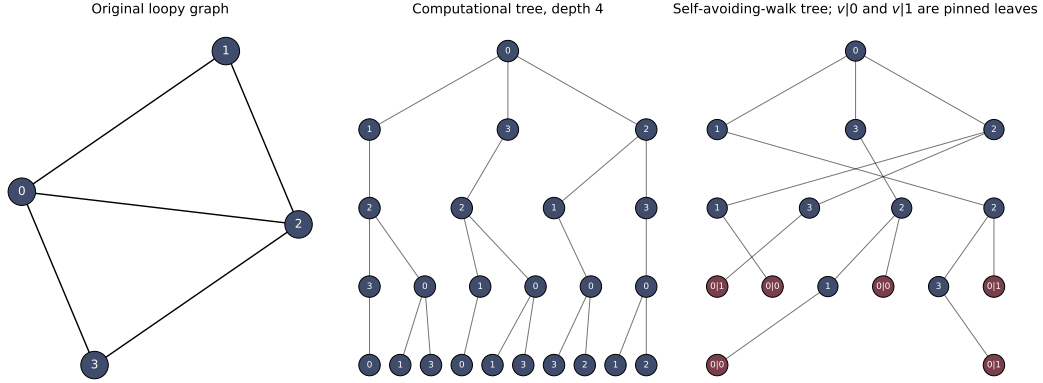


Figure 5.5: A loopy graph, its finite computation tree, and its self-avoiding tree rooted at the same vertex. The computation tree retains nonbacktracking walks up to a chosen depth. The self-avoiding tree terminates when a walk would revisit an earlier vertex and records the closure through a boundary condition.

5.3.2 Self-avoiding tree for the hard-core model

The exact theorem of Weitz [210] concerns the hard-core model, which represents independent sets. Let $x_i \in \{0, 1\}$ indicate whether vertex i is occupied, and let $\lambda > 0$ be the activity:

$$p(\mathbf{x}) \propto \lambda^{\sum_{i \in V} x_i} \prod_{\{i, j\} \in \mathcal{E}} \mathbf{1}(x_i x_j = 0).$$

The indicator enforces that adjacent vertices cannot both be occupied.

Construct the self-avoiding-walk tree $T_{\text{SAW}}(\mathcal{G}, r)$ from walks beginning at r . A branch is terminated when the next step would revisit a vertex already on the walk. Fix a local ordering of the neighbors of every original vertex. At a cycle-closing leaf, compare the edge that closes the cycle with the edge by which the repeated vertex was first entered. Pin the leaf occupied if the closing edge is higher in the local ordering and unoccupied otherwise.

Theorem 5.3.1 (Self-avoiding-tree identity [210]). *For the hard-core model on $\mathcal{G}$, the root occupation ratio*

$$\frac{p_{\mathcal{G}}(x_r = 1)}{p_{\mathcal{G}}(x_r = 0)} \tag{5.34}$$

is exactly equal to the corresponding ratio at the root of $T_{\text{SAW}}(\mathcal{G}, r)$ with the cycle-closing leaves pinned by the ordering rule above.

The literal occupied/unoccupied pinning rule in the theorem is tailored to the hard-core constraint, but the self-avoiding-tree and computation-tree program is not limited to that model. Weitz-style tree recursions and correlation-decay arguments have been extended to anti-ferromagnetic two-state spin systems with soft interactions, including the anti-ferromagnetic Ising model with an external field [183, 125, 126]. For larger alphabets, computation-tree methods have been developed for list colorings and discrete Markov random fields [73]; broader

multistate and Holant formulations require model-dependent recursive couplings or bounded-treewidth subroutines rather than one universal scalar leaf-pinning rule [223]. Thus the organizing idea—replace a loopy neighborhood by a recursively generated tree and analyze the propagation of boundary information—is general, while the exact boundary object and the conditions guaranteeing useful correlation decay depend on the model.

Constructive proof idea. Start from the ratio in Eq. (5.34), represented as the quotient of two partition functions with the root pinned occupied and unoccupied, respectively. Replicate the root once for every incident edge. In the numerator all replicas inherit the occupied value, while in the denominator they inherit the unoccupied value. Multiply numerator and denominator by a telescoping sequence of otherwise identical partition functions in which the replica values are changed one at a time. The original ratio becomes a product of ratios, each attached to one branch leaving the root.

This node splitting opens every cycle that passes through the root. Apply the same operation to each newly exposed vertex. At every stage, numerator and denominator differ only at the current root of a subproblem; all other duplicated factors cancel between consecutive ratios. The resulting structures contain more nodes but fewer closed walks. Repeating the construction generates the tree of self-avoiding walks.

When an attempted extension returns to a vertex already present on the current walk, no further copy is created. Instead, the associated telescoping factor is represented by a pinned leaf. The fixed local neighbor ordering determines whether this leaf is pinned occupied or unoccupied, and this choice makes the two neighboring factors in the telescoping product cancel. After all branches terminate, the product of the exact tree recursions is identical to the original root ratio. The full algebraic construction and the deterministic approximation scheme based on truncating this tree are given in [210]. $\square$

Problem 5.1 (Beyond the hard-core self-avoiding tree). *Consider first a binary pairwise model with strictly positive node factors $\phi_i(x_i)$ and edge factors $\psi_{ij}(x_i, x_j)$. Derive the vector or likelihood-ratio recursion for a rooted tree and investigate how a cycle-closing branch of the self-avoiding tree should transmit its boundary contribution. Test the proposed construction against exact enumeration on a triangle and a four-cycle, and identify which properties of the hard-core model made scalar occupied/unoccupied pinning sufficient.*

Next consider a q -state model, such as list coloring or a general discrete Markov random field. Formulate a computation-tree recursion in terms of q -component messages, determine what boundary data must be retained at a repeated vertex, and compare a truncated-tree approximation with exact marginals on small graphs. Use [183, 125, 73, 223] to distinguish the binary soft-interaction extension from genuinely multistate constructions. State clearly whether your proposed tree gives an exact finite identity or only an approximation controlled by correlation decay.

Companion notebook: Conditioning, computation trees, and self-avoiding trees

The notebook `ch05/ch05_04_tree_transformations.ipynb` constructs finite computation and self-avoiding trees, applies the Weitz boundary rule, and compares the root marginal with exact enumeration on several small graphs.

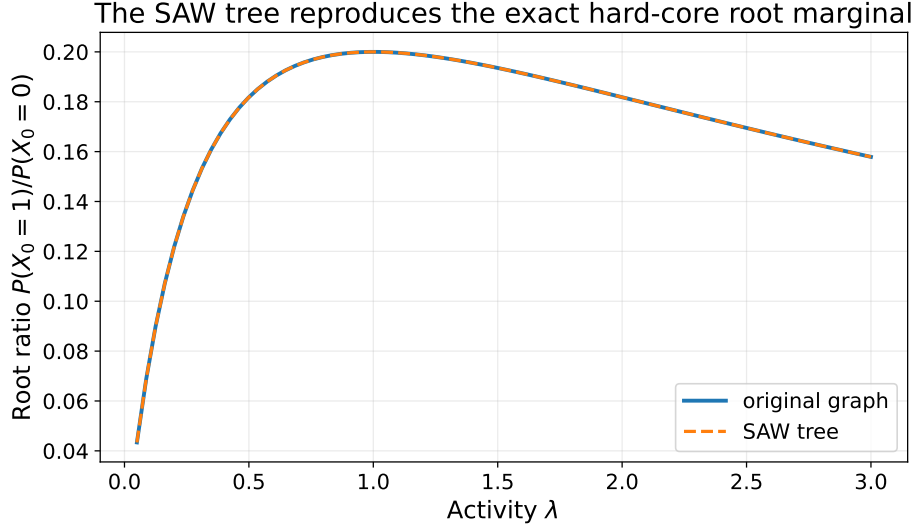


Figure 5.6: Exact check of the self-avoiding-tree identity on small hard-core models. The direct root occupation ratio from enumeration agrees with the tree recursion after the cycle-closing boundary conditions are imposed.

Exercise 5.3.1. For a triangle and a four-cycle, construct the self-avoiding tree by hand for two different neighbor orderings. Verify that the pinned leaves change but the root ratio does not.

Exercise 5.3.2. Run ordinary loopy BP for n synchronous iterations on a small pairwise model. Construct the depth- n computation tree with boundary messages matching the initialization and verify equality of the root beliefs.

5.4 What the transformations accomplish

The constructions in this chapter separate three questions that are often conflated. A gauge asks which local tensor basis makes a contraction easiest. A reparametrization asks how the same probability distribution can be redistributed among local beliefs. A graph transformation asks which enlarged, conditioned, or unfolded graph exposes an exact or approximate algorithm. The transformations are therefore not preprocessing tricks; they are coordinates on spaces of algorithms.

Problem 5.2 (Choosing a tractable gauge). For a small binary normal factor graph, formulate an optimization over paired gauge matrices that attempts to make selected transformed factors sparse or low rank while enforcing Eq. (5.2). Compare the resulting contraction cost with the original representation. Discuss whether nonnegativity should be imposed and how that restriction changes the attainable gauges.

Problem 5.3 (Reparametrization as algorithm design). Start from a loopy factor graph and a family of positive functions $R_{A \rightarrow a}$. Design an objective that favors marginal consistency while

controlling numerical conditioning of the local beliefs. Derive stationary equations, compare them with BP, and test whether different reparametrizations lead to different convergence behavior even though they represent the same global distribution.

Problem 5.4 (Tree transformations for structured generation). *Combine cutset conditioning with the exact conditional-generation procedure of Chapter 4. Select a cutset that leaves a tree, compute exact tree conditionals for each cutset assignment, and design a generator that samples first the cutset and then the tree. Analyze the computational tradeoff between cutset size, repeated inference, and caching.*

Chapter 6

Variational Inference: Free Energies, Bounds, and Message Passing

Reader's Guide.

Why this chapter? Exact elimination fails when its separator functions become too large. Variational inference attacks the same difficulty from a different direction: instead of carrying an exponentially large exact global distribution, optimize over a simpler family of distributions or over collections of local beliefs. The price is approximation; the benefit is that locality survives and often leads directly to message-passing algorithms.

What you should know before starting. The essential prerequisites are the Gibbs variational principle from Chapter 4, exact tree messages from Chapter 3, and the idea of locally equivalent representations from Chapter 5. No prior statistical-mechanics course is assumed. “Energy,” “entropy,” and “free energy” are used as mathematical names for the three terms already introduced in the exact variational principle.

The simplest warning example: independence cannot represent correlation. Let two binary variables have

$$p(0,0) = p(1,1) = \frac{1}{2}, \quad p(0,1) = p(1,0) = 0.$$

Each variable separately is uniform, but the pair is perfectly correlated. A mean-field trial law $q(x_1, x_2) = q_1(x_1)q_2(x_2)$ cannot reproduce this joint distribution: choosing uniform marginals necessarily assigns probability $1/4$ to all four pairs. This is the first approximation in the chapter in its most transparent form. Mean field keeps a globally valid but overly simple joint law. Bethe inference repairs part of the missing structure by keeping explicit beliefs on interacting regions and enforcing their local consistency.

A hierarchy to keep in mind. The chapter repeatedly asks how much dependence to retain. Product beliefs are cheapest but discard correlations; Bethe beliefs retain the graph’s original local correlations; tree-reweighted constructions average over tractable tree structures and can provide bounds; Kikuchi/generalized belief propagation introduces larger overlapping regions. Moving up the hierarchy costs more but can represent more of the dependence that exact inference would retain.

What you should be able to do after reading. You should be able to derive and interpret mean-field coordinate updates; construct the Bethe free energy and local-consistency constraints; derive belief-propagation fixed-point equations from stationarity; distinguish a variational objective from an algorithm used to optimize it; understand what tree reweighting certifies; and see generalized belief propagation as the same logic applied to larger regions.

How to use the companion notebooks. The notebooks in `notebooks/ch06/` are organized as failure tests as much as demonstrations. Begin with mean field on a small Ising model and change coupling strength until independence becomes visibly poor. Then verify Bethe/belief-propagation exactness on a tree before introducing a loop. Use the tree-reweighted notebook to distinguish an approximation from a certified bound, and the Kikuchi notebook to see what is gained by enlarging regions. Whenever possible, compare against exact enumeration from Chapter 3.

Chapter 4 introduced the exact Gibbs variational principle and showed that inference can be viewed as optimization over the full probability simplex. That representation is exact but exponentially large. The purpose of this chapter is to replace the global distribution by tractable collections of marginal or regional beliefs while retaining as much of the graphical structure as possible.

The chapter develops a hierarchy:

$$\begin{aligned} \text{product beliefs} &\longrightarrow \text{locally consistent node-factor beliefs} \longrightarrow \\ &\text{tree-reweighted beliefs} \longrightarrow \text{multilevel region beliefs.} \end{aligned}$$

The first step gives mean-field inference; the second gives the Bethe free energy and belief propagation; the third gives certified tree-reweighted bounds; and the last gives the Kikuchi approximation and generalized belief propagation. The same hierarchy also links finite-temperature inference to the zero-temperature MAP relaxations introduced in Chapter 4.

We use the normal factor graph (NFG) representation first because its edge variables make the entropy-counting logic particularly transparent. We then translate the main results to ordinary factor graphs (FGs), where most machine-learning implementations are formulated. The gauge and reparametrization viewpoints developed in Chapter 5 will provide a third interpretation of the same fixed-point equations. Chapter 7 will subsequently use these variational objects to motivate approximate elimination, and Chapter 9 will quantify the loop corrections omitted by Bethe and generalized Bethe approximations.

For a normalized trial distribution $q(\mathbf{x})$, define its marginals on NFG nodes and edges by

$$q_a(\mathbf{x}_a) := \int_{\mathbf{x} \setminus \mathbf{x}_a} q(\mathbf{x}), \quad a \in \mathcal{V}, \quad (6.1)$$

$$q_\alpha(x_\alpha) := \int_{\mathbf{x} \setminus x_\alpha} q(\mathbf{x}), \quad \alpha \in \mathcal{E}. \quad (6.2)$$

We use the letter q for an actual global trial distribution and $\mathfrak{B}$ for a collection of approximate beliefs that may be only locally consistent. This distinction is important: mean field optimizes over globally realizable product distributions, whereas Bethe and Kikuchi methods generally optimize over relaxed marginal polytopes that may contain beliefs with no global joint realization.

The five notebooks accompanying this chapter reuse the Ising chains, cycles, and region hierarchies introduced in Chapters 2–5. They compare exact, mean-field, Bethe, tree-reweighted, Kikuchi, and MAP-LP calculations, and save all figures in `ch06/figs/`.

6.1 Mean-field approximation

Mean field in one sentence. Replace “all variables may depend on one another” by “chosen blocks are independent,” and then choose the best distribution within that restricted family. The optimization may still be nonconvex, but every candidate is a legitimate global distribution. This is why the mean-field free energy gives a one-sided bound on the exact partition function.

6.1.1 Product-family restriction in a normal factor graph

For an NFG, the elementary variables live on edges. The basic mean-field family assumes that these edge variables are independent:

$$q^{(\text{MF})}(\mathbf{x}) := \prod_{\alpha \in \mathcal{E}} \mathfrak{B}_\alpha(x_\alpha). \quad (6.3)$$

Every factor $\mathfrak{B}_\alpha$ is nonnegative and normalized, so Eq. (6.3) defines a valid global probability distribution. Marginalizing the product with respect to all variables except x_α returns $\mathfrak{B}_\alpha(x_\alpha)$ exactly. Mean field therefore restricts the feasible family but does not relax global realizability.

The feasible set is a Cartesian product of probability simplices:

$$\mathbf{B}^{(\text{MF})} := (\mathfrak{B}_\alpha(x_\alpha) \mid \alpha \in \mathcal{E}), \quad (6.4)$$

$$\mathcal{D}^{(\text{MF})} := \left\{ \mathbf{B}^{(\text{MF})} \left| \begin{array}{ll} \mathfrak{B}_\alpha(x_\alpha) \geq 0, & \alpha \in \mathcal{E}, \ x_\alpha \in \mathcal{X}_\alpha, \\ \int_{x_\alpha} \mathfrak{B}_\alpha(x_\alpha) = 1, & \alpha \in \mathcal{E}. \end{array} \right. \right\}. \quad (6.5)$$

The expected energy under the product distribution is

$$\begin{aligned} E^{(\text{MF})}(\mathbf{B}^{(\text{MF})}) &:= \sum_{a \in \mathcal{V}} \int_{\mathbf{x}_a} E_a(\mathbf{x}_a) \prod_{\alpha \in e(a)} \mathfrak{B}_\alpha(x_\alpha) \\ &= -\frac{1}{\beta} \sum_{a \in \mathcal{V}} \int_{\mathbf{x}_a} \log f_a(\mathbf{x}_a) \prod_{\alpha \in e(a)} \mathfrak{B}_\alpha(x_\alpha). \end{aligned} \quad (6.6)$$

Here $e(a)$ denotes the set of directed edge incidences at node a , as defined in Appendix 23.

6.1.2 Mean field and MAP

Restricting the exact expected-energy formulation of MAP to the product family gives

$$\bar{E}^{(\text{MF})} := \min_{\mathbf{B}^{(\text{MF})} \in \mathcal{D}^{(\text{MF})}} E^{(\text{MF})}(\mathbf{B}^{(\text{MF})}). \quad (6.7)$$

At first sight this looks like a relaxation or approximation. In fact, its *global* optimum is exact.

Proposition 6.1.1 (Product-family MAP is globally exact). *For finite alphabets,*

$$\bar{E}^{(\text{MF})} = \bar{E} := \min_{\mathbf{x} \in \mathcal{X}} E(\mathbf{x}). \quad (6.8)$$

Proof. Every product distribution satisfies $\mathbb{E}_q[E] \geq \min_{\mathbf{x}} E(\mathbf{x})$. Conversely, for every configuration $\mathbf{x}^*$, the delta distribution $q(\mathbf{x}) = \prod_{\alpha} \delta(x_\alpha - x_\alpha^*)$ belongs to the product family. Choosing a MAP configuration therefore attains $\min_{\mathbf{x}} E(\mathbf{x})$. $\square$

The difficulty is algorithmic rather than representational. The function in Eq. (6.6) is generally multilinear and nonconvex, so local coordinate methods can stop at nonglobal stationary points. Global minimizers may lie on the boundary of $\mathcal{D}^{(\text{MF})}$ —indeed exact MAP solutions are delta distributions located at vertices of the product polytope. This clarifies why “mean-field MAP” is not normally used as a practical exact solver despite Proposition 6.1.1.

6.1.3 Mean-field free energy and the lower bound on the partition function

Let

$$H(\mathfrak{B}_\alpha) := - \int_{x_\alpha} \mathfrak{B}_\alpha(x_\alpha) \log \mathfrak{B}_\alpha(x_\alpha).$$

The entropy of the product distribution is additive:

$$S^{(\text{MF})}(\mathbf{B}^{(\text{MF})}) := \sum_{\alpha \in \mathcal{E}} H(\mathfrak{B}_\alpha). \quad (6.9)$$

The mean-field free energy and partition-function estimate are

$$F^{(\text{MF})}(\mathbf{B}^{(\text{MF})}) := E^{(\text{MF})}(\mathbf{B}^{(\text{MF})}) - \beta^{-1} S^{(\text{MF})}(\mathbf{B}^{(\text{MF})}), \quad (6.10)$$

$$-\beta^{-1} \log Z^{(\text{MF})} := \min_{\mathbf{B}^{(\text{MF})} \in \mathcal{D}^{(\text{MF})}} F^{(\text{MF})}(\mathbf{B}^{(\text{MF})}). \quad (6.11)$$



Because the optimization is the exact Gibbs variational problem restricted to a subset of valid global distributions,

$$Z^{(\text{MF})} \leq Z, \quad -\beta^{-1} \log Z^{(\text{MF})} \geq -\beta^{-1} \log Z. \quad (6.12)$$

The certificate is rigorous, but the bound can be loose when correlations are strong. The limitation is structural: Eq. (6.3) assigns zero mutual information to every pair of distinct edge variables. Generalized or structured mean-field methods enlarge the independent blocks while keeping the global trial distribution tractable [98, 218, 202].

The mean-field feasible set $\mathcal{D}^{(\text{MF})}$ is convex, but the objective need not be. The negative entropy contribution is convex, whereas the expected energy in Eq. (6.6) is generally multilinear. Practical solvers therefore use coordinate descent, block-coordinate descent, gradient methods with simplex parameterizations, or automatic differentiation. Each exact coordinate update decreases the free energy, but convergence is only to a stationary point unless additional convexity is available.

6.1.4 Factor-graph and pairwise forms

For an ordinary factor graph with variables x_i and factors $f_A(\mathbf{x}_A)$, the product family is

$$q^{(\text{MF})}(\mathbf{x}) = \prod_{i \in \mathcal{V}_1} q_i(x_i).$$

The free energy becomes

$$F^{(\text{MF})}(q) = -\beta^{-1} \sum_{A \in \mathcal{V}_2} \mathbb{E}_{\prod_{i \in A} q_i} [\log f_A(\mathbf{x}_A)] - \beta^{-1} \sum_{i \in \mathcal{V}_1} H(q_i).$$

Holding every factor except q_i fixed gives the coordinate update

$$q_i^{\text{new}}(x_i) \propto \exp \left(\sum_{A \ni i} \mathbb{E}_{q_{A \setminus i}} [\log f_A(\mathbf{x}_A) \mid x_i] \right). \quad \text{👉}$$

This equation is often called the coordinate-ascent variational-inference update, even though in free-energy language it is coordinate descent.

For the pairwise Ising model

$$p(\mathbf{x}) \propto \exp \left(\beta \sum_{(i,j) \in \mathcal{E}} J_{ij} x_i x_j + \beta \sum_i h_i x_i \right), \quad x_i \in \{-1, +1\},$$

write $m_i = \mathbb{E}_q[x_i]$. The mean-field fixed-point equations are

$$m_i = \tanh \left(\beta h_i + \beta \sum_{j \in \partial i} J_{ij} m_j \right). \quad \text{👉}$$

They are easy to implement, but multiple solutions can appear when interactions are strong.

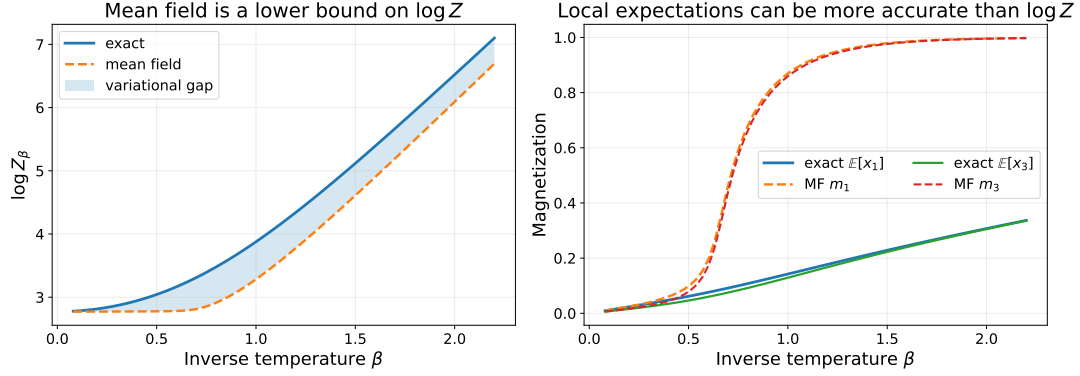


Figure 6.1: Exact and mean-field results for the four-spin cycle. The product-family optimum gives a certified lower bound on $\log Z$; local magnetizations may remain relatively accurate even when the global normalization gap becomes substantial. Generated by `ch06/ch06_01_mean_field_variational_inference.ipynb`.

Companion notebook: Non-convex mean-field landscape, dependence on initialization

Notebook `ch06/ch06_01_mean_field_variational_inference.ipynb` verifies Eq. (6.12), visualizes the nonconvex mean-field landscape, and compares coordinate updates from several initializations. The same code can be modified to implement block mean field.

Exercise 6.1.1. For the four-spin cycle in Fig. 6.1, replace the four singleton factors by two blocks, (x_1, x_2) and (x_3, x_4) . Derive the block-coordinate update and verify numerically that the structured mean-field bound is no worse than the fully factorized bound.

6.2 Bethe approximation in a normal factor graph

Why Bethe is the next step beyond mean field. The correlated two-bit example in the reader’s guide fails under a product law because no object remembers the pair correlation. Bethe inference introduces beliefs on factors/edges in addition to singleton beliefs and asks neighboring beliefs to agree on their overlaps. On a tree these local pieces are sufficient to reconstruct a genuine global distribution exactly. On a graph with loops, the same local-consistency conditions remain computationally attractive but no longer guarantee that the beliefs come from one global joint law; this is where approximation enters.

Mean field ignores all correlations. Bethe inference retains each node belief $\mathfrak{B}_a(x_a)$ and each shared edge belief $\mathfrak{B}_\alpha(x_\alpha)$, enforcing local marginal consistency. Its entropy counting is inherited from the exact tree factorization in Chapter 3.

6.2.1 Tree reconstruction and the Bethe ansatz

On an NFG tree, a globally consistent distribution is reconstructed from its node and edge marginals as

$$q(\mathbf{x}) = \frac{\prod_{a \in \mathcal{V}} q_a(\mathbf{x}_a)}{\prod_{\alpha \in \mathcal{E}} q_\alpha(x_\alpha)}. \quad (6.13)$$

On a loopy graph, Bethe uses the same algebraic expression as an ansatz:

$$\mathfrak{B}^{(\text{BP})}(\mathbf{x}) := \frac{\prod_{a \in \mathcal{V}} \mathfrak{B}_a(\mathbf{x}_a)}{\prod_{\alpha \in \mathcal{E}} \mathfrak{B}_\alpha(x_\alpha)}.$$

The local beliefs are required to be consistent, but the reconstructed function need not be normalized and need not reproduce the beliefs when marginalized. This is the essential difference from mean field.

Define the NFG local or Bethe polytope

$$\mathbf{B}^{(\text{BP})} := (\mathfrak{B}_a(\mathbf{x}_a) \mid a \in \mathcal{V}) \cup (\mathfrak{B}_\alpha(x_\alpha) \mid \alpha \in \mathcal{E}), \quad (6.14)$$

$$\mathcal{D}^{(\text{BP})} := \left\{ \mathbf{B}^{(\text{BP})} \left| \begin{array}{ll} \mathfrak{B}_a(\mathbf{x}_a) \geq 0, & a \in \mathcal{V}, \mathbf{x}_a \in \mathcal{X}_a, \\ \mathfrak{B}_\alpha(x_\alpha) = \int_{\mathbf{x}_a \setminus x_\alpha} \mathfrak{B}_a(\mathbf{x}_a), & \alpha \in \mathcal{E}, a \in v(\alpha), \\ \int_{\mathbf{x}_a} \mathfrak{B}_a(\mathbf{x}_a) = 1, & a \in \mathcal{V}. \end{array} \right. \right\}. \quad (6.15)$$

Edge normalization follows from the last two lines. The exact marginal polytope is projected into this space and then relaxed by dropping the existence of a global joint distribution:

$$\mathbb{M}(\mathcal{G}) \subseteq \mathcal{D}^{(\text{BP})}. \quad (6.16)$$

Equality holds for trees, whereas loops generally make the inclusion strict [202].

6.2.2 Bethe energy and entropy

The Bethe energy is

$$E^{(\text{BP})}(\mathbf{B}^{(\text{BP})}) := \sum_{a \in \mathcal{V}} \int_{\mathbf{x}_a} E_a(\mathbf{x}_a) \mathfrak{B}_a(\mathbf{x}_a) = -\beta^{-1} \sum_{a \in \mathcal{V}} \int_{\mathbf{x}_a} \log f_a(\mathbf{x}_a) \mathfrak{B}_a(\mathbf{x}_a). \quad (6.17)$$

The Bethe entropy is the node entropy minus the overlap entropy:

$$S^{(\text{BP})}(\mathbf{B}^{(\text{BP})}) := \sum_{a \in \mathcal{V}} H(\mathfrak{B}_a) - \sum_{\alpha \in \mathcal{E}} H(\mathfrak{B}_\alpha). \quad (6.18)$$

The subtraction has a direct degrees-of-freedom interpretation. Multiplying the node marginals counts every shared edge variable twice, once from each endpoint. Dividing by the edge marginal removes one copy. On a tree, the same accounting gives the exact joint entropy.

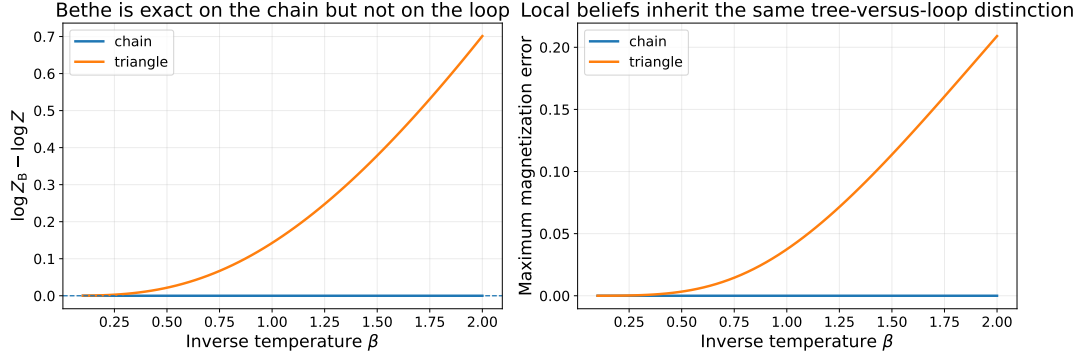


Figure 6.2: Bethe inference on a chain and a triangle. The tree calculation is exact to numerical precision. The single loop produces an error in both $\log Z$ and local magnetizations, and the error grows as interactions strengthen. Generated by `ch06/ch06_02_bethe_and_loopy_bp.ipynb`.

For a pairwise model, Eq. (6.18) has the particularly informative form

$$S^{(\text{BP})} = \sum_i H(\mathfrak{B}_i) - \sum_{(i,j) \in \mathcal{E}} I_{ij}(\mathfrak{B}_{ij}),$$

where

$$I_{ij}(\mathfrak{B}_{ij}) := \sum_{x_i, x_j} \mathfrak{B}_{ij}(x_i, x_j) \log \frac{\mathfrak{B}_{ij}(x_i, x_j)}{\mathfrak{B}_i(x_i) \mathfrak{B}_j(x_j)}$$

is the edge mutual information. Bethe starts with independent-node entropy and subtracts the correlation information assigned to each edge. A tree has exactly enough edges for this accounting to be globally consistent; a loop can overcount correlated information.

The Bethe free energy and partition-function estimate are

$$F^{(\text{BP})}(\mathbf{B}^{(\text{BP})}) := E^{(\text{BP})}(\mathbf{B}^{(\text{BP})}) - \beta^{-1} S^{(\text{BP})}(\mathbf{B}^{(\text{BP})}), \quad (6.19)$$

$$-\beta^{-1} \log Z^{(\text{BP})} := \min_{\mathbf{B}^{(\text{BP})} \in \mathcal{D}^{(\text{BP})}} F^{(\text{BP})}(\mathbf{B}^{(\text{BP})}). \quad (6.20)$$

Proposition 6.2.1 (Exactness on trees). *If the NFG is a tree and all beliefs are globally consistent, then the Bethe entropy equals the exact entropy, the Bethe free energy equals the exact Gibbs free energy, and $Z^{(\text{BP})} = Z$.*

On general graphs, $F^{(\text{BP})}$ is usually nonconvex and $Z^{(\text{BP})}$ has no universal upper- or lower-bound sign. Important special structures are known. For perfect matching, the corresponding Bethe free energy is convex [198]; for log-supermodular models, the Bethe partition function is a lower bound on the exact one [170]; and sufficient entropy-counting conditions can make broader fractional free energies convex [87, 84]. These statements concern different properties—convexity, accuracy, and bound direction—and should not be conflated.

Fig. 6.2 separates two facts that are sometimes conflated: Bethe is structurally exact on a tree, while the same stationary equations on a loopy graph define an approximation whose error depends on interaction strength and topology.

6.2.3 Stationarity gives belief propagation

Introduce multipliers $\eta_{a \rightarrow \alpha}(x_\alpha)$ for the edge-to-node consistency constraints and constants ψ_a for node normalization. The augmented functional is

$$\begin{aligned} L^{(\text{BP})}(\mathbf{B}^{(\text{BP})}, \eta, \psi) &:= F^{(\text{BP})}(\mathbf{B}^{(\text{BP})}) + \sum_{a \in \mathcal{V}} \sum_{\alpha \in e(a)} \int_{x_\alpha} \eta_{a \rightarrow \alpha}(x_\alpha) \left(\mathfrak{B}_\alpha(x_\alpha) - \int_{\mathbf{x}_a \setminus x_\alpha} \mathfrak{B}_a(\mathbf{x}_a) \right) \\ &\quad + \sum_{a \in \mathcal{V}} \psi_a \left(\int_{\mathbf{x}_a} \mathfrak{B}_a(\mathbf{x}_a) - 1 \right). \end{aligned} \quad (6.21)$$

Interior stationarity with respect to node and edge beliefs gives

$$\mathfrak{B}_a(\mathbf{x}_a) \propto f_a(\mathbf{x}_a) \prod_{\alpha \in e(a)} M_{b \rightarrow a}(x_\alpha), \quad (6.22)$$

$$\mathfrak{B}_\alpha(x_\alpha) \propto M_{a \rightarrow b}(x_\alpha) M_{b \rightarrow a}(x_\alpha), \quad \alpha = (a, b), \quad (6.23)$$

where exponentiated Lagrange multipliers have been collected into positive messages. Enforcing consistency yields the NFG sum-product equations

$$M_{a \rightarrow b}(x_\alpha) \propto \int_{\mathbf{x}_a \setminus x_\alpha} f_a(\mathbf{x}_a) \prod_{\beta = (c, a) \in e(a), \beta \neq \alpha} M_{c \rightarrow a}(x_\beta). \quad (6.24)$$

After convergence, the beliefs are recovered from

$$\begin{aligned} \mathfrak{B}_a(\mathbf{x}_a) &= \frac{f_a(\mathbf{x}_a) \prod_{\alpha = (b, a) \in e(a)} M_{b \rightarrow a}(x_\alpha)}{\int_{\mathbf{x}_a} f_a(\mathbf{x}_a) \prod_{\alpha = (b, a) \in e(a)} M_{b \rightarrow a}(x_\alpha)}, \\ \mathfrak{B}_\alpha(x_\alpha) &= \frac{M_{a \rightarrow b}(x_\alpha) M_{b \rightarrow a}(x_\alpha)}{\int_{x_\alpha} M_{a \rightarrow b}(x_\alpha) M_{b \rightarrow a}(x_\alpha)}. \end{aligned} \quad (6.25)$$

Thus every positive BP fixed point is an interior stationary point of the constrained Bethe free energy [222, 221]. The converse holds modulo message-normalization freedoms. A stationary point need not be the global Bethe minimum, and a parallel update need not converge.

Companion notebook: Exactness of a chain, Bethe entropy decomposition

The notebook `ch06/ch06_02_bethe_and_loopy_bp.ipynb` verifies exactness on a chain, evaluates the mutual-information decomposition of Bethe entropy, and compares damped and undamped updates on a frustrated triangle.

6.2.4 Belief propagation as gauge fixing

Chapter 5 introduced NFG gauges whose paired matrices satisfy Eq. (5.2). Select a distinguished symbol 0_α on every edge and define gauge beliefs through Eqs. (5.7) and (5.8). The BP gauge is the choice for which these node and edge beliefs are marginally consistent:

$$\int_{\mathbf{x}_a \setminus x_\alpha} \mathfrak{B}_a^{(\text{BP})}(\mathbf{x}_a) = \mathfrak{B}_\alpha^{(\text{BP})}(x_\alpha), \quad a \in v(\alpha). \quad (6.26)$$

This is the gauge-language form of Eq. (6.24).

6.2.4.1 Consistency of edge and node beliefs

The ground-state rows of the BP gauge can be identified with normalized messages:

$$G_{a \rightarrow b}^{(\text{BP})}(0_\alpha, x_\alpha) \propto M_{a \rightarrow b}(x_\alpha).$$

The paired-gauge normalization then gives the edge belief, while the transformed node factor gives the node belief. Eq. (6.26) fixes the ground-state rows; the remaining gauge rows retain the freedom required to form a complete inverse-transpose pair. This residual freedom becomes useful in the loop expansion of Chapter 9.

6.2.4.2 No-loose-end coloring rules

Call $x_\alpha = 0_\alpha$ uncolored and $x_\alpha \neq 0_\alpha$ colored. The BP gauge makes every transformed configuration with exactly one colored edge incident to some node vanish:

$$\left. \begin{array}{l} x_\alpha \neq 0_\alpha, \\ x_\beta = 0_\beta, \quad \beta \in e(a) \setminus \{\alpha\} \end{array} \right\} \implies Z^{(\text{G})}(\mathbf{x}) = 0. \quad (6.27)$$

Using Eq. (5.6), this global cancellation is equivalent to the local conditions

$$\int_{\mathbf{x}'_a} f_a(\mathbf{x}'_a) G_{a \rightarrow b}^{(\text{BP})}(x_\alpha, x'_\alpha) \prod_{\beta=(c,a) \in e(a) \setminus \{\alpha\}} G_{c \rightarrow a}^{(\text{BP})}(0_\beta, x'_\beta) = 0, \quad x_\alpha \neq 0_\alpha. \quad (6.28)$$

Combining these equations with gauge orthogonality leaves an equation involving only ground-state rows:

$$G_{a \rightarrow b}^{(\text{BP})}(0_\alpha, x_\alpha) = \frac{\int_{\mathbf{x}_a \setminus x_\alpha} f_a(\mathbf{x}_a) \prod_{\beta=(c,a) \in e(a) \setminus \{\alpha\}} G_{c \rightarrow a}^{(\text{BP})}(0_\beta, x_\beta)}{\int_{\mathbf{x}_a} f_a(\mathbf{x}_a) \prod_{\beta=(c,a) \in e(a)} G_{c \rightarrow a}^{(\text{BP})}(0_\beta, x_\beta)}. \quad (6.29)$$

Eq. (6.29) is a normalized version of the BP update. The no-loose-end rule is the mechanism that removes all dangling subgraphs from the exact loop series.

6.2.4.3 Least-sensitive gauge

The transformed ground contribution $Z^{(\text{G})}(\mathbf{0})$ depends on the ground-state gauge rows. Subject to Eq. (5.10), its stationary gauges satisfy

$$\frac{\delta Z^{(\text{G})}(\mathbf{0})}{\delta G_{a \rightarrow b}(0_\alpha, x_\alpha)} = 0. \quad (6.30)$$

The variation in Eq. (6.30) is taken subject to the gauge constraints in Eq. (5.10). Introducing one multiplier for each edge normalization and evaluating the variation gives Eq. (6.29). BP is therefore simultaneously a marginal-consistency gauge, a no-loose-end gauge, and a stationary gauge for the leading transformed contribution.

6.2.5 Belief propagation as fixing reparametrizations

The ground-state gauge row and the positive reparametrization function of Chapter 5 can be identified:

$$G_{a \rightarrow b}^{(\text{BP})}(0_\alpha, x_\alpha) = R_{a \rightarrow b}^{(\text{BP})}(x_\alpha). \quad (6.31)$$

The associated beliefs are

$$\mathfrak{B}_a^{(\text{BP})}(\mathbf{x}_a) := \frac{f_a(\mathbf{x}_a) \prod_{\alpha=(b,a) \in e(a)} R_{b \rightarrow a}^{(\text{BP})}(x_\alpha)}{\int_{\mathbf{x}_a} f_a(\mathbf{x}_a) \prod_{\alpha=(b,a) \in e(a)} R_{b \rightarrow a}^{(\text{BP})}(x_\alpha)}, \quad (6.32)$$

$$\mathfrak{B}_\alpha^{(\text{BP})}(x_\alpha) := \frac{R_{a \rightarrow b}^{(\text{BP})}(x_\alpha) R_{b \rightarrow a}^{(\text{BP})}(x_\alpha)}{\int_{x_\alpha} R_{a \rightarrow b}^{(\text{BP})}(x_\alpha) R_{b \rightarrow a}^{(\text{BP})}(x_\alpha)}, \quad (6.33)$$

$$\int_{\mathbf{x}_a \setminus x_\alpha} \mathfrak{B}_a^{(\text{BP})}(\mathbf{x}_a) = \mathfrak{B}_\alpha^{(\text{BP})}(x_\alpha). \quad (6.34)$$

The reparametrization identity remains exact for every positive R ; Bethe enters only when the reparametrized local beliefs are required to satisfy Eq. (6.34). This distinction between an exact transformation and an approximate consistency closure is central to Chapter 9.

6.2.6 Zero temperature and the NFG local LP

Removing the entropy term from Eq. (6.19) gives the local-polytope relaxation of MAP:

$$\bar{E}^{(\text{BP})} := \min_{\mathbf{B}^{(\text{BP})} \in \mathcal{D}^{(\text{BP})}} E^{(\text{BP})}(\mathbf{B}^{(\text{BP})}). \quad (6.35)$$

Because $\mathcal{D}^{(\text{BP})}$ contains the projection of the exact marginal polytope,

$$\bar{E}^{(\text{BP})} \leq \bar{E}. \quad (6.36)$$

The notation is potentially counterintuitive: this is a lower bound on the minimum energy and therefore an upper bound on the maximum log probability. The relaxation can be fractional on loopy graphs.

6.3 Bethe approximation in a factor graph

Most algorithmic presentations use an FG with variable nodes $i \in \mathcal{V}_1$, factor nodes $A \in \mathcal{V}_2$, and incidence edges $(i, A) \in \mathcal{E}$. Let d_i be the degree of variable node i . Define singleton and factor marginals by

$$\mathfrak{B}_i(x_i) := \int_{\mathbf{x} \setminus x_i} \mathfrak{B}(\mathbf{x}), \quad (6.37)$$

$$\mathfrak{B}_A(\mathbf{x}_A) := \int_{\mathbf{x} \setminus \mathbf{x}_A} \mathfrak{B}(\mathbf{x}). \quad (6.38)$$

The tree reconstruction formula becomes

$$\mathfrak{B}^{(\text{BP})}(\mathbf{x}) = \frac{\prod_{A \in \mathcal{V}_2} \mathfrak{B}_A(\mathbf{x}_A)}{\prod_{i \in \mathcal{V}_1} \mathfrak{B}_i(x_i)^{d_i-1}}. \quad (6.39)$$

6.3.1 Bethe free energy and the local polytope

The FG local polytope is

$$\mathbf{B}^{(\text{BP})} := (\mathfrak{B}_i(x_i) \mid i \in \mathcal{V}_1) \cup (\mathfrak{B}_A(\mathbf{x}_A) \mid A \in \mathcal{V}_2), \quad (6.40)$$

$$\mathcal{D}^{(\text{BP})} := \left\{ \mathbf{B}^{(\text{BP})} \left| \begin{array}{ll} \mathfrak{B}_A(\mathbf{x}_A) \geq 0, & A \in \mathcal{V}_2, \mathbf{x}_A \in \mathcal{X}_A, \\ \mathfrak{B}_i(x_i) = \int_{\mathbf{x}_A \setminus x_i} \mathfrak{B}_A(\mathbf{x}_A), & (i, A) \in \mathcal{E}, \\ \int_{x_i} \mathfrak{B}_i(x_i) = 1, & i \in \mathcal{V}_1. \end{array} \right. \right\}. \quad (6.41)$$

As in the NFG case,

$$\mathbb{M}(\mathcal{G}) \subseteq \mathcal{D}^{(\text{BP})}. \quad (6.42)$$

The FG Bethe energy, entropy, free energy, and optimum are

$$E^{(\text{BP})}(\mathbf{B}^{(\text{BP})}) := -\beta^{-1} \sum_{A \in \mathcal{V}_2} \int_{\mathbf{x}_A} \mathfrak{B}_A(\mathbf{x}_A) \log f_A(\mathbf{x}_A), \quad (6.43)$$

$$S^{(\text{BP})}(\mathbf{B}^{(\text{BP})}) := \sum_{A \in \mathcal{V}_2} H(\mathfrak{B}_A) - \sum_{i \in \mathcal{V}_1} (d_i - 1) H(\mathfrak{B}_i), \quad (6.44)$$

$$F^{(\text{BP})}(\mathbf{B}^{(\text{BP})}) := E^{(\text{BP})}(\mathbf{B}^{(\text{BP})}) - \beta^{-1} S^{(\text{BP})}(\mathbf{B}^{(\text{BP})}), \quad (6.45)$$

$$-\beta^{-1} \log Z^{(\text{BP})} := \min_{\mathbf{B}^{(\text{BP})} \in \mathcal{D}^{(\text{BP})}} F^{(\text{BP})}(\mathbf{B}^{(\text{BP})}). \quad (6.46)$$

Eq. (6.44) is equivalent to the mutual-information form

$$S^{(\text{BP})} = \sum_{i \in \mathcal{V}_1} H(\mathfrak{B}_i) - \sum_{A \in \mathcal{V}_2} D_{\text{KL}} \left(\mathfrak{B}_A \parallel \prod_{i \in A} \mathfrak{B}_i \right),$$

where the factor multi-information measures dependence internal to region A .

6.3.2 BP equations for beliefs and messages

The constrained Lagrangian is

$$\begin{aligned} L^{(\text{BP})} := & F^{(\text{BP})} + \sum_{(i,A) \in \mathcal{E}} \int_{x_i} \eta_{A \rightarrow i}(x_i) \left(\mathfrak{B}_i(x_i) - \int_{\mathbf{x}_A \setminus x_i} \mathfrak{B}_A(\mathbf{x}_A) \right) \\ & + \sum_{A \in \mathcal{V}_2} \psi_A \left(\int_{\mathbf{x}_A} \mathfrak{B}_A(\mathbf{x}_A) - 1 \right). \end{aligned} \quad (6.47)$$

Interior stationarity gives

$$\mathfrak{B}_A(\mathbf{x}_A) \propto f_A(\mathbf{x}_A) \prod_{i \in A} M_{i \rightarrow A}(x_i), \quad (6.48)$$

$$\mathfrak{B}_i(x_i) \propto \prod_{A \in \partial i} M_{A \rightarrow i}(x_i). \quad (6.49)$$

It is convenient to use two message types:

$$M_{i \rightarrow A}(x_i) := \prod_{B \in \partial i \setminus \{A\}} M_{B \rightarrow i}(x_i), \quad (6.50)$$

$$M_{A \rightarrow i}(x_i) \propto \int_{\mathbf{x}_A \setminus x_i} f_A(\mathbf{x}_A) \prod_{j \in A \setminus \{i\}} M_{j \rightarrow A}(x_j). \quad (6.51)$$

The beliefs are

$$\mathfrak{B}_i(x_i) \propto \prod_{A \in \partial i} M_{A \rightarrow i}(x_i), \quad (6.52)$$

$$\mathfrak{B}_A(\mathbf{x}_A) \propto f_A(\mathbf{x}_A) \prod_{i \in A} M_{i \rightarrow A}(x_i). \quad (6.53)$$

Combining the two message types gives the one-line factor-to-variable update

$$M_{A \rightarrow i}(x_i) \propto \int_{\mathbf{x}_A \setminus x_i} f_A(\mathbf{x}_A) \prod_{j \in A \setminus \{i\}} \prod_{B \in \partial j \setminus \{A\}} M_{B \rightarrow j}(x_j). \quad (6.54)$$

The legacy one-message notation is recovered by writing $\widetilde{M}_{A \rightarrow i} = M_{A \rightarrow i}$ and substituting Eq. (6.50):

$$M_{i \rightarrow A}(x_i) = \prod_{B \in \partial i \setminus \{A\}} \widetilde{M}_{B \rightarrow i}(x_i), \quad (6.55)$$

$$\mathfrak{B}_i(x_i) \propto \prod_{A \in \partial i} \widetilde{M}_{A \rightarrow i}(x_i), \quad (6.56)$$

$$\mathfrak{B}_A(\mathbf{x}_A) \propto f_A(\mathbf{x}_A) \prod_{i \in A} \prod_{B \in \partial i \setminus \{A\}} \widetilde{M}_{B \rightarrow i}(x_i), \quad (6.57)$$

$$\widetilde{M}_{A \rightarrow i}(x_i) \propto \int_{\mathbf{x}_A \setminus x_i} f_A(\mathbf{x}_A) \prod_{j \in A \setminus \{i\}} \prod_{B \in \partial j \setminus \{A\}} \widetilde{M}_{B \rightarrow j}(x_j). \quad (6.58)$$

These are the usual sum-product equations [116, 221].

6.3.3 BP as a gauge fixing

Converting an FG to an NFG introduces equality factors, as explained in Section 2.2.1. Applying the BP gauge of Section 6.2.4 to the normalized graph and then contracting the equality factors produces Eqs. (6.50)–(6.51). In the FG picture, message rescaling is the residual gauge freedom: multiplying one message by a positive constant changes neither normalized beliefs nor consistency. Fixing one normalization per directed incidence removes this numerical degree of freedom but has no probabilistic content.

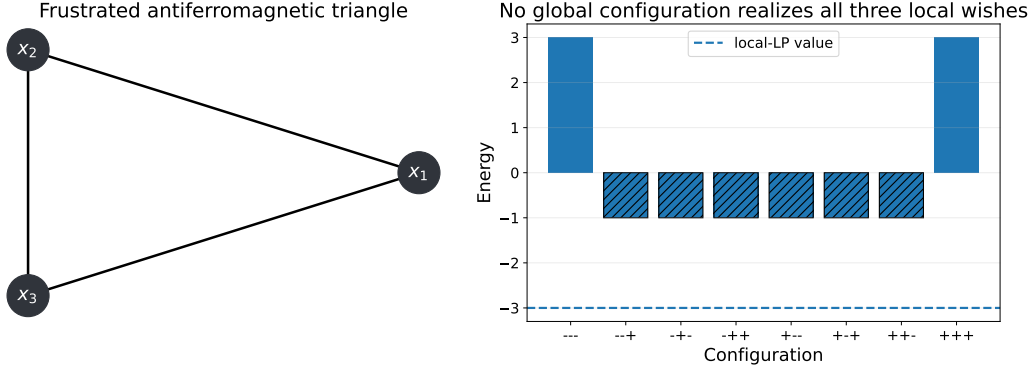


Figure 6.3: A frustrated antiferromagnetic triangle. Every global spin configuration has energy at least -1 , but the local LP assigns perfect anticorrelation to every edge and attains -3 . This is the smallest transparent example of a local-to-global consistency gap. Generated by `ch06/ch06_05_zero_temperature_and_map_lp.ipynb`.

6.3.4 Primal-domain LP relaxation of MAP

The zero-temperature FG local relaxation is

$$\bar{E}^{(\text{BP})} := \min_{\mathbf{B}^{(\text{BP})} \in \mathcal{D}^{(\text{BP})}} \sum_{A \in \mathcal{V}_2} \int_{\mathbf{x}_A} E_A(\mathbf{x}_A) \mathfrak{B}_A(\mathbf{x}_A), \quad (6.59)$$

$$\bar{E}^{(\text{BP})} \leq \bar{E}. \quad (6.60)$$

The objective is linear and the feasible region is a polytope. Fractional extreme points reflect local beliefs that cannot be assembled into a global configuration.

6.3.5 Dual-domain LP relaxation and reparametrization

Introduce functions $\eta_{A \rightarrow i}(x_i)$ satisfying

$$\sum_{A \in \partial i} \eta_{A \rightarrow i}(x_i) = 0, \quad i \in \mathcal{V}_1, \quad x_i \in \mathcal{X}_i.$$

The LP dual can be written as

$$\bar{E}^{(\text{BP})} = \max_{\eta, \psi} \sum_{A \in \mathcal{V}_2} \psi_A, \quad (6.61)$$

$$\psi_A \leq E_A(\mathbf{x}_A) - \sum_{i \in A} \eta_{A \rightarrow i}(x_i), \quad A \in \mathcal{V}_2, \quad \mathbf{x}_A \in \mathcal{X}_A,$$

with the zero-sum constraints above. Eliminating ψ gives

$$\bar{E}^{(\text{BP})} = \max_{\eta: \sum_{A \in \partial i} \eta_{A \rightarrow i} = 0} \sum_{A \in \mathcal{V}_2} \min_{\mathbf{x}_A} \left(E_A(\mathbf{x}_A) - \sum_{i \in A} \eta_{A \rightarrow i}(x_i) \right). \quad (6.62)$$

Strong LP duality makes Eqs. (6.59) and (6.62) equal.

The same zero-sum functions reparametrize the exact energy:

$$E(\mathbf{x}) = \sum_{A \in \mathcal{V}_2} \left(E_A(\mathbf{x}_A) - \sum_{i \in A} \eta_{A \rightarrow i}(x_i) \right). \quad (6.63)$$

Therefore

$$\max_{\eta} \sum_A \min_{\mathbf{x}_A} \tilde{E}_A(\mathbf{x}_A) \leq \min_{\mathbf{x}} \sum_A \tilde{E}_A(\mathbf{x}_A) = \bar{E}. \quad (6.64)$$

The LP dual seeks a reparametrization that makes the sum of independent local minima as large as possible. This is the optimization counterpart of the reparametrization picture in Chapter 5.

6.3.6 Message-domain representation of the Bethe free energy

Because the Bethe problem is generally nonconvex, eliminating beliefs by stationarity does not automatically produce a strong convex dual. It does, however, give a useful reduced functional of messages. Define

$$M_{i \rightarrow A}(x_i) := \prod_{B \in \partial i \setminus \{A\}} M_{B \rightarrow i}(x_i), \quad (6.65)$$

$$Z_A(M) := \int_{\mathbf{x}_A} f_A(\mathbf{x}_A) \prod_{i \in A} M_{i \rightarrow A}(x_i), \quad (6.66)$$

$$Z_i(M) := \int_{x_i} \prod_{A \in \partial i} M_{A \rightarrow i}(x_i), \quad (6.67)$$

$$Z_{iA}(M) := \int_{x_i} M_{i \rightarrow A}(x_i) M_{A \rightarrow i}(x_i).$$

The corresponding belief collection is

$$\mathbf{B}^{(\text{BP})}(M) := (\mathfrak{B}_i(M), \mathfrak{B}_A(M)), \quad (6.68)$$

with normalized beliefs given by Eqs. (6.52) and (6.53). At a BP fixed point,

$$\log Z^{(\text{BP})}(M) = \sum_{A \in \mathcal{V}_2} \log Z_A(M) + \sum_{i \in \mathcal{V}_1} \log Z_i(M) - \sum_{(i,A) \in \mathcal{E}} \log Z_{iA}(M), \quad (6.69)$$

$$F^{(\text{BP})}(\mathbf{B}^{(\text{BP})}(M)) = -\beta^{-1} \log Z^{(\text{BP})}(M). \quad (6.70)$$

The expression is invariant under positive message rescalings. Away from fixed points it is best viewed as a stationary message-domain parameterization, not as a guaranteed global dual optimum.

6.4 Tree-reweighted variational inference

Tree reweighting: use exact trees as building blocks. Bethe applies a tree-exact entropy formula directly to a loopy graph. Tree-reweighted inference instead represents the loopy model through a convex combination of spanning-tree parameterizations and averages their exact tree entropies with edge appearance probabilities. The construction is more conservative, but that conservatism buys a certificate: for suitable weights it produces an upper bound on the log partition function rather than only an uncontrolled approximation.

The Bethe approximation is exact on one tree but not certified on a loopy graph. Tree-reweighted (TRW) inference replaces the single informal tree approximation by a convex mixture of exact tree models. Convexity of the log-partition function then yields an upper bound [201, 202]. We state the standard construction for a connected pairwise factor graph: unary factors are present in every tree, whereas each pairwise factor is present only when its edge belongs to that tree. Higher-order analogues replace spanning trees by tractable decomposable subgraphs or hypertrees.

Let $\mathcal{T}$ be a collection of spanning trees and let $\rho(T) \geq 0$ satisfy $\sum_T \rho(T) = 1$. For each tree, choose tree-supported log-potentials $\theta_A^{(T)}$ such that

$$f_A^{(T)}(\mathbf{x}_A) := \exp\left(\theta_A^{(T)}(\mathbf{x}_A)\right), \quad (6.71)$$

$$\sum_{T \ni A} \rho(T) \theta_A^{(T)}(\mathbf{x}_A) = \log f_A(\mathbf{x}_A). \quad (6.72)$$

Convexity of $\log Z$ gives

$$\log Z \leq \sum_{T \in \mathcal{T}} \rho(T) \log Z_T(\theta^{(T)}). \quad (6.73)$$

Optimizing the parameter split gives

$$\log Z \leq \Phi^{(\text{TRW})}(\rho) := \min_{\{\theta^{(T)}\}: (6.72)} \sum_T \rho(T) \log Z_T(\theta^{(T)}), \quad (6.74)$$

$$\log Z^{(\text{TRW})}(\rho) := \Phi^{(\text{TRW})}(\rho). \quad (6.75)$$

Each tree distribution factorizes exactly:

$$q_T(\mathbf{x}) \propto \prod_{A \in T} f_A^{(T)}(\mathbf{x}_A), \quad (6.76)$$

$$q_T(\mathbf{x}) = \frac{\prod_{A \in T} q_A^{(T)}(\mathbf{x}_A)}{\prod_i q_i^{(T)}(x_i)^{d_i(T)-1}}. \quad (6.77)$$

Agreement of the tree marginals produces one common collection of locally consistent pseudo-marginals. For pairwise models, define the edge-appearance probability

$$\rho_{ij} := \sum_{T \ni (i,j)} \rho(T).$$

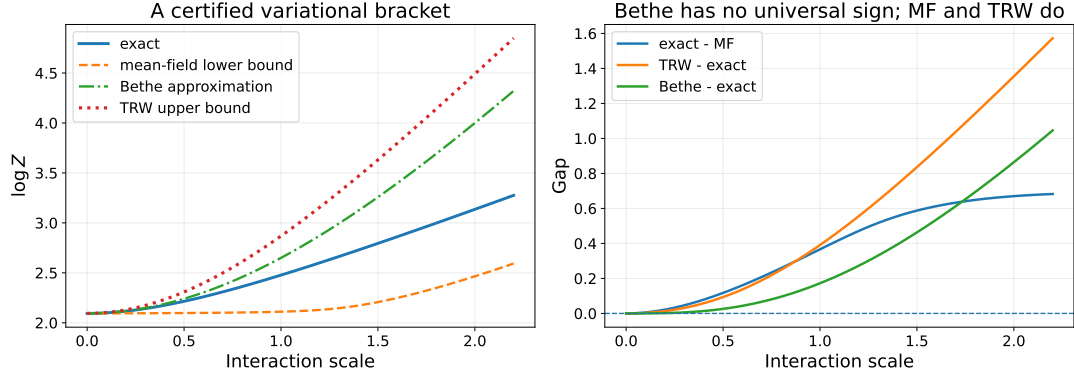


Figure 6.4: Mean-field, Bethe, and tree-reweighted approximations for a three-spin model. Mean field lower-bounds $\log Z$ and TRW upper-bounds it. The Bethe error can have either sign. Generated by `ch06/ch06_03_tree_reweighted_bounds.ipynb`.

The standard TRW variational form is

$$\log Z \leq \log Z^{(\text{TRW})}(\rho), \quad (6.78)$$

$$\log Z^{(\text{TRW})}(\rho) = \max_{\mathbf{B} \in \mathcal{D}^{(\text{BP})}} \left(\sum_A \mathbb{E}_{\mathfrak{B}_A} [\log f_A] + S^{(\text{TRW})}(\mathbf{B}; \rho) \right), \quad (6.79)$$

$$S^{(\text{TRW})}(\mathbf{B}; \rho) = \sum_i H(\mathfrak{B}_i) - \sum_{(i,j) \in \mathcal{E}} \rho_{ij} I_{ij}(\mathfrak{B}_{ij}), \quad (6.80)$$

$$\rho_{ij} = \mathbb{P}_{T \sim \rho}((i, j) \in T). \quad (6.81)$$

Unlike the ordinary Bethe entropy, the TRW entropy is concave for ρ in the spanning-tree polytope, so the variational optimization is convex.

In exponential-family notation, the tree log-partition functions admit the Legendre–Fenchel representation

$$\log Z_T(\theta^{(T)}) = \sup_{\mu^{(T)} \in \mathbb{M}(T)} \left(\langle \theta^{(T)}, \mu^{(T)} \rangle + H_T(\mu^{(T)}) \right). \quad (6.82)$$

This is the moment-space route from Eq. (6.73) to Eq. (6.79).

The bound can be tightened by optimizing the tree distribution:

$$\log Z \leq \min_{\rho \in \mathbb{T}(\mathcal{G})} \log Z^{(\text{TRW})}(\rho), \quad (6.83)$$

where $\mathbb{T}(\mathcal{G})$ is the spanning-tree polytope [66, 51]. The inner optimization is over pseudo-marginals; the outer optimization is over tree appearance probabilities.

The certified bracket in Fig. 6.4 is the principal conceptual advantage of combining mean field and TRW: the two approximations approach the same exact normalization from opposite sides, whereas Bethe supplies no universal direction.

Companion notebook: Uniform mixture of spanning trees

The triangle has three spanning trees. The notebook `ch06/ch06_03_tree_reweighted_bounds.ipynb` constructs their uniform mixture, derives $\rho_{ij} = 2/3$, and solves the local variational problems numerically to verify the certified bracket $\log Z^{(\text{MF})} \leq \log Z \leq \log Z^{(\text{TRW})}$.

6.5 Message-passing algorithms

Objective first, update rule second. It is useful to keep two levels separate. A variational principle specifies *what* quantity should be optimized and over which feasible beliefs. A message-passing rule specifies *how* we try to reach a stationary point or optimum using local computations. On trees these views meet cleanly. On loopy graphs, convergence of an update rule, correctness of a fixed point, and global optimality of the underlying variational problem are three distinct questions.

A variational formulation identifies the objective and feasible set; it does not by itself choose a solver. For small problems, generic constrained optimizers are appropriate. For large sparse graphs, message-passing algorithms exploit locality and parallelism. Their numerical behavior depends on convexity, update scheduling, damping, and the geometry of the fixed point.

6.5.1 Loopy sum-product

A standard parallel update is

$$\begin{aligned} M_{i \rightarrow A}^{(n)}(x_i) &\propto \prod_{B \in \partial i \setminus \{A\}} M_{B \rightarrow i}^{(n)}(x_i), \\ M_{A \rightarrow i}^{(n+1)}(x_i) &\propto \int_{\mathbf{x}_A \setminus x_i} f_A(\mathbf{x}_A) \prod_{j \in A \setminus \{i\}} M_{j \rightarrow A}^{(n)}(x_j). \end{aligned} \tag{6.84}$$

Messages can be normalized after every update because only their relative values matter. Damping replaces a new normalized message $\widehat{M}$ by a convex combination of old and new messages. On a tree, an inward-outward schedule terminates after finitely many passes; on a loopy graph, convergence is not automatic.

Sufficient convergence conditions have been obtained using computation trees, correlation decay, and contraction estimates [208, 189, 87, 143, 95]. These conditions are sufficient rather than necessary. Convex free-energy constructions provide a different route to guaranteed convergence [84]; specialized models such as matching admit stronger results [198].

6.5.2 Min-sum and the zero-temperature limit

Write positive messages in exponential form

$$M_{A \rightarrow i}(x_i) = \exp(-\beta m_{A \rightarrow i}(x_i)).$$

Using

$$-\beta^{-1} \log \sum_x \exp(-\beta u(x)) \longrightarrow \min_x u(x) \quad (\beta \rightarrow \infty),$$

the sum-product update becomes min-sum:

$$m_{A \rightarrow i}^{(n+1)}(x_i) = \min_{\mathbf{x}_A \setminus x_i} \left(E_A(\mathbf{x}_A) + \sum_{j \in A \setminus \{i\}} \sum_{B \in \partial j \setminus \{A\}} m_{B \rightarrow j}^{(n)}(x_j) \right) + \text{const.}$$

The additive constant is a message normalization. This limit explains the formal transition from sum-product to min-sum, but it does not guarantee convergence or exact MAP recovery. Even a converged min-sum fixed point may correspond to a fractional local-LP solution. Convex fractional entropies and norm-product algorithms provide continuous finite-temperature paths whose zero-temperature limits solve specified LP relaxations with convergence guarantees [84].

6.5.3 Min-sum diffusion

Min-sum diffusion (MSD) is a coordinate-ascent method for the MAP-LP dual, developed from work of Schlesinger and later generalized to higher-order factors [176, 175, 212, 213, 214]. Reproduce Eq. (6.62) as

$$\begin{aligned} \bar{E}^{(\text{BP})} = \max_{\eta} \sum_{A \in \mathcal{V}_2} \min_{\mathbf{x}_A} \left(E_A(\mathbf{x}_A) - \sum_{i \in A} \eta_{A \rightarrow i}(x_i) \right), \\ \text{subject to } \sum_{A \in \partial i} \eta_{A \rightarrow i}(x_i) = 0. \end{aligned} \quad (6.85)$$

The lower-bound property follows from

$$\min_{\mathbf{x}} \sum_A \tilde{E}_A(\mathbf{x}_A) \geq \sum_A \min_{\mathbf{x}_A} \tilde{E}_A(\mathbf{x}_A). \quad (6.86)$$

An elementary MSD step selects an incidence (i, A) and first computes, for every $B \in \partial i \setminus \{A\}$,

$$\eta_{B \rightarrow i}^{\text{new}}(x_i) = \min_{\mathbf{x}_B \setminus x_i} \left(E_B(\mathbf{x}_B) - \sum_{j \in B \setminus \{i\}} \eta_{B \rightarrow j}(x_j) \right).$$

It then enforces the zero-sum constraint through

$$\eta_{A \rightarrow i}^{\text{new}}(x_i) = - \sum_{B \in \partial i \setminus \{A\}} \eta_{B \rightarrow i}^{\text{new}}(x_i). \quad (6.87) \quad \text{👉}$$

The dual objective cannot decrease under this block update. Indeed, after using the zero-sum constraint, the objective change reduces to an instance of

$$\min_x \left(u(x) + \sum_k v_k(x) \right) - \min_x u(x) - \sum_k \min_x v_k(x) \geq 0.$$

Thus every iterate supplies a valid lower bound on the exact MAP energy. Convergence and exactness require additional assumptions; the cited literature analyzes several schedules and agreement conditions.

6.5.4 Distributed and large-scale implementations

Message updates are local but memory traffic, scheduling, and synchronization dominate at scale. Distributed algorithms can update factors asynchronously, prioritize large residuals, and decompose the local polytope into blocks. Schwing and collaborators developed distributed inference for convex free energies and globally convergent algorithms for MAP-LP relaxations [180, 181]. Modern implementations additionally use accelerator-friendly tensor contractions, automatic differentiation for local subproblems, and sparse batching. These engineering choices do not alter the variational objective, but they can determine whether a theoretically attractive formulation is usable on a very large graph.

Companion notebook: Frustration exposes the MAP-LP gap

In the notebook `ch06/ch06_05_zero_temperature_and_map_lp.ipynb` frustrated triangle, shown in Fig. 6.3, separates three notions: the exact soft Gibbs law, its zero-temperature MAP limit, and the fractional local-LP optimum. The notebook also verifies every local consistency constraint of the fractional solution.

6.6 Generalized belief propagation and Kikuchi free energies

Generalized BP: enlarge the unit of local reasoning. If single factors and variables do not capture the important correlations, group several of them into overlapping regions. The beliefs now live on these larger regions and must agree on their overlaps. The bookkeeping is heavier, but the principle is unchanged: approximate the inaccessible global entropy by a signed combination of tractable regional entropies, then enforce local consistency. Ordinary Bethe/belief propagation is recovered when the chosen regions are only the original factors and variables.

Bethe retains factors and their singleton overlaps. Generalized belief propagation (GBP) retains a hierarchy of larger regions and their intersections. The construction combines the region reparametrizations of Chapter 5, the junction-tree exactness of Chapter 3, and the variational stationary-point principle of this chapter [103, 222, 221].

6.6.1 Cycle-free region diagrams

Let $\mathcal{E}^h$ be a family of regions partially ordered by inclusion, with counting numbers $c(g)$ defined recursively by the Möbius rule in Eq. (5.27). If the region diagram is cycle-free and the region beliefs are globally consistent, then

$$p(\mathbf{x}) = \prod_{g \in \mathcal{E}^h} p_g(\mathbf{x}_g)^{c(g)}. \quad (6.88)$$

Eq. (6.88) generalizes both the tree Bethe factorization and the junction-tree formula. The counting numbers ensure that every variable and every factor has total counting weight one.

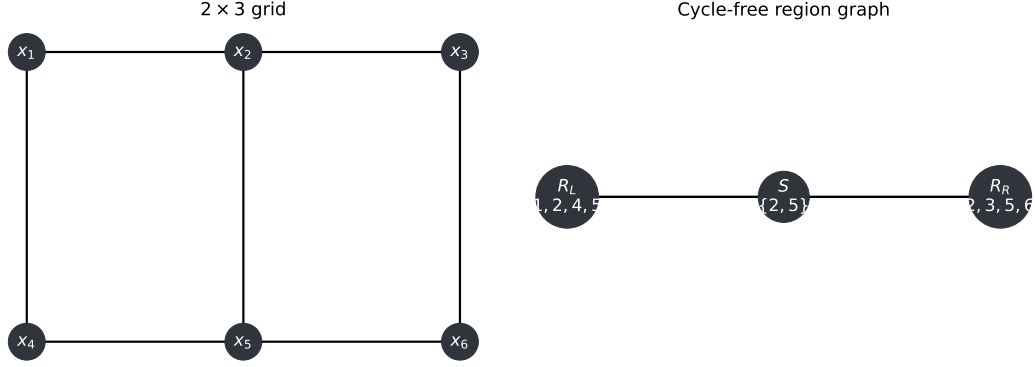


Figure 6.5: A 2×3 grid and a cycle-free region graph with two plaquette regions and their separator. All loops of the original graph are internal to a maximal region, so exact region-to-region message passing is possible. Generated by `ch06/ch06_04_kikuchi_and_gbp.ipynb`.

For Fig. 6.5, let $R_L = \{1, 2, 4, 5\}$, $R_R = \{2, 3, 5, 6\}$, and $S = R_L \cap R_R = \{2, 5\}$. The counting numbers are $c(R_L) = c(R_R) = 1$ and $c(S) = -1$, giving

$$p(\mathbf{x}) = \frac{p_{R_L}(\mathbf{x}_{R_L})p_{R_R}(\mathbf{x}_{R_R})}{p_S(\mathbf{x}_S)}.$$

The corresponding entropy identity is $H(p) = H(p_{R_L}) + H(p_{R_R}) - H(p_S)$.

6.6.2 Kikuchi free energy

On a loopy region diagram, replace the exact marginals in Eq. (6.88) by locally consistent region beliefs:

$$\mathfrak{B}^{(\text{GBP})}(\mathbf{x}) := \prod_{g \in \mathcal{E}^h} \mathfrak{B}_g(\mathbf{x}_g)^{c(g)}. \quad (6.89)$$

The region-belief collection and its local polytope are

$$\mathbf{B}^{(\text{GBP})} := (\mathfrak{B}_g(\mathbf{x}_g) \mid g \in \mathcal{E}^h), \quad (6.90)$$

$$\mathcal{D}^{(\text{GBP})} := \left\{ \mathbf{B}^{(\text{GBP})} \left| \begin{array}{ll} \mathfrak{B}_g(\mathbf{x}_g) \geq 0, & g \in \mathcal{E}^h, \\ \mathfrak{B}_{\underline{g}}(\mathbf{x}_{\underline{g}}) = \int_{\mathbf{x}_g \setminus \mathbf{x}_{\underline{g}}} \mathfrak{B}_g(\mathbf{x}_g), & \underline{g} \in \text{child}(g), \\ \int_{\mathbf{x}_g} \mathfrak{B}_g(\mathbf{x}_g) = 1, & g \in \mathcal{E}^h. \end{array} \right. \right\}. \quad (6.91)$$

Assign every original factor to one region containing its scope and let $E_g(\mathbf{x}_g)$ be the sum of the energies assigned to region g . We also define the corresponding regional factor $\psi_g(\mathbf{x}_g) :=$

$\exp(-\beta E_g(\mathbf{x}_g))$. The Kikuchi energy, entropy, free energy, and partition estimate are

$$E^{(\text{GBP})}(\mathbf{B}^{(\text{GBP})}) := \sum_{g \in \mathcal{E}^h} \int_{\mathbf{x}_g} E_g(\mathbf{x}_g) \mathfrak{B}_g(\mathbf{x}_g), \quad (6.92)$$

$$S^{(\text{GBP})}(\mathbf{B}^{(\text{GBP})}) := \sum_{g \in \mathcal{E}^h} c(g) H(\mathfrak{B}_g), \quad (6.93)$$

$$F^{(\text{GBP})}(\mathbf{B}^{(\text{GBP})}) := E^{(\text{GBP})}(\mathbf{B}^{(\text{GBP})}) - \beta^{-1} S^{(\text{GBP})}(\mathbf{B}^{(\text{GBP})}), \quad (6.94)$$

$$-\beta^{-1} \log Z^{(\text{GBP})} := \min_{\mathbf{B}^{(\text{GBP})} \in \mathcal{D}^{(\text{GBP})}} F^{(\text{GBP})}(\mathbf{B}^{(\text{GBP})}). \quad (6.95) \quad \text{🔗}$$

The Bethe approximation is the special case whose regions are factors and variables. Larger regions retain more dependence but increase storage and contraction cost exponentially in region size.

6.6.3 Generalized BP equations and message passing

For a region g , let $\text{Des}^+(g)$ contain g and all descendants. Define the set of boundary messages entering this descendant subgraph from outside. A parent-to-child region-graph parameterization is

$$\mathfrak{B}_g(\mathbf{x}_g) \propto \left(\prod_{r \in \text{Des}^+(g)} \psi_r(\mathbf{x}_r) \right) \left(\prod_{(p \rightarrow c): p \notin \text{Des}^+(g), c \in \text{Des}^+(g)} M_{p \rightarrow c}(\mathbf{x}_c) \right), \quad (6.96)$$

$$\mathfrak{B}_{\underline{g}}(\mathbf{x}_{\underline{g}}) = \int_{\mathbf{x}_g \setminus \mathbf{x}_{\underline{g}}} \mathfrak{B}_g(\mathbf{x}_g), \quad \underline{g} \in \text{child}(g). \quad (6.97)$$

Stationary points of the constrained Kikuchi free energy satisfy these consistency equations [221]. A practical update can multiply a parent-to-child message by a damped ratio of the parent marginal to the current child belief:

$$M_{g \rightarrow \underline{g}}^{\text{new}}(\mathbf{x}_{\underline{g}}) \propto M_{g \rightarrow \underline{g}}(\mathbf{x}_{\underline{g}}) \left(\frac{\int_{\mathbf{x}_g \setminus \mathbf{x}_{\underline{g}}} \mathfrak{B}_g(\mathbf{x}_g)}{\mathfrak{B}_{\underline{g}}(\mathbf{x}_{\underline{g}})} \right)^\gamma, \quad 0 < \gamma \leq 1. \quad \text{🔗}$$

At a fixed point, the ratio is constant and Eq. (6.97) holds. Other GBP schedules and message parameterizations are possible; the invariant content is regional marginal consistency.

For the cycle-free hierarchy in Fig. 6.5, the updates reduce to ordinary junction-tree sum-product:

$$M_{R_L \rightarrow S}(\mathbf{x}_S) = \int_{\mathbf{x}_{R_L} \setminus \mathbf{x}_S} \psi_{R_L}(\mathbf{x}_{R_L}), \quad M_{R_R \rightarrow S}(\mathbf{x}_S) = \int_{\mathbf{x}_{R_R} \setminus \mathbf{x}_S} \psi_{R_R}(\mathbf{x}_{R_R}),$$

and

$$\mathfrak{B}_S(\mathbf{x}_S) \propto M_{R_L \rightarrow S}(\mathbf{x}_S) M_{R_R \rightarrow S}(\mathbf{x}_S).$$

The alternation of multiplication and marginalization explains the name sum-product.

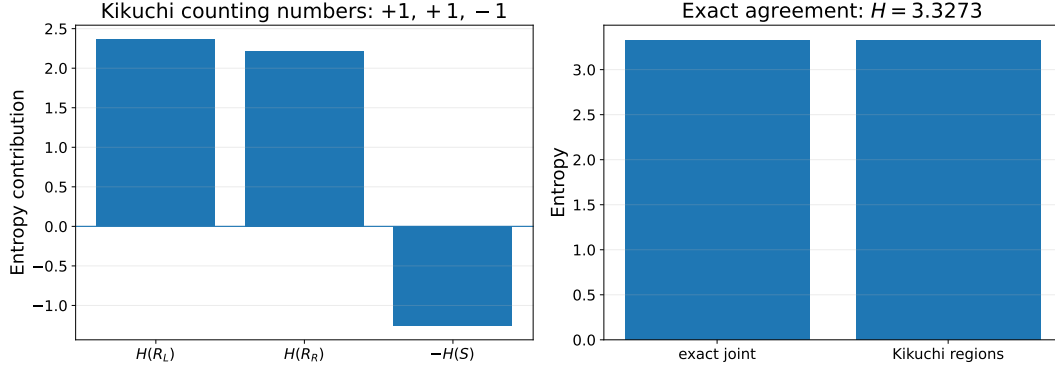


Figure 6.6: Kikuchi entropy counting on the cycle-free two-plaquette region graph. The maximal regions have counting number +1 and the separator has counting number -1 . Their combination exactly reproduces the global entropy. Generated by `ch06/ch06_04_kikuchi_and_gbp.ipynb`.

6.6.4 Generalized BP as reparametrization

Chapter 5 introduced exact region reparametrizations $R_{g \rightarrow \underline{g}}(\underline{x}_{\underline{g}})$. Replacing the GBP messages in Eq. (6.96) by these reparametrization functions produces the exact reparametrized region factors of Eq. (5.26). GBP fixes the otherwise free reparametrization by imposing Eq. (6.97). Thus:

- the reparametrization identity is exact for every admissible R ;
- the Kikuchi variational approximation is approximate on a loopy region graph;
- GBP selects a reparametrization whose regional beliefs are locally consistent.

This separation is essential when the selected reparametrization is later inserted into exact cluster-cumulant or loop expansions [211, 226].

Fig. 6.6 makes the Möbius logic concrete: the two maximal-region entropies double-count the separator, and the negative separator counting number removes exactly one copy.

Companion notebook: Two-plaquette hierarchy tests

The notebook `ch06/ch06_04_kikuchi_and_gbp.ipynb` verifies the regional factorization, entropy identity, partition function, separator marginal, and region-to-region messages for the two-plaquette hierarchy.

6.7 What the variational hierarchy buys us

The approximations in this chapter should not be ranked along a single axis. They trade different resources and guarantees:

Method	Retained beliefs	Global guarantee	Typical solver	Main limitation
Mean field	Independent blocks	Lower bound on $\log Z$	Coordinate or gradient methods	Misses inter-block dependence
Bethe/BP	Factors and overlaps	Exact on trees; no universal bound	Sum-product	Nonconvexity and convergence
TRW	Tree-reweighted local beliefs	Upper bound on $\log Z$	Convex message passing	Often more conservative marginals
MAP-LP	Zero-temperature local beliefs	Lower bound on MAP energy	LP, dual ascent, diffusion	Fractional optima
Kikuchi/GBP	Multilevel regions	Exact on cycle-free region graphs	Regional message passing	Exponential region cost

The modern structured-GenAI interpretation is immediate. A variational family specifies which dependencies a learned approximation is allowed to represent; messages or amortized neural maps solve the associated local consistency problem; and richer region structures offer a controlled way to inject known scientific interactions. Later chapters will replace hand-derived messages by learned functions in selected settings, but the underlying approximation geometry remains the one developed here.

Exercise 6.7.1. *For each row of the table, identify whether the method restricts the family of globally realizable distributions, relaxes global consistency, modifies entropy counting, or combines more than one of these operations.*

Problem 6.1. *Design a variational approximation for the four-spin cycle that interpolates continuously between mean field and Bethe by assigning a fractional coefficient ρ to each edge mutual information. Use an entropy-decomposition criterion, or a numerical Hessian study supplemented by an analytic argument, to determine a defensible interval of ρ for which the entropy approximation is concave on the local polytope. Compare accuracy, convergence, and bound direction numerically. A problem is intentionally more open-ended than an exercise: modeling choices and a defensible computational strategy are part of the requested solution.*

Problem 6.2. *Construct a region hierarchy for a 3×3 Ising grid. Compare at least three choices of maximal regions under a fixed memory budget. Evaluate Bethe and Kikuchi approximations against exact enumeration or a high-accuracy reference, and explain which loops each hierarchy resolves explicitly.*

Problem 6.3. *Formulate a neural message parameterization that preserves positivity and invariance under message rescaling. Train it to imitate exact junction-tree messages on small graphs, then test whether its fixed points decrease a known variational free energy on larger loopy graphs. Separate empirical convergence from any variational guarantee.*

Chapter 7

Approximate Elimination: Mini-Buckets, Gauges, and Renormalization

Reader's Guide.

Why this chapter? Exact elimination is easy to state but can become impossible to execute: before eliminating one variable we may have to construct a table over many of its still-uneliminated neighbors. This chapter asks a practical question: *what can we deliberately forget, compress, or re-express so that elimination remains feasible?* Unlike Chapter 6, which replaces the exact distribution by a variational family, the methods here keep the sequential elimination architecture and modify individual contractions.

The simplest mental picture: one bucket is one temporary table. Suppose all factors involving x_a have been collected in one bucket. Exact elimination first multiplies them, so every surviving neighbor can still interact through the common value of x_a , and only then sums over x_a . If this temporary table is too large, a mini-bucket method splits the factors into smaller groups and eliminates within each group separately. The computation becomes cheaper because the groups no longer communicate fully. The approximation error is precisely the price of that lost communication.

The accuracy knob. The parameter usually called the i -bound limits the largest scope retained inside a mini-bucket. Small i -bound means small tables and aggressive approximation; increasing it keeps more induced dependence and eventually recovers exact bucket elimination on a fixed finite problem. It is therefore useful to read this chapter as a controlled memory–accuracy tradeoff rather than as a collection of unrelated algorithms.

A hierarchy to keep in mind. Bucket elimination is exact. Mini-bucket elimination splits an oversized contraction and uses a simple inequality. Weighted mini-buckets replace

the hard sum/max split by Hölder-weighted contractions. Gauge optimization asks whether an equivalent representation gives a tighter bound. Mini-bucket renormalization instead tries to compensate directly for the coupling lost by the split. These are progressively more sophisticated answers to the same local question: how should one approximate a contraction that is too large to carry exactly?

How to use the companion notebooks. Begin with `ch07/ch07_01_bucket_elimination_and_minibuckets.ipynb`. First change only the elimination order and observe that the exact answer stays fixed while the largest temporary table changes. Then fix the order and vary the i -bound. Only after this exact-versus-approximate comparison is clear should you move to Hölder weights, gauges, and renormalization. The most informative plots are not just accuracy at one parameter value, but the crossover from a regime where a small mini-bucket is adequate to one where the discarded dependence becomes dominant.

Chapter 3 showed that exact elimination is conceptually simple but can create factors whose scope grows with the induced width. This chapter keeps the elimination logic and controls the growth. The resulting methods occupy a useful middle ground between exact dynamic programming and the iterative variational methods of Chapter 6: they are finite sequences of local contractions, require no fixed-point convergence, and expose an explicit accuracy–complexity parameter.

The central object is a bucket associated with the next variable in an elimination order. Exact bucket elimination multiplies all factors in the bucket and sums over the variable. Approximate schemes split the bucket and replace the lost coupling between the pieces by one of three mechanisms:

1. a certified inequality, as in mini-bucket and weighted mini-bucket elimination;
2. an invariant change of representation, as in gauge-optimized weighted mini-buckets;
3. an optimized low-rank compensation, as in mini-bucket renormalization.

These alternatives connect directly to the transformations of Chapter 5, the variational bounds of Chapter 6, and the tensor-contraction viewpoint developed further in Chapter 9.

We work with finite alphabets and nonnegative factors. Continuous-variable analogues are possible when the local integrations and approximations are tractable, but the discrete setting makes the bound and complexity statements transparent. Bucket elimination and mini-buckets follow Dechter [58, 60]; weighted mini-buckets follow Liu and Ihler [131]; gauge optimization and bucket renormalization follow [3, 4].

7.1 Bucket Elimination Algorithm

Bucket elimination, also called variable elimination (BE), [58, 109] is a method for computing the partition function exactly by summing up variables sequentially. First, BE assumes a fixed elimination ordering $o = [a_1, \dots, a_n]$ among variables nodes $a \in \mathcal{V}_1$. Then BE groups factors by placing each factor f_A , where $A \in \mathcal{V}_2$, in the “bucket” β_a , where a is the earliest element in

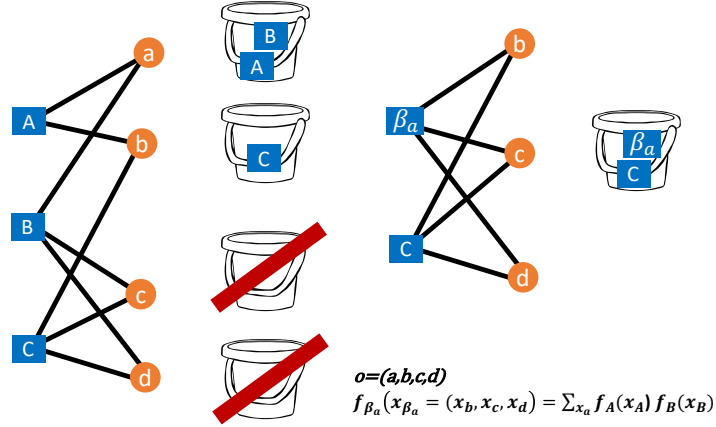


Figure 7.1: Illustration of a single step of the Bucket Elimination algorithm.

o , s.t. $(a, A) \in \mathcal{E}$. Next, BE eliminates the variable by marginalizing the product of factors in the bucket, i.e.,

$$f_{\beta_a}(\mathbf{x}_{\beta_a}) = \sum_{x_a} \prod_{A \in \beta_a} f_A(\mathbf{x}_A), \quad \forall \mathbf{x}_{\beta_a} = (x_b | b \in \beta_a, b \neq a), \quad (7.1)$$

where $b \in \beta_a$ indicates that node b share an edge with a factor which was placed in the bucket β_a . Finally, the newly generated function f_{β_a} is inserted into another bucket corresponding to its earliest (remaining) node in the elimination order, o . This process is easily seen as applying a distributive property: groups of factors corresponding to buckets are summed over sequentially, and then the newly created factor (with the variable just eliminated removed) is assigned to another bucket. The procedure is illustrated in Fig. 7.1.

The computational cost of BE is exponential in the number of uneliminated variables in the bucket, which is called *induced width* of the graph given the elimination order. The induced width depends on the elimination order. Minimizing the largest induced scope over all orders gives the treewidth (up to the conventional one-variable offset); see Chapter 3. Finding an optimal order is itself computationally hard, so practical solvers use graph-based heuristics such as min-fill and weighted min-fill.

BE, just described, is summarized in the Algorithm 9.

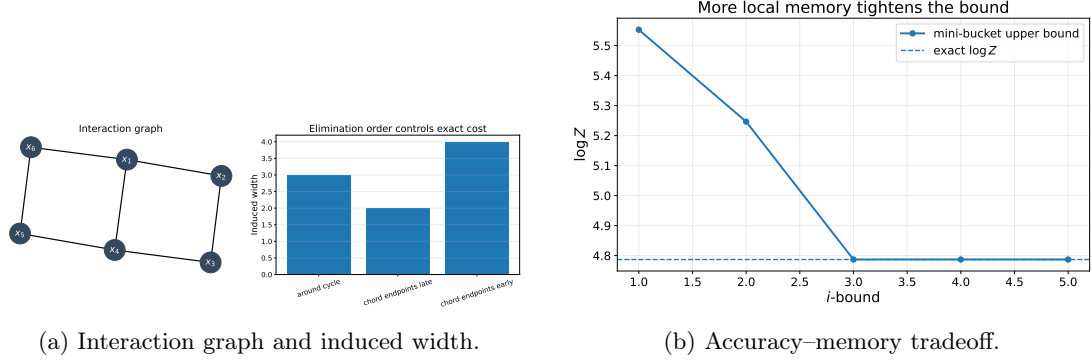


Figure 7.2: Exact and approximate elimination on the same six-variable model. The left panel shows that the elimination order changes the largest intermediate factor. The right panel shows that increasing the i -bound tightens the mini-bucket upper bound until exact elimination is recovered. Generated by `ch07/ch07_01_bucket_elimination_and_minibuckets.ipynb`.

Algorithm 9 BE for computing Z

- 1: **Input:** $\mathcal{G} = (\mathcal{V}_1, \mathcal{V}_2, \mathcal{E})$, set of factors $\mathcal{F} = (f_A(\mathbf{x}_A) | \forall A \in \mathcal{V}_2)$ and elimination order $o = [a_1, \dots, a_n]$.
- 2: **for** a in o **do**
- 3: Assign factors containing x_a to the bucket associated with a : $\beta_a = (A | \forall (a, A) \in \mathcal{E})$
- 4: Generate new factor associated with elimination of x_a according to:

$$f_{\beta_a}(\mathbf{x}_{\beta_a}) = \sum_{x_a} \prod_{A \in \beta_a} f_A(\mathbf{x}_A), \quad \forall \mathbf{x}_{\beta_a}.$$

- 5: Update the factor list: $\mathcal{F} \leftarrow \mathcal{F} \setminus (f_A(\mathbf{x}_A) | \forall A \in \beta_a) \cup f_{\beta_a}(\mathbf{x}_{\beta_a})$
 - 6: Respective updates of $\mathcal{G} = (\mathcal{V}_1, \mathcal{V}_2, \mathcal{E})$
 - 7: **Output:** $Z = \prod_{A \in \mathcal{V}_2} f_A(\mathbf{x}_A)$
-

Companion notebook: Elimination order controls exact cost

Notebook `ch07/ch07_01_bucket_elimination_and_minibuckets.ipynb` implements exact factor-table elimination on a six-variable graph. It verifies that all elimination orders return the same partition function while producing different induced widths, and then introduces the mini-bucket accuracy-memory tradeoff.

A memory limit by itself does not characterize approximation quality. The same notebook therefore keeps the graph, elimination order, and i -bound fixed while scaling all interactions. Figure 7.3 shows the complementary failure direction: a fixed local memory budget that is adequate for weakly coupled variables can become very loose as correlations strengthen.

Exercise 7.1.1. For the model in the notebook, construct the interaction graph produced after each elimination step. Compare min-degree, min-fill, and random elimination orders using the maximum intermediate scope and the total number of table entries created.

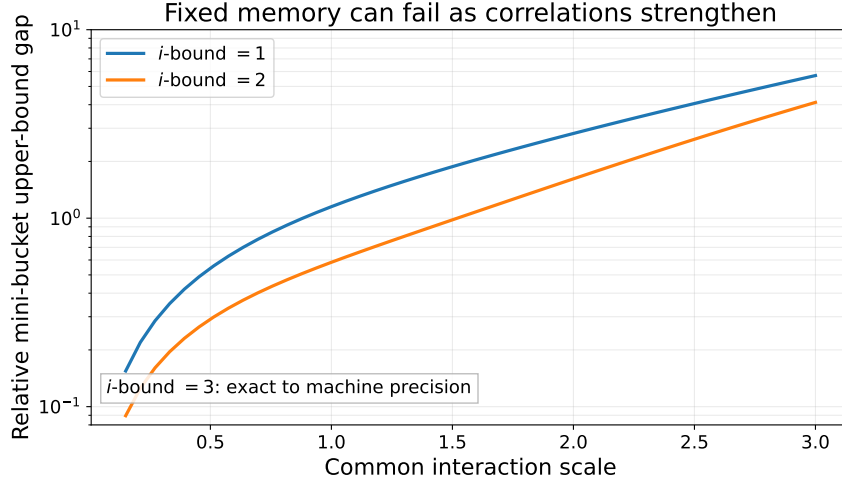


Figure 7.3: Mini-bucket stress test. At fixed i -bound, increasing the common interaction scale can enlarge the relative upper-bound gap by orders of magnitude. Increasing i -bound delays the breakdown because more induced interactions are retained explicitly; for this small instance i -bound = 3 recovers exact elimination to machine precision.

What is being approximated when a bucket is split? It helps to imagine making several copies of the variable to be eliminated. Exact elimination forces every factor in the bucket to use the *same* x_a . After splitting, mini-bucket r effectively works with its own copy $x_a^{(r)}$. The equality tying these copies together has been relaxed. The resulting smaller contractions are tractable, but they no longer preserve all correlations induced through x_a . The max/sum construction below converts that relaxation into a computable upper bound.

7.2 Mini-Bucket Elimination Algorithm

Mini-Bucket Elimination (MBE) Algorithm, introduced in [60], achieves lower complexity than the BE algorithm approximating a step of the BE algorithm by splitting each bucket into several smaller “mini-buckets”. See Fig. (7.4). Formally, for each variable a in the elimination order o , the bucket β_a is partitioned into r_a mini-buckets, $\forall \ell = 1, \dots, r_a : \beta_a^{(\ell)}$, such that $\beta_a = \bigcup_{\ell=1}^{r_a} \beta_a^{(\ell)}$ and $\forall \ell_1, \ell_2 : \beta_a^{(\ell_1)} \cap \beta_a^{(\ell_2)} = \emptyset$. Next, MBE generates new factors according to the following modification of the BE rules (7.1):

$$f_{\beta_a^{(\ell)}}(\mathbf{x}_{\beta_a^{(\ell)}}) = \max_{x_a} \prod_{A \in \beta_a^{(\ell)}} f_A(\mathbf{x}_A), \quad \forall \ell = 1, \dots, r_a - 1, \quad \forall \mathbf{x}_{\beta_a^{(\ell)}}, \quad (7.2)$$

$$f_{\beta_a^{(r_a)}}(\mathbf{x}_{\beta_a^{(r_a)}}) = \sum_{x_a} \prod_{A \in \beta_a^{(r_a)}} f_A(\mathbf{x}_A), \quad \forall \mathbf{x}_{\beta_a^{(r_a)}}, \quad (7.3)$$

where $\forall \ell = 1, \dots, r_a : \mathbf{x}_{\beta_a^{(\ell)}} = (x_b | b \in \beta_a, b \neq a^{(\ell)})$. Other steps are equivalent to that of BE. Observe that MBE replaces the exact marginalization of the bucket in (7.1) by its upper bound,

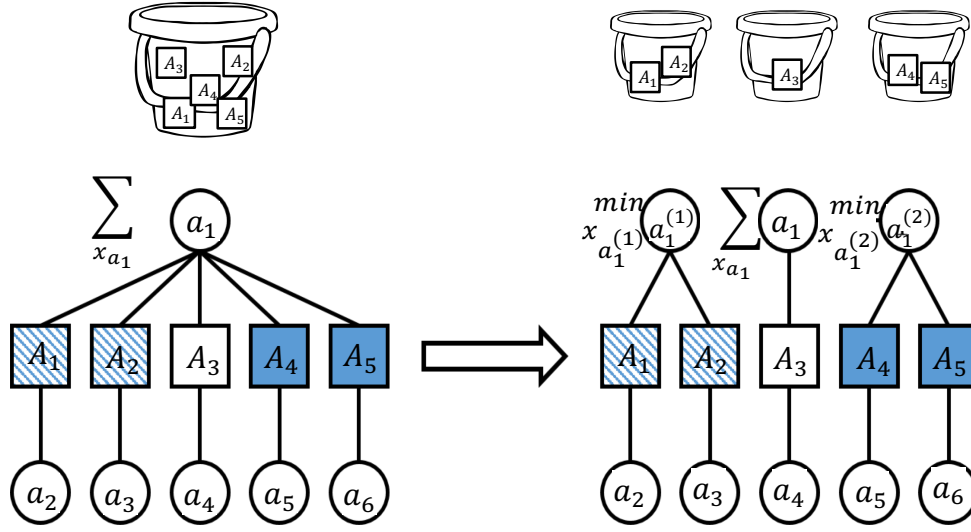


Figure 7.4: Illustration of an elementary step of the mini-bucket elimination. Bucket associated with node a_1 is shown on the top left. The bucket includes five factors $A_1, \dots, A_5$. Split of the bucket, β_{a_1} into three non-overlapping mini-buckets, $\beta_{a_1}^{(1)} = (A_1, A_2)$, $\beta_{a_1}^{(2)} = (A_4, A_5)$ and $\beta_{a_1}^{(3)} = (A_3)$, is shown on the top right. Node a_1 is replicated into three nodes, $a_1^{(1)}, a_1^{(2)}, a_1^{(3)} = a_1$, where we select replica #3 as the main one (thus not upper-indexed). The essence of the mini-bucket elimination consists in “decoupling” $x_{a_1}^{(1)}, x_{a_1}, x_{a_1}^{(2)}$ (ignoring their dependence) and then maximizing over $x_{a_1}^{(1)}$ and $x_{a_1}^{(2)}$ and summing over x_{a_1} , according to Eqs. (7.2) and Eq. (7.3), respectively.

i.e., $\sum_{x_a} \prod_{A \in \beta_a} f_A \leq \prod_{\ell=1}^{r_a} f_{\beta_a^{(\ell)}}$, and hence yields an upper bound on Z . We can also get a lower bound on Z replacing max by min in (7.2).

Let us stress that one has to choose mini-buckets for MBE carefully as their sizes may be critical for complexity and accuracy: smaller mini-buckets may be better for speed, but worse in accuracy. MBE depends on the induced width parameter *ibound* setting the maximal size of a mini-bucket, i.e., $|\mathcal{V}(\beta_a^{(\ell)})| \leq \text{ibound} + 1$. Estimate for the time complexity of the MBE is, $O(d^{\text{ibound}+1} |\mathcal{E}| \cdot \max_{A \in \mathcal{E}} |A|)$, where we accounted for the fact that the maximum number of the mini-buckets is bounded by $|\mathcal{E}| \max_{A \in \mathcal{E}} |A|$.

Why introduce Hölder weights? Ordinary mini-buckets make a rather discrete choice: one piece is summed and the others are maximized. Weighted mini-buckets replace that corner choice by a continuous family of generalized sums. The Hölder weights become optimization variables: they do not change which factors were split, but they can change how tight the resulting bound is. Thus there are now two distinct design decisions—*how to partition a bucket* and *how to weight the pieces*.

7.3 Weighted Mini-Bucket Elimination Algorithms

Weighted Mini-Bucket Elimination Algorithm, introduced in [131] generalizes the MBE algorithm. The weights are associated with each mini-bucket and then we take advantage of the Hölder's inequality [81]. We therefore start describing the method by providing some background for the latter.

Let $(\psi_a(x), a = 1, \dots, r_a)$ be a set of functions defined on discrete variable x , and $\mathbf{w} = (w_1, \dots, w_{r_i})$ be a vector of *Hölder weights*. Define the *weighted absolute summation*:

$$\sum_x^{\circ} \psi_a(x) := \left(\sum_x |\psi_a(x)|^{1/w_a} \right)^{w_a}.$$

For $w_a > 0$, this is the vector ℓ_p norm with $p = 1/w_a$, raised to the power w_a . The limiting convention $w_a = 0$ gives $\max_x |\psi_a(x)|$. If $w_a > 0$ for all $a \geq 1$, then Hölder's inequality implies that

$$\sum_x^{\circ} \prod_{i=1}^{r_i} \psi_a(x) \leq \prod_{a=1}^{r_a} \sum_x^{\circ} \psi_a(x), \quad (7.4) \quad \text{👉}$$

where the weights satisfy $w_0 = \sum_a w_a$. In the elimination application we set $w_0 = 1$. If only one weight is positive, e.g., $w_1 > 0$ and $w_a < 0$ for all $i > 1$, we have the reverse Hölder's inequality:

$$\sum_x^{\circ} \prod_{i=1}^{r_i} \psi_a(x) \geq \prod_{a=1}^{r_a} \sum_x^{\circ} \psi_a(x).$$

Algorithm 10 WMBE for bounding Z

- 1: **Input:** $\mathcal{G} = (\mathcal{V}_1, \mathcal{V}_2, \mathcal{E})$, set of factors $\mathcal{F} = (f_A | \forall A \in \mathcal{V}_2)$, elimination order $o = (a_1, \dots, a_n)$ and bound on the bucket size *ibound*.
- 2: **for** a in o **do**
- 3: Assign factors containing x_a to the bucket associated with a : $\beta_a = (A | \forall (a, A) \in \mathcal{E})$
- 4: Partition the bucket into r_a mini-buckets, $(\beta_a^{(r)} | r = 1, \dots, r_a)$, such that, $\forall r$: $\text{card}(\beta_a^{(r)}) \leq \text{ibound}$
- 5: Assign weights $(w_1, \dots, w_{r_a})$, such that $\sum_r w_r = 1$
- 6: **for** $r \leftarrow 1, \dots, r_a$ **do**
- 7: Generate new factor associated with elimination of x_a according to:

$$f_{\beta_a}^{(r)}(\mathbf{x}_{\beta_a}^{(r)}) = \sum_x^{\circ} \prod_{A \in \beta_a^{(r)}} f_A(\mathbf{x}_A), \quad \forall \mathbf{x}_{\beta_a}^{(r)}.$$

- 8: Update the factor list: $\mathcal{F} \leftarrow \mathcal{F} \setminus (f_A(\mathbf{x}_A) | \forall A \in \beta_a^{(r)}) \cup f_{\beta_a}^{(r)}(\mathbf{x}_{\beta_a}^{(r)})$
 - 9: Respective updates of $\mathcal{G} = (\mathcal{V}_1, \mathcal{V}_2, \mathcal{E})$
 - 10: **Output:** $Z = \prod_{A \in \mathcal{V}_2} f_A(\mathbf{x}_A)$
-

WMBE modifies BE by applying the Hölder's inequality whenever the size of a bucket, i.e., length of $\mathbf{x}_{\beta_a}$, exceeds some given parameter called *ibound*. In this case, WMBE splits the

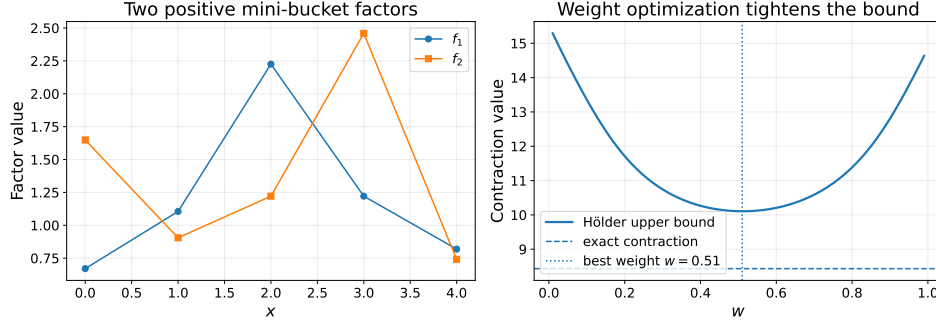


Figure 7.5: A Hölder weight is a continuous design variable. Equal weights are generally not optimal, while the zero-weight limit approaches the max operation used in ordinary mini-bucket elimination. Generated by `ch07/ch07_02_holder_weights.ipynb`.

bucket into multiple ‘mini-buckets’, and weighted absolute summation replaces (7.1), i.e.,

$$\sum_{x_a} \prod_{A \in \beta_a} f_A(x_A) \leq \prod_{r=1}^{r_a} \left(\sum_{x_a}^{w_r} \prod_{A \in \beta_a^{(r)}} f_A(x_A) \right), \quad (7.5)$$

for all x_{β_a} , where Hölder weights satisfy $\sum_r w_r = 1$, $\forall r : w_r > 0$, $\beta_a = \bigcup_r \beta_a^{(r)}$, and $\beta_a^{(r)}$ are disjoint for all r . We then generate multiple new factors according to

$$f_{\beta_a^{(r)}}(x_{\beta_a^{(r)}}) = \sum_{x_a}^{w_r} \prod_{A \in \beta_a^{(r)}} f_A(x_A), \quad \forall x_{\beta_a^{(r)}},$$

and redistribute them among the remaining buckets (according to the standard rule – choose bucket which contains the first insert into other buckets). By construction, WMBE yields an upper bound for the partition function Z . One can use the same idea to derive a lower bound for Z using the reverse Hölder’s inequality. We summarize WMBE in the Algorithm 10.

Notice that not only the weighted version of the mini-bucket is more general than its original version, but it also allows a more compact description because all mini-buckets are equivalent – stated in terms of the *weighted absolute summation* putting what used to be summation and maximization in the original formulation on equal footing.

For some additional discussion of experiments with splitting buckets into mini-buckets and redistribution of weights see [131].

Companion notebook: Hölder weights tighten a split bucket

Notebook `ch07/ch07_02_holder_weights.ipynb` isolates a two-mini-bucket contraction. It verifies the generalized Hölder upper bound, optimizes the weight continuously, and shows how the zero-weight limit recovers a maximum.

Exercise 7.3.1. *Extend the notebook to three positive factors. Optimize weights on the simplex and compare the optimum with equal weights and with the sum/max corner corresponding to ordinary MBE.*

A new idea: improve the approximation without changing the exact model. A gauge transformation changes the numerical representation of the factors while leaving the exact partition function invariant. Exact elimination therefore does not care which gauge is used. An *approximate* elimination can care a great deal, because its bound is applied after the representation has been split. WMBE-G exploits this asymmetry: transform first, approximate second, and optimize the otherwise redundant gauge degrees of freedom to make the approximation tighter.

7.4 Gauge Transformed Weighted Mini-Bucket Elimination Algorithm

In this Section, we follow [3] and describe the gauge-transformed optimization scheme, in short WMBE-G, to improve WMBE upper bound for the partition function Z derived in [131]. WMBE-G improves the WMBE bound of [131] by optimizing over the large family of *gauge transformed* GMs to find the tightest WMBE bound possible.

In order to describe the optimization formulation for tightening the WMBE bound, we first observe that (7.5) can be reformulated into

$$\sum_{x_a} \prod_{A \in \beta_a} f_A(\mathbf{x}_A) \leq \sum_{x_a^{(1)}}^{w_1} \cdots \sum_{x_a^{(r_a)}}^{w_{r_a}} \prod_{r=1}^{r_a} \prod_{A \in \beta_a^{(r)}} f_A(\mathbf{x}_A \setminus x_a, x_a^{(r)}) \quad (7.6)$$

where we simply label each term within the brackets on the right hand side of (7.5) differently according to respective mini-bucket, thus arriving at r_a replicas of x_a , $x_a^{(1)}, \dots, x_a^{(r_a)}$ so that respective factors no longer share the split variable. Applying (7.6) repeatedly we arrive at the following re-statement of the Algorithm 10 which can also be termed WMBE bound:

$$Z \leq Z_{\text{WMBE}} = \sum_{\bar{x}_n}^{\bar{w}_n} \cdots \sum_{\bar{x}_1}^{\bar{w}_1} \prod_{A \in \mathcal{V}_2} f_A(\bar{\mathbf{x}}_A), \quad (7.7) \quad \text{👉}$$

where (7.7), $\bar{\mathbf{x}} = (\bar{x}_1, \dots, \bar{x}_{\bar{n}})$ and $\bar{\mathbf{w}} = (\bar{w}_1, \dots, \bar{w}_{\bar{n}})$ indicate the ‘split’ version of variables and associated Hölder weights, indexed according to appearance of respective node in the extended (as including mini-buckets) elimination order $\bar{o} = (a_1^{(1)}, \dots, a_1^{(r_{a_1})}, \dots, a_n^{(1)}, \dots, a_n^{(r_{a_n})})$. In other words, the WMBE bound provided by the Algorithm 10 can be viewed according to Eq. (7.7) as a *weighted absolute summation over product of factors in a new GM*.

Let us emphasize that unlike the original partition function (i.e. expression on the left hand side of (7.7)) the weighted absolute summation (i.e. expression on the right hand side of (7.7)) is tractable with respect to *ibound* since at most d^{ibound} terms are counted for each weighted absolute summation associated with a mini-bucket.

Our next step consists in replacing the right hand side of (7.7) by its *gauged* version and then minimizing the upper bound over gauges. This *Gauge Transformation* (GT) will be done in three steps.

First, we *normalize* the GM, as described in Section 2.2.1 (see also Fig. 2.2) and transition from the Factor Graph (FG) representation to the Normal Factor Graph (NFG) representation. The *normalization* consists in replicating nodal variables, associated with $\mathcal{V}_1$ at all the edges of the bi-partite graph, $\mathcal{G} = (\mathcal{V}_1, \mathcal{V}_2, \mathcal{E})$, and then associating the “=” factors with the set of

(originally) variable nodes, $\mathcal{V}_1$. Therefore, starting from this point and below in this Section we will utilize the NFG representation in parallel with the FG representation (and will also mix notations, if it does not obscure meaning):

$$\begin{aligned} P(\mathbf{x}) &= Z^{-1} \prod_{a \in \mathcal{V}} f_a(\mathbf{x}_a) = Z^{-1} \left(\prod_{a \in \mathcal{V}_1} f_a(\mathbf{x}_a) \right) \left(\prod_{A \in \mathcal{V}_2} f_A(\mathbf{x}_A) \right), \\ Z &= \sum_{\mathbf{x}} \prod_{a \in \mathcal{V}} f_a(\mathbf{x}_a) = \sum_{\mathbf{x}} \left(\prod_{a \in \mathcal{V}_1} f_a(\mathbf{x}_a) \right) \left(\prod_{A \in \mathcal{V}_2} f_A(\mathbf{x}_A) \right), \\ \forall a \in \mathcal{V}_1 : \quad f_a(\mathbf{x}_a) &= \begin{cases} 1, & \forall A, B \text{ s.t. } (a, A), (a, B) \in \mathcal{E}, \ x_{aA} = x_{aB} \\ 0, & \text{otherwise} \end{cases}, \end{aligned}$$

where $\mathcal{V} = \mathcal{V}_1 \cup \mathcal{V}_2$ and $\forall a \in \mathcal{V} : \mathbf{x}_a = (x_{ab} | \forall b \in \mathcal{V}, \text{ s.t. } (a, b) \in \mathcal{E}), \forall a \in \mathcal{V}_1 : \mathbf{x}_a = (x_{aA} | \forall A \in \mathcal{V}_2, \text{ s.t. } (a, A) \in \mathcal{E}), \forall A \in \mathcal{V}_2 : \mathbf{x}_A = (x_{aA} | \forall a \in \mathcal{V}_1 \text{ s.t. } (a, A) \in \mathcal{E})$.

Then, second, we follow Section 5.1.1 and apply the *Gauge Transformation* (GT) to the left hand side of (7.7). We remind that in the case of the d -dimensional discrete alphabet (space) considered here, the GTs are defined by the following set of invertible $d \times d$ matrices, $\{G_{ab} | (a, b) \in \mathcal{E}_d\}$, associated with directed edges (thus two per each undirected edge of the original graph) termed *gauges*:

$$\forall (a, b) \in \mathcal{E}_d : \quad G_{ab} = \begin{pmatrix} G_{ab}(1, 1) & \cdots & G_{ab}(1, d) \\ \vdots & \ddots & \vdots \\ G_{ab}(d, 1) & \cdots & G_{ab}(d, d) \end{pmatrix}.$$

The transformed GM with respect to the gauges $\mathbf{G} = (G_{ab} : (a, b) \in \mathcal{E}_d)$, consists of modified factors $(\tilde{f}_a | \forall a \in \mathcal{V})$ computed as follows:

$$\forall a \in \mathcal{V} : \quad \tilde{f}_a(\mathbf{x}_a; \mathbf{G}_a) = \sum_{\mathbf{x}'_a} f_a(\mathbf{x}'_a) \prod_{(a, b) \in \mathcal{E}} G_{ab}(x_{ab}, x'_{ab}), \quad (7.8)$$

where $\mathbf{G}_a = (G_{ab} | \forall b \in \mathcal{V}, \text{ s.t. } (a, b) \in \mathcal{E})$. Here, the gauges must satisfy the following *gauge constraints*:

$$\forall (a, b) \in \mathcal{E} : \quad G_{ba}^\top G_{ab} = \mathbb{I}, \quad (7.9)$$

where $\mathbb{I}$ is the identity matrix. With these constraints, the partition function is invariant thus leading us to the following GT version of (7.7)

$$Z = \sum_{\mathbf{x}} \prod_{a \in \mathcal{V}} \tilde{f}_a(\mathbf{x}_a; \mathbf{G}_a) \leq Z_{\text{WMBE}}^{(G)} = \sum_{\bar{x}_n}^{\bar{w}_n} \cdots \sum_{\bar{x}_1}^{\bar{w}_1} \prod_{a \in \mathcal{V}} \tilde{f}_a(\bar{\mathbf{x}}_a; \mathbf{G}_a). \quad (7.10)$$

Finally, third, minimizing the right hand side of (7.10) we are able to present our main gauge-optimized formulation:

$$\min_{\mathbf{G}} \quad Z_{\text{WMBE}}^{(G)}, \quad \text{s.t.} \quad G_{ba}^\top G_{ab} = \mathbb{I}, \quad \forall (a, b) \in \mathcal{E}. \quad (7.11)$$

What follows below is description of the WMBE-G algorithm and implementation details resolving (7.11).

First of all, note that the gauge constraint can be removed simply by expressing one of the two gauge matrices (per edge) via its counter-part, e.g. substituting, G_{Aa} , where $a \in \mathcal{V}_1$, $A \in \mathcal{V}_2$, $(a, A) \in \mathcal{E}$, by $G_{Aa} = (G_{aA}^\top)^{-1}$. Then, the constraints in (7.11) are explicitly removed and it can be solved by any unconstrained optimization solver. This path was chosen in [3] where we thus optimize over gauges iterating a standard gradient descent step, minimizing the objective in Eq. (7.11), then followed by additional updates of the factors.

Our next remark is about relation of the WMBE-G to the original WMBE from [131]. It is natural to initialize algorithm solving (7.11) with identity matrix for gauges because just this yields immediately the WMBE bound. Indeed, $f_a(\mathbf{x}_a) = \tilde{f}_a(\mathbf{x}_a; \mathbb{I}_a)$, where $\mathbb{I}_a = (G_{ab} = \mathbb{I} | \forall b \in \mathcal{V}, \text{ s.t. } (a, b) \in \mathcal{E})$.

Let us assume that the aforementioned explicit resolution of the gauge constraint and identity matrix initialization are already in place, and discuss a step of the iterative procedure. Update of the gauges becomes

$$\forall (a, A) \in \mathcal{E}, \text{ s.t. } a \in \mathcal{V}_1, A \in \mathcal{V}_2, \forall x', x'' : \quad G_{aA}(x', x'') \leftarrow G_{aA}(x', x'') - \Delta \frac{\partial \log Z_{\text{WMBE}}}{\partial G_{aA}(x', x'')}, \quad (7.12)$$

$$\frac{\partial \log Z_{\text{WMBE}}}{\partial G_{aA}(x', x'')} = \sum_{\bar{\mathbf{x}}_a \setminus \bar{\mathbf{x}}_{aA}} q(\bar{\mathbf{x}}_a \setminus \bar{\mathbf{x}}_{aA}, x'') \frac{f_a(\bar{\mathbf{x}}_a \setminus \bar{\mathbf{x}}_{aA}, x')}{f_a(\bar{\mathbf{x}}_a \setminus \bar{\mathbf{x}}_{aA}, x'')} - \sum_{\bar{\mathbf{x}}_A \setminus \bar{\mathbf{x}}_{aA}} q(\bar{\mathbf{x}}_A \setminus \bar{\mathbf{x}}_{aA}, x') \frac{f_A(\bar{\mathbf{x}}_A \setminus \bar{\mathbf{x}}_{aA}, x'')}{f_A(\bar{\mathbf{x}}_A \setminus \bar{\mathbf{x}}_{aA}, x'')},$$

where Δ is the step size (which may also be optimized, see [3] for details), and q is an ‘auxiliary distribution’ defined as

$$q(\bar{\mathbf{x}}) = \prod_{k=1}^{\bar{n}} q(\bar{x}_k | \bar{x}_{k+1:\bar{n}}), \quad q(\bar{x}_k | \bar{x}_{k+1:n}) \propto \left(\sum_{\bar{x}_{k-1}}^{\bar{w}_{k-1}} \cdots \sum_{\bar{x}_1}^{\bar{w}_1} \prod_{a \in \mathcal{V}} f_a(\bar{\mathbf{x}}_a) \right)^{1/\bar{w}_k}.$$

Next we update conjugated gauges and factors,

$$\begin{aligned} \forall a \in \mathcal{V}_1, \forall A \in \mathcal{V}_2, \text{ s.t. } (a, A) \in \mathcal{E} : \quad & G_{Aa} \leftarrow (G_{aA}^\top)^{-1}, \\ \forall a \in \mathcal{V}_1 : \quad & f_a(\mathbf{x}_a) \leftarrow \tilde{f}_a(\mathbf{x}_a; \mathbf{G}_a), \\ \forall A \in \mathcal{V}_2 : \quad & f_A(\mathbf{x}_A) \leftarrow \tilde{f}_A(\mathbf{x}_A; \mathbf{G}_A), \end{aligned}$$

and reset all the gauges for the next iteration, $\forall a \in \mathcal{V}_1, \forall A \in \mathcal{V}_2, \text{ s.t. } (a, A) \in \mathcal{E} : G_{aA} = G_{Aa} \leftarrow \mathbb{I}$.

As established in [3] empirically this style of update leads to a monotonic improvement of the WMBE bound at every step and also converges. Each iteration $t = 1, \dots, T$ results in a sequence of updated factors.

Note that one can use naïve gradient descent, i.e., update gauges only, i.e. without resetting them to the identity matrix as above. However, we prefer to update factors only because it allows us to simplify the gradient formulation and thus save on not repeating intuitively redundant updates of gauges. This iterative approach to resolving the optimization formulation (7.11) is summarized in the Algorithm 11.

Algorithm 11 Gauged WMBE for bounding Z

-
- 1: **Input:** $\mathcal{G} = (\mathcal{V}_1, \mathcal{V}_2, \mathcal{E})$, set of factors $\mathcal{F} = (f_A | \forall A \in \mathcal{V}_2)$ (in the FG format), elimination order $o = (a_1, \dots, a_n)$ and bound on the bucket size *ibound*.
 - 2: Run Algorithm 10.
 - 3: Initialize gauges: $\forall a \in \mathcal{V}_1, \forall A \in \mathcal{V}_2, \text{ s.t. } (a, A) \in \mathcal{E} : G_{aA} = G_{Aa} = \mathbb{I}$.
 - 4: **for** $t = 1, 2, \dots, T$ **do**
 - 5: **for** $\forall a \in \mathcal{V}_1, \forall A \in \mathcal{V}_2, \text{ s.t. } (a, A) \in \mathcal{E}$ **do**
 - 6: Update G_{aA} according to (7.12).
 - 7: $G_{Aa} \leftarrow (G_{aA}^\top)^{-1}$
 - 8: $\forall \mathbf{x}_a : f_a(\mathbf{x}_a) \leftarrow \tilde{f}_a(\mathbf{x}_a; \mathbf{G}_a)$
 - 9: $\forall \mathbf{x}_A : f_A(\mathbf{x}_A) \leftarrow \tilde{f}_A(\mathbf{x}_A; \mathbf{G}_A)$.
 - 10: Reset gauges $G_{aA}, G_{Aa} \leftarrow \mathbb{I}$.
 - 11: **Output:** $Z_{\text{WMBE-G}} = \prod_{a \in \mathcal{V}} f_a(\mathbf{x}_a)$
-

Some additional comments on the Algorithm 11 are in order.

One can also utilize various ideas from [131] in order to improve the efficiency and power of the proposed optimization. Thus, computation of the auxiliary marginals $q(\mathbf{x}_a)$ in (7.12) can be efficiently carried out by a message-passing scheme proposed by the authors. Moreover, one can jointly optimize the Hölder weights $\tilde{\mathbf{w}}$ in addition to $\mathbf{G}$ using the auxiliary distribution during optimization of (7.11). In the experiments reported in [3], we utilize both the message-passing algorithm and the joint optimization involving $\tilde{\mathbf{w}}$ using the log-gradient step suggested in [131].

Our next remark is about the order of elimination which is yet another freedom that one may exploit in order to tighten the WMBE bound. However, and consistently with what was reported in [131] optimization of the elimination order is difficult. Hence in [3] the order was chosen at random.

Final comment is about the freedom in splitting a bucket into mini-buckets. Consistently with the fact that in the NFG format the strategy reduces to a fixed split process: whenever *ibound* is exceeded, a variable x is split in two parts $x^{(1)}, x^{(2)}$, and adjacent factors are assigned separately.

From bounding to compensating. The previous methods obtain tractability by relaxing a large contraction and then tightening a bound. Mini-bucket renormalization takes a different view: if the split removed a coupling, insert a low-complexity correction chosen to imitate the effect of the missing interaction. The guiding analogy is low-rank compression of a large matrix or tensor—we accept a restricted representation, but optimize that representation against the exact local object before continuing the elimination.

7.5 Mini-Bucket Renormalization Algorithm

In this Section and also in the following Section we follow [4] and discuss what is coined the Mini-Bucket Renormalization (MBR).

Main essence of the MBR is in utilizing ideas of *renormalization* from theoretical physics. We would like to *renormalize* a bucket within the Bucket Elimination approach. More accurately, we would like to use the renormalization ideas when breaking a bucket into mini-buckets. The renormalization (also called renormalization group) originates from 1960'ies, and its most recent

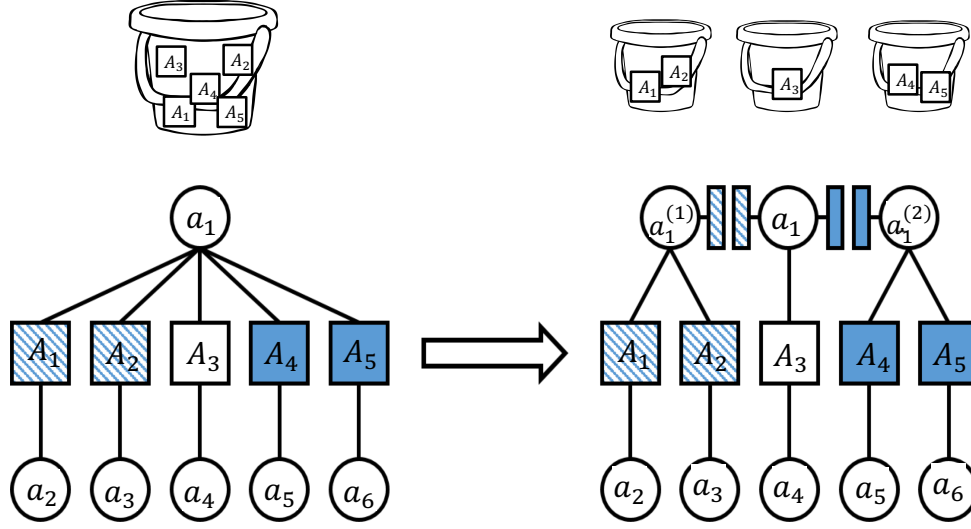


Figure 7.6: Illustration of the renormalization process for splitting into the mini-buckets. Notations are equivalent to these shown in Fig. (7.4) under exception of the new rules for splitting of the a_1 variables. We introduce summation over the new variables weighted with a product of two new equality factors, $\sum_{x_{a_1}^{(1)}, x_{a_1}^{(2)}} \mathbb{1}(x_{a_1} = x_{a_1}^{(1)}) \mathbb{1}(x_{a_1} = x_{a_1}^{(2)}) \cdots$, and then approximate each equality factor, splitting it into a product of two factors each dependent on its own variable, $\forall \ell = 1, 2 : \mathbb{1}(x_{a_1} = x_{a_1}^{(\ell)}) \approx g_{a_1}^{(\ell)}(x_{a_1}) g_{a_1^{(\ell)}}(x_{a_1}^{(\ell)})$. Four new g -dependent factors are shown as rectangles. Optimization over the g factors, i.e. *renormalization*, is done according to Eqs. (7.13).

relevant reincarnation which became popular in computational condensed matter physics is known under the name of the *Tensor Renormalization Group* (TRG) method, see e.g. [124] and references therein.

For each variable a in the elimination order o , MBR partitions a bucket β_a into r_a distinct mini-buckets, $\beta_a^{(\ell)}$, $\ell = 1, \dots, r_a$, with the maximal size of each of the mini-buckets bounded by *ibound*. Then for $\ell = 1, \dots, r_a - 1$, the mini-bucket $\beta_a^{(\ell)}$ gets $a^{(\ell)}$ replica of the node a , and the last, r_a -th, replica get the original node a . We *renormalize* the replicas, introducing local factors $g_a^{(\ell)}$ and $g_{a^{(\ell)}}$ which are expected to approximate factors f_A of the GM better than the MBE scheme of [60].

Specifically, the local compensating/renormalizing factors, g , are chosen by solving the following $r_a - 1$ independent optimizations (see also Fig. 7.6 for illustration)

$$\forall \ell = 1, \dots, r_a - 1 : \min_{g_a^{(\ell)}, g_{a^{(\ell)}}} \sum_{\mathbf{x}_{\beta_a^{(\ell)}}} \left(\tilde{f}_{\beta_a^{(\ell)}}(\mathbf{x}_{\beta_a^{(\ell)}}) - \prod_{A \in \beta_a^{(\ell)}} f_A(\mathbf{x}_{\beta_a^{(\ell)}}) \right)^2, \quad (7.13)$$

where $\tilde{f}_{\beta_a^{(\ell)}}$ are renormalized factors of the mini-bucket $\beta_a^{(\ell)}$ dependent on $\mathbf{x}_{\beta_a^{(\ell)}}$:

$$\tilde{f}_{\beta_a^{(\ell)}}(\mathbf{x}_{\beta_a^{(\ell)}}) = g_a^{(\ell)}(x_a^{(\ell)}) \sum_{x'} g_{a^{(\ell)}}(x') \prod_{A \in \beta_a^{(\ell)}} f_A(\mathbf{x}_A \setminus x_a, x').$$

We show below that (7.13) can be solved efficiently. Once all the mini-buckets are processed, factors can be summed over the variables $x_{a(1)}, \dots, x_{a(r_a-1)}$ and x_a separately (r_a mini-buckets per bucket a), i.e., introduce new factors as follows:

$$\ell = 1, \dots, r_a - 1 : \quad f_{\beta_a^{(\ell)}}(\mathbf{x}_{\beta_a^{(\ell)}}) = \sum_{x'} g_a^{(\ell)}(x') \prod_{A \in \beta_a^{(\ell)}} f_A(\mathbf{x}_A \setminus x_a^{(\ell)}, x'), \quad (7.14)$$

$$f_{\beta_a^{(r_a)}}(\mathbf{x}_{\beta_a^{(r_a)}}) = \sum_{x_a} \prod_{\ell=1}^{r_a-1} g_a^{(\ell)}(x_a^{(\ell)}) \prod_{A \in \beta_a^{(r_a)}} f_A(\mathbf{x}_A). \quad (7.15)$$

Resulting factors are then added to its corresponding mini-bucket and repeat until all the buckets are processed, like in the cases of BE and MBE. One can check that

$$\prod_{\ell=1}^{r_a} f_{\beta_a^{(\ell)}}(\mathbf{x}_{\beta_a^{(\ell)}}) = \sum_{x_a} f_{\beta_a^{r_a}}(\mathbf{x}_{\beta_a^{(r_a)}}) \prod_{\ell=1}^{r_a-1} \tilde{f}_{\beta_a^{(\ell)}}(\mathbf{x}_{\beta_a^{(\ell)}}) \approx \sum_{x_a} \prod_{\ell=1}^{r_a} f_{\beta_a^{(\ell)}}(\mathbf{x}_{\beta_a^{(\ell)}}),$$

from (7.13) and MBR indeed approximates BE. Formal description of the MBR is given in the Algorithm 12.

Algorithm 12 Mini-bucket renormalization (MBR)

- 1: **Input:** GM, $(\mathcal{V}_1^\dagger, \mathcal{V}_2^\dagger, \mathcal{E}^\dagger, \mathcal{F}^\dagger)$, elimination order o and induced width bound $ibound$.
 - 2: $\mathcal{F} \leftarrow \mathcal{F}^\dagger$
 - 3: **for** a in o **do**
 - 4: $\beta_a \leftarrow (A | \forall A \in \mathcal{V}_2, \text{ s.t. } (a, A) \in \mathcal{E})$
 - 5: Split the bucket β_a into r_a mini-buckets $(\beta_a^{(\ell)} | \ell = 1, \dots, r_a)$ such that $|\mathcal{V}(\beta_a^{(\ell)})| \leq ibound + 1$ for $\ell = 1, \dots, r_a$.
 - 6: **for** $\ell = 1, \dots, r_a - 1$ **do**
 - 7: Generate compensating factors $g_a^{(\ell)}, g_{a^{(\ell)}}$ according to (7.13).
 - 8: Generate new factor $f_{\beta_a^{(\ell)} \setminus \{a\}}$ according to (7.14).
 - 9: Generate new factor $f_{\beta_a^{r_a} \setminus \{a\}}$ according to (7.15).
 - 10: **for** $\ell = 1, \dots, r_a$ **do**
 - 11: $\mathcal{F} \leftarrow \mathcal{F} \cup \{f_{\beta_a^{(\ell)} \setminus \{a\}}\} \setminus \beta_a^{(\ell)}$
 - 12: **Output:** $Z = \prod_{A \in \mathcal{V}_2} f_A(\mathbf{x}_A)$.
-

Companion notebook: Low-rank compensation repairs a split variable

Notebook `ch07/ch07_03_low_rank_renormalization.ipynb` matricizes a positive local tensor, computes its best rank-one approximation, and visualizes the residual. The example

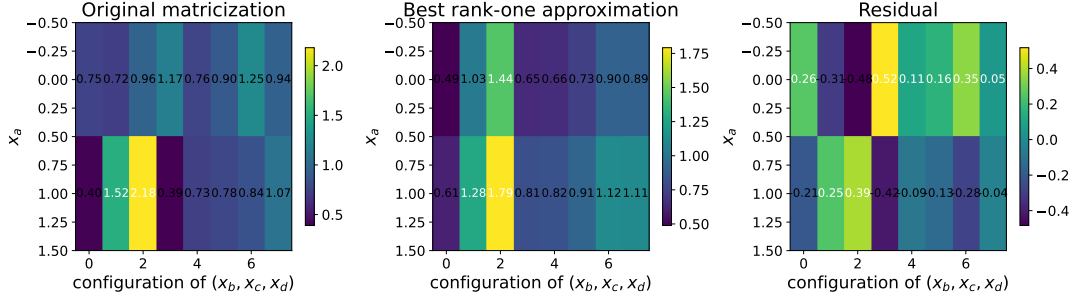


Figure 7.7: Local renormalization as rank-one compression. The leading singular triplet captures the best Frobenius-norm approximation to the matricized bucket tensor; the residual identifies the dependence discarded by the split. Generated by `ch07/ch07_03_low_rank_renormalization.ipynb`.

makes the relation between mini-bucket renormalization, bond-dimension truncation, and tensor-network compression explicit.

The singular-value calculation also supplies a natural failure diagnostic. A single favorable tensor can make rank-one compensation look nearly exact even when the approximation has no general guarantee. The companion notebook therefore deforms a positive matricized factor continuously from a rank-one case toward a two-direction case. Figure 7.8 shows that the discarded singular direction grows together with the rank-one approximation error. Thus the local spectral ratio is not merely a compression statistic: it can be used to decide when the assumed bond dimension is no longer adequate.

7.5.1 Efficiency, Complexity and Global Perspectives

Let us now discuss efficiency and complexity of the MBR algorithm. The optimization (7.13) is related to the rank-1 approximation on $f_{\beta_a^{(e)}}$, which can be solved efficiently via (truncated) singular value decomposition (SVD). Specifically, let $\mathbf{M}$ be a $d \times d^{|\mathcal{V}(\beta_a^{(e)})|-1}$ matrix representing $f_{\beta_a^{(e)}}$ as follows:

$$\mathbf{M} \left(x_a, \sum_{b \in \mathcal{V}(\beta_a^{(e)}), b \neq a} x_b \prod_{c \in \mathcal{V}(\beta_a^{(e)}), c > b} d \right) = f_{\beta_a^{(e)}}(\mathbf{x}_{\beta_a^{(e)}}). \quad (7.16)$$

The local least-squares problem is a rank-one matrix approximation:

$$\min_{\mathbf{r}_1, \mathbf{r}_2} \|\mathbf{M} - \mathbf{r}_1 \mathbf{r}_2^T\|_F.$$

If $\mathbf{M} = U \text{diag}(\sigma) V^T$, the Eckart–Young theorem gives the optimum $\sigma_1 u_1 v_1^T$. One convenient scaling is $\mathbf{r}_1 = \sqrt{\sigma_1} u_1$ and $\mathbf{r}_2 = \sqrt{\sigma_1} v_1$. Because the matricization is generally rectangular, the left and right singular vectors are different objects; the legacy claim that they coincide was incorrect. Positivity of the original tensor does not by itself guarantee an entrywise-positive right singular vector, although sign choices and nonnegative low-rank variants can be useful when

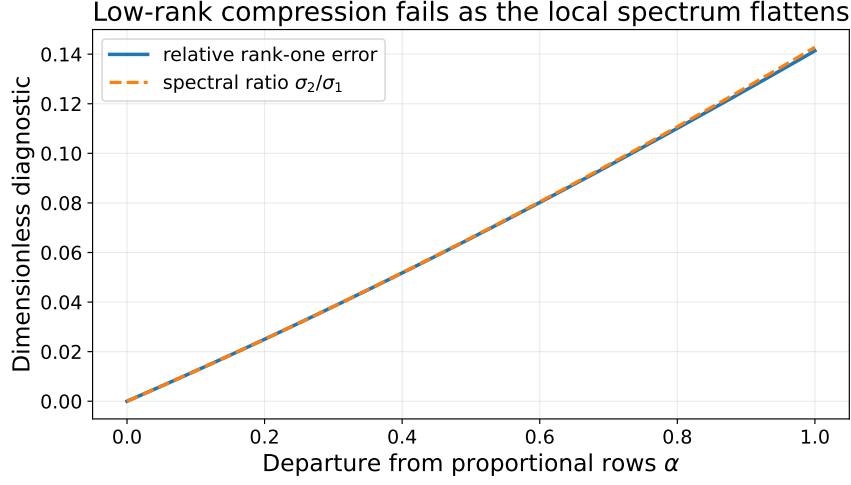


Figure 7.8: Stress test for local low-rank renormalization. The tensor is deformed from a rank-one regime toward a less compressible regime. The best rank-one error grows together with the first neglected singular-value ratio σ_2/σ_1 , providing an internal diagnostic of impending approximation failure.

positivity of compensating factors is required. Due to the above observations, the complexity of (7.13) is $N_{\text{SVD}}(\mathbf{M})$ that denotes the complexity of SVD for matrix $\mathbf{M}$. Therefore, we arrive at the following estimate of the overall complexity

$$O(N_{\text{SVD}}(\mathbf{M}) \cdot T) = O\left(N_{\text{SVD}}(\mathbf{M}) \cdot |\mathcal{E}| \cdot \max_{A \in \mathcal{E}} |A|\right),$$

where $N_{\text{SVD}}(\mathbf{M}) = O(d^{\text{bound}+2})$ in general, but typically much smaller if we utilize mature SVD solvers.

We conclude this Section and Chapter discussing a graph-global prospective of the MBR algorithm and other algorithms stated in terms of buckets and mini-buckets. We have two points to make in this context.

First, even though the MBR algorithm is stated as a semi-group, that is as a sequence of elimination of variables thus reducing a graph at every step, we can also re-state it in alternative terms of building a sequence of GMs over larger graphs incorporating replicas of the eliminated nodes. This “replication”, as opposed “elimination” prospective on MBR is illustrated in Fig. (7.9).

Second, MBR just discussed only considers local neighborhood for renormalizing mini-buckets to approximate a single marginalization process of BE. An improvement which generalizes local optimization rules for selecting renormalization (7.13), coined Global-Bucket Renormalization (GBR) Algorithm was suggested in [4]. Reader interested in details of the GBR, and also extensive experimental validation of all the algorithms mentioned in this Chapter of the book and some other basic GM inference algorithm (e.g. BP), is advised to check [4]. We have decided not to include details of the GBR algorithm in the book because we believe today (a number of years after publication of [4]) that there are number of other ways to account for

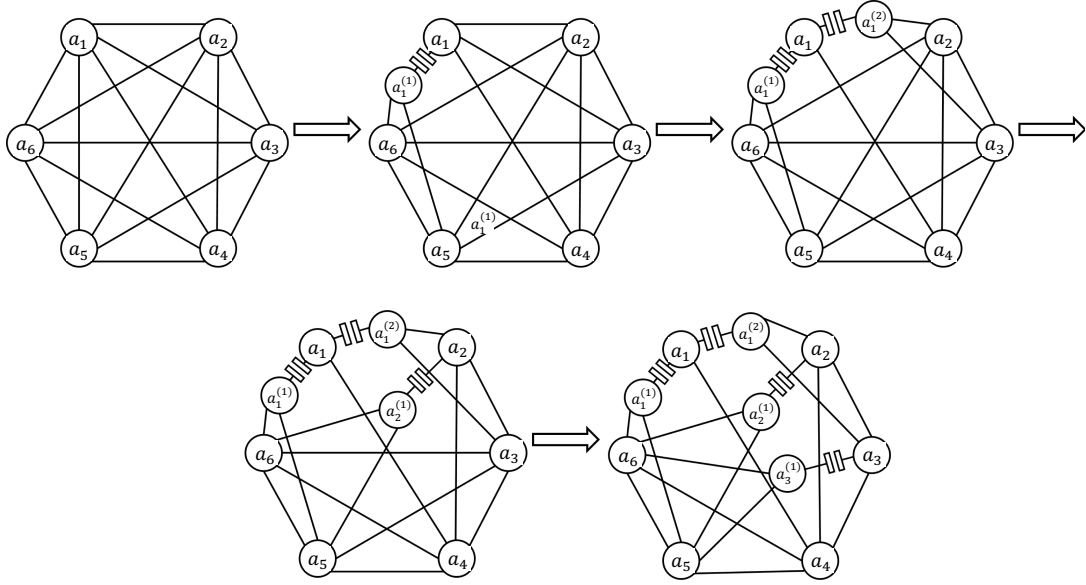


Figure 7.9: Illustration of the Mini Bucket Renormalization (MBR) algorithm as a reduction to the basic Bucket Elimination (BE) algorithm. We work here with the example of a pair-wise GM (factors are associated with edges of the graph, thus not shown) defined over complete six nodes graph. This “replication” version of the MBR works in three steps. First, we introduce the elimination order, $o = (a_1, \dots, a_6)$ and $ibound = 2$ and introduce a sequence of graph transformations – transforming at each iteration “current” GM to an “updated” one by splitting variable. This sequence of the GM transformations is shown in the Figure. Second, we find optimal renormalization parameters associated with the new singleton factors – shown in the Figure as rectangles – re-solving Eqs. (7.13). (See text for details.) Third, we apply the Bucket Elimination (BE) algorithm with the elimination order, $\tilde{o} = (a_1^{(1)}, a_1^{(2)}, a_1, a_2^{(1)}, a_2, a_3^{(1)}, a_3, a_4, a_5, a_6)$, to the final GM (in the aforementioned sequence) with the induced width 2, which is the final GM in the sequence found at the first step (and with the factors found at the second step).

the GM-global effects when deciding on the local renormalizations. Moreover, we are convinced that an even better (empirical and possibly theoretical) improvement can be achieved in the future by merging renormalization and gauge-transformation ideas discussed above.

7.6 What approximate elimination buys us

The methods of this chapter share an elimination order and a local complexity cap, but they use different replacements for the dependence destroyed by splitting:

Method	Replacement for a full bucket	Guarantee	Main design variables
MBE	sums and limiting maxima	upper or lower bound	order, partition, i -bound
WMBE	weighted power sums	Hölder bound	weights plus MBE choices
WMBE-G	WMBE after gauge transformation	optimized bound	gauges and weights
MBR	low-rank compensating factors	approximation, not generally a bound	rank, local norm, order

The comparison also clarifies the relation to later chapters. MCMC in Chapter 8 trades deterministic bounds for asymptotically exact stochastic estimates. Loop calculus in Chapter 9 starts from a fixed-point approximation and organizes exact corrections by graphical substructures. Approximate elimination instead compresses the computation as it proceeds.

Problem 7.1. *Design an adaptive mini-bucket method that increases the local i -bound only when a computable split diagnostic exceeds a threshold. Possible diagnostics include the optimized Hölder gap, the first neglected singular value, or disagreement between gauges. State what can be certified and test the proposal on the Chapter 2 Ising examples.*

Problem 7.2. *Formulate a hybrid algorithm that alternates a global gauge update with local low-rank renormalization. Explain which quantities remain invariant, which quantities are bounded, and which steps are heuristic. Discuss how automatic differentiation could be used without obscuring the graphical structure.*

Chapter 8

Markov Chain Monte Carlo: Invariance, Mixing, and Exact Sampling

Reader's Guide.

Why this chapter? Exact generation in Chapter 4 repeatedly solves conditional-inference problems. Markov chain Monte Carlo takes a different route: construct an easy random move that, when repeated, spends the correct long-run fraction of time in every state. The target distribution is therefore specified indirectly through a *dynamics* rather than generated in one exact pass.

The simplest example: a two-state random walk. Imagine states 0 and 1. If the chain jumps $0 \rightarrow 1$ with probability a and $1 \rightarrow 0$ with probability b , then the long-run occupancy satisfies $\pi(0)a = \pi(1)b$. Many different pairs (a, b) can produce the same ratio $\pi(1)/\pi(0)$. This already separates two ideas that should not be confused: the *stationary distribution* tells us where the chain spends time after equilibration, while the transition probabilities determine how quickly it gets there and how correlated successive samples are.

Three questions, in this order. For every MCMC method ask: (i) does the proposed transition preserve the desired law? (ii) can the chain reach all relevant states and forget periodic structure? (iii) does it do so rapidly enough to be computationally useful? Detailed balance is one convenient answer to the first question, not a synonym for MCMC and not a guarantee of rapid mixing.

Two construction patterns. Gibbs sampling redraws a variable from its exact conditional distribution. Metropolis–Hastings proposes a move from an easier kernel and corrects it by an accept/reject probability. Both can preserve the same target distribution while having very different temporal memory. Coupling from the past later adds a different notion

of exactness: not an asymptotically equilibrated trajectory, but a provably exact equilibrium sample.

How to use the companion notebooks. Use the notebooks in `notebooks/ch08/` to separate stationary correctness from computational efficiency. First verify the invariant distribution on a tiny state space by direct matrix calculation. Then compare total-variation decay, autocorrelation, and effective sample size. Finally use CFTP to see what an exact-sampling certificate looks like and annealed importance sampling to see how a sequence of intermediate distributions can make a hard normalization problem easier.

Chapter 4 introduced generation as one of the central computational tasks. Exact decimation produces independent samples but repeatedly calls a conditional-inference oracle. Markov chain Monte Carlo (MCMC) takes the complementary route: it constructs a local stochastic dynamics whose invariant distribution is the desired graphical-model law. Samples along one trajectory are correlated, and the practical question becomes whether the chain forgets its initialization rapidly enough.

This chapter separates three logically distinct issues:

1. **invariance**: does the transition rule preserve the target law?
2. **ergodicity**: is the invariant law unique and reached from every initial state?
3. **mixing and estimation**: how many transitions are needed for accurate expectations?

For finite chains, irreducibility and aperiodicity provide a clean answer to the first two questions; quantitative convergence requires spectral, coupling, conductance, or functional-inequality estimates. Standard references include [123, 169, 190].

8.1 Finite-state Markov chains

Let $\mathcal{S}$ be a finite state space. We use the column-stochastic convention

$$K(y \mid x) \geq 0, \quad \sum_{y \in \mathcal{S}} K(y \mid x) = 1. \quad (8.1)$$

If $\pi_t(x) = \mathbb{P}(X_t = x)$, then

$$\pi_{t+1} = K\pi_t, \quad \pi_{t+\tau} = K^\tau \pi_t. \quad (8.2)$$

A distribution π is *stationary* when $K\pi = \pi$.

Definition 8.1.1. A finite chain is *irreducible* if every state can reach every other state with positive probability in some number of steps. It is *aperiodic* if the common return-time period of its states is one.

Theorem 8.1.2 (Finite ergodic theorem). If a finite Markov chain is irreducible, it has a unique stationary distribution. If it is also aperiodic, then $K^t \pi_0 \rightarrow \pi$ for every initial distribution π_0 .

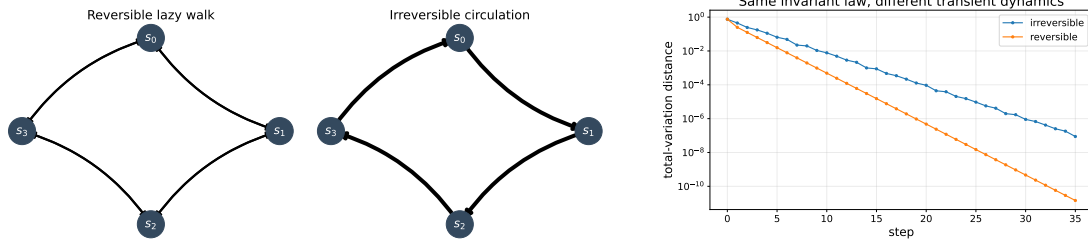


Figure 8.1: Two transition mechanisms with the same invariant law. In the two network panels, arrows show transition direction and line width is proportional to the corresponding transition probability. The reversible walk has equal clockwise and counterclockwise fluxes and therefore no stationary circulation; the irreversible walk has visibly asymmetric transition strengths and carries a directed current. Their transient convergence can nevertheless differ. Generated by `ch08/ch08_01_stationarity_and_mixing.ipynb`.

The uniqueness question is central in the general theory of Markov chains. In MCMC it is usually handled during algorithm design: self-loops give aperiodicity, and local proposals are chosen so the state graph is connected. The difficult issue is normally not whether convergence eventually occurs, but how slowly it may occur.

A standard metric is total variation,

$$\|\mu - \nu\|_{\text{TV}} = \frac{1}{2} \sum_{x \in \mathcal{S}} |\mu(x) - \nu(x)|.$$

The mixing time at tolerance ε is

$$t_{\text{mix}}(\varepsilon) = \min \left\{ t : \max_{x \in \mathcal{S}} \|K^t \delta_x - \pi\|_{\text{TV}} \leq \varepsilon \right\}.$$

For reversible chains, the spectral gap controls asymptotic relaxation; conductance and coupling bounds often provide more geometric information [123].

Companion notebook: The invariant law does not determine the dynamics

Notebook `ch08/ch08_01_stationarity_and_mixing.ipynb` constructs reversible and irreversible chains with the same stationary law. It visualizes probability circulation and compares total-variation convergence from the same initial state.

8.2 Detailed balance, global balance, and probability currents

A chain is reversible with respect to π when

$$K(y | x)\pi(x) = K(x | y)\pi(y) \quad \text{for all } x, y. \quad (8.3)$$

Detailed balance immediately implies stationarity after summing over x . Stationarity itself requires only global balance,

$$\sum_x K(y | x) \pi(x) = \pi(y). \quad (8.4)$$

The antisymmetric stationary current

$$J(y, x) = K(y | x) \pi(x) - K(x | y) \pi(y)$$

vanishes for a reversible chain. An irreversible chain may have nonzero circulating currents while preserving the same π . This freedom is algorithmically important: detailed balance is a convenient construction principle, not a requirement of MCMC.

Exercise 8.2.1. For the two four-state chains in the companion notebook, compute every current $J(y, x)$ and verify the node-wise conservation law $\sum_y J(y, x) = 0$.

From an invariant law to an implementable local move. For a graphical model, the full target $P(\mathbf{x})$ may require an intractable normalization constant, but a one-variable conditional often depends only on neighboring factors and its local normalization is cheap. Gibbs sampling exploits exactly this asymmetry. Metropolis–Hastings, in the next section, is even more general: it needs only ratios of unnormalized probabilities. These are two recurring algorithmic tricks in probabilistic computation—replace an inaccessible global normalization by a tractable local normalization or by a ratio in which the constant cancels.

8.3 Gibbs and heat-bath sampling

Let the target distribution be a positive graphical-model law

$$P(\mathbf{x}) = Z^{-1} \prod_{A \in \mathcal{V}_2} f_A(\mathbf{x}_A).$$

Choose a variable a and retain all other coordinates. The exact conditional is local:

$$P(x_a | \mathbf{x}_{-a}) = \frac{\prod_{A \ni a} f_A(x_a, \mathbf{x}_{A \setminus a})}{\sum_{x'_a} \prod_{A \ni a} f_A(x'_a, \mathbf{x}_{A \setminus a})}. \quad (8.5) \quad \text{👉}$$

A random-scan Gibbs step selects a and redraws x_a from Eq. (8.5). Because the update is an exact conditional draw, it satisfies detailed balance. Block Gibbs sampling replaces one variable by a tractable subset.

8.4 Metropolis–Hastings sampling

Suppose the target is known up to normalization, $P(x) \propto p(x)$, and let $q(y | x)$ be a proposal kernel. Metropolis–Hastings accepts a proposal y with probability

$$\alpha(x, y) = \min \left(1, \frac{p(y)q(x | y)}{p(x)q(y | x)} \right). \quad (8.6) \quad \text{👉}$$

Algorithm 13 Random-scan Gibbs sampling

-
- 1: **Input:** factors, initial configuration $\mathbf{x}^{(0)}$, number of updates T .
 - 2: **for** $t = 0, \dots, T - 1$ **do**
 - 3: choose a from a prescribed scan distribution;
 - 4: draw $x_a^{(t+1)} \sim P(\cdot \mid \mathbf{x}_{-a}^{(t)})$ using Eq. (8.5);
 - 5: keep all other coordinates unchanged.
 - 6: **Output:** the correlated trajectory $\mathbf{x}^{(0:T)}$.
-

Algorithm 14 Metropolis–Hastings sampling

-
- 1: **Input:** unnormalized target p , proposal q , initial state $x^{(0)}$.
 - 2: **for** $t = 0, \dots, T - 1$ **do**
 - 3: draw $y \sim q(\cdot \mid x^{(t)})$;
 - 4: draw $u \sim \text{Uniform}(0, 1)$;
 - 5: **if** $u < \alpha(x^{(t)}, y)$ **then**
 - 6: $x^{(t+1)} \leftarrow y$;
 - 7: **else**
 - 8: $x^{(t+1)} \leftarrow x^{(t)}$.
-

The transition kernel includes the rejected probability as a self-loop. The identity

$$p(x)q(y \mid x)\alpha(x, y) = p(y)q(x \mid y)\alpha(y, x) \quad (8.7)$$

proves detailed balance without knowing Z [139, 83].

For an Ising model with energy

$$E(\mathbf{x}) = - \sum_{(a,b) \in \mathcal{E}} J_{ab} x_a x_b - \sum_a h_a x_a,$$

a symmetric single-spin proposal gives acceptance probability

$$\alpha(\mathbf{x}, \mathbf{x}^{(a)}) = \min(1, \exp(-\beta \Delta_a E)), \quad \Delta_a E = 2x_a \left(h_a + \sum_{b \in \partial a} J_{ab} x_b \right).$$

This corrects the sign and field-index errors in the legacy Glauber formula. Heat-bath Glauber dynamics instead uses the conditional Bernoulli probability and is a Gibbs update.

Companion notebook: Local rules, common equilibrium, different memory

Notebook `ch08/ch08_02_gibbs_and_metropolis.ipynb` compares Gibbs and single-spin Metropolis dynamics on an Ising ring. Exact enumeration verifies the stationary law, while traces and autocorrelations reveal different effective sample sizes.

A sample count is not an information count. If N states were independent, averaging them would give the familiar $N^{-1/2}$ Monte Carlo scaling. Along an MCMC trajectory the states are correlated. The integrated autocorrelation time measures how many consecutive

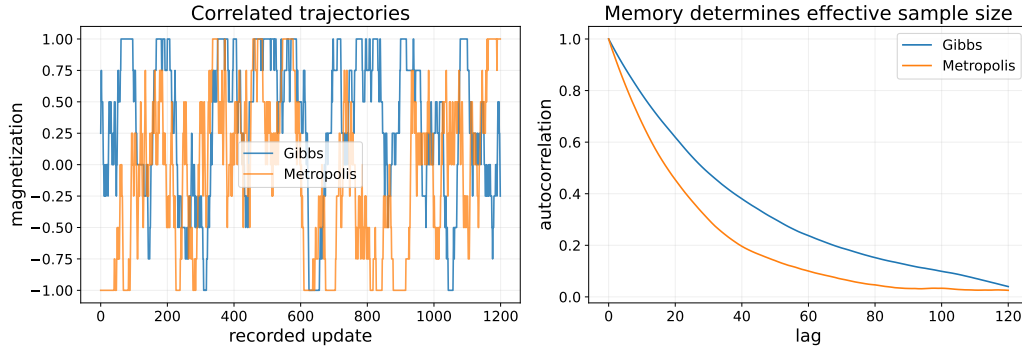


Figure 8.2: Gibbs and Metropolis trajectories have the same equilibrium target but different temporal memory. Autocorrelation, rather than the raw number of recorded states, controls the effective amount of information. Generated by `ch08/ch08_02_gibbs_and_metropolis.ipynb`.

states behave roughly like one independent draw, and N_{eff} converts trajectory length into an approximate information-equivalent sample size. This is why two correct samplers with the same invariant law can have dramatically different practical value.

8.5 From trajectories to estimators

After discarding an initial transient, an expectation is estimated by

$$\hat{\mathbb{E}}[g] = \frac{1}{N} \sum_{t=1}^N g(X_t).$$

Correlation inflates variance. For a stationary scalar series with normalized autocorrelation $\rho(k)$, the integrated autocorrelation time and effective sample size are

$$\tau_{\text{int}} = 1 + 2 \sum_{k \geq 1} \rho(k), \quad N_{\text{eff}} \approx \frac{N}{\tau_{\text{int}}}.$$

Trace plots, multiple initializations, split-chain diagnostics, and autocorrelation estimates are useful, but no finite collection of empirical diagnostics proves convergence in an unknown difficult instance. Whenever possible, one should exploit model-specific contraction, monotonicity, spectral, or conductance estimates.

Problem 8.1. *Construct two MCMC kernels for the frustrated triangle of Chapter 6: one single-spin chain and one block chain. Compare acceptance, autocorrelation, and transition-graph bottlenecks as temperature decreases.*

Why “from the past”? Ordinary coupling starts several chains now and waits for them to meet. Conditioning on an unusually early meeting can bias the state at the meeting time. CFTP reverses the logic: fix the present time, run the same random update maps from farther and farther in the past, and stop only when every admissible initial state has been

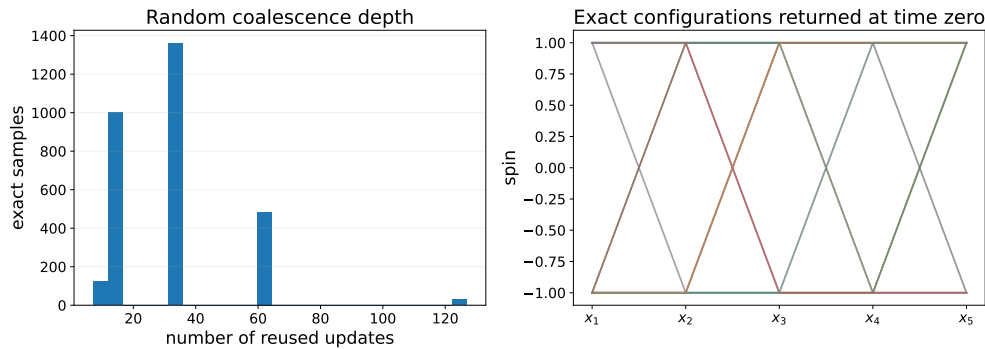


Figure 8.3: Coupling from the past extends the random update history until the extremal trajectories coalesce. The lookback depth is random; the state at time zero is an exact equilibrium sample. Generated by `ch08/ch08_03_coupling_from_the_past.ipynb`.

forced to the same present state. The coalescence event is then a certificate that the present state has exactly the stationary law.

8.6 Coupling from the past and exact sampling

Coupling uses the same random update map for several initial states. Once trajectories coalesce, they evolve identically. Running all states forward until coalescence is biased toward unusually early meeting points. Coupling from the past (CFTP) removes this bias: start at time $-T$, reuse the same random maps up to time zero, and double T until all possible initial states coalesce before the present [163, 123].

Tracking every initial state is normally impossible. For an attractive binary model, the partial order $-1 < +1$ and monotone heat-bath updates imply that trajectories started from the all-minus and all-plus states bound all others. Coalescence of these two extremal chains certifies an exact sample. In Ising language, attractiveness corresponds to ferromagnetic couplings $J_{ab} \geq 0$; related monotonicity and correlation inequalities reappear in the attractive-model discussions later in the book.

Companion notebook: Exact samples emerge from monotone coalescence

Notebook `ch08/ch08_03_coupling_from_the_past.ipynb` implements monotone CFTP for a ferromagnetic Ising chain. The returned samples agree with exact marginals, and the random lookback depth quantifies the difficulty of coalescence.

Exercise 8.6.1. Show that the heat-bath update is monotone when every incident coupling is nonnegative. Give a three-spin frustrated counterexample where the extremal-chain ordering is not preserved.

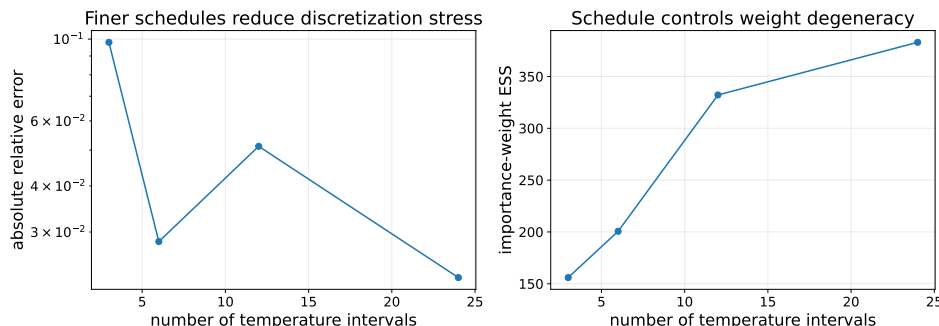


Figure 8.4: Annealed importance sampling on a small Ising model. A finer temperature schedule reduces abrupt distributional changes and improves the effective sample size of the importance weights. Generated by `ch08/ch08_04_annealed_importance_sampling.ipynb`.

8.7 Annealing, partition functions, and randomized approximation

MCMC also enters normalization through a sequence of intermediate distributions

$$P_k(x) \propto p_0(x)^{1-\beta_k} p_1(x)^{\beta_k}, \quad 0 = \beta_0 < \cdots < \beta_K = 1.$$

Simulated annealing uses the path to seek low-energy states. Annealed importance sampling estimates a ratio of partition functions by accumulating incremental weights while applying invariant transitions at each temperature [152]. The method is asymptotically valid, but a finite empirical implementation should not be confused with a fully polynomial randomized approximation scheme (FPRAS).

An FPRAS for a positive quantity returns a relative-error approximation with high probability in time polynomial in the input size, $1/\varepsilon$, and $\log(1/\delta)$. Proving such a guarantee requires quantitative mixing and variance control along the entire annealing path. The classical Jerrum–Sinclair program for attractive counting problems illustrates how MCMC, self-reducibility, and annealing combine to yield complexity guarantees [97].

Companion notebook: An annealing schedule controls weight degeneracy

Notebook `ch08/ch08_04_annealed_importance_sampling.ipynb` estimates the partition function of a small Ising model. It compares schedules through relative error and importance-weight effective sample size.

Problem 8.2. Choose a sequence of inverse temperatures using a target overlap or effective-sample-size criterion. Compare the resulting schedule with a uniform schedule and explain which additional estimates would be needed for a polynomial-time theorem.

8.8 Connections forward

Three ideas from this chapter recur in modern generative modeling. First, a transition kernel can preserve a desired marginal without being reversible. Second, a sequence of kernels defines a path-space probability and can be controlled or time-reversed. Third, exactness of a stationary law does not imply computational usefulness without a quantitative mixing statement. These distinctions will be central when the book turns from equilibrium MCMC to diffusion, bridge, and decision-based generative processes.

Chapter 9

Loop Calculus: Exact Corrections Beyond Belief Propagation

Reader's Guide.

Why this chapter? Belief propagation is exact on a tree because there is no alternative route by which information can circulate back to its starting point. Once loops are present, the Bethe/BP construction remains a useful reference but misses global consistency effects. Loop calculus does not discard BP; it asks for an exact accounting identity for what BP leaves out.

The simplest picture: tree plus one extra edge. Start from a tree, where $Z = Z^{\text{BP}}$. Add one edge that closes a single cycle. BP can still be run, but the new cycle creates one genuinely global correlation channel. Loop calculus writes the exact partition function schematically as

$$Z = Z^{\text{BP}}(1 + \text{corrections from closed subgraphs}).$$

For a general loopy graph, the correction is a sum over generalized loops—subgraphs with no dangling degree-one vertex. The absence of loose ends is not cosmetic: the BP gauge/reparametrization makes every term with a loose end vanish.

Reference plus correction. This chapter is an early and exact instance of a principle used repeatedly later in the book. Choose a tractable reference object, here the BP/Bethe approximation; preserve an explicit correction rather than pretending the reference is exact; then reorganize, truncate, sample, or learn the correction when the full exact sum is too expensive. The same logic will reappear in importance sampling, Feynman–Kac correction, and structured generative models.

Two derivations, one identity. Reparametrization changes how the same factors are written; gauge fixing changes the local basis used in their contraction. At a BP fixed point both can be arranged so that the zeroth-order term is exactly the BP contribution

and the surviving nonzero terms are loops. Do not read the two derivations as competing algorithms—they are complementary coordinate systems for the same exact decomposition.

How to read the later sections. First understand the ordinary BP loop series and verify that it collapses to one on a tree. Then view generalized-BP loop series as the same idea with larger reference regions. Cluster cumulants and resummations should be read as ways of reorganizing many overlapping loop terms, not as a change in the exact target. The final tensor-network discussion makes explicit that the entire construction is a structured contraction problem.

Belief propagation is exact on trees and often useful on loopy graphs, but Chapter 6 emphasized that a loopy BP fixed point is only a stationary point of the Bethe approximation. This chapter answers a sharper question: once a BP solution is available, can the exact partition function be written as the BP estimate times an explicit graphical correction? Loop calculus answers yes.

The construction has two equivalent starting points. BP reparametrization rewrites the same probability law in terms of locally consistent beliefs; BP gauge fixing chooses a basis in which every contribution containing a loose end vanishes. Expanding the remaining product leaves only subgraphs with no degree-one vertices. These factor loops or generalized loops provide exact correction terms [46, 47, 48, 215].

The chapter develops four increasingly aggregated views:

1. **loop series** in normal and ordinary factor graphs;
2. **gauge and reparametrization** derivations, including nonbinary extensions;
3. **generalized-BP loop corrections** on region hierarchies;
4. **cluster cumulants** and other resummations that organize many overlapping loops.

This is the bridge from the transformation theory of Chapter 5 and the Bethe/Kikuchi approximations of Chapter 6 to tensor-network and quantum-classical contraction methods later in the book.

Companion notebook: One cycle produces one exact correction

Notebook `ch09/ch09_01_single_cycle_loop_correction.ipynb` verifies the zero-field Ising identity $Z = Z_{\text{BP}}(1 + \prod_{e \in C} \tanh J_e)$ on a single cycle and follows the correction as the couplings grow.

9.1 Loop Series derived via Reparametrizations in NFG Representation

Let us return to discussion of Section 5.1.2 and consider specific reparametrization correspondent to BP described in Section 6.2.5, specifically by Eqs. (6.32, 6.33, 6.34).

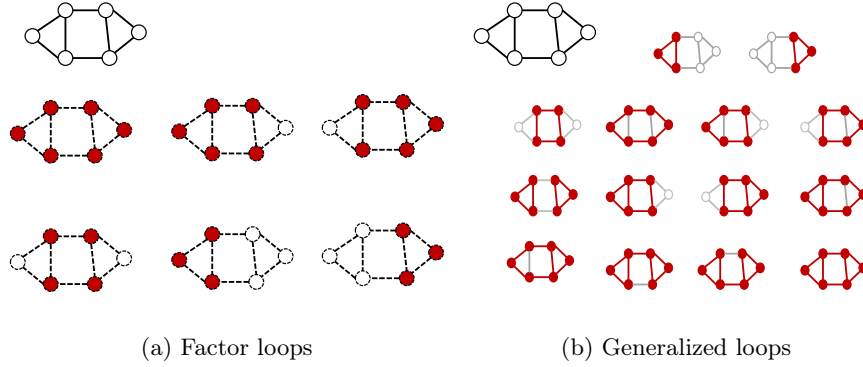


Figure 9.1: Loop Series illustrated on a NFG example. The full graph, shown in the left upper corner (of both sub-figures) generates six *factor loops*, shown in Subfigure (a) vs fourteen *generalized loops* shown in Subfigure (b). Generalized loops, correspondent to the same factor loop, have identical set of factor nodes (marked solid red in the Figure). Generalized loops and factor loops enumerate terms of the Loop Series stated in terms of gauges and reparametrizations described in Section 9.2 and Section 9.1 respectively.

Extending Eqs. (5.15,5.16,5.18) from the case of a general reparametrization to the BP-reparametrization we arrive at

$$Z = Z^{(\text{BP})} \mathcal{Z}^{(\text{BP})}, \quad (9.1) \quad \text{👉}$$

$$Z^{(\text{BP})} = \prod_{a \in \mathcal{V}} \left(\int_{\mathbf{x}_a \in \mathcal{X}_a} f_a(\mathbf{x}_a) \prod_{\alpha_d \in v(a)} R_{\alpha_d}^{(\text{BP})}(x_\alpha) \right), \quad (9.2)$$

$$\mathcal{Z}^{(\text{BP})} = \int_{\mathbf{x} \in \mathcal{X}} \left(\prod_{\alpha \in \mathcal{E}} \mathfrak{B}_\alpha^{(\text{BP})}(x_\alpha) \right) \prod_{a \in \mathcal{V}} \frac{\mathfrak{B}_a^{(\text{BP})}(\mathbf{x}_a)}{\prod_{\alpha_d \in e(a)} \mathfrak{B}_{\alpha_d}^{(\text{BP})}(x_\alpha)}, \quad (9.3)$$

where $Z^{(\text{BP})}$ is the BP-estimation for the partition function and $\mathcal{Z}^{(\text{BP})}$ is the reparameterized partition function which we also refer to in the following as the *Loop Series* derived via reparametrizations.

Let us now combine Eqs. (6.32,6.33,6.34,9.1,9.2,9.3) and represent integrand in the reparameterized partition function, $\mathcal{Z}^{(\text{BP})}$, as a product over factor node terms

$$\mathcal{Z}^{(\text{BP})} = \int_{\mathbf{x} \in \mathcal{X}} \left(\prod_{\alpha \in \mathcal{E}} \mathfrak{B}_\alpha^{(\text{BP})}(x_\alpha) \right) \prod_{a \in \mathcal{V}} \left(1 + \phi_a^{(\text{BP})}(\mathbf{x}_a) \right), \quad (9.4)$$

$$\forall a \in \mathcal{V}, \forall \mathbf{x}_a \in \mathcal{X}_a : \phi_a^{(\text{BP})}(\mathbf{x}_a) := \frac{\mathfrak{B}_a^{(\text{BP})}(\mathbf{x}_a) - \prod_{\alpha_d \in e(a)} \mathfrak{B}_{\alpha_d}^{(\text{BP})}(x_\alpha)}{\prod_{\alpha_d \in e(a)} \mathfrak{B}_{\alpha_d}^{(\text{BP})}(x_\alpha)}. \quad (9.5)$$

where each term is represented as $1 + \phi_a^{(\text{BP})}(\mathbf{x}_a)$.

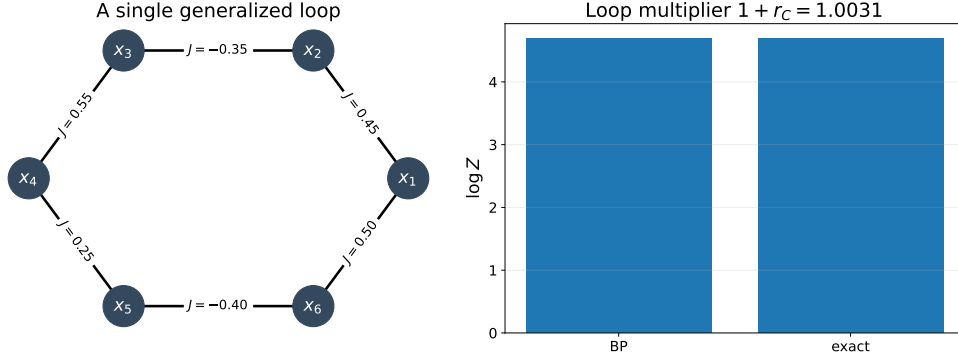


Figure 9.2: For a zero-field single cycle, the exact-to-BP ratio is one explicit loop multiplier. The graph panel labels the couplings; the bar panel verifies the corrected partition function. Generated by `ch09/ch09_01_single_cycle_loop_correction.ipynb`.

Next, we expand the product over the nodes in the integrand of Eq. (9.5) into a series. Each term in the series is associated with a coloring/markings of a node: if a node contributes unity, we say that the node is uncolored, and if the node contributes ϕ_a , we call it colored. A subset of colored factor nodes describes contribution of the respective colored term into the series.

The remarkable feature of the BP-reparametrizations (which also extends to BP-gauges) is that many of the contributions in the series vanish, i.e. are zero identically. Specifically, any contribution which has at least one colored node, linked to other colored nodes through less than two (one or none) edges, is zero identically. This feature of the series in Eq. (9.5) is a simple corollary of the BP construction. To prove it consider, first, the case of an isolated colored node, a , i.e. a node which has no link to any of the other colored nodes. Then, integration over variables associated with edges linked to the colored node results in

$$\sim \int_{\mathbf{x}_a} \left(\mathfrak{B}_a^{(\text{BP})}(\mathbf{x}_a) - \prod_{\alpha_d \in e(a)} \mathfrak{B}_{\alpha_d}^{(\text{BP})}(\mathbf{x}_{\alpha_d}) \right) = 0,$$

where we use the fact that both $\mathfrak{B}_a^{(\text{BP})}$ and $\mathfrak{B}_{\alpha_d}^{(\text{BP})}$ are normalized beliefs. (We also see immediately that presence of an isolated colored node will turn any contribution into zero even in the case of a general, i.e. not necessarily BP, gauge.) Next, consider the case of a node, a , linked to only one other colored node, b , distinct from a , via a directed edge, from a to b , α_d . Summing over variables adjusted to a , one arrives at zero again,

$$\sim \int_{\mathbf{x}_a} \frac{\left(\mathfrak{B}_a^{(\text{BP})}(\mathbf{x}_a) - \prod_{\alpha_d} \mathfrak{B}_{\alpha_d}^{(\text{BP})}(\mathbf{x}_{\alpha_d}) \right)}{\mathfrak{B}_{\alpha_d}^{(\text{BP})}(\mathbf{x}_{\alpha_d})} = \int_{\mathbf{x}_{\alpha_d}} \frac{\left(\int_{\mathbf{x}_a \setminus \mathbf{x}_{\alpha_d}} \mathfrak{B}_a^{(\text{BP})}(\mathbf{x}_a) - \mathfrak{B}_{\alpha_d}^{(\text{BP})}(\mathbf{x}_{\alpha_d}) \right)}{\mathfrak{B}_{\alpha_d}^{(\text{BP})}(\mathbf{x}_{\alpha_d})} = 0,$$

where we utilized the consistency relations (6.34) between the edge and factor beliefs valid only for BP-reparametrizations.

The resulting expression for the Loop Series (LS) derived via reparametrizations becomes

$$\mathcal{Z}^{(\text{BP})} = \frac{Z}{Z^{(\text{BP})}} = 1 + \sum_{\mathcal{V}^{(\text{f})} \in \Upsilon^{(\text{f})}} \int_{\mathbf{x} \in X} \left(\prod_{\alpha \in \mathcal{E}} \mathfrak{B}_{\alpha}^{(\text{BP})}(\mathbf{x}_{\alpha}) \right) \prod_{a \in \mathcal{V}^{(\text{f})}} \phi_a^{(\text{BP})}(\mathbf{x}_a), \quad (9.6)$$

where $\mathcal{V}^{(\epsilon)}$ is a subset of $\mathcal{V}$, $\mathcal{V}^{(\epsilon)} \subseteq \mathcal{V}$, such that for any node, a from $\mathcal{V}^{(\epsilon)}$ there exist at least two distinct directed edges starting at a and ending at nodes of $\mathcal{V}^{(\epsilon)}$, i.e. formally:

$$\forall a \in \mathcal{V}^{(\epsilon)} : \exists \alpha_d, \beta_d \in e(a), \alpha_d \neq \beta_d, v(\alpha), v(\beta) \in \mathcal{V}^{(\epsilon)}. \quad (9.7) \quad \text{📖}$$

We call valid $\mathcal{V}^{(\epsilon)}$ a factor loop of the $\mathcal{G}$, and $\Upsilon^{(\epsilon)}$ denotes the set of all possible factor loops of $\mathcal{G}$. Factor loops (six of them) are shown for an exemplary graph in Fig. 9.1a.

Two remarks about the factor based Loop Series (LS) are in order.

First, the representation of the factor loop series according to Eqs. (9.6,9.7) as a sum over loop factor-graphs $\mathcal{V}^{(\epsilon)}$, can be restated as a sum over binary vector with each component associated with a factor node. In fact, this alternative representation of the factor loops via binary variables will be used in Section 9.3.1 to describe LS derived via reparametrizations in the FG Representation

Second, the LS derived via gauges introduced in [46, 47] and discussed in the (following) Section 9.2, is different from the LS derived via reparametrizations. Specifically, LS derived via gauges will be presented in terms of an expansion over other type of loops, so-called *generalized* loops defined in terms of a subset of edges such that each node neighboring any edge from the generalized loop has at least two directed edges leaving the node which belong to the generalized loop. Therefore, relation between the factor loops and the generalized loops is as follows: any factor loop has at least one associated generalized loop. In general, the number of generalized loops associated with a given factor loop may be significant, typically growing exponentially with the size of the factor loop.

However, it is important to emphasize that this advantage of the reparametrization approach “compactness” over the gauge approach did not come for free. Indeed even though each term in the binary (regardless of the alphabet of the original GM) factor loop series is expressed in terms of the BP-solution – the expression is not loop-local. In other words complexity of computing a factor loop contribution may grow with the size of the loop significantly (faster than linearly with the loop size, and possibly exponentially). This is in contrast with the gauge transformation and generalized loop approach, we are about to discuss in the next Section, resulting in the Loop Series which can be interpreted as a partition function of a new (gauged) GM which is graph-local.

What the gauge derivation adds. In the reparametrization derivation the BP beliefs make the cancellation visible probabilistically. In the gauge derivation the same cancellation is engineered algebraically by choosing local basis transformations. A useful analogy is choosing coordinates in linear algebra so that unwanted components become orthogonal to the retained direction. Here the BP gauge makes every “open” excitation with a dangling end integrate to zero, leaving only closed graphical objects.

9.2 Loop Series derived via Gauges in NFG Representation

An alternative path to expressing the exact partition function of NFG GM is through using gauge-dependent Eqs. (5.5,5.6), with the general gauge, G , replaced by the BP gauge described by either of the following three (consistent with each other) formulas Eqs. (6.26), Eqs. (6.29) or

Eqs. (6.30). This results in

$$\mathcal{Z}^{(\text{BP})} = \frac{Z}{Z^{(\text{BP})}} = 1 + \sum_{\mathcal{G}^{(\mathfrak{g})} \subseteq \Upsilon^{(\mathfrak{g})}} \int_{\mathbf{x}^{(\mathfrak{g})} \in \mathcal{X}^{(\mathfrak{g})}} \prod_{a \in \mathcal{V}^{(\mathfrak{g})}} \frac{\int_{\mathbf{x}'_a \in \mathcal{X}'_a} f_a(\mathbf{x}'_a) \prod_{\alpha_d \in e(a)} G_{\alpha_d}(x_{\alpha_d}, x'_{\alpha_d})}{\int_{\mathbf{x}'_a \in \mathcal{X}'_a} f_a(\mathbf{x}'_a) \prod_{\alpha_d \in e(a)} G_{\alpha_d}(0, x'_{\alpha_d})}, \quad (9.8)$$

where $\mathcal{G}^{(\mathfrak{g})} = (\mathcal{V}^{(\mathfrak{g})}, \mathcal{E}^{(\mathfrak{g})})$ is a subgraph of $\mathcal{G}$ of the special *generalized loop* type defined as follows: generalized loop should contain at least one node; any node, a , of the generalized loop, $\mathcal{G}^{(\mathfrak{g})}$, has at least two directed edges leaving a , i.e. formally, $\forall a \in \mathcal{V}^{(\mathfrak{g})} \exists \alpha_d, \beta_d \in e(a), \alpha_d, \beta_d \in \mathcal{V}^{(\mathfrak{g})}, \alpha_d \neq \beta_d$. $\Upsilon^{(\mathfrak{g})}$ in Eq. (9.8) is the set of all generalized loops of $\mathcal{G}$. $\mathcal{X}^{(\mathfrak{g})}$ in Eq. (9.8) is built as a direct product of the sets of all edge variables of the respective generalized loop, $\mathcal{V}^{(\mathfrak{g})}$, however each excluding the 0/ground-state (0 state is a state picked arbitrarily as a reference when introducing gauges), i.e. $\mathcal{X}^{(\mathfrak{g})} := \cup_{\alpha \in \mathcal{E}^{(\mathfrak{g})}} (\mathcal{X}_\alpha \setminus 0)$.

Generalized loops (fourteen of them) are shown for an exemplary graph in Fig. 9.1b.

Few remarks are in order.

First, of the three equivalent representation via gauge-dependent Eqs. (6.26, 6.29, 6.30), the equation built on coloring of edges ¹, i.e. Eq. (6.29), guarantees explicitly that any subgraph/coloring of $\mathcal{G}$ which is not a generalized loop gives zero contributions to $\mathcal{Z}^{(\text{BP})}$.

Second, notice that the sums in Eq. (9.6) and in Eq. (9.8) are significantly different. It is a sum over *generalized loops* in the latter, while the summation is over the *factor loops* in the former. Any generalized loop corresponds to a factor loop, while a factor loop (as only fixing a set of nodes associated with factors) generates at least one but typically many more than one (exponential in the number of nodes) generalized loops. Note, however, that in the special case of the pairwise model (2.14) of an arbitrary alphabet the variety of possible generalized loops is reduced, and the factor loop to generalized loop relation becomes one-on-one. (The two families are isomorphic.) To prove this statement one, first of all, needs to represent pairwise model as a FG model (this is achieved by associating edges with factor nodes of the FG model) and then normalize FG model into respective NF model (see Section 2.2.1 for details). The resulting NFG model is stated over the extended bipartite graph (one part of nodes associated with original variable nodes and the other with newly formed factor nodes). Then, one observes that any factor loop of the extended bipartite graph generates a unique (one and only one) generalized loop.

Third, the factor loop contributions to Eq. (9.6) depend solely on the BP-reparametrizations, $R^{(\text{BP})}(x')$, equivalent to the uncolored, $G^{(\text{BP})}(0; x')$, BP-gauges while the generalized loop contributions in Eq. (9.8) depends explicitly on the colored gauges, i.e. $G(x \neq 0, x')$. According to Eqs. (6.29) the BP constraints fixes unambiguously the uncolored gauges, $G(0, x')$. The colored part of the BP-gauges does depend on the uncolored portion, according to the skew-orthogonality condition (5.2), however in general, when the alphabet size is larger than two, there are more conditions than number of colored BP-gauges. In this regards the case of the binary alphabet, is special, as only in this case the colored BP-gauges are fixed unambiguously within the BP-gauge procedure. This explains why we have separate discussions below of the binary alphabet and of the general q -ary alphabet (with $q > 2$).

¹We remind that description of the graph coloring can be found in Section 6.2.4.2.

9.2.1 Binary Alphabet

In the case of the binary alphabet, $\forall \alpha \in \mathcal{E}$, $|\mathcal{X}_\alpha| = 2$, each generalized loop, $\mathcal{G}^{(\mathfrak{g})}$ contributing the LS derived via gauges (9.8) generates a unique colored vector $\mathbf{x}^{(\mathfrak{g})} = (x_\alpha = 1 | \forall \alpha \in \mathcal{E}^{(\mathfrak{g})})$, so that the two sums (more accurately, one sum and one integration turned into sum) on the rhs of Eq. (9.8) can be combined in one sum over generalized loops. Then, straightforward algebraic manipulations of the terms under the sum(s) in Eq. (9.8), utilizing the BP Eqs. (6.26,6.29,6.30) results in the following explicit formula for the binary LS derived via gauges

$$\mathcal{Z}^{(\text{BP})} = \frac{Z}{\mathcal{Z}^{(\text{BP})}} = 1 + \sum_{\mathcal{G}^{(\mathfrak{g})} \subseteq \Upsilon^{(\mathfrak{g})}} r_{\mathcal{G}^{(\mathfrak{g})}}, \quad (9.9)$$

$$\forall \mathcal{G}^{(\mathfrak{g})} \subset \Upsilon^{(\mathfrak{g})} : \quad r_{\mathcal{G}^{(\mathfrak{g})}} = \frac{\prod_{a \in \mathcal{V}^{(\mathfrak{g})}} \mu_a}{\prod_{\alpha_d \in \mathcal{V}^{(\mathfrak{g})}} (1 - (m_\alpha)^2)}, \quad (9.10)$$

$$\forall \alpha \in \mathcal{E}^{(\mathfrak{g})} : \quad m_\alpha = \sum_{x_\alpha = \pm 1} \mathfrak{B}_\alpha^{(\text{BP})}(x_\alpha)(1 - 2x_\alpha) \quad (9.11)$$

$$\forall a \in \mathcal{V}^{(\mathfrak{g})} : \quad \mu_a = \sum_{\mathbf{x}_a} \mathfrak{B}_a^{(\text{BP})}(\mathbf{x}_a) \prod_{\alpha_d \in e(a)} (1 - 2x_\alpha - m_\alpha). \quad (9.12)$$

The remarkable feature of this compact expression is that if the BP Eqs. (6.26,6.29,6.30) are solved and thus the BP-beliefs are found, the complexity of evaluating any term in LS derived via gauges, $r_{\mathcal{G}^{(\mathfrak{g})}}$, is linear in the size of the generalized loop $\mathcal{G}^{(\mathfrak{g})}$.

Relation to high-temperature and dual expansions. For a zero-field binary Ising model, expanding $\exp(J_{ab}x_ax_b) = \cosh(J_{ab})(1 + x_ax_b \tanh J_{ab})$ shows that spin summation removes every edge subset with an odd-degree vertex. The surviving even subgraphs are precisely the polygon configurations of the classical high-temperature expansion. On a single cycle this leaves one correction, as in the companion notebook. Loop calculus extends this mechanism to fields, higher-order factors, and arbitrary BP beliefs: “even degree” is replaced by the more general no-loose-end cancellation. On planar lattices the polygon expansion also underlies Kramers–Wannier-type primal–dual transformations [114]; loop calculus should be viewed as a belief-adapted generalization rather than as a separate unrelated expansion.

9.2.2 q -ary Alphabet

For an alphabet of size $q > 2$, each edge has $q - 1$ nontrivial gauge directions. A loop term therefore consists of a leaf-free subgraph together with a compatible nonzero color on every selected edge. Summation over these colorings produces the *loop tower* of [40]; the binary construction is recovered when only one nontrivial color exists.

Mori’s information-geometric formulation expresses the same corrections using centered sufficient statistics, local covariance tensors, and expectation coordinates [146]. Gauge colors can be interpreted as a basis in the tangent space of the local probability simplex, while information geometry provides basis-independent contractions. For continuous alphabets the finite color sum becomes an expansion in centered functions, for example orthogonal polynomials or eigenfunctions of local covariance operators. Convergence and basis selection are then genuine analytic questions rather than finite combinatorics.

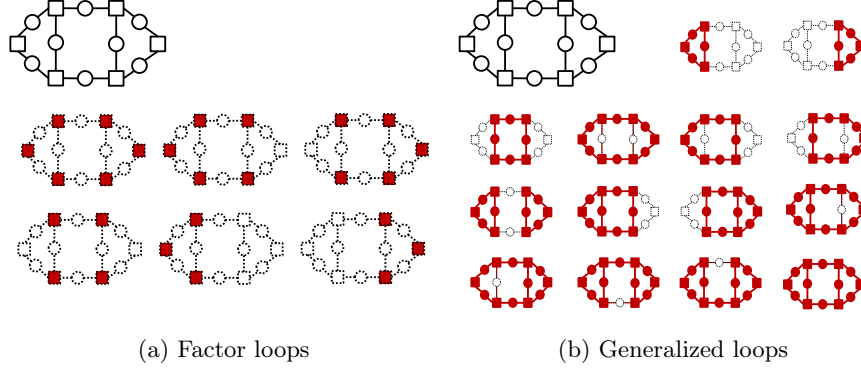


Figure 9.3: Loop series illustrated on a FG example. The full graph, shown in the left upper corner (of both sub-figures) generates six factor loops, shown in Subfigure (a) and fourteen generalized loops shown in Subfigure (b). Generalized loops, correspondent to the same reparametrization loop, have identical set of colored factors (marked red squares in the Figure). Generalized loops and factor loops enumerate terms of the Loop Series in the FG representation of gauge and reparametrization settings described in Section 9.3.1 and Section 9.3.2 respectively.

9.3 Loop Series in the Factor Graph Representations

In this Section we give a brief (and rather formal) discussion to how one extends the Loop Series from the Normal Factor Graph (NFG) representation to the Factor Graph (FG) representation.

9.3.1 Loop Series derived via BP Reparametrization in FG Representation

Let us now return back to reparametrizations in the Factor Graph (FG) representation described in Section 5.1.3 and to BP-equations described in Section 6.3.2 and combine the two approaches to derive the Loop Series expression. Specifically, we force the general reparametrization, i.e. R -, functions, introduced in Section 5.1.3, to take the special form of the BP-message, i.e. M -, functions introduced in Section 6.3.2. Formally, we replace R -functions in Eq. (5.23) by M -functions, describing relations between the marginal BP beliefs according to Eqs. (6.52, 6.53, 6.51), therefore arriving at

$$Z = Z^{(\text{BP})} \mathcal{Z}^{(\text{BP})}, \quad Z^{(\text{BP})} := \frac{\prod_{A \in \mathcal{V}_2} \left(\int_{\mathbf{x}_A \in \mathcal{X}_A} f_A(\mathbf{x}_A) \prod_{a \in \mathcal{V}; (a, A) \in \mathcal{E}} M_{A \rightarrow a}^{(\text{BP})}(x_a) \right)}{\prod_{a \in \mathcal{V}_1} \left(\int_{x_a \in \mathcal{X}_a} \prod_{A \in \mathcal{V}_2; (a, A) \in \mathcal{E}} M_{A \rightarrow a}^{(\text{BP})}(x_a) \right)}^{d_a - 1}, \quad (9.13)$$

$$\mathcal{Z}^{(\text{BP})} := \int_{\mathbf{x} \in \mathcal{X}} \left(\prod_{a \in \mathcal{V}_1} \mathfrak{B}_a^{(\text{BP})}(x_a) \right) \prod_{A \in \mathcal{V}_2} \frac{\mathfrak{B}_A^{(\text{BP})}(\mathbf{x}_A)}{\prod_{a \in \mathcal{V}_1; (a, A) \in \mathcal{E}} \mathfrak{B}_a^{(\text{BP})}(x_a)}, \quad (9.14)$$

where we use the same notations (e.g. for upper indexes and message functions) as introduced in the case of the FG model in Section 9.1.

To build the Loop Series we follow the logic of the RPT scheme introduced in Section 9.1: single out uncolored (unity in the integrand) and colored (ϕ in the integrand) contributions in $\mathcal{Z}^{(\text{BP})}$

$$\mathcal{Z}^{(\text{BP})} = \int_{\mathbf{x} \in \mathcal{X}} \left(\prod_{a \in \mathcal{V}_1} \mathfrak{B}_a^{(\text{BP})}(x_a) \right) \prod_{A \in \mathcal{V}_2} \left(1 + \phi_A^{(\text{BP})}(\mathbf{x}_A) \right), \quad (9.15)$$

$$= \sum_{\boldsymbol{\sigma} \in \{0,1\}^{|\mathcal{V}_2|}} z(\boldsymbol{\sigma}), \quad (9.16)$$

$$\forall A \in \mathcal{V}_2, \forall \mathbf{x}_A \in \mathcal{X}_A : \phi_A^{(\text{BP})}(\mathbf{x}_A) := -1 + \frac{\mathfrak{B}_A^{(\text{BP})}(\mathbf{x}_A)}{\prod_{a \in \mathcal{V}_1; (a,A) \in \mathcal{E}} \mathfrak{B}_a^{(\text{BP})}(x_a)}, \quad (9.17)$$

$$\forall \boldsymbol{\sigma} \in \{0,1\}^{|\mathcal{V}_2|} : z(\boldsymbol{\sigma}) := \int_{\mathbf{x} \in \mathcal{X}} \left(\prod_{a \in \mathcal{V}_1} \mathfrak{B}_a^{(\text{BP})}(x_a) \right) \prod_{A \in \mathcal{V}_2} \left(\phi_A^{(\text{BP})}(\mathbf{x}_A) \right)^{\sigma_A}, \quad (9.18)$$

where the binary vector $\boldsymbol{\sigma}$ is drawn from the set, $(\sigma_A = \{0,1\} | \forall A \in \mathcal{V}_2)$.

Continuing with the strategy, we observe that the series (9.16) over $\boldsymbol{\sigma}$ has many terms, $z(\boldsymbol{\sigma})$, equal to zero identically due to special properties of the BP solution. Specifically, the analysis (which is very similar to the one presented above in the case of the NFG representation, therefore not presented here) shows that the only nonzero contributions, $z(\boldsymbol{\sigma})$, in the series are associated with $\boldsymbol{\sigma}$ forming a factor loop.

Set of the factor loops are illustrated on an exemplary FG model in Fig. 9.3a.

Exercise 9.3.1. Consider the homogeneous Ising model with only pairwise interactions over a fully connected graph of

- (a) three nodes ($N = 3$, triangle), and
- (b) four nodes ($N = 4$, tetrad).

The probability distribution of the state $\mathbf{x} = (x_i = \pm 1 | i = 1, \dots, N)$ in the model is given by:

$$p(\mathbf{x}|J) = \frac{\exp\left(J \sum_{i,j=1}^N x_i x_j\right)}{Z(J)}, \quad Z(J) = \sum_{\mathbf{x}'} \exp\left(J \sum_{i,j=1}^N x'_i x'_j\right),$$

where $Z(J)$ is the partition function and J is a real-valued parameter (positive or negative).

Compute (analytically, numerically, or both):

1. The dependence of $Z(J)$ on J , the only remaining parameter beyond N .
2. Pairwise and singleton beliefs, as well as the partition function estimate within the BP approximation: $\mathfrak{B}_i^{(\text{BP})}(x_i|J)$, $\mathfrak{B}_{ij}^{(\text{BP})}(x_i, x_j|J)$, and $Z^{(\text{BP})}(J)$. Is the solution unique for all values of J ? When is $Z^{(\text{BP})}(J)$ lower/upper bound for Z ? Is the tree/BP expression for the joint probability distribution in terms of marginals exact at any values of J ?

Hint: Derive or extract the BP equations from the formulas above (in the text). The BP equations can be viewed as defining a fixed point of the Bethe free energy, imposing consistency on beliefs (considered as re-parametrizations), or as "no loose end" conditions within the gauge transformation approach.

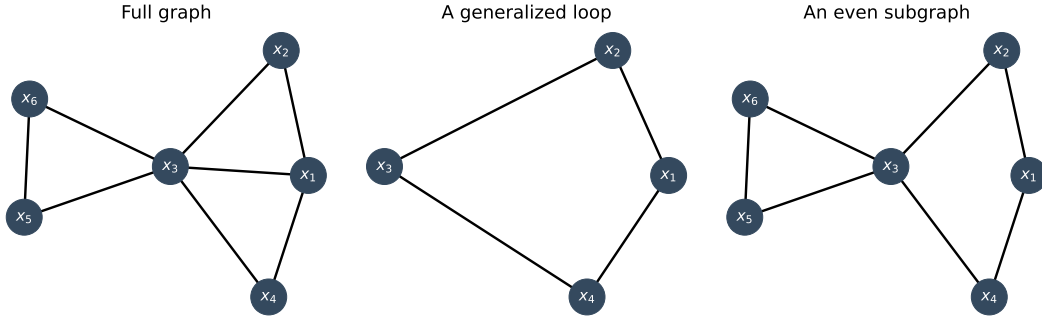


Figure 9.4: The full graph, a generalized loop, and an even subgraph. Only vertices incident to the selected edges are shown in the two subgraph panels. Loop calculus requires the leaf-free condition; the zero-field high-temperature expansion imposes the stronger even-degree condition. Generated by `ch09/ch09_02_generalized_loop_enumeration.ipynb`.

3. (Bonus) Analyze the loop series for this case: classify the generalized loops, find the contributions associated with different generalized loops as functions of J , and verify that $Z = Z^{(BP)} \times \text{Loop Series}$ holds for all J .

9.3.2 Loop Series derived via Gauges in FG Representation

Normalize the factor graph by replicating every variable occurrence and inserting an equality factor, as in Section 2.2.1. Apply the paired edge gauges of Section 5.1.1 and choose the BP gauge. Summation over an equality node contracts the colored indices associated with all copies of the original variable. Terms with only one selected incident edge vanish; hence every retained variable and factor node has degree at least two in the selected bipartite subgraph. These are precisely the generalized loops illustrated in Fig. 9.3.

The factor-node contribution is a centered moment tensor of the BP factor belief. The variable-node contribution is the corresponding centered equality tensor. Contracting these local tensors over the selected edges gives the generalized-loop weight. This is algebraically equivalent to the BP-reparametrization series in the preceding subsection, but the gauge derivation makes the tensor-network structure and the freedom of basis explicit.

Companion notebook: Leaf-free subgraphs organize exact corrections

Notebook `ch09/ch09_02_generalized_loop_enumeration.ipynb` enumerates generalized loops and even subgraphs on a small graph. It demonstrates the distinction between the general no-degree-one condition and the stronger parity condition of the zero-field high-temperature expansion.

Exercise 9.3.2. Modify the notebook to enumerate factor loops in a normal factor graph. Map each factor loop to all compatible generalized loops and compare their aggregate weights.

Changing the reference changes the residual loops. Ordinary BP treats factors and variables as the basic regions. Generalized belief propagation already absorbs selected clus-

ters into a richer reference approximation. The corresponding loop correction therefore starts *after* those regional correlations have been accounted for. The exact target is unchanged; what changes is the division between “reference” and “correction.” This is the same design freedom that later motivates choosing better tractable references before applying an exact or learned correction.

9.4 Loop Series based on Generalized Belief Propagation

We will carry on here the line started in Section 5.1.3, where the reparametrization framework associated with a hypegraph diagram of the original GM was introduced, and then extended in Section 6.6.4 discussing how to transition from a general reparametrization to the special reparametrization associated with the Generalized Belief Propagation (GBP) Eqs. (6.96, 6.97) – what we call in the following the GBP-reparametrizations.

Basic equations expressing partition function of the original GM exactly in terms of a solution of the GBP Eqs. (6.96, 6.97) (if there are more than one solution, we can pick any of them) are

$$Z = Z^{(\text{GBP})} \mathcal{Z}^{(\text{GBP})}, \quad (9.19)$$

$$Z^{(\text{GBP})} := \prod_{g \in \mathcal{E}^{(\text{h})}} \left(\int_{\mathbf{x}_g \in \mathcal{X}_g} \prod_{\underline{g} \in \text{Des}^+(g)} f_{\underline{g}}(\mathbf{x}_{\underline{g}}) \prod_{g' \in \text{par}(\underline{g}) \setminus \text{Des}^+(g)} M_{g' \rightarrow \underline{g}}^{(\text{GBP})}(\mathbf{x}_{\underline{g}}) \right)^{c_g}, \quad (9.20)$$

$$\mathcal{Z}^{(\text{GBP})} := \int_{\mathbf{x} \in \mathcal{X}} \left(\prod_{g \in \mathcal{E}^{(\text{h})}} \left(\mathfrak{B}_g^{(\text{GBP})}(\mathbf{x}_g) \right)^{c_g} \right), \quad (9.21)$$

where we change from reparametrizations, R , to messages, M (utilizing their equivalence established in Section 6.6.4). We assume that the GBP Eqs. (6.96, 6.97) are solved, and that the GBP estimate for the partition function (9.20) is found. Our further focus will be on discussing the GBP-reparameterized partition function, $\mathcal{Z}^{(\text{GBP})}$, stated in Eq. (9.21).

We will describe below an approach to resolving Eq. (9.21) which consists in generalizing approach of Section 9.1 resulting in the Loop Series derived via reparametrizations for BP to the case of GBP.

First, we modify Eq. (9.21), replacing it with the expression consistent with the alternative reparametrization Eq. (5.31)

$$\mathcal{Z}^{(\text{GBP})} = \int_{\mathbf{x} \in \mathcal{X}} \left(\prod_{g \in \mathcal{E}^{(\text{h})}} \prod_{g' \subseteq g} \left(\mathfrak{B}_{g'}^{(\text{GBP})}(\mathbf{x}_{g'}) \right)^{\mu(g', g)} \right) \quad (9.22)$$

$$= \int_{\mathbf{x} \in \mathcal{X}} \left(\prod_{a \in \mathcal{V}^{(\text{h})}} \mathfrak{B}_a^{(\text{GBP})}(\mathbf{x}_a) \right) \left(\prod_{g \in (\mathcal{E}^{(\text{h})} \setminus \mathcal{V}^{(\text{h})})} \prod_{g' \subseteq g} \left(\mathfrak{B}_{g'}^{(\text{GBP})}(\mathbf{x}_{g'}) \right)^{\mu(g', g)} \right). \quad (9.23)$$

Second, we rewrite Eq. (9.23)

$$\mathcal{Z}^{(\text{GBP})} = \int_{\mathbf{x} \in \mathcal{X}} \left(\prod_{a \in \mathcal{V}^{(\text{h})}} \mathfrak{B}_a^{(\text{GBP})}(x_a) \right) \prod_{g \in (\mathcal{E}^{(\text{h})} \setminus \mathcal{V}^{(\text{h})})} \left(1 + \phi_g^{(\text{GBP})}(\mathbf{x}_g) \right), \quad (9.24)$$

$$\forall g \in \mathcal{E}^{(\text{h})}, \forall \mathbf{x}_g \in \mathcal{X}_g : \phi_g^{(\text{GBP})}(\mathbf{x}_g) := -1 + \prod_{g' \subseteq g} \left(\mathfrak{B}_{g'}^{(\text{GBP})}(\mathbf{x}_{g'}) \right)^{\mu(g', g)}, \quad (9.25)$$

and interpret it as a series where each hyperedge, which is not a node, either contributes 1 (uncolored contribution) or $\phi_g^{(\text{GBP})}(\mathbf{x}_g)$ (colored contribution)

$$\mathcal{Z}^{(\text{GBP})} = \sum_{\boldsymbol{\sigma} \in \{0,1\}^{|\mathcal{E}^{(\text{h})} \setminus \mathcal{V}^{(\text{h})}|}} z(\boldsymbol{\sigma}), \quad (9.26)$$

$$z(\boldsymbol{\sigma}) := \int_{\mathbf{x} \in \mathcal{X}} \left(\prod_{a \in \mathcal{V}^{(\text{h})}} \mathfrak{B}_a^{(\text{GBP})}(x_a) \right) \prod_{g \in (\mathcal{E}^{(\text{h})} \setminus \mathcal{V}^{(\text{h})})} \left(\phi_g^{(\text{GBP})}(\mathbf{x}_g) \right)^{\sigma_g}, \quad (9.27)$$

where $\boldsymbol{\sigma} := (\sigma_g = 0, 1 | \forall g \in (\mathcal{E}^{(\text{h})} \setminus \mathcal{V}^{(\text{h})}))$. Notice that $z(\mathbf{0}) = 1$.

Third, like in the case of the related analysis conducted above for $\mathcal{Z}^{(\text{BP})}$, we expect that many (but far from all) contributions to the GBP-based σ -series in Eq. (9.26) return zero due to the special properties of the GBP. In fact, generalizing Eq. (9.18) we are able to make the following statement.

Theorem 9.4.1. *We say that a hyperedge is degree one, if all of its children are (super)nodes. $z(\boldsymbol{\sigma})$, defined in Eq. (9.27), is zero, $z(\boldsymbol{\sigma}) = 0$, if there exists a colored superedge of degree one, $\exists g \in \text{Par}(\mathcal{V}^{(\text{h})})$ where $\sigma_g = 1$, such that for all of its children (which are nodes), but possibly one, there exists no other colored superedge, g' , they contribute to with $\sigma_{g'} = 1$.*

Proof. To prove the Theorem 9.4.1 it suffices, first, to prove it for an isolated colored hyperedge of degree one, i.e. hyperedge of degree one which does not share any node with any other colored hyperedge, and, second, to prove it for a colored hyperedge of degree one which shares exactly one node with another colored hyperedge (the latter may be of any degree).

In the first case of an isolated colored hyperedge, $g \in \mathcal{E}^{(\text{h})} \setminus \mathcal{V}^{(\text{h})}$, so far of an arbitrary degree, multiplicative contribution to the respective $z(\boldsymbol{\sigma})$ associated with the integration/summation over $\mathbf{x}_g$ is

$$\begin{aligned} & \propto \int_{\mathbf{x}_g \in \mathcal{X}_g} \left(\prod_{a \in g} \mathfrak{B}_a^{(\text{GBP})}(x_a) \right) \left(-1 + \prod_{g' \subseteq g} \left(\mathfrak{B}_{g'}^{(\text{GBP})}(\mathbf{x}_{g'}) \right)^{\mu(g', g)} \right) \\ & = -1 + \int_{\mathbf{x}_g \in \mathcal{X}_g} \left(\prod_{a \in g} \mathfrak{B}_a^{(\text{GBP})}(x_a) \right) \left(\prod_{g' \subseteq g} \left(\mathfrak{B}_{g'}^{(\text{GBP})}(\mathbf{x}_{g'}) \right)^{\mu(g', g)} \right). \end{aligned} \quad (9.28)$$

In general, i.e. when g is a hyperedge of degree higher than one (i.e. it is separated from at least one node by at least two generations) Eq. (9.28) is not zero. However, when g is a hyperedge of degree one $\prod_{g' \subseteq g} \left(\mathfrak{B}_{g'}^{(\text{GBP})}(\mathbf{x}_{g'}) \right)^{\mu(g', g)}$ reduces to $\left(\mathfrak{B}_g^{(\text{GBP})}(\mathbf{x}_g) \right) / \left(\prod_{a \in g} \mathfrak{B}_a^{(\text{GBP})}(x_a) \right)$ and

then the integral/sum on the right hand side of Eq. (9.28) becomes, $\int_{\mathbf{x}_g \in \mathcal{X}_g} \mathfrak{B}_{g'}^{(\text{GBP})}(\mathbf{x}_{g'}) = 1$,

then resulting in the cancellation, $z(\boldsymbol{\sigma}) = 0$. Notice that the same cancellation will be observed for any other reparametrizations over the multigraph and not only in the case of the GBP-reparametrizations.

Consider now the second case correspondent to a colored hyperedge of degree sharing exactly one node, say node b , with some other hyperedge. Integating/summing over $\mathbf{x}_g \setminus x_b$ one arrives at the following contribution to the respective $z(\boldsymbol{\sigma})$

$$\begin{aligned} & \propto \int_{\mathbf{x}_g \setminus x_b} \left(\prod_{a \in g} \mathfrak{B}_a^{(\text{GBP})}(x_a) \right) \left(-1 + \frac{\mathfrak{B}_g^{(\text{GBP})}(\mathbf{x}_g)}{\prod_{a \in g} \mathfrak{B}_a^{(\text{GBP})}(x_a)} \right) F(x_b, \mathbf{x} \setminus \mathbf{x}_g) \\ & = \int_{x_b} \left(-\mathfrak{B}_b^{(\text{GBP})}(x_b) + \int_{\mathbf{x}_g \setminus x_b} \mathfrak{B}_g^{(\text{GBP})}(\mathbf{x}_g) \right) F(x_b, \mathbf{x} \setminus \mathbf{x}_g), \end{aligned} \quad (9.29)$$

where F aggregates dependency on the components of $\mathbf{x}$ other than $\mathbf{x}_g \setminus x_b$. We observe that Eq. (9.29) is obviously zero for any (unspecified) F , because the expression under parenthesis in the integrand of Eq. (9.29) is zero due to GBP consistency condition for the marginal beliefs associated with x_b . $\square$

Why resum loops into clusters? The literal loop series can contain many strongly overlapping terms, so truncating by the number or size of individual loops may be a poor approximation. A cluster cumulant groups correlated corrections and assigns to a cluster only the part not already explained by its proper subclusters. This is analogous to connected-correlation or cumulant expansions in probability and statistical mechanics: reorganize the exact information so that truncation follows collective structure rather than raw term count.

9.5 Cluster Cumulant Expansion

In this Section we follow the ideas originally suggested in [211].

We split our discussion in two. First, in Section 9.5.1 we discuss construction of the Cumulant Expansion (CE) based on the Belief Propagation (BP) solution. Second, we generalize it in Section 9.5.2 and present CE based on the Generalized Belief Propagation (GBP).

9.5.1 CCE based on Belief Propagation

In this Subsection we get back to the discussion of $\mathcal{Z}^{(\text{BP})}$, that is the BP-renormalized partition function (which we also call Loop Series) derived in Section 9.3.1 and build, following [211], *Cluster Cumulant Expansion* (CCE) approximating $\mathcal{Z}^{(\text{BP})}$.

Consider collection of subsets of factor nodes of $\mathcal{G}$, which we denote with $g = (A_1, A_2, \dots)$, $g \subset \mathcal{V}_2$, and define a partially ordered set (poset), $\mathcal{E}^{(h)}$, of such collections, $\mathcal{E}^{(h)} = (g_1, g_2, \dots)$, where relations between the collections is via inclusion, i.e. $g_1 \leq g_2$ iff $g_1 \subseteq g_2$, and $g_1 < g_2$ iff $g_1 \subset g_2$. Let us also associate with each collection, g , respective subgraph of $\mathcal{G}$, $\mathcal{G}_g := (\mathcal{V}_{1;g}, \mathcal{V}_{2;g}, \mathcal{E}_g)$, where $\mathcal{V}_{2;g} := (A \in g)$, $\mathcal{V}_{1;g} := (a \in \mathcal{V}_1 \mid \text{s.t. } A \in g, (a, A) \in \mathcal{E})$, and $\mathcal{E}_g := ((a, A) \in \mathcal{E} \mid \text{s.t. } A \in$

$\mathcal{V}_2$, $a \in \mathcal{V}_1$). Finally, for each collection we introduce respective BP-reparameterized partition function:

$$\mathcal{Z}_g^{(\text{BP})} := \int_{\mathbf{x}_g \in \mathcal{X}_g} \left(\prod_{a \in \mathcal{V}_{1;g}} \mathfrak{B}_a^{(\text{BP})}(x_a) \right) \prod_{A \in \mathcal{V}_{2;g}} \frac{\mathfrak{B}_A^{(\text{BP})}(\mathbf{x}_A)}{\prod_{a \in \mathcal{V}_{1;g}; (a,A) \in \mathcal{E}_g} \mathfrak{B}_a^{(\text{BP})}(x_a)}, \quad (9.30)$$

where $\mathcal{X}_g := \cup_{A \in \mathcal{V}_{2;g}} \mathcal{X}_A$. Note that for $g = \mathcal{V}_2$, we recover Eq. (9.14), i.e. $\mathcal{Z}_{\mathcal{V}_2} = \mathcal{Z}^{(\text{BP})}$.

It will be convenient (for what follows) to restate Eq. (9.30) in the hypergraph terms:

$$\forall g \in \mathcal{E}^{(\text{h})} : \quad \mathcal{Z}_g^{(\text{BP})} = \int_{\mathbf{x}_g \in \mathcal{X}_g} \prod_{g' \subseteq g} \prod_{g'' \subseteq g'} \left(\mathfrak{B}_{g''}^{(\text{BP})}(\mathbf{x}_{g''}) \right)^{\mu(g'', g')}. \quad (9.31)$$

(Note that derivation of this expression is equivalent to the derivation which led us to Eq. (9.22) in the case of GBP and $g = \mathcal{V}_2$.)

Looking at $\mathcal{Z}_g^{(\text{BP})}$ from the prospective of the Loop Calculus, the following question seems natural: Are there contributions, $\mathcal{Z}_g^{(\text{BP})} (\log \mathcal{Z}_g^{(\text{BP})})$, which are unity (zero)? And indeed, the following statement answers this question affirmatively.

Theorem 9.5.1. *Any g with $\mathcal{G}_g$ which is a tree (a single-connected graph) returns zero for $\log \mathcal{Z}_g^{(\text{BP})}$.*

Proof. If $\mathcal{G}_g$ is a tree then

$$P(\mathbf{x}_g) = \left(\prod_{a \in \mathcal{V}_{1;g}} \mathfrak{B}_a^{(\text{BP})}(x_a) \right) \prod_{A \in \mathcal{V}_{2;g}} \frac{\mathfrak{B}_A^{(\text{BP})}(\mathbf{x}_A)}{\prod_{a \in \mathcal{V}_{1;g}; (a,A) \in \mathcal{E}_g} \mathfrak{B}_a^{(\text{BP})}(x_a)},$$

is a valid probability distribution, thus the normalization, i.e. $\mathcal{Z}_g^{(\text{BP})}$, is unity and therefore the result follows. $\square$ $\square$

Moreover, the theorem also results in a number of useful corollaries:

- $\log(\mathcal{Z}_g^{(\text{BP})})$ is zero if g consists of a single factor node.
- If $\mathcal{G}_g$ contains “dangling” branches than these can be removed from the considerations (marginalized over), i.e. $\mathcal{Z}_g^{(\text{BP})} = \mathcal{Z}_{g'}^{(\text{BP})}$, where $\mathcal{G}_{g'}$ is derived from $\mathcal{G}_g$ removing the dangling edges.

The latter statement suggests to keep in our analysis only g associated with the “core subgraphs” of $\mathcal{G}$, i.e. the factor loops.

Next step in the CCE consists in analyzing relations between $\log(\mathcal{Z}_g^{(\text{BP})})$ and respective cumulants, C_g . We consider logarithms of $\mathcal{Z}_g^{(\text{BP})}$ and represent them, by analogy with Kikuchi approach to entropy in statistical physics [103], also discussed in Section 6.6.2, as the ζ -function type of series over the respective cumulants

$$\forall g \in \mathcal{E}^{(\text{h})} : \quad \log \left(\mathcal{Z}_g^{(\text{BP})} \right) = \sum_{g' \subseteq g} C_{g'} = \sum_{g' \subseteq g} \zeta(g', g) C_{g'}, \quad (9.32)$$

where $\zeta(g', g)$ is an element of the ζ -matrix (function) defined in Eq. (4.14). Eq. (9.32) should be viewed as defining the cumulants. Möbius inversion of Eq. (9.32) results in

$$\forall g \in \mathcal{E}^{(h)} : \quad C_g = \sum_{g' \subseteq g} \mu(g', g) \log \mathcal{Z}_{g'}^{(\text{BP})}, \quad (9.33)$$

where the μ -terms are elements of the Möbius matrix defined in Eq. (4.15).

Given the definition of cumulants Eq. (9.32) and all the factor loop considerations above we arrive at the following statements (Theorem 3 of [211]):

Theorem 9.5.2. *Cluster cumulant of a superedge g returns zero, i.e. $C_g = 0$, if its factor sub-graph, $\mathcal{G}_g$, is singly connected (i.e. forms a tree).*

Proof. It follows from Eq. (9.33) that C_g is a linear combination of $\log \mathcal{Z}_g^{(\text{BP})}$ and $\log \mathcal{Z}_{g'}^{(\text{BP})}$, where $g' < g$. Using Theorem 9.5.1 together with the fact that every sub-graph of a tree is a smaller tree (or a forest of trees) and that at the lowest level of the poset we have $C_{g'} = \mathcal{Z}_{g'}^{(\text{BP})}$, the result follows by induction. $\square$

Essence of the BP-based cumulant expansion consists in approximating $\log \mathcal{Z}^{(\text{BP})}$ as follows

$$\log \mathcal{Z}^{(\text{BP})} \approx \sum_{g \in \mathcal{E}^{(h)}} C_g = \sum_{g \in \mathcal{E}^{(h)}} \sum_{g' \subseteq g} \mu(g', g) \log \mathcal{Z}_{g'}^{(\text{BP})}, \quad (9.34)$$

that is ignoring cumulants associated with hyperedges larger than contained in $\mathcal{E}^{(h)}$.

Why is it a good idea to drop/ignore cumulants associated with larger sub-graphs? We come up with this suggestion, following the authors of [211], based on the following chain of heuristic arguments (then resulted in theorems):

- Möbius and Kikuchi transformation, originally from statistical mechanics [194, 117, 147], and then resulted in Generalized Belief Propagation [220], were built based on the intuition of perturbative analysis of a complex (multi-particle) systems in the regime of weak coupling of parts (sub-systems) applied to a “logarithm of a sum (or integral)” type of object, like free energy and log partition function.
- We have seen similar intuition embedded in empirical analysis of different contributions in the Loop Series and methods developed to re-sum the Loop Series, e.g. explored in [80, 78]. If BP provides a reasonable starting approximation then we expect that the larger the graph in the Loop Series the smaller contribution it provides to the respective partition function. Indeed, and exaggerating the story a bit, if a basic element of a graph, node or edge, contributes ϵ to a term in the Loop Series associated with a graph containing the element, then we expect to get ϵ^M for a graph consisting of M such (or similar) elements. That is the contribution decreases with increase in M if ϵ is sufficiently small. (Note, that for this argument to work ϵ may need to be much smaller than unity to overcome the combinatorial factor, associated with the growth in the number of similar $O(\epsilon^M)$ contributions.)
- We may expect that at least some contributions to $\mathcal{Z}^{(\text{BP})}$ (i.e. to the Loop Series) from a large subgraph may be decomposed into a product of contributions associated with weakly connected components of the subgraph. For example, consider a case when the

Loop Series only contains four contributions – (a) unity contribution, associated with the BP itself, (b) a contribution, z_A , associated with a subgraph, A , build from disconnected or weakly coupled loops, A_1 and A_2 , which can thus be split (exactly or approximately) into product, $\approx z_{A_1} z_{A_2}$; and (c) contributions, z_{A_1} and z_{A_2} , associated with A_1 and A_2 , respectively. Then, we estimate for the LS, $\mathcal{Z}^{(\text{BP})} = 1 + z_A + z_{A_1} + z_{A_2} \approx 1 + z_{A_1} z_{A_2} + z_{A_1} + z_{A_2} \approx (1 + z_{A_1})(1 + z_{A_2})$, where it was assumed that $z_{A_1}, z_{A_2} \ll 1$. When we restate this estimate in terms of the log-partition functions associated with A , A_1 and A_2 , discussed in this Subsection, we arrive at, $\log \mathcal{Z}^{(\text{BP})} = \log \mathcal{Z}_A^{(\text{BP})} \approx \log(\mathcal{Z}_{A_1}^{(\text{BP})}) + \log(\mathcal{Z}_{A_2}^{(\text{BP})})$. Moreover, we observe that the cumulant associated with A , vanishes, i.e. $C_A = 0$, while $C_{A_1}, C_{A_2} \neq 0$. Mathematical version (thus with proofs) of the last argument leads to the following statement (Theorem 4 of [211]).

Theorem 9.5.3. *Cluster cumulant of a superedge g returns zero, i.e. $C_g = 0$, if its factor sub-graph, $\mathcal{G}_g$, is disconnected.*

Proof. If the graph $\mathcal{G}_g$ is disconnected into $\mathcal{G}_{g_1}$ and $\mathcal{G}_{g_2}$ its probability factorizes, as well as the respective partial (reparameterized) partition function, $\mathcal{Z}_g^{(\text{BP})} = \mathcal{Z}_{g_1}^{(\text{BP})} \mathcal{Z}_{g_2}^{(\text{BP})}$. Then, Eq. (9.32) results in

$$\begin{aligned} \log \mathcal{Z}_g^{(\text{BP})} &= \sum_{g' < g_1} C_{g'} + \sum_{g' < g_2} C_{g'}, \\ C_g &= \log \mathcal{Z}_g^{(\text{BP})} - \sum_{g' < g} C_{g'} \\ &= \sum_{g' < g_1} C_{g'} + \sum_{g' < g_2} C_{g'} - \sum_{g' < g} C_{g'}. \end{aligned}$$

Clusters $g' < g$ in the poset fall into one of three categories: either (a) $C_{g'} \leq C_{g_1}$, or (b) $C_{g'} \leq C_{g_2}$, or (c) $(C_{g'} > C_{g_1}) \wedge (C_{g'} > C_{g_2}) \wedge (C_{g'} < C_g)$. We now proceed by induction. In the cases (a) and (b) the cumulants will cancel in the expression for C_g . Sufficiently small clusters will have no cumulants in the case (c), and so must be zero too. Larger clusters cumulants in the case (c) correspond to smaller clusters that must also be disconnected, which by the inductive arguments must vanish. $\square$

Using CCE to approximate the reparameterized partition function, $\mathcal{Z}^{(\text{BP})}$, is additionally reinforced by the following consideration (Theorem 5 of [211]). Consider a poset $\mathcal{E}^{(h)}$ that includes a factor A with unit potential, i.e. $f_A = 1$, and a cluster g that contains factor, A . Assume also that there exists another cluster, $g' < g$, containing the same set of factors as g , however excluding A , i.e. $g' = g \setminus A$. Then, $C_g = 0$. Importance of this statement becomes obvious when we consider factors which are approximately independent of the respective variables, i.e. $f_A \approx 1$. In this case we expect, by the continuity argument, that $C_g \approx 0$, if cluster, $g' = g \setminus A$ is in the poset. Hence, a cluster-cumulant will only significantly differ from zero if it introduces new dependencies that were not already captured by lower order cumulants. This result strengthens the interpretation of CCE as an expansion over orders of statistical dependency.

Example 9.5.4. *Illustrate construction of the BP-based cumulant truncation schemes of different levels on example of a small graph.*

Solution. *Consider example of a graph and its set of generalized loops illustrated in Fig. 9.5b. We show in Fig. 9.5a respective set of the factor loops, which we choose as hyperedges in the*

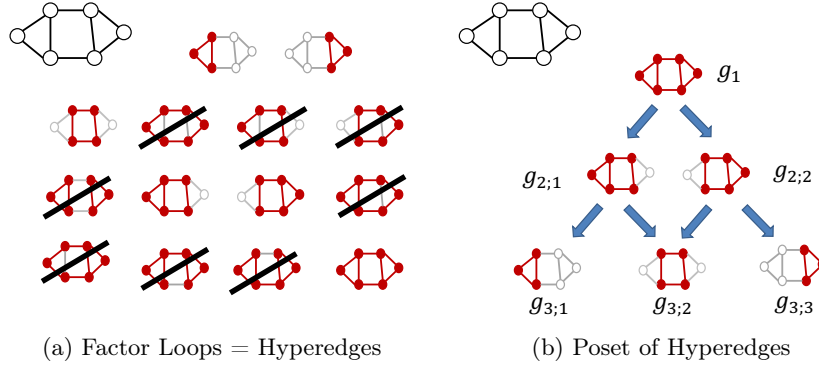


Figure 9.5: Construction of cumulants is illustrated on an exemplary poset. Subfigure (a) shows factor loops (not crossed) which form a subset of generalized loops. Factor loops may be selected as hyperedges in the poset used in the Cluster Cumulant Expansion based on BP. Subfigure (b) shows relation between the hyperedges within the poset.

poset. Fig. 9.5b illustrates the poset. We can express partial partition functions, $\mathcal{Z}_g^{(BP)}$, associated with the factor loops, g via respective cumulants, C_g :

$$\begin{aligned} \log \mathcal{Z}_{g_{3,1}}^{(BP)} &= C_{g_{3,1}}, \quad \log \mathcal{Z}_{g_{3,2}}^{(BP)} = C_{g_{3,2}}, \quad \log \mathcal{Z}_{g_{3,3}}^{(BP)} = C_{g_{3,3}}, \\ \log \mathcal{Z}_{g_{2,1}}^{(BP)} &= C_{g_{2,1}} + C_{g_{3,1}} + C_{g_{3,2}}, \quad \log \mathcal{Z}_{g_{2,2}}^{(BP)} = C_{g_{2,2}} + C_{g_{3,2}} + C_{g_{3,3}}, \\ \log \mathcal{Z}_{g_1}^{(BP)} &= C_{g_1} + C_{g_{2,1}} + C_{g_{2,2}} + C_{g_{3,1}} + C_{g_{3,2}} + C_{g_{3,3}}. \end{aligned}$$

Then the respective Möbius inversion, expressing the cumulants via the partial partition functions, becomes

$$\begin{aligned} C_{g_{3,1}} &= \log \mathcal{Z}_{g_{3,1}}^{(BP)}, \quad C_{g_{3,2}} = \log \mathcal{Z}_{g_{3,2}}^{(BP)}, \quad C_{g_{3,3}} = \log \mathcal{Z}_{g_{3,3}}^{(BP)}, \\ C_{g_{2,1}} &= \log \mathcal{Z}_{g_{2,1}}^{(BP)} - \log \mathcal{Z}_{g_{3,1}}^{(BP)} - \log \mathcal{Z}_{g_{3,2}}^{(BP)}, \quad C_{g_{2,2}} = \log \mathcal{Z}_{g_{2,2}}^{(BP)} - \log \mathcal{Z}_{g_{3,2}}^{(BP)} - \log \mathcal{Z}_{g_{3,3}}^{(BP)}, \\ C_{g_1} &= \log \mathcal{Z}_{g_1}^{(BP)} - \log \mathcal{Z}_{g_{2,1}}^{(BP)} - \log \mathcal{Z}_{g_{2,2}}^{(BP)} + \log \mathcal{Z}_{g_{3,2}}^{(BP)}. \end{aligned}$$

9.5.2 CCE based on Generalized Belief Propagation

Eq. (9.22) for the GBP-reparameterized partition function, $\mathcal{Z}^{(GBP)}$, is the starting point here. Inspired by the product-factorized form of the integrand in these equations, and extending the CCE approach based on BP (described in the preceding Subsection), we introduce the partition function associated with each individual hyperedge, g ,

$$\mathcal{Z}_g^{(GBP)} := \int_{\mathbf{x}_g \in \mathcal{X}_g} \prod_{g' \subseteq g} \prod_{g'' \subseteq g'} \left(\mathfrak{B}_{g''}^{(GBP)}(\mathbf{x}_{g''}) \right)^{\mu(g'', g')}. \quad (9.35)$$

By analogy with hyperedges associated with factor loops used to build CCE based on BP, described in Section 9.5.2, we would like to extend the poset, $\mathcal{E}^{(h)}$, of hyperedges used to construct GBP, into $\mathcal{E}^{(h+)}$, $\mathcal{E}^{(h)} \subset \mathcal{E}^{(h+)}$ adding some more complex/larger hyperedges. Following

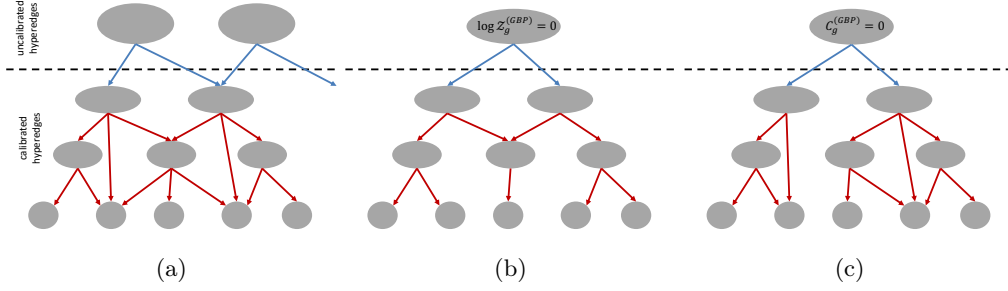


Figure 9.6: CCE of a region graph. Gray regions are hyperedges. Red arrows show relations within the poset. (a) Separation into calibrated hyperedges, i.e. accounted for in GBP, (above dashed line) and uncalibrated hyperedges which are added to CCE (below dashed line). Any uncalibrated hyperedge is a parent to calibrated hyperedges it contains. (b) Tree-structured uncalibrated hyperedge, g , for which, $\log Z_g^{(GBP)} = 0$. (c) Uncalibrated hyperedge, g , for which respective region graph, $\mathcal{G}_g$, is disconnected, then $C_g^{(GBP)} = 0$.

[211], we call hyperedges, $g \in \mathcal{E}^{(h)}$, *calibrated*, and hyperedges, $g \in \mathcal{E}^{(h+)} \setminus \mathcal{E}^{(h)}$, *uncalibrated*. The construction is illustrated in Fig. (9.6).

Like in the case of CCE based on BP we will work with cumulants, defined by the following GBP-versions of Eqs. (9.32) and its inverse Eq. (9.33):

$$\forall g \in \mathcal{E}^{(h+)} : \log \left(Z_g^{(GBP)} \right) = \sum_{g' \subseteq g} C_{g'}^{(GBP)} = \sum_{g' \subseteq g} \zeta(g', g) C_{g'}^{(GBP)}, \quad (9.36)$$

$$\forall g \in \mathcal{E}^{(h+)} : C_g^{(GBP)} = \sum_{g' \subseteq g} \mu^{(GBP)}(g', g) \log Z_{g'}^{(GBP)}. \quad (9.37)$$

We will also use the same strategy to approximate the GBP reparameterized partition function, $Z^{(GBP)}$, thus replacing Eq. (9.34) by

$$\log Z^{(GBP)} \approx \sum_{g \in \mathcal{E}^{(h+)}} C_g^{(GBP)} = \sum_{g \in \mathcal{E}^{(h+)}} \sum_{g' \subseteq g} \mu^{(GBP)}(g', g) \log Z_{g'}^{(GBP)}. \quad (9.38)$$

Notice, however, that even though the strategy is the same (as for CCE based on BP solution) the specific values of the $\mu^{(GBP)}(g', g)$ coefficient may be different from $\mu(g', g)$, even for g , and thus $g' \leq g$, which are calibrated. Similar to how it worked in the BP-based case, there are some cancellations (i.e. zeros) for the respective, $C_g^{(GBP)}$, coefficients illustrated in Fig. (9.6) and detailed in the following theorem (aggregate of Theorem 5 and Theorem 6 from [211]) which we present here without proof.

Theorem 9.5.5. Define a subgraph, $\mathcal{E}_g$ associated with descendants of g which are in $\mathcal{E}^{(h)}$, i.e. $\mathcal{E}_g = (g' < g | g \in \mathcal{E}^{(h)})$, and the counting numbers $C^{(GBP)}(g', g)$ for superedges $g' \in \mathcal{E}_g$:

$$C^{(GBP)}(g', g) = 1 - \sum_{g'' \in \mathcal{E}_g \cap \text{an}(g')} C^{(GBP)}(g'', g).$$

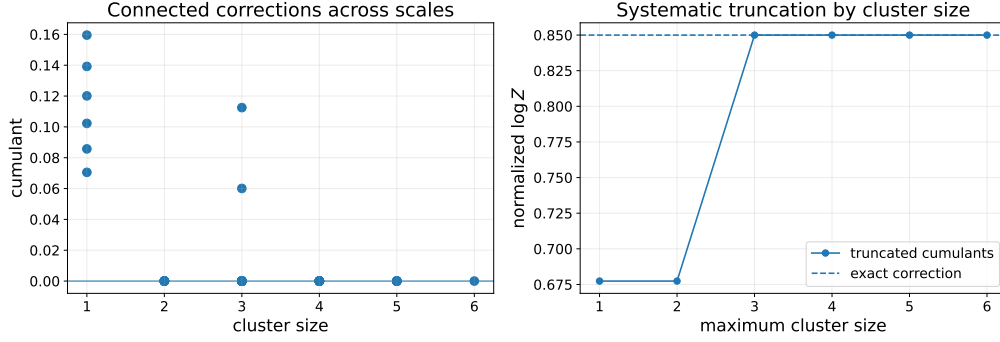


Figure 9.7: Cluster cumulants separate connected corrections across scales. The right panel illustrates systematic truncation by maximum cluster size; adaptive truncations may instead rank clusters by estimated contribution. Generated by `ch09/ch09_03_cluster_cumulants.ipynb`.

1. If hyperedges contributing to $\mathcal{E}_g$ show $C^{(GBP)}(g', g) \neq 0$ and $\mathcal{E}_g$ is singly connected (i.e. if it forms a directed tree as shown in Fig. 9.6b), then the GBP partial log partition function of g and the respective cumulant are zero, $\log \mathcal{Z}_g^{(GBP)} = C_g^{(GBP)} = 0$.
2. If hyperedges contributing to $\mathcal{E}_g$ show $C^{(GBP)}(g', g) \neq 0$ and $\mathcal{E}_g$ is disconnected (as shown in Fig. 9.6c), then the GBP-cumulant of g is zero, $C_g^{(GBP)} = 0$.

Choosing calibrated and correction regions. The calibrated hierarchy $\mathcal{E}^{(h)}$ should contain structures whose internal inference is affordable and whose strongest short loops are thereby absorbed into GBP. Candidate maximal regions can be selected from chordless cycles, high-mutual-information clusters, tensor-contraction bottlenecks, or application-specific physical motifs. The correction hierarchy $\mathcal{E}^{(h+)}$ should then prioritize unions of calibrated regions that are connected, multiply connected, and likely to have large residual dependence. Size alone is a poor ordering criterion: a large weakly coupled cluster may contribute less than a small frustrated one. Practical adaptive schemes can rank candidates using boundary correlations, preliminary cumulant estimates, or disagreement between overlapping regional beliefs. Guaranteeing a controlled remainder while avoiding combinatorial explosion remains open.

Companion notebook: Möbius inversion isolates connected cluster corrections

Notebook `ch09/ch09_03_cluster_cumulants.ipynb` computes exact subgraph partition functions for two overlapping cycles, performs Möbius inversion, verifies cancellation for a disconnected cluster, and studies truncation by cluster size.

9.6 Tree-Re-Weighting as Re-Parametrization(s)

The TRW approach described in Section 6.4, resulted in the upper bound for the partition function, which can be presented as a product over a set of spanning trees

$$Z \leq Z^{(TRW)} = \prod_{T \in \mathcal{T}} \left(Z^{(T)}(\theta^{(*;T)}) \right)^{\rho^{(T)}}, \quad (9.39)$$

$$Z^{(T)}(\theta^{(*;T)}) := \int_x \prod_{A \in \mathcal{V}_2, T} (f_A(\mathbf{x}_A))^{\theta_A^{(*;T)}}, \quad (9.40)$$

where the set of spanning trees $\mathcal{T}$ is sufficiently large (to guarantee that $\text{supp}(\rho) = \mathcal{V}_2$) but finite; ρ is either the optimal solution of Eq. (6.83), or at least a valid solution from the interior of the spanning tree polytope, described by conditions in Eq. (6.83) (to guarantee upper bound); and details on how to compute $\theta^{(*)}$, as well as implicitly described $\{\hat{\mathfrak{B}}_A\}$ and $\{\hat{\mathfrak{B}}_a\}$, were also provided in Section 6.4. Notice, that the two expressions on the right hand side of Eqs. (9.40) describe two alternative representations of the spanning tree partition function.

An important observation, which will allow us to relate the exact partition function to the TRW approach, starts from expressing the joint probability distribution of the original loopey GM via the probability distribution of the GM projected to a particular spanning tree, T ,

$$\begin{aligned} P(\mathbf{x}) &= \frac{\prod_{A \in \mathcal{V}_2} f_A(\mathbf{x}_A)}{Z} = P^{(T)}(\mathbf{x}|\theta^{(*;T)}) \frac{Z^{(T)}(\theta^{(*;T)})}{Z} \prod_{A \in \mathcal{V}_2} (f_A(\mathbf{x}_A))^{1-\theta^{(*;T)}} \\ &= \frac{\prod_{A \in \mathcal{V}_2, T} \hat{\mathfrak{B}}_A(\mathbf{x}_A|\theta^{(*;T)})}{\prod_{a \in \mathcal{V}_1} \left(\hat{\mathfrak{B}}_a(\mathbf{x}_a|\theta^{(*;T)}) \right)^{q_a^{(T)}-1}} \frac{Z^{(T)}(\theta^{(*;T)})}{Z} \prod_{A \in \mathcal{V}_2} (f_A(\mathbf{x}_A))^{1-\theta^{(*;T)}}, \end{aligned} \quad (9.41)$$

where dependence on $\theta^{(*;T)}$ emphasizes conditioning to the respective optimal value. Summing/integrating Eq. (9.41) over $\mathbf{x}$, and taking into account that $P(\mathbf{x})$ is properly normalized we arrive at the following relation between Z and $Z^{(T)}$

$$\begin{aligned} Z &= Z^{(T)}(\theta^{(*;T)}) \int_x \frac{\prod_{A \in \mathcal{V}_2, T} \hat{\mathfrak{B}}_A(\mathbf{x}_A|\theta^{(*;T)})}{\prod_{a \in \mathcal{V}_1} \left(\hat{\mathfrak{B}}_a(\mathbf{x}_a|\theta^{(*;T)}) \right)^{q_a^{(T)}-1}} \prod_{A \in \mathcal{V}_2} (f_A(\mathbf{x}_A))^{1-\theta^{(*;T)}} \\ &= Z^{(T)}(\theta^{(*;T)}) \mathbb{E}_{P^{(T)}(\cdot|\theta^{(*;T)})} \left[\prod_{A \in \mathcal{V}_2} (f_A(\mathbf{x}_A))^{1-\theta^{(*;T)}} \right]. \end{aligned} \quad (9.42)$$

The relation was derived via sequence of transformations of the Re-Parametrization type, set up in Section 5.1.3, and therefore it allows to interpret $Z^{(T)}$ as an approximation for Z linked to the spanning tree Re-Parametrization, and then the remaining multiplicative factor in Eq. (9.42), defined as an expectation over the spanning tree probability distribution, as a correction/mismatch.

Two remarks are in order. First, the derivation just completed can be generalized to any $\theta^{(T)}$, which is not necessarily the optimal one, $\theta^{(*;T)}$. Second, in general, $Z^{(T)}$ does not upper bound Z .

According to what was proven in Section 6.4, $Z^{(TRW)}$, of Eq. (9.39) provides an upper bound for Z , therefore suggesting the following natural generalization of Eq. (9.42)

$$\begin{aligned} Z &= \prod_{T \in \mathcal{T}} Z^{\rho(T)} = \prod_{T \in \mathcal{T}} \left(Z^{(T)}(\theta^{(*;T)}) \mathbb{E}_{P^{(T)}(\cdot | (\theta^{(*;T)})} \left[\prod_{A \in \mathcal{V}_2} (f_A(\mathbf{x}_A))^{1-\theta^{(*;T)}} \right] \right)^{\rho(T)} \\ &= Z^{(TRW)} \prod_{T \in \mathcal{T}} \left(\mathbb{E}_{P^{(T)}(\cdot | (\theta^{(*;T)})} \left[\prod_{A \in \mathcal{V}_2} (f_A(\mathbf{x}_A))^{1-\theta^{(*;T)}} \right] \right)^{\rho(T)}. \end{aligned} \quad (9.43)$$

In summary, $Z^{(TRW)}$, allows interpretation in terms of $|\mathcal{T}|$ Re-Parametrizations, each associated with a spanning tree in the $\mathcal{T}$ set, and Eq. (9.43) gives an explicit relation between the exact partition function Z and its upper-bound, $Z^{(TRW)}$. To compute the multiplicative correction, and therefore improve the estimate for Z provided by $Z^{(TRW)}$, requires evaluating expectations over the (easy to sample) spanning tree probability.

9.6.1 Fractional interpolation between tree reweighting and belief propagation

The BP and TRW constructions above provide two different structured references for the same partition function. A natural question is whether one can move continuously between them while preserving an exact correction identity. Fractional Belief Propagation (FBP) provides precisely such an interpolation for Ising models [15]. Introduce a scalar $\lambda \in [0, 1]$ and let $Z^{(\lambda)}$ denote the partition-function estimate obtained from the corresponding fractional variational problem, with

$$Z^{(0)} = Z^{(TRW)}, \quad Z^{(1)} = Z^{(BP)}.$$

For attractive Ising models the fractional estimate is monotone and satisfies

$$Z^{(TRW)} \geq Z^{(\lambda)} \geq Z^{(BP)}, \quad \lambda \in [0, 1], \quad (9.44)$$

and there exists a unique value λ_* for which

$$Z = Z^{(\lambda_*)}. \quad (9.45)$$

Thus, within this attractive family, one can regard the unknown exact answer as selecting a single scalar location between two variational references. This does not make λ_* directly available on a new instance, but it changes what must be estimated.

More importantly for the logic of this book, the fractional construction retains an exact reference–correction decomposition for *every* λ ,

$$Z = Z^{(\lambda)} \tilde{Z}^{(\lambda)}, \quad (9.46)$$

where $\tilde{Z}^{(\lambda)}$ is represented as an expectation over a node-independent product law built from the fractional singleton marginals [15]. Equation (9.46) is a static realization of the reference–correction principle in Eq. (1.6): λ changes the reference and therefore the statistical difficulty of evaluating the residual, but not the exactness of the identity.

Two distinct questions should therefore be separated. The first is *structural*: which λ makes $Z^{(\lambda)}$ exact? The second is *algorithmic*: which λ makes the correction easiest to estimate without knowing Z ? The first is answered by λ_* ; the second suggests criteria based on empirical variance, concentration, or transfer across related instances. These criteria need not select the same λ .

Advanced connection: From a scalar interpolation to learned reference selection

The bounded scalar λ_* is an unusually clean target for neuralization. On small instances one can compute Z exactly, label λ_* , and ask whether graph-local statistics and BP beliefs predict it on new instances. Chapter 15 uses this as the simplest example of learning a *reference parameter* rather than learning a partition function directly.

Problem 9.1 (Selecting a fractional reference without knowing Z). *For a family of small attractive Ising models, compute λ_* by enumeration and compare it with a value selected by minimizing the empirical variance of an unbiased Monte Carlo estimator of $\tilde{Z}^{(\lambda)}$. Study how the two values change with coupling strength, external field, graph density, and loop density. Then repeat the experiment outside the attractive regime and state explicitly which statements based on Eq. (9.44) no longer have a guarantee.*

9.7 Re-summing Loop Series

The exact series is combinatorial, so practical use requires selecting or resumming a small subset. Gómez, Mooij, and Kappen proposed truncations based on short or strongly weighted generalized loops and showed that corrections can substantially improve BP when the dominant loops are identifiable [80, 78]. The central difficulty is overlap: adding individually large loop terms may double-count structures whose aggregate effect is better represented by a connected cluster. Cluster cumulants provide one answer; determinant or zeta-function resummations provide another for special binary or planar structures [206]. An effective modern strategy is adaptive: generate candidate loops from residual correlations, merge strongly overlapping candidates, and stop when the estimated tail is below the desired tolerance.

Problem 9.2. *Develop an adaptive loop-selection rule for the examples of Chapters 6 and 7. Compare selection by length, absolute loop weight, and incremental cumulant. Report both accuracy and enumeration cost.*

9.7.1 Loop series as a measure to sample, and as a device for proofs

The previous subsection treats the loop series as a sum to be truncated or resummed. Two other uses of the same exact expansion are important because they illustrate that an exact correction need not be enumerated term by term.

Sampling the correction. Ahn, Chertkov, and Shin construct MCMC algorithms whose state space is built from generalized loops rather than from the original configurations [2]. The construction is explicitly BP aware: the graphical model is first reparametrized at a nontrivial BP fixed point, so the deterministic approximation supplies the coordinates in which the stochastic correction is sampled. For the 2-regular part of the series they combine a worm-type sampler with rejection; for the full generalized-loop series they use a cycle basis to obtain a rejection-free construction. On planar binary pairwise models the 2-regular contribution can also be computed through a Pfaffian reduction, providing a useful exact benchmark.

The signs of loop weights matter. Generalized-loop corrections are not, in general, a positive probability measure, so Monte Carlo sampling requires a representation that handles positive

and negative contributions rather than pretending that the signed series is an ordinary Gibbs law. This is a static version of the weight-degeneracy issue that appears later on path space.

Using the expansion in a proof. Loop calculus is also useful when the desired output is not a numerical correction but a theorem. Chandrasekaran, Chertkov, Gamarnik, Shah, and Shin analyze the Bethe approximation to the number of independent sets by combining a convergent BP-like scheme with a loop-series control argument [39]. Two points are particularly instructive. First, an algorithm produces only an ε -approximate stationary point, whereas the basic loop identity is naturally written at an exact Bethe stationary point; the proof therefore controls the perturbation introduced by approximate stationarity. Second, the exponentially large family of generalized loops is bounded through model-specific combinatorial structure rather than enumerated.

This separates two notions that are easy to conflate: an expansion can be *exact* but computationally useless, or it can become useful after sampling, truncation, resummation, or a model-specific bound controls the residual.

Remark 9.7.1 (From static loop weights to path weights). *The loop sampler replaces a combinatorial sum by an expectation over loop objects. Feynman–Kac methods later replace a path integral by an expectation over weighted trajectories. The mathematical objects differ, but the computational question is parallel: does the chosen reference place enough mass where the correction is important, or is the estimate dominated by rare high-weight events?*

9.8 Beyond BP Gauges

BP gauges are special because they annihilate every loose-end contribution. Other gauges can be optimized for different objectives, including variational lower bounds or concentration of the remaining tensor contraction [1]. This suggests a broader program: choose a gauge jointly with a truncation family so that omitted loop or tensor components are small in a computable norm. Gauge choice can also be neuralized, but the invariant constraint must remain exact. The distinction is important: a learned basis may improve compression, while only paired inverse-transpose transformations guarantee that the partition function is unchanged.

Problem 9.3. *For a small normal factor graph, parameterize invertible edge gauges and minimize the sum of squared weights of all generalized loops longer than a chosen cutoff. Compare the resulting gauge with the BP gauge and with a gauge optimized for a mean-field lower bound.*

9.9 From loop calculus to tensor networks and quantum graphical models

A normal factor graph can be read literally as a tensor network: a factor is a local tensor, an edge variable is a bond index, and the partition function is the full contraction. Under this reading, the gauge transformations of Chapter 5 become changes of basis on internal bonds. The paired transformation

$$G_{ab}^T G_{ba} = I$$

leaves the complete contraction invariant because the two basis changes cancel on the contracted bond. Positivity is not required for this algebraic statement.

This observation has become practically useful. Belief propagation can be used to select tensor-network gauges, and BP-based gauging has been developed as a preprocessing and optimization tool for structured, unstructured, and infinite tensor networks [191]. Generalized belief propagation extends the same idea to overlapping regions and provides a hierarchy of approximate tensor-network contractions [192]. Thus the progression developed earlier in the book,

$$\text{BP} \longrightarrow \text{GBP} \longrightarrow \text{loop corrections},$$

can be read either as probabilistic inference or as a hierarchy of contraction schemes.

What survives without positivity? For a classical graphical model with positive factors, BP beliefs may be interpreted as approximate marginals and the Bethe free energy has a probabilistic variational meaning. A quantum-amplitude tensor network may instead contain negative or complex entries. Marginal and entropy interpretations then disappear, while multilinearity, paired gauges, exact contraction identities, and algebraic loop expansions remain. This distinction explains why the gauge derivation of loop calculus is more portable than a derivation that relies entirely on probability semantics.

Companion notebook: Gauge invariance survives signed and complex tensor entries

Notebook `ch09/ch09_04_tensor_gauge_beyond_positivity.ipynb` contracts the same small tensor network before and after paired gauge transformations for positive, signed, and complex tensors. It verifies exact invariance up to numerical precision and illustrates how poor gauge conditioning can amplify floating-point error without changing the exact algebra.

Advanced connection: Quantum stars as a controlled complementary expansion

The driven quantum-star construction provides a useful quantum counterpart of exact tree elimination followed by controlled corrections [43]. Leaves can be eliminated exactly, while the residual quantum dynamics is organized in powers of inverse coordination $1/d$. The comparison with loop calculus is instructive precisely because the organizing parameters are different: loop calculus orders corrections by graph topology, whereas the quantum-star hierarchy is controlled by coordination and dynamical concentration.

Problem 9.4 (Gauge selection for tensor contraction). *For a small normal factor graph, compare the BP gauge with a gauge chosen to minimize a fixed-rank bond-truncation error. Repeat after allowing selected tensor entries to become negative or complex. Report the exact contraction, the truncated contraction, and the residual in each gauge. Which parts of the BP interpretation disappear when positivity is lost, and which algebraic statements remain exact?*

9.10 What loop calculus adds to the inference hierarchy

Loop calculus does not replace BP, GBP, tree-reweighting, or approximate elimination. It supplies a diagnostic and correction layer. BP chooses a locally consistent reference representation; loop terms identify precisely which global structures the reference misses. GBP moves selected

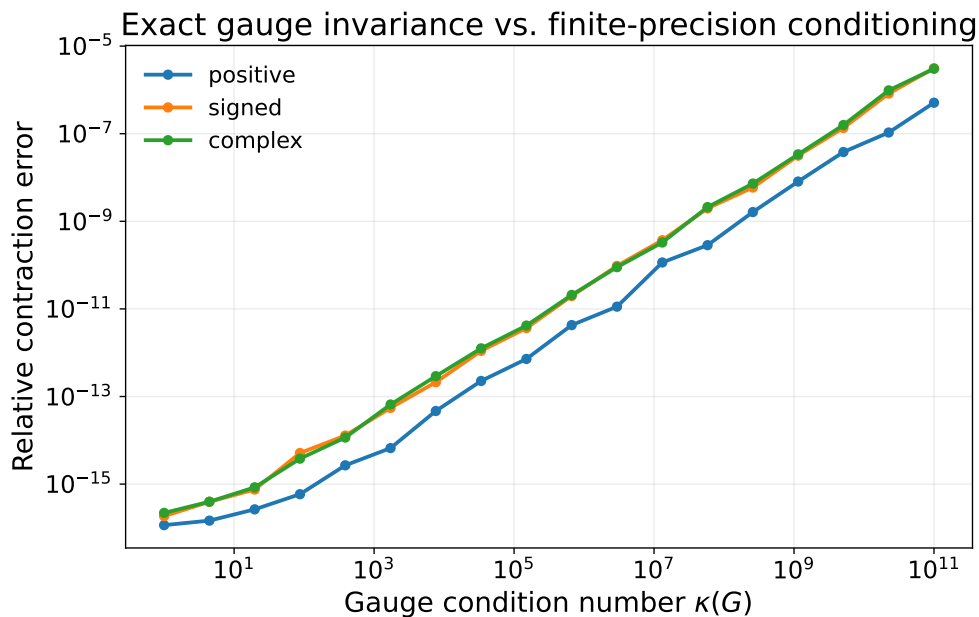


Figure 9.8: Paired tensor-network gauges preserve the exact contraction for positive, signed, and complex tensors. Finite-precision error grows only when the gauge becomes badly conditioned, separating algebraic invariance from numerical conditioning. Generated by `ch09/ch09_04_tensor_gauge_beyond_positivity.ipynb`.

loops into the reference itself; cumulants organize the remaining overlaps; gauges change the basis in which corrections are expressed. This hierarchy is especially natural for structured generative models, where a learned local approximation may be accurate on common configurations but fail on rare global constraints. Loop and cluster corrections provide a route to detect and repair those failures without discarding the local model.

Chapter 10

Graphical Model Learning

Reader's Guide.

Why this chapter? Until now the factors were given and computation happened *inside* the model. Learning reverses the arrow: samples are given and the model itself must be reconstructed. Three different unknowns should be kept separate—the graph topology, numerical factor parameters on a chosen graph, and latent variables that are not observed at all. They lead to different algorithms even when they use the same graphical-model language.

Three simple inverse problems. For tree structure learning, each candidate edge receives a data-derived score—mutual information—and the global problem becomes a maximum-spanning tree. For sparse Ising parameter learning, conditioning on one spin turns a difficult globally normalized model into a tractable local logistic-type problem. For latent-variable learning, missing data create an inference problem inside the parameter-fitting problem; Expectation–Maximization alternates between the two.

A recurring computational obstacle: the partition function. Maximum likelihood for an energy-based graphical model differentiates through $\log Z(\theta)$. The gradient therefore asks for expectations under the current model, which can be as hard as the inference problems of earlier chapters. Much of this chapter can be read as different ways around that obstacle: exploit a tree, replace the global likelihood by local conditionals, use a screening objective, or approximate the inner model expectation by sampling.

What to compare when evaluating a learning method. Do not conflate parameter error, graph-recovery error, likelihood, and downstream inference quality. A method may recover edges but estimate couplings poorly, or fit likelihood well while yielding an inconvenient representation for later inference. The notebooks therefore compare statistical recovery with computational cost and, where possible, with the behavior of the learned model in subsequent inference tasks.

How to use the companion notebooks. Start with Chow–Liu because it cleanly separates sampling noise from optimization: once pairwise mutual informations are estimated, the combinatorial step is exact. Then use the sparse-Ising notebooks to see why local objectives avoid repeated partition-function evaluation. Finally, when reading EM and contrastive divergence, explicitly identify the inner inference/sampling step and ask what happens if it is only approximate.

Learning reverses the direction of the problems developed so far. In Chapters 2–9 we assumed that the graph and its factors were known and asked for partition functions, marginals, MAP configurations, samples, or controlled approximations. Here the probability law is observed only through data, and the graph, parameters, or latent representation must be inferred.

Three themes organize the chapter. First, when the target graph is a tree, the Chow–Liu construction turns structure learning into a maximum-spanning-tree problem. Second, for general sparse Ising models, local convex objectives such as pseudo-likelihood and interaction screening avoid repeated evaluation of the global partition function. Third, hidden variables return us to inference inside learning: Expectation–Maximization, energy-based gradients, contrastive divergence, and Restricted Boltzmann Machines all require an inner probabilistic calculation. This last theme anticipates the later neuralization and structured-generative-model parts of the book.

10.1 Learning a Spanning Tree: the Chow–Liu Algorithm

The factorization of a tree distribution derived in Section 3.2.1.1 immediately suggests an inverse problem: given independent samples of $\mathbf{x} = (x_1, \dots, x_N)$, can we infer the tree that best explains their dependence structure? Chow and Liu showed that this apparently combinatorial search has an exact polynomial-time solution [52].

For a candidate tree $T = (\mathcal{V}, \mathcal{E}_T)$, form the tree distribution from the exact one- and two-variable marginals of the target law P ,

$$P_T(\mathbf{x}) = \frac{\prod_{(a,b) \in \mathcal{E}_T} P(x_a, x_b)}{\prod_{a \in \mathcal{V}} P(x_a)^{q_a - 1}}, \quad (10.1)$$

where q_a is the degree of node a in T . We measure the discrepancy by the Kullback–Leibler divergence

$$D(P \parallel P_T) = \int_{\mathbf{x}} P(\mathbf{x}) \log \frac{P(\mathbf{x})}{P_T(\mathbf{x})} \geq 0. \quad (10.2)$$

Substituting Eq. (10.1) and marginalizing the terms that depend only on one edge or one node gives

$$D(P \parallel P_T) = - \sum_{(a,b) \in \mathcal{E}_T} I(X_a; X_b) + \sum_{a \in \mathcal{V}} S(X_a) - S(\mathbf{X}),$$

with mutual information

$$I(X_a; X_b) = \int_{x_a, x_b} P(x_a, x_b) \log \frac{P(x_a, x_b)}{P(x_a)P(x_b)}.$$

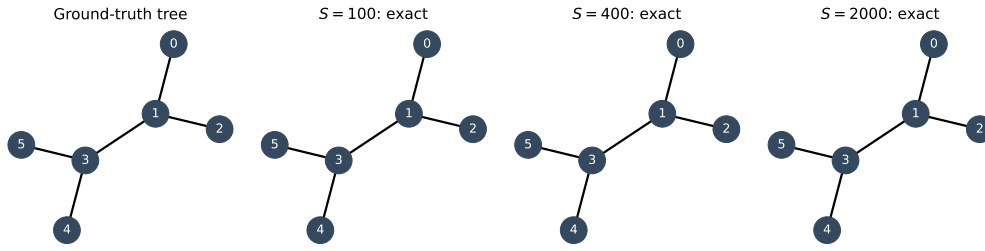


Figure 10.1: Chow–Liu structure learning on a tree Ising model. Edge labels show the pairwise mutual-information estimates used by the maximum-spanning-tree algorithm. As the sample size increases, the empirical ordering of informative edges stabilizes and the reconstructed tree approaches the ground-truth topology.

The entropy terms independent of T drop out. Hence the optimal tree maximizes the total edge mutual information,

$$T^* = \arg \max_{T \text{ tree}} \sum_{(a,b) \in \mathcal{E}_T} I(X_a; X_b).$$



This is exactly a maximum-spanning-tree problem. Kruskal’s greedy algorithm therefore returns the optimum [52].

Algorithm 15 Chow–Liu maximum-information spanning tree

- 1: Estimate $I(X_a; X_b)$ for every pair $a < b$ and sort the edges in decreasing order.
 - 2: $\mathcal{E}_T \leftarrow \emptyset$.
 - 3: **for** each candidate edge (a, b) in sorted order **do**
 - 4: **if** adding (a, b) does not create a cycle **then**
 - 5: $\mathcal{E}_T \leftarrow \mathcal{E}_T \cup \{(a, b)\}$.
 - 6: **if** $|\mathcal{E}_T| = |\mathcal{V}| - 1$ **then**
 - 7: **break**.
 - 8: **return** $T = (\mathcal{V}, \mathcal{E}_T)$.
-

Two interpretations are useful. If the data-generating model is itself tree structured and the mutual informations are known exactly, the construction recovers an optimal tree representation. For a loopy distribution, the same calculation returns the tree distribution minimizing $D(P \parallel P_T)$ within the Chow–Liu family. With finite data, the mutual informations are plug-in estimates and the recovered structure is itself random; this statistical layer motivates the prediction-centric discussion in Section 10.3.

Companion notebook: Learning structure from samples by mutual information

Notebook `ch10/ch10_01_chow_liu_structure_learning.ipynb` generates samples from a six-spin tree Ising model, estimates all pairwise mutual informations, reconstructs the maximum-information tree, and studies how the recovery probability grows with sample size.

Exercise 10.1.1. Use the notebook to replace the tree data-generating model by the same graph with one additional weak edge. Compare the recovered Chow–Liu tree with the original tree and explain which edge is sacrificed by the KL-optimal tree approximation as the new coupling increases.

Local learning as “many small regressions” instead of one global normalization.

For an Ising model, the conditional law of X_a given all other spins depends only on the field at node a and its incident couplings; the global partition function cancels. Pseudo-likelihood therefore fits one conditional model per node. Interaction screening uses a different local objective with the same strategic advantage. The conceptual move is important: locality is not merely an approximation device here—under suitable assumptions these local population objectives identify the exact global parameters.

10.2 Efficient Local Learning of Graphical Models

Maximum-likelihood learning in an exponential family matches empirical and model sufficient statistics, but the model expectation generally requires the same difficult partition-function or marginal calculations studied in the preceding chapters. Trees are a notable exception. A second route is therefore to replace the global likelihood by local objectives whose normalization involves only one variable. This section develops two such constructions for sparse Ising models: regularized pseudo-likelihood and interaction screening.

Thanks to recent breakthrough in the field [34, 19, 164, 173, 31, 141, 28, 200, 133, 216] (see also references therein) we are able to answer the question affirmatively and moreover constructively. In what follows we describe two constructive approaches to reconstruction of the Ising model. First approach, described in 10.2.1, is based on empirical estimations of the so-called *pseudo-log-likelihoods*. Second approach, described in Section 10.2.2, is based on empirical estimations of the so-called *interaction screenings*.

The most significant feature of the two new approaches to learning consists in substitution of the global objective — the log-likelihood described by Eq. (4.21) — by many local (and also convex !!) optimizations each associated with a node of the GM.

Quite remarkably, this substitution allows not only to achieve tractability, because computation of the partition function for any value of the candidate GM parameters is no longer required, but it also guarantees exactness, i.e. exact reconstruction of the GM parameters in the asymptotic limit of infinitely many samples. Moreover, the two local approaches are “doubly” efficient, as they (both) yield accurate structural estimations of the graph and of the GM parameters

1. with the number of samples which is logarithmic in the number of nodes in the graph;
2. with computational efforts which are polynomial in N .

Even though the learning approach discussed in this Section applies broadly to learning any GM over a finite alphabet, we will follow [133] and focus in this Section solely on the Ising model. We will adopt notations of Section 2.4.2 but also absorb the temperature parameter, β , into the singleton, $\mathbf{h}^* = (h_a^* | a \in \mathcal{V})$, and pair-wise, $\mathbf{J}^* = (J_{ab}^* | (a, b) \in \mathcal{E})$, vectors (of parameters), therefore assuming that i.i.d. samples are generated from the *ground truth* probability

distribution described by the following (minor) modification of Eq. (2.16)

$$\forall \mathbf{x} = (\pm 1)^{|\mathcal{V}|}, \quad P(\mathbf{x} | \mathbf{J}^*, \mathbf{h}^*) = \frac{1}{Z} \exp \left(\sum_{a \in \mathcal{V}} h_a^* x_a + \sum_{(a,b) \in \mathcal{E}} J_{ab}^* x_a x_b \right). \quad (10.3)$$

As shown below we are able to estimate complexity of the GM learning (i.e. to estimate number of samples and computational efforts needed to achieve a preset prediction quality) without imposing any additional constraints (on graph or factors). Let us briefly review recent pre-history which led researchers to these generic – universal approximation – statements.

The problem of designing a universal learning algorithm with polynomial computational complexity that achieves exact graph topology reconstruction for arbitrary Ising models in all regimes was posed in [142] and resolved affirmatively by Bresler in [34, 29]. The biggest challenges addressed in [34, 29] were the low temperature regime and long-range correlations, which are known to be particularly difficult for learning. Nonetheless, the computational cost of these algorithms is still high, and scales as a polynomial of high degree in the number of nodes [34], or double exponential in the maximum node-degree $d_{\max}$ and in the maximum interaction strength [29]. Moreover, both algorithms require prior information on the bounds on the interaction strengths, i.e. positive α and β such that $\alpha \leq |J_{ab}| \leq \beta$ for all $(a, b) \in \mathcal{E}$, as well as the knowledge of $d_{\max}$.

In an attempt to determine the *optimal* number of samples needed for reconstructing the graph, information-theoretic bounds were derived in [173]. Let us emphasize, following discussion of [133], three salient features of these bounds. First, the *optimal* number of samples S_{opt} for perfect graph recovery scales exponentially with the maximum interaction value and node-degree, $S_{\text{opt}} \propto e^{c\gamma}$, where $\gamma = \beta d_{\max} + h_{\max}$ and $h_{\max}$ denotes an upper bound on the absolute values of magnetic fields. Although it was shown that $c \in [1, 4]$, the precise value of c remained unknown. This range of c was referred to in [133] (see also below, e.g. Fig. 10.5) as to the *optimal regime* with respect to the dependence of the number of samples on γ . Intuitively, this exponential scaling requirement can be attributed to the typical waiting time for collecting sufficient number of “non-trivial” samples, i.e., those that are different from the ground states. This waiting time is more pronounced in the low temperature regime when γ is large. Second, for finite $d_{\max}$ the dependence on the number of variables N is very weak: $S_{\text{opt}} \propto \log N$. This logarithmic dependence represents the amount of information needed for hypothesis testing over the set of $C_N^{d_{\max}}$ candidate neighborhoods of a given vertex [224]. Third, the number of required configurations grows as α decreases, since it is difficult to distinguish between the presence of a very weak coupling and its absence. In particular, in the limit of small α , $S_{\text{opt}} \propto 1/\alpha^2$.

In what follows, we discuss two exact methods for solving the problem of learning the Ising model. The first method is based on the *Regularized Pseudo-Likelihood Estimator* (RPLE) of [165] supplemented with a post-optimization parameter thresholding procedure. We reproduce the proof from [133] in Section 10.2.1 stating that this ingredient makes the RPLE exact and practically efficient. The second algorithm, that we discuss in Section 10.2.2, is an exact estimator – called *Regularized Interaction Screening Estimator* (RISE) – based on the *Interaction Screening* method introduced in [200]. Following [200, 133] we will also comment briefly on the proof of RISE efficiency which is similar to how the efficiency bounds were established for RPLE. Section 10.2.3 concludes our discussion of the local, exact and efficient learning algorithms with highlights and brief summary of empirical comparison of the aforementioned algorithms reported in [133]. We will see that RISE, as well as its log-RISE version, outperforms RPLE and requires a number of samples lying within the information-theoretic *optimal regime*.

10.2.1 Regularized Pseudo-Likelihood Estimator

A widely used approach aiming at achieving the optimal scalings is traced back to [20] but in the form we use it here was suggested in [165], where estimation of model parameters is performed based on the so-called pseudo-likelihood acting as a surrogate for the intractable log-likelihood function. The method is based on minimixing a set of local (!) *Regularized Pseudo-Likelihood Estimators* (RPLE). Each of the RPLE estimators can be interpreted as a regularized probability of a single spin, x_a , conditioned on the remaining $N - 1$ spins, $\mathbf{x}_{\sim a} = (x_b | \forall b \in \mathcal{V}, b \neq a)$, in the system:

$$\forall a \in \mathcal{V}: (\hat{\mathbf{J}}_a, \hat{h}_a)^{(\text{RPLE})} = \arg \min_{(\mathbf{J}_a, h_a)} \mathcal{L}_a(\mathbf{J}_a, h_a), \quad (10.4)$$

$$\mathcal{L}_a(\mathbf{J}_a, h_a) = \mathbb{E} \left[\log \left(1 + e^{-2x_a(h_a + \sum_{b \neq a} J_{ab} x_b)} \right) \right]_S + \lambda \sum_{b \neq a} |J_{ab}|, \quad (10.5)$$

$\mathbf{J}_a = (J_{ab} | \forall b \neq a)$ and h_a are the optimization parameters (vector and scalar, respectively); and the low index S in the *expectation* indicates that the expectation is *empirical*, i.e. estimated from S samples, $(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(S)})$:

$$\mathbb{E}[f(\mathbf{x})]_S = \frac{1}{S} \sum_{s=1}^S f(\mathbf{x}^{(s)}).$$

The first term on the right hand side of Eq. (10.5) is the (minus) **pseudo-(log)-likelihood**, $-\mathbb{E}[\log(P(x_a | \mathbf{x}_{\sim a}; \mathbf{J}_a, h_a))]_S$, thus defined as the minus log-likelihood of the node-marginal probability considered as the function of $\mathbf{J}_a$ and h_a . Standard use of the pseudo-log-likelihood in statistical theory is as an approximation to the log-likelihood, defined in Eq. (4.21). However, and quite-remarkably, the use of the pseudo-log-likelihood is exact (not approximate !!) for reconstructing parameters of the GM. Specifically, in the case of the Ising model we observe that

$$(\mathbf{J}_a^*, h_a^*) = \arg \min_{(\mathbf{J}_a, h_a)} \mathbb{E}_{P^*} [-\log P_{(\mathbf{J}_a, h_a)}(X_a | \mathbf{X}_{\sim a})]. \quad (10.6)$$

The population identity is most transparent as a conditional cross-entropy statement. Subtracting the objective at the truth gives

$$\mathbb{E}_{\mathbf{X}_{\sim a} \sim P^*} D(P^*(X_a | \mathbf{X}_{\sim a}) \| P_{(\mathbf{J}_a, h_a)}(X_a | \mathbf{X}_{\sim a})) \geq 0. \quad (10.7)$$

Consequently the true conditional law is a population minimizer; under the usual identifiability/nondegeneracy conditions the corresponding local parameter vector is unique. Differentiating the unregularized population loss gives the score identity

$$\nabla_{\mathbf{J}_a, h_a} \mathcal{L}_a = \mathbb{E}_{P^*} [(\tanh H_a(\mathbf{X}) - X_a) \mathbf{z}_a], \quad H_a(\mathbf{X}) = h_a + \sum_{b \neq a} J_{ab} X_b, \quad (10.8)$$

where $\mathbf{z}_a = (1, (X_b)_{b \neq a})$ collects the intercept and covariates. At the true parameters, $\mathbb{E}[X_a | \mathbf{X}_{\sim a}] = \tanh H_a^*(\mathbf{X})$, so the score vanishes. This argument also makes clear why pseudo-likelihood can be statistically exact without ever evaluating the global partition function.

The second term on the right hand side of Eq. (10.5) is the sparsity promoting ℓ_1 regularization term. The term is important as it discourages the minimizer $\hat{\mathbf{J}}_a$ from being dense by effectively pushing the interaction values toward zero whenever an edge is absent.

We are now ready to report the main result of [133] concerning the Algorithm implementing optimization (10.4).

For the sake of simplicity, clarity of the presentation and without loss of generality, we choose to consider, in this and also in the following Section, the case where the magnetic fields (singleton contributions) are set to zero.

Theorem 10.2.1 (RPLE error). *Let S be the number of i.i.d. samples from the Ising model (10.3) with $N = |\mathcal{V}|$ variables, bounded degree d and the maximum coupling $\beta = \max_{(a,b) \in \mathcal{E}} |J_{ab}^*|$. The reconstruction error on the couplings (in the neighborhood of node a) of the RPLE with regularization parameter $\lambda = c_1 \sqrt{\log(N^2/\epsilon)/S}$ is bounded with the probability $1 - \epsilon/N$ as*

$$\left\| \hat{\mathbf{J}}_a^{\text{RPLE}} - \mathbf{J}_a^* \right\|_2 \leq C_d e^{4\beta d} \sqrt{\frac{\log(N^2/\epsilon)}{S}}, \quad (10.9)$$

where C_d depends only polynomially on d , and c_1 is a constant.

Prior to discussing main idea of the Theorem's proof – let us clarify its practical significance.

1. The control of the error on the reconstructed couplings is important because it allows to *reconstruct the structure of the graph in the GM exactly with a high probability*. Indeed, if the error on the right hand side of Eq. (10.9) is smaller than (say) $\alpha/2$, where $\alpha = \min_{(a,b) \in \mathcal{E}} |J_{ab}^*|$, it becomes possible to reconstruct the structure of the neighborhood of node a by declaring that the edges whose reconstructed coupling is less than $\alpha/2$ (in absolute value) are absent. Indeed, repeating this procedure over N neighborhoods and making use of the union bound, also known as [Boole's inequality](#), we guarantee that the probability of a global miss-assignment is smaller than the sum of the probabilities of the respective local miss-assignments, i.e. smaller than $N\epsilon/N = \epsilon$. Therefore, the RPLE algorithm returns exact reconstruction of the graph with the probability $1 - \epsilon$. Notice, that this (union bound) line of arguments is the reason for choosing the $1 - \epsilon/N$ factor in the local (each neighborhood) condition of the Theorem 10.2.1.
2. Furthermore, the Theorem 10.2.1 allows to *estimate the number of samples S^* which is sufficient to obtain a fixed error on the couplings, and hence to recover the structure of the graph*. The result for RPLE is $S^* \propto e^{8\beta d} \log N$. Notice, however, that this expressions for the error is not tight. It represents the upper bounds on the actual required number of samples. Detailed numerical experiments reported in [133] (and briefly discussed below) revealed that the scalings of S^* in practice is better than the theoretically predicted one.

Below we sketch main elements of the Theorem 10.2.1's proof. (The reader is advised to check [133] for details.)

Proof. (of Theorem 10.2.1) To bound the distance between the true parameters of the model $\mathbf{J}_a^*$ and their estimated counterparts $\hat{\mathbf{J}}_a$ for finite S , we use proof strategy based on constructing a quadratic lower-bound of the objective function centered around $\mathbf{J}_a^*$. An explicit form of the quadratic lower-bound $Q_{\mathcal{L}_a}$ for the RPLE objective function (10.5), which satisfies $\mathcal{L}_a(\mathbf{J}_a) \geq Q_{\mathcal{L}_a}(\mathbf{J}_a)$, can be found. Let us discuss it briefly (see [200] for details). The idea is that the distance $\|\delta \mathbf{J}\|_2 \equiv \|\hat{\mathbf{J}}_a - \mathbf{J}_a^*\|_2$ can be estimated using this explicit form of $Q_{\mathcal{L}_a}$ and the fact that the estimator is convex. This quadratic lower-bound is approximated by a second-order Taylor expansion of $\mathcal{L}_a(\hat{\mathbf{J}}_a) = \mathcal{L}_a(\mathbf{J}_a^* + \delta \mathbf{J})$ around $\mathbf{J}_a^*$:

$$Q_{\mathcal{L}_a}(\mathbf{J}_a^* + \delta \mathbf{J}) = \mathcal{L}_a(\mathbf{J}_a^*) + \mathbb{E} \left[\delta \mathbf{J}^T \cdot \nabla \mathcal{L}_a(\mathbf{J}_a^*) \right] + \frac{1}{2} \mathbb{E} \left[\delta \mathbf{J}^T \cdot \nabla^2 \mathcal{L}_a(\mathbf{J}_a^*) \delta \mathbf{J} \right]. \quad (10.10)$$

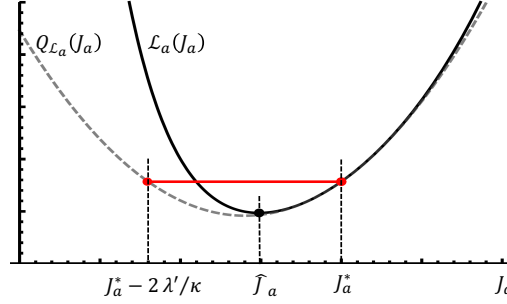


Figure 10.2: Schematic (one dimensional) illustration of the quadratic approximation of the RPLE error function. The objective function, $\mathcal{L}_a(\mathbf{J}_a)$, is shown in black. The quadratic lower-bound expansion around $\mathbf{J}_a^*$, $Q_{\mathcal{L}_a}$, is the gray dashed line. The estimated distance (upper bound) between $\mathbf{J}_a^*$ and $\hat{\mathbf{J}}_a$ is indicated by the red line starting in $\mathbf{J}_a^*$. As $\mathcal{L}_a(\mathbf{J}_a)$ is convex, the minimum point is ensured to be enclosed between the quadratic lower-bound and the red line, which results in the estimation of the difference, $\|\hat{\mathbf{J}}_a - \mathbf{J}_a^*\|_2$, as explained in the text.

Since the estimator $\mathcal{L}_a(\mathbf{J}_a)$ reaches its minimum at $\hat{\mathbf{J}}_a$, we have $\mathcal{L}_a(\mathbf{J}_a^*) \geq \mathcal{L}_a(\hat{\mathbf{J}}_a)$ (where the equality occurs for $S \rightarrow \infty$, where $\hat{\mathbf{J}}_a$ coincides with $\mathbf{J}_a^*$). Because $\mathcal{L}_a \geq Q_{\mathcal{L}_a}$, the convex sublevel set of $\mathcal{L}_a$ corresponding to the value $\mathcal{L}_a(\mathbf{J}_a^*)$ is contained in the convex sublevel set of $Q_{\mathcal{L}_a}$, and thus the minima $\hat{\mathbf{J}}_a$ must lie within this region too:

$$\hat{\mathbf{J}}_a \in (\mathbf{J}_a | \mathcal{L}_a(\mathbf{J}_a) \leq \mathcal{L}_a(\mathbf{J}_a^*)) \subseteq (\mathbf{J}_a | Q_{\mathcal{L}_a}(\mathbf{J}_a) \leq Q_{\mathcal{L}_a}(\mathbf{J}_a^*)). \quad (10.11)$$

As a result, the distance $\|\hat{\mathbf{J}}_a - \mathbf{J}_a^*\|_2$ can be upper bounded by the diameter of the convex region on the right hand side of (10.11). Geometrical intuition behind the arguments is illustrated in Fig. 10.2 as a one-dimensional, thus schematic, sketch. Here, the quadratic expansion (10.10) reads: $Q_{\mathcal{L}_a}(\mathbf{J}_a^* + \delta \mathbf{J}) = \mathcal{L}_a(\mathbf{J}_a^*) + \lambda' \delta \mathbf{J} + \frac{1}{2} \kappa (\delta \mathbf{J})^2$. This function takes the value $\mathcal{L}_a(\mathbf{J}_a^*)$ at two points: $\delta \mathbf{J} = 0$ and $\delta \mathbf{J} = -2\lambda'/\kappa$. The distance between the estimated and the true parameters can hence be estimated as

$$\|\hat{\mathbf{J}}_a - \mathbf{J}_a^*\|_2 \leq \frac{2\lambda'}{\kappa}. \quad (10.12)$$

In the high-dimensional setting, λ' represents the largest component of the gradient $\nabla \mathcal{L}$, and κ is the smallest eigenvalue of the Hessian matrix $\nabla^2 \mathcal{L}$, both evaluated at the point $\mathbf{J}_a^*$. Given this proof strategy, we need to estimate λ' and κ in order to recover the precise statement of the Theorem.

Estimation of λ' : It is straightforward to compute the gradient of the pseudo-likelihood objective (10.5):

$$\nabla \mathcal{L}_a(\mathbf{J}_a^*) = \mathbb{E} \left[\mathbf{x}_{\sim a} \left(\tanh \left(\sum_{b \sim a} J_{ab}^* x_b \right) - x_a \right) \right]_S, \quad (10.13)$$

where $\mathbf{x}_{\sim a}$ denotes the vector containing all spins but x_a , and ∂a denotes the set of neighbors of node a . As all components of the gradient with respect to $\mathbf{J}_a^*$ are upper-bounded (in absolute value) by 2 we use [Hoeffding's concentration inequality](#) [93] to show that any given gradient

component is bounded with high probability

$$\mathbb{P} \left(\left| \frac{\partial}{\partial J_{ac}} \mathcal{L}_a(J^*) \right| \geq \sqrt{\frac{8}{S}} t \right) \leq c_1 e^{-t^2}, \quad (10.14)$$

where $c_1 > 0$ is a constant. The inequality (10.14) means that the gradient components lie in an interval estimated as $\sqrt{8t}/\sqrt{S}$. Moreover, the probability that these gradient components lie outside of this interval, and are away by a multiplicative factor t decreases exponentially in t^2 . By choosing $t = \sqrt{\log(N^2/\epsilon)}$, we limit the right hand side in (10.14) by ϵ/N^2 . This shows that with the probability of at least $1 - \epsilon/N^2$ any given gradient components is upper-bounded

$$\left| \frac{\partial}{\partial J_{ac}} \mathcal{L}_a(J^*) \right| \leq c_1 \sqrt{\frac{\log(N^2/\epsilon)}{S}}. \quad (10.15)$$

Recall that there are $N - 1$ components of the gradient vector. Taking the [union bound](#) over them, we can guarantee that the maximum over these $N - 1$ gradient components is of the same order as in (10.15) with probability $1 - \epsilon/N$. Therefore, the quantity λ' defined above can be estimated as

$$\lambda' \propto \sqrt{\frac{\log(N^2/\epsilon)}{S}}. \quad (10.16)$$

Estimation of κ : The Hessian matrix of the pseudo-likelihood function $\mathcal{L}$ can be found by the following direct computation

$$\nabla^2 \mathcal{L}_a(J^*) = \mathbb{E} \left[x_{\sim a} x_{\sim a}^\top \left(1 - \tanh \left(\sum_{b \sim a} J_{ab}^* x_b \right)^2 \right) \right]_S. \quad (10.17)$$

Using the inequality $1 - \tanh(x)^2 \geq \exp(-2|x|)$ and the fact that $|\sum_{b \sim a} J_{ab}^* x_b| \leq \beta d$, we show that the Hessian is lower-bounded in the positive semi-definite sense, i.e.

$$\nabla^2 \mathcal{L}_a(J^*) \succeq e^{-2\beta d} C_S, \quad (10.18)$$

where the matrix C_S is the empirical covariance matrix

$$C_S = \mathbb{E} [x_{\sim a} x_{\sim a}^\top]_S. \quad (10.19)$$

In expectation C_S is equal to the covariance matrix $C = \mathbb{E} [x_{\sim a} x_{\sim a}^\top]$ for which all eigenvalues are bigger than $a_C e^{-2\beta d}$ [142], where a_C is a constant depending polynomially on d . However, already from the expression (10.16) we see that S scales as $\log N$ in order to guarantee the constant error on the couplings. In this so-called high-dimensional regime $S \propto \log N$, the empirical covariance matrix possesses only $\mathcal{O}(\log N)$ non-zero eigenvalues. The reason for C_S to be severely rank deficient is that C_S is the sum of S rank-one matrices $x_{\sim a}^{(s)} x_{\sim a}^{(s)\top}$, $s = 1, \dots, S$. Therefore the rank of C_S can not exceed S .

This problem is circumvented by the presence of the ℓ_1 penalty term in the optimization formulation of the RPLE (10.4). It turns out that if the penalty parameter λ is greater than the largest component of the gradient (10.16) (which explains why we choose similar notations for λ and λ'), the only relevant eigenvalues correspond to eigenvectors that are sparse. (The reader is advised to check see [153] for a more precise statement.) Such eigenvalues are called

restricted eigenvalues as they correspond to the minimum of the quadratic form associated with C_S restricted to the sector of sparse vectors. An intuitive explanation of this property is that perturbations of $\mathbf{J}_a^*$ with $\delta \mathbf{J}$ which are not sparse drastically change the value of the ℓ_1 penalty. Therefore a non-sparse perturbation $\delta \mathbf{J}$ increases the value of the pseudo-likelihood objective $\mathcal{L}$ with ℓ_1 penalty even though it may not change the value of the pseudo-likelihood objective alone, which discourages such directions of perturbation.

It remains to verify that restricted eigenvalues of C_S are with high probability bounded-away from zero. A technical proof of this statement can be found in [200] where it is shown that with high probability C_S has all its restricted eigenvalues greater than $\frac{1}{2}a_C e^{-2\beta d}$. Combining this bound with (10.18), we get the following estimation of κ :

$$\kappa = \frac{1}{2}a_C e^{-4\beta d}. \quad (10.20)$$

Now using the expression (10.12), we finally determine the error between the couplings and their estimated counterpart as the ratio between (10.16) and (10.20):

$$\left\| \hat{\mathbf{J}}_a^{\text{RPLE}} - \mathbf{J}_a \right\|_2 \leq \frac{2\lambda'}{\kappa} \propto e^{4\beta d} \sqrt{\frac{\log(N^2/\epsilon)}{S}}. \quad (10.21)$$

This final inequality represents the statement of the Theorem. $\square$

10.2.2 Learning via Interaction Screening

Another exact reconstruction algorithm termed the *Regularized Interaction Screening Estimator* (RISE) was introduced in [200, 133].

The RISE is based on the minimization of the *Interaction Screening Objective* (ISO)

$$I_a(\mathbf{J}_a, h_a) = \mathbb{E} \left[\exp \left(-x_a \left(h_a + \sum_{b \neq a} J_{ab} x_b \right) \right) \right]_S, \quad (10.22)$$

over the probe pair-wise vector of coupling parameters, $\mathbf{J}_a$, and the probe singleton parameter (magnetic field), h_a for a given spin x_a . The ISO, as its name suggests, is constructed based on the property of the *interaction screening*. This property guarantees that in the limit of large number of samples the unique minimizer of the convex ISO objective (10.22) is achieved at $(\mathbf{J}_a, h_a) = (\mathbf{J}_a^*, h_a^*)$, meaning that the true interactions present in the model are fully “screened”. Indeed, the ISO is an empirical average of the inverse of the factors in the Gibbs measure; if $\mathcal{F}_a(\mathbf{J}_a, h_a) = \exp(\sum_{b \neq a} J_{ab} x_a x_b + h_a x_a)$, then $I(\mathbf{J}_a, h_a) = \mathbb{E} [\mathcal{F}_a^{-1}(\mathbf{J}_a, h_a)]_S$. In the limit of large number of samples, $S \rightarrow \infty$: $I(\mathbf{J}_a, h_a) \rightarrow I^*(\mathbf{J}_a, h_a) = \mathbb{E} [1/\mathcal{F}_a(\mathbf{J}_a, h_a)]$. The derivative of the ISO corresponds to weighted pairwise correlations, $\partial I^*/\partial J_{ab} = -\mathbb{E} [x_a x_b / \mathcal{F}_a(\mathbf{J}_a, h_a)]$, and this sheds light on its key property. When $(\mathbf{J}_a, h_a) = (\mathbf{J}_a^*, h_a^*)$, $\partial I^*/\partial J_{ab}|_{\mathbf{J}_a^*, h_a^*} = 0$, meaning that the minimum of ISO is achieved at $(\mathbf{J}_a, h_a) = (\mathbf{J}_a^*, h_a^*)$ as $S \rightarrow \infty$. See Fig. 10.3 for illustration of how optimization of the ISO objective will look like at large, but still finite, S .

Like in the case of the RPLE construction, discussed above in Section 10.2.1, the ISO should be appended with the ℓ_1 regularizer in order to promote sparsity [200], thus resulting in the *Regularized Interaction Screening Estimator* introduced in [200]

$$(\hat{\mathbf{J}}_a^{\text{RISE}}, \hat{h}_a^{\text{RISE}}) = \arg \min_{(\mathbf{J}_a, h_a)} (I_a(\mathbf{J}_a, h_a) + \lambda \|\mathbf{J}_a\|_1). \quad (10.23)$$



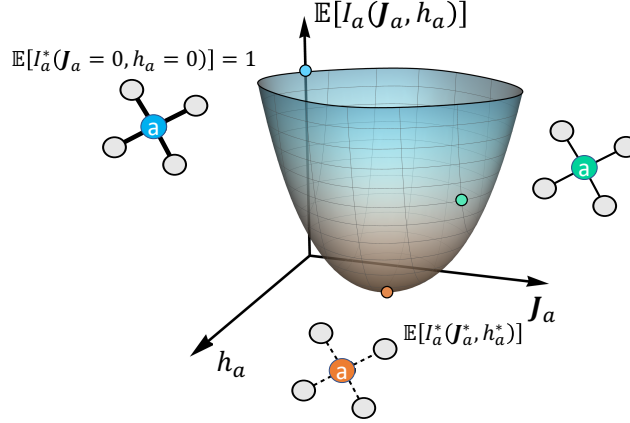


Figure 10.3: **Interaction Screening Objective for different probe values of model parameters in the large S limit.** The ISO is an empirical average of the inverse of the factors in the Gibbs measure, and its screening property becomes apparent in the limit of large number of samples. Changing the value of the probing parameters $(\mathbf{J}_a, h_a)$ in the ISO alters the effective interaction strength of x_a with its neighbors. This mechanism is schematically represented in the figure where the value of ISO for different values of probing parameters is depicted. When the probing parameters are equal to the true ones (J_a^*, h_a^*) , the ISO completely screens this interaction making x_a effectively independent of its neighbors. With some analysis, this can be shown to be equivalent to the ISO attaining its minimum at the true parameters of the model.

Analysis of the RISE is formally stated in the form of a Theorem which is an almost exact twin of Theorem 10.2.1, with the only correction being in the factor, $e^{3\beta d}$ replacing $e^{4\beta d}$ in Eq. (10.9). We refer interested reader to consult supplementary material section of [133] for details of the RISE (error) Theorem, which are very similar to the proof of Theorem 10.2.1, presented above in 10.2.1 (also based on the discussion of [133]). (Notice that the factors 3 and 4 in the exponents of the RISE and the RPLE, respectively, are likely to be an artifact of the employed proof techniques and is not tight as indicated by the computational experiments reported in [200, 133].)

Moreover it was also shown in [133] empirically that a log- version of RISE

$$(\hat{\mathbf{J}}_a, \hat{h}_a) = \arg \min_{(\mathbf{J}_a, h_a)} \left[\log I_a(\mathbf{J}_a, h_a) + \lambda \|\mathbf{J}_a\|_1 \right], \quad (10.24)$$

Companion notebook: Local convex learning without a partition function

Notebook `ch10/ch10_02_local_learning_estimators.ipynb` compares nodewise pseudo-likelihood and interaction-screening objectives on a sparse Ising model. It visualizes recovered couplings and the reduction of local parameter error as the number of samples increases.

called *log-RISE* improves performance of RISE.

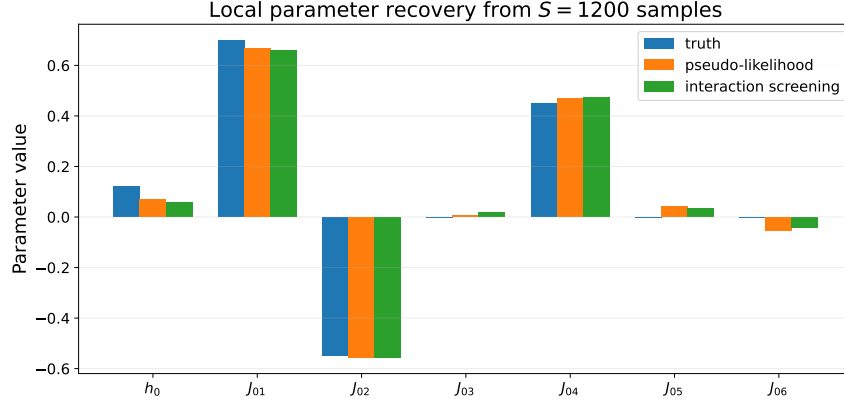


Figure 10.4: Local reconstruction of Ising couplings from samples. Each nodewise problem is

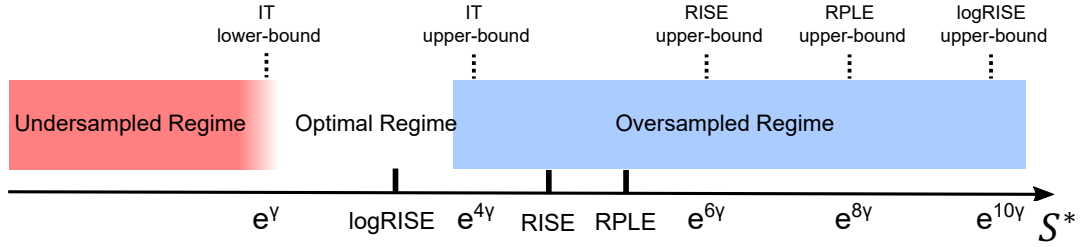


Figure 10.5: **Theoretical and empirical worst-case scaling of S^* with respect to γ .** This figure summarizes the main theoretical and empirical results of [133] for learning the Ising model. The red region represents the undersampled regime where the number of samples is insufficient for perfect graph reconstruction from the information theory perspective. The existence of an exact algorithm, albeit with an exponential computational complexity, has been proven for $S \propto e^{4\gamma}$, and thus represents an upper bound on the optimal number of samples S_{opt} that must lie in the white region, named the *optimal regime*. The quantities $e^{6\gamma}$ and $e^{8\gamma}$ and $e^{10\gamma}$ denote the theoretical upper bounds on the scaling for the RISE, the RPLE and the logRISE, respectively, all reported in [133]. However, these bounds are not tight, and the worst-case empirical scalings observed in the numerical experiments of [133] were much lower; these values are indicated in the chart as “RISE”, “RPLE”, and “logRISE” and correspond to $e^{4.5\gamma}$, $e^{5.2\gamma}$ and $e^{3.8\gamma}$, respectively (see the Supplementary Text of [133] for additional details on the scaling of the RISE). Remarkably, the empirical scaling for the logRISE lies within the *optimal regime*.

The analysis of RPLE and RISE reported in the preceding sections suggests the following three step implementation scheme:

1. Given S samples, we find the minimizer of the objective (10.5) or (10.24), respectively, at each node $a \in \mathcal{V}$, and obtain a collection of estimated parameters $(\hat{\mathbf{J}}_a, \hat{H}_a)$. Given that both estimators are convex, any appropriate convex optimization method can be used to find the minimizer of the objective function, the simplest one being a plain gradient descent supplemented with an additional projection step due to non-differentiability of the ℓ_1 regularization term. (Thus, results reported in [133] were based on using Ipopt optimization software [23].)
2. When a sufficient number of samples were used we observe (see Fig. 2A of [133]) that reconstructed $|\hat{J}_{ab}|$ are split in two groups, of inferred couplings that are close to zero and of those with significantly bigger intensities in absolute value, separated by a well-defined gap. We threshold the inferred couplings below the observed gaps to zero. The edges associated with the remaining non-zero couplings form the reconstructed graph $\hat{\mathcal{G}}$.
3. We optimize the unregularized objective, i.e. setting $\lambda = 0$, but only over the couplings corresponding to the edges in $\hat{\mathcal{G}}$, and obtain our final estimates $(\hat{\mathbf{J}}_a, \hat{h}_a)$.

Extensive empirical analysis of RPLE and RISE was complemented in [133] by analysis of the third algorithm called logRISE:

$$(\hat{\mathbf{J}}_a, \hat{h}_a) = \arg \min_{(\mathbf{J}_a, h_a)} \left[\log I_a(\mathbf{J}_a, h_a) + \lambda \|\mathbf{J}_a\|_1 \right]. \quad (10.25)$$

As the name suggests we use here not the ISO itself but its logarithm to form the log-RISE objective (10.25). Obviously, in the absence of the regularizer (for $\lambda = 0$), taking the logarithm of the ISO does not change its minimizer (thus guarantying exactness of the log-RISE algorithm). However, this difference is crucial for non-zero values of the regularization term, which suggests that logRISE might have better properties for the reconstruction problem due to a particular form of its first and second derivatives (see the Supplementary Text of [133] for additional explanations and details). Unfortunately, the proof techniques used in [133] for deriving bounds on scaling for the RPLE and the RISE results in the expressions which are less tight when applied to the estimator logRISE (since it no longer can be represented in a form of finite functional sum over individual samples): the number of required samples for logRISE in the worst case scales as $S^* \propto \exp(10\gamma) \log N / \alpha^2$.

Given looseness of the theoretical analysis (specifically well pronounced in the case of logRISE, but also significant in the case of RISE and RPLE), [133] relied on advanced empirical/numerical assessment of the algorithms' performance and comparison. We limit our discussion here of the advanced numerical analysis of the three algorithms to reproducing Fig. 10.5 from [133] which summarizing observations for theoretical and empirical worst-case scaling of S^* with respect to γ . (We remind that $\gamma = \beta d_{max} + h_{max}$, d_{max} is the maximum node degree, and α and β are upper bounds on elements of $\mathbf{J}$, i.e. $\alpha \leq |J_{ab}| \leq \beta$ is the lower bound.)

10.3 Infer-Learning

Given that inference is often the ultimate goal while information about stochastic system comes via samples, we naturally aim to formulate and resolve the challenge of *infer-learning*, i.e.

prediction-centric learning: evaluating marginal probabilities of interest directly from the samples, somehow bypassing the subtask of the GM (graph and factor) learning.

This problem of combining GM learning (graph and factor estimation) and inference (prediction) has some but relatively limited coverage in the literature. The paper of Wainwright [203], with a self-expressing title – *Estimating the “Wrong” Graphical Model: Benefits in the Computation-Limited Setting* – sets a good starting point for our discussion. Working under the restriction of limited computational budget allocated for inference [203] suggested a joint method in which the same convex variational relaxation is used to construct a GM estimator and to perform approximate inference (marginalization for the prediction step). The key conclusion of the paper is that in the computation-limited setting, using an inconsistent parameter estimator (i.e., an estimator that returns the “wrong” model even in the infinite data limit) is provably beneficial, since *the resulting errors can partially compensate for errors made by using an approximate prediction technique*. Notice that this approach was focused on specific marginals, e.g. marginal associated with a node or pair of nodes

This infer-learning problem was further discussed in [85]. (Note that *inference* was the term used in [85] to describe the approach. We will continue to use the term infer-learning as stating the combination of inference and learning in a more expressive way.) The problem was made tractable through additional assumptions – looking for model classes where inference is not tractable, but approximate inference with approximation bounds is. Specifically, the authors of [85] choose to work with Ising models with high girth and correlation decay amenable to asymptotically accurate inference via BP (message-passing) algorithms. An extended Chow-Liu algorithm, first making a structure prediction and then producing parameter estimation, was suggested. Here the extension means that graphs with a finite girth (and not trees) are learned according to the rules of the Chow-Liu algorithm (like BP, exact over trees, is applied for inference over loopy graphs, thus loopy-BP). Under proper structural and factor assumptions for GM (standard in the papers/books working within the setting of the correlation decay and finite girth, see e.g. [149]) it was proven that the reconstructed graph includes all the edges of the actual graph while mutual information associated with the redundant edges (miss-predicted) is sufficiently small.

The infer-learning setting, now limited primarily to trees, was recently addressed in [101, 32, 22, 26, 55]. Collectively these papers have advanced our understanding of the finite-sample analysis of the Chow-Liu algorithm [52] (see Section 10.1), showing that infer-learning is essentially a settled question for fully observed tree-structured GMs. Let us briefly review these significant contributions.

Bresler and Karzand has pioneered the prediction centric approach to learning (thus infer-learning) in [101, 32], where a (node) local measure – so-called local total variation distance – was used to derive a bound on sample complexity of the Chow-Liu algorithm. It was shown in [101, 32] that an ϵ -error in approximating exact probability distribution of an Ising model over tree, measured in terms of the (node) local total variational distance can be achieved with finite probability applying Chow-Liu to estimates based on $O(\max(\epsilon^{-2}, \exp(2\beta)) \log |\mathcal{V}|)$ i.i.d. samples, where $\beta = \max_{(a,b)} |J_{ab}|$. Most importantly, it was shown in [101, 32] that provable accurate prediction-centric learning can be achieved with a “wrong” graph/tree (consistently with what was observed empirically before, e.g. in [203]). With regards to the (low temperature regime, $\beta \rightarrow \infty$) deficiency — it was corrected in [26], where a new algorithm, coined Chow-Liu ++, was introduced. The new algorithm combines elements of the Chow-Liu algorithm with tree-metric reconstruction method to efficiently and optimally learn the Ising model with finite probability over tree under prediction-centric objective/loss (thus infer-learning).

A finite sample guarantees for the Chow-Liu algorithm to learn an approximate tree-structured GM were derived in [22]. Here a global (and not local as in [101, 32]) Kullback Leibler divergence was the approximation measure. It was shown that if a probability is tree-structured then the Chow-Liu algorithm with the plugin-estimator for mutual information with $O(|\mathcal{V}|/\epsilon)$ i.i.d. samples outputs with finite probability an ϵ -approximate (in KL distance) tree with finite probability. We count this result as contributing to progress in infer-learning (and not just learning) because even though the predicted graph (tree) is not correct, the resulting inference – KL divergence – is minimized. Notice that the result of [22] does not depend on the maximum strength of interaction, i.e. it also applies to the low-temperature regime which was a challenge for the approach of [101, 32].

Another graph-global perspective on the infer-learning over trees was advanced in [55]. Like in [22] an ϵ -approximate tree learning is achieved in the general setting – without assumptions on the strength of interaction (at any temperatures), however with another metric – total variation distance (global version of what was used in [101, 32]) – and with slightly more $O(|\mathcal{V}| \log(|\mathcal{V}|)/\epsilon)$ i.i.d. samples.

A principally new difficulty: the variables needed for learning are not observed. With hidden variables, even evaluating the complete-data sufficient statistics requires an inference step. Expectation–Maximization makes this nesting explicit: infer a distribution over the hidden variables using the current parameters (E-step), then fit parameters as though those inferred sufficient statistics were available (M-step). Contrastive divergence and related energy-based methods replace another exact inner expectation—the model expectation—by a short sampling computation. Thus learning becomes an outer optimization wrapped around an inner inference or sampling engine.

10.4 Learning with Hidden Variables

Hidden variables make the learning problem recursive: evaluating the objective or its gradient requires inference over variables that were never observed. Let X denote observed variables, Y hidden variables, and $p(X, Y | \theta)$ a parametric joint model. For observations $x^{(1)}, \dots, x^{(S)}$, the average observed-data log-likelihood is

$$\mathcal{L}(\theta) = \frac{1}{S} \sum_{s=1}^S \log p(x^{(s)} | \theta) = \frac{1}{S} \sum_{s=1}^S \log \int dy p(x^{(s)}, y | \theta), \quad (10.26)$$

and learning seeks

$$\theta^* = \arg \max_{\theta} \mathcal{L}(\theta). \quad (10.27)$$

The integral over Y is the new source of difficulty. The next two constructions handle it in complementary ways: EM introduces an explicit posterior over hidden variables, whereas energy-based gradients compare statistics under data-conditioned and model-generated distributions.

10.4.1 Expectation–Maximization

Expectation–Maximization (EM) alternates inference over hidden variables with parameter optimization [12, 62]. Introduce a separate normalized auxiliary density $q_s(y)$ for each observation.

Jensen's inequality gives

$$\log p(x^{(s)} | \theta) = \log \int dy q_s(y) \frac{p(x^{(s)}, y | \theta)}{q_s(y)} \quad (10.28)$$

$$\geq \int dy q_s(y) \log \frac{p(x^{(s)}, y | \theta)}{q_s(y)} := \mathcal{F}_s(q_s, \theta). \quad (10.29)$$

The gap is

$$\log p(x^{(s)} | \theta) - \mathcal{F}_s(q_s, \theta) = D(q_s(Y) \parallel p(Y | x^{(s)}, \theta)),$$

so the bound becomes tight at the exact posterior. Standard EM therefore uses

$$q_s^{(t+1)}(y) = p(y | x^{(s)}, \theta^{(t)}), \quad (10.30) \quad \text{👉}$$

$$\theta^{(t+1)} = \arg \max_{\theta} \frac{1}{S} \sum_{s=1}^S \mathbb{E}_{q_s^{(t+1)}} [\log p(x^{(s)}, Y | \theta)]. \quad (10.31) \quad \text{👉}$$

Each exact E-step makes the bound tight at the current parameter, and the M-step cannot decrease the observed-data likelihood. EM need not find the global maximum: latent models are typically nonconvex and can have multiple stationary points. If exact posterior inference is unavailable, replacing q_s by a tractable variational family yields variational EM, directly connecting this chapter to Chapter 6. The same ELBO architecture is neuralized in variational auto-encoders, where an encoder network amortizes the map $x \mapsto q_{\phi}(y | x)$.

10.4.2 Energy-based gradients and contrastive divergence

For an energy-based visible model

$$p_{\theta}(x) = \frac{\exp(-E_{\theta}(x))}{Z(\theta)}, \quad (10.32)$$

the gradient of the average log-likelihood separates into a data term and a model term,

$$\nabla_{\theta} \mathcal{L}(\theta) = -\mathbb{E}_{p_{\text{data}}} [\nabla_{\theta} E_{\theta}(X)] + \mathbb{E}_{p_{\theta}} [\nabla_{\theta} E_{\theta}(X)]. \quad (10.33)$$

The second expectation requires sampling from the current model, returning us to the MCMC machinery of Chapter 8. Gradient ascent uses

$$\theta^{(t+1)} = \theta^{(t)} + \varepsilon \nabla_{\theta} \mathcal{L}(\theta^{(t)}). \quad (10.34)$$

Contrastive divergence replaces an equilibrium model sample by a short Markov chain initialized at the data [88, 38]. If p_k is the distribution after k transition steps starting from the empirical law, the CD- k surrogate is

$$g_{\text{CD-k}}(\theta) = -\mathbb{E}_{p_0} [\nabla_{\theta} E_{\theta}(X)] + \mathbb{E}_{p_k} [\nabla_{\theta} E_{\theta}(X)]. \quad (10.35)$$

As $k \rightarrow \infty$ this approaches the exact likelihood gradient under ergodicity; for finite k it is generally biased. Its historical importance is algorithmic: short chains can provide a useful learning signal even when equilibration is expensive.

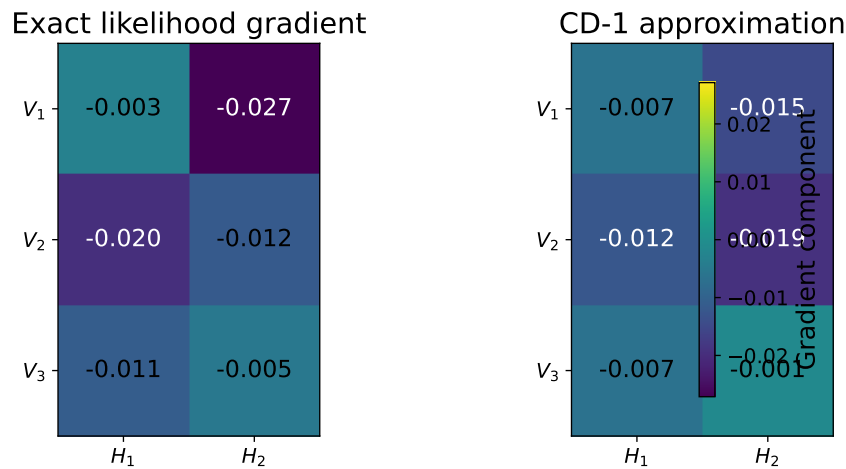


Figure 10.6: Exact likelihood gradient and the CD-1 approximation in a tiny Restricted Boltzmann Machine. Short-chain contrastive divergence can point in a similar direction while remaining a biased approximation to the equilibrium gradient.

Companion notebook: Hidden-variable learning: exact EM and short-chain gradients

Notebook `ch10/ch10_03_hidden_variables_em_and_cd.ipynb` first fits a small latent Bernoulli mixture by EM and then uses an exactly enumerable Restricted Boltzmann Machine to compare the true likelihood gradient with CD-1. Exact enumeration makes the finite- k bias visible rather than hiding it behind Monte Carlo noise.

10.4.3 Restricted Boltzmann Machines, deep belief networks, and neuralization

A Restricted Boltzmann Machine (RBM) is a bipartite undirected graphical model whose visible and hidden units interact across layers but not within a layer. Conditional on the visible layer, the hidden units are independent, and conversely; this conditional factorization makes block Gibbs sampling particularly simple. Stacking RBMs and using layerwise contrastive-divergence pretraining led historically to deep belief networks and to an early successful route for training deep neural architectures [89, 90]. In the language of this book, the important conceptual step is *neuralization*: a graphical-model energy is represented by increasingly expressive learned functions while the probability semantics, latent-variable structure, and sampling questions remain explicit.

The recent theory of learning RBMs also illustrates how graphical structure changes statistical difficulty. General-sign latent-variable models can encode complicated dependencies and are difficult to identify without assumptions. For attractive or ferromagnetic RBMs, monotonicity and positive dependence create exploitable structure; several reconstruction results use influences, covariances, and submodularity-like properties to recover neighborhoods or latent in-

teractions efficiently [33, 30, 77]. These results reinforce a theme that will recur in the scientific-GenAI part of the book: expressive neural parameterization need not eliminate structure; when the structure is retained, it can provide both algorithms and guarantees.

Exercise 10.4.1. *For the RBM in the companion notebook, increase one visible–hidden coupling while keeping the others fixed. Compare the exact likelihood gradient, CD-1, and CD-5. Explain the observed change using the mixing discussion of Chapter 8.*

Problem 10.1. *Design a prediction-centric learning objective for a graphical model in which only one scientifically meaningful marginal, rather than the complete parameter vector, is required. State what must be inferred during training, which approximation you would reuse at deployment time, and how you would evaluate whether parameter error and inference error compensate or reinforce each other. Relate your proposal to the infer-learning viewpoint of Section 10.3.*

Part II

Scientific Applications and Special Cases

Chapter 11

Applications: Structure as the Common Language

Reader's Guide.

How to read an applications chapter. Do not try to memorize eight unrelated domains. For every example ask the same translation questions: What are the variables? What makes them interact? Which factors are hard constraints and which are soft preferences? Is the target a static distribution, a conditional distribution, or a path law? Which task is required—MAP, marginals, normalization, learning, or generation? Once these questions are answered, the application becomes a concrete instance of the abstract machinery developed earlier.

The easiest recurring distinction: model versus computation. The same probability object can support very different computational tasks. In particle tracking, for example, the same matching model leads to an easy MAP assignment, difficult soft marginals, and a permanent as its partition function. In coding, the same parity constraints can be used for exact decoding, LP decoding, or probabilistic soft decoding. Throughout this chapter, separate the *scientific structure* from the *algorithm chosen to compute with it*.

Stress-test rather than showcase. Each application should be pushed beyond the regime where a standard approximation obviously works. Increase frustration, constraint density, noise, coupling, or another physically meaningful hardness parameter and compare against an exact small-instance reference whenever possible. The failure point is useful information: it tells us what a later neuralized approximation or structured generative method would actually need to improve.

A bridge to the second half of the book. The examples below are not disposable illustrations. Several become recurring laboratories later: matching for hard combinatorial structure, Ising/Gaussian models for controlled probabilistic structure, trajectory planning for path-space models, and language/coding examples for discrete generation. When later

chapters introduce learned messages, diffusion, bridges, or autoregressive generation, return to the same six-question translation rather than treating the modern method as a new subject detached from the original model.

The preceding chapters developed graphical models as a mathematical language for representing and computing with structured probability. This chapter changes perspective. We ask how the same structure-emphasizing graphical model methodology and approaches arise in several exemplary scientific, engineering, and information-processing domains.

The examples are deliberately heterogeneous (but also subjective). Particle tracking leads to matching; error correction to parity constraints; Boolean satisfiability to hard factors; robot planning to path-space distributions; infrastructure networks to Gaussian inference and physical constraints; social contagion to a dynamic graphical model; biological systems to trees, latent variables, and sparse networks; and sequence labeling to conditional random fields. The point is not that all of these problems should be solved by one algorithm. The point is that each becomes more transparent once we identify the same six ingredients:

1. the **structure** — graph, hierarchy, geometry, dynamics, or hard constraints;
2. the **probability object** — a joint law, conditional law, posterior, or path measure;
3. the principal **inference or optimization task**;
4. what is known and what must be **learned**;
5. how samples, trajectories, or decisions are **generated**;
6. what can be checked independently as a **verification criterion**.

The examples serve a second purpose which will become increasingly important in the remainder of the book. Graphical-model language is often native to these applications, and the inference, optimization, sampling, and learning methods developed so far can be remarkably effective. At the same time, the resulting computational problems are genuinely difficult. Exact methods may scale exponentially, relaxations can cease to be tight, iterative approximations can fail to converge or return inaccurate beliefs, and sampling may become prohibitively slow. These difficulties need not wait for asymptotically large systems: a small controlled example can often be pushed from an easy regime into a difficult one simply by changing coupling strength, noise level, constraint density, conditioning, or another physically meaningful parameter.

We therefore adopt a *stress-test principle* for the examples in this chapter and, when appropriate, throughout the book. An approximation should not be demonstrated only where it works. We also vary a hardness parameter far enough to expose a crossover and, whenever possible, a clear failure regime. The comparison should use an exact or independently certified reference on small instances and should distinguish external error from any internal diagnostic or certificate supplied by the method. Failure is not an embarrassment to be hidden: it identifies the computational frontier that a specialized algorithm, a neuralized surrogate, or a structured generative method must actually move.

This viewpoint also provides a bridge to the AI-era material later in the book. Neuralization and modern generative techniques are introduced not because classical graphical-model methods are obsolete, but because they offer new ways to amortize repeated computation, learn difficult closures or corrections, construct scalable proposals, and combine exact structure with flexible

approximators. We will return to many of the examples below in that role, inserting learned approximators and generative dynamics into the graphical constructions rather than appending them from outside.

11.1 Physics: from equilibrium factors to stochastic dynamics

The physics examples below are intentionally selective rather than a survey of graphical models in physics. They emphasize two problems closely connected to the author's own research trajectory: equilibrium spin models and their stochastic dynamics, and particle association/tracking motivated by Lagrangian descriptions of complex flows. Many other natural examples could have been chosen; these two are used because they connect particularly directly to the mathematical constructions developed earlier and to the stochastic and generative extensions developed later.

11.1.1 Glauber dynamics

The Ising model of Section 2.4.2 is a static probability law. Glauber dynamics turns that law into a local stochastic process whose stationary distribution is the Ising measure [76]. Consider binary spins $x_a \in \{-1, +1\}$ with

$$P(\mathbf{x}) = \frac{1}{Z} \exp \left(\sum_{(a,b) \in \mathcal{E}} J_{ab} x_a x_b + \sum_{a \in \mathcal{V}} h_a x_a \right).$$

For a single-site heat-bath update at node a , define the local field

$$H_a(\mathbf{x}_{\setminus a}) = h_a + \sum_{b \in \partial a} J_{ab} x_b.$$

Conditional on all other spins, the updated spin is drawn from

$$P(x'_a = s \mid \mathbf{x}_{\setminus a}) = \frac{\exp(s H_a)}{2 \cosh H_a}, \quad s \in \{-1, +1\}.$$

Thus the static factors determine a dynamic transition kernel. This particular update is simply Gibbs sampling, so it preserves the Gibbs distribution and connects directly to the invariance and mixing questions of Chapter 8. The same distinction between a *target distribution* and a *dynamics designed to sample it* will become central when we move from graphical models to diffusion, bridge, and controlled generative processes.

A useful neighboring example is a Gaussian graphical model. There both the target law and local conditional updates are Gaussian. Ising/Glauber and Gaussian/Gaussian therefore provide two elementary laboratories for the later transition from static graphical structure to learned stochastic dynamics.

Exercise 11.1.1. *Derive the heat-bath update above directly from the conditional probability $P(x_a \mid \mathbf{x}_{\setminus a})$. Then write the corresponding one-spin Metropolis–Hastings acceptance probability and verify detailed balance for both kernels.*

11.1.2 Particle tracking as a perfect-matching model

Consider N visually indistinguishable particles observed at two times $t_1 < t_2$. Let $\mathbf{r}_a(t_1)$ and $\mathbf{r}_b(t_2)$ denote the two point clouds. A dynamical model of the underlying flow supplies a nonnegative transition weight

$$w_{ab} = p(b \leftarrow a),$$

measuring how plausible it is that particle a in the first snapshot became particle b in the second. Introduce binary assignment variables $x_{ab} \in \{0, 1\}$. Because every observed particle must have exactly one predecessor and one successor, the posterior over assignments has the factor-graph form

$$P^{(\text{PM})}(\mathbf{x}) = \frac{1}{Z_{\text{PM}}} \prod_{a=1}^N \mathbb{1}\left(\sum_{b=1}^N x_{ab} = 1\right) \prod_{b=1}^N \mathbb{1}\left(\sum_{a=1}^N x_{ab} = 1\right) \prod_{a,b=1}^N w_{ab}^{x_{ab}}. \quad (11.1)$$

For a complete bipartite graph,

$$Z_{\text{PM}} = \text{perm}(W), \quad W = (w_{ab}),$$

so the same model simultaneously exposes three different tasks:

- **MAP tracking:** maximize $\sum_{a,b} x_{ab} \log w_{ab}$ over perfect matchings;
- **soft tracking:** compute edge marginals $P(x_{ab} = 1)$;
- **normalization:** compute the permanent of W .

The MAP problem is a weighted bipartite matching problem and its standard assignment LP is integral. The soft problem is substantially harder: the permanent is a canonical difficult partition function and is revisited in Section 13.1.

The same construction extends to the general minimum-weight perfect-matching problem on a non-bipartite graph. If $x_e \in \{0, 1\}$ denotes whether edge e is selected, the energy and node constraints can be written

$$E(\mathbf{x}) = \sum_{e \in E} c_e x_e, \quad (11.2)$$

$$\sum_{e \in \delta(v)} x_e = 1, \quad v \in V. \quad (11.3)$$

For bipartite graphs the degree-constraint polytope is integral; for general graphs additional odd-set or blossom inequalities are required. This is precisely the kind of distinction between an exact direct LP and a potentially loose local graphical relaxation emphasized in Chapter 12. The graphical-model formulation and its loop corrections were developed in [42].

Companion notebook: Particle association: MAP matching and soft marginals

Notebook `ch11/ch11_01_particle_tracking_matching.ipynb` constructs two small particle snapshots, builds local transition likelihoods, enumerates the exact matching posterior, computes edge marginals, and compares them with the MAP assignment. See also Figs. (11.1, 11.2).

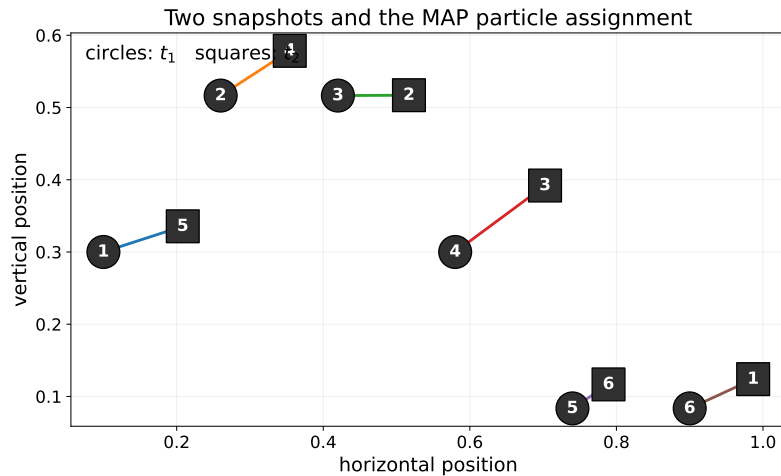


Figure 11.1: Two particle snapshots (circles and squares) and the MAP association. The graphical-model variables are assignment edges; the one-to-one constraints couple otherwise local transition likelihoods into a global matching problem.

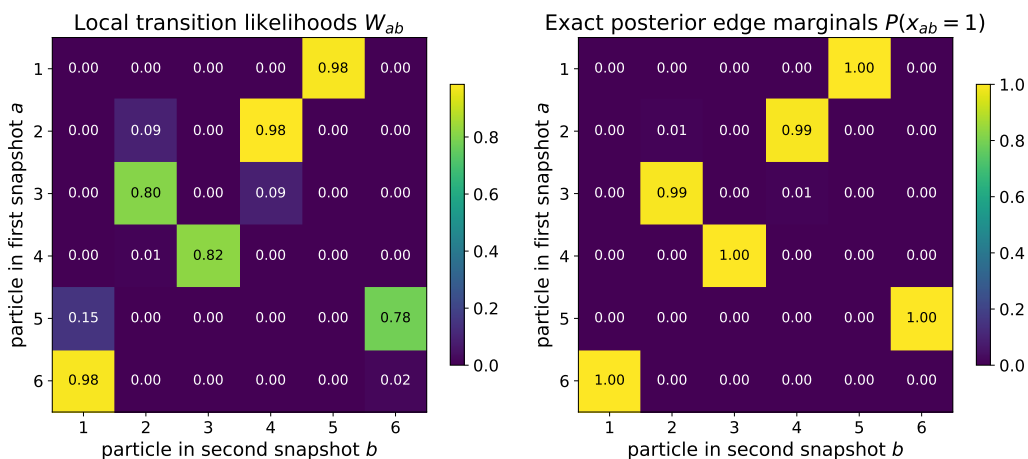


Figure 11.2: Local transition likelihoods and exact posterior edge marginals for the particle-association model. Hard matching constraints can sharpen or redirect local likelihood information.

Exercise 11.1.2. Increase the diffusion scale used to construct w_{ab} in the companion notebook. Compare the entropy of the matching posterior, the MAP assignment, and the edge marginals. Identify a regime in which the MAP assignment remains unchanged while the uncertainty of the soft inference problem changes substantially.

11.2 Information theory: graphical error-correcting codes

A linear binary code is a particularly clean example of a graphical model in which almost all structure is carried by hard factors. Let $H \in \{0, 1\}^{M \times N}$ be a parity-check matrix over $\text{GF}(2)$. The code is

$$\mathcal{C} = \{\mathbf{x} \in \{0, 1\}^N : H\mathbf{x} = 0 \pmod{2}\}, \quad (11.4)$$

and its dimension is $L = N - \text{rank}_{\text{GF}(2)}(H)$, not necessarily $N - M$ if the checks are linearly dependent. The Tanner graph is the bipartite factor graph connecting bit i to check α whenever $H_{\alpha i} = 1$ [167, 136].

Suppose a codeword passes through a memoryless channel and y_i is the observation associated with bit x_i . The posterior is

$$P(\mathbf{x} | \mathbf{y}) = \frac{1}{Z(\mathbf{y})} \prod_{i=1}^N p_{\text{ch}}(y_i | x_i) \prod_{\alpha=1}^M \mathbb{1} \left(\sum_{i \in \partial \alpha} x_i = 0 \pmod{2} \right). \quad (11.5)$$

The word-MAP decoder is

$$\mathbf{x}^{(\text{MAP})} = \arg \max_{\mathbf{x} \in \mathcal{C}} P(\mathbf{x} | \mathbf{y}), \quad (11.6)$$

whereas bitwise MAP decoding uses the marginal posterior

$$x_i^{(\text{bMAP})} = \arg \max_{s \in \{0, 1\}} P(x_i = s | \mathbf{y}). \quad (11.7)$$

These two estimators optimize different loss functions and need not return a mutually consistent word.

The same factorization is the natural domain for sum-product decoding. On a cycle-free Tanner graph BP is exact; on practical LDPC graphs it becomes loopy BP, whose success is one of the historically important demonstrations of message passing far beyond trees. Chapters 6 and 9 explain the variational and loop-correction viewpoints behind this phenomenon.

At zero temperature one obtains LP decoding [67]. Let

$$\gamma_i = \log \frac{p_{\text{ch}}(y_i | 0)}{p_{\text{ch}}(y_i | 1)}$$

be the channel log-likelihood ratio, and let $\mathcal{P}(H)$ denote the fundamental polytope obtained by intersecting the local parity-check polytopes. The LP decoder solves

$$\min_{\mathbf{f} \in \mathcal{P}(H)} \sum_{i=1}^N \gamma_i f_i. \quad (11.8) \quad \text{👉}$$

If the optimizer is integral, it is a codeword and has the maximum-likelihood certificate property. Fractional pseudocodewords expose exactly the relaxation gap discussed more generally in Chapter 12.

Companion notebook: LDPC decoding: exact marginals and belief propagation

Notebook `ch11/ch11_02_ldpc_decoding.ipynb` uses a small Hamming-style parity-check code, sends a codeword through an additive Gaussian channel, computes the exact posterior

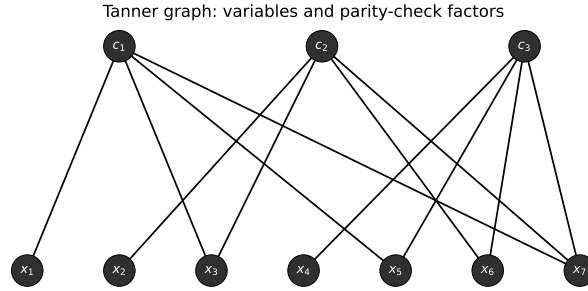


Figure 11.3: Tanner graph used in the decoding example. Variable nodes (bottom row) represent code bits; check nodes (top row) enforce local parity constraints.

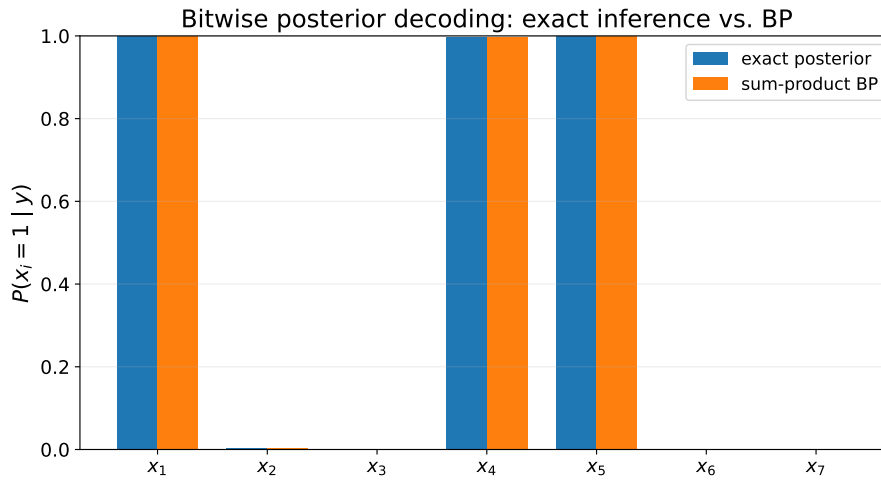


Figure 11.4: Bitwise posterior decoding on a small linear code at one moderate-noise operating point: exact enumeration versus sum-product BP. This first comparison is deliberately benign; Fig. 11.5 pushes the same inference problem into regimes where loopy BP is no longer accurate.

by codeword enumeration, and compares exact bit marginals with sum-product BP.

A single successful operating point is not enough to characterize an approximate decoder. The companion notebook therefore fixes one standardized noise direction and sweeps the channel standard deviation σ . It also repeats BP after adding a *redundant* parity check obtained as a binary sum of two existing checks. The redundant check leaves the code, and hence the exact posterior, unchanged, but it changes the Tanner graph on which loopy BP operates. Figure 11.5 shows both effects. At low noise the exact posterior is concentrated and both BP representations are essentially exact. Around the intermediate-noise crossover the bit-marginal error becomes substantial; in this small example the redundant-check representation reaches a maximum bit-marginal error of about 0.33, even though it represents exactly the same code. At still larger noise the exact posterior itself becomes broad, separating approximation error from intrinsic

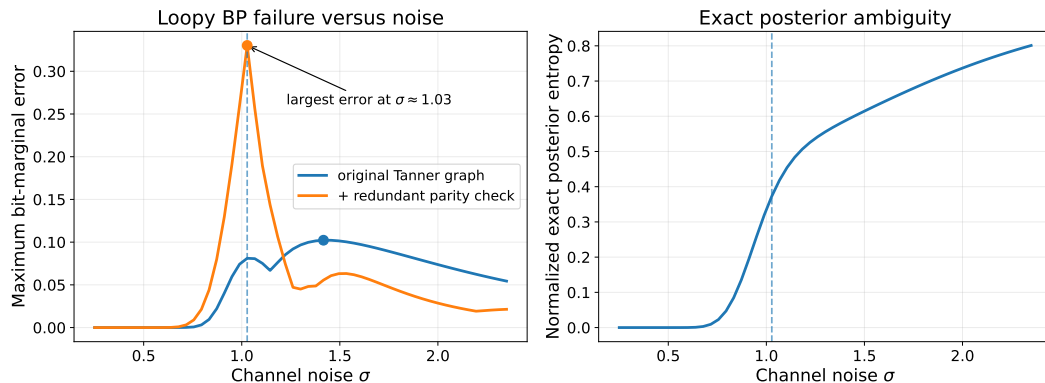


Figure 11.5: Stress test of loopy sum-product decoding. Left: maximum bit-marginal error relative to exact enumeration as the channel-noise standard deviation σ is increased. The two BP curves correspond to Tanner graphs defining the same code; the second adds one redundant parity check and can fail much more strongly in the crossover regime. Right: normalized entropy of the exact codeword posterior, which quantifies the intrinsic ambiguity of the decoding problem independently of BP.

ambiguity of the decoding problem.

This experiment illustrates two distinct failure mechanisms that will recur later in the book. First, approximation quality can change sharply with a physical or statistical control parameter such as SNR. Second, for loopy algorithms, two factor graphs representing the same exact probability law need not be *approximation-equivalent*. The latter observation is especially important when messages are later unfolded or neuralized: learning should not obscure the dependence of the approximate algorithm on the chosen representation.

Exercise 11.2.1. Use the stress-test portion of the notebook to locate the value of σ at which each Tanner-graph representation has its largest marginal error. At that value compare exact bit marginals, BP marginals, bitwise hard decisions, and word MAP. Then vary damping and iteration count. Which failures are reduced by solver tuning, and which persist because the loopy fixed point itself is inaccurate?

The inverse problem: designing the code. Decoding assumes that H is given. Designing a code asks for the graph itself: choose a sparse parity-check structure with good distance, favorable iterative-decoding thresholds, manageable finite-length trapping sets, and an acceptable encoding/decoding cost. These requirements pull in different directions, and the resulting problem is a large combinatorial design problem rather than a single solved optimization template. Modern LDPC design therefore relies heavily on ensembles, density-evolution ideas, protographs, and structured constructions [167]. From the viewpoint of this book it is a particularly attractive *structure-learning/design* challenge: the object being optimized is the graph on which inference will later run.

Problem 11.1. Formulate a small code-design problem in which the Tanner graph is the decision variable. Choose at least two competing criteria — for example minimum distance and short-cycle suppression, or BP decoding error and check sparsity. Explain which properties are

local graph statistics and which require global inference or enumeration. Test the formulation for a fixed small (N, L) where exhaustive graph search is still possible.

11.3 Computer science: satisfiability as a hard-factor model

Boolean satisfiability is a canonical example in which a factor graph is not merely a convenient picture: it is the problem statement. Let $x_a \in \{0, 1\}$ and let each clause A involve K variables. For every incident pair (a, A) let $y_{aA} \in \{0, 1\}$ specify the value of x_a that satisfies the corresponding literal. Then the K -SAT clause factor is

$$f_A^{(K\text{-SAT})}(\mathbf{x}_A) = \mathbb{1}(\exists a \in \partial A : x_a = y_{aA}),$$

and the unnormalized graphical model is

$$\tilde{P}(\mathbf{x}) = \prod_{A \in \mathcal{V}_2} f_A^{(K\text{-SAT})}(\mathbf{x}_A). \quad (11.9) \quad \text{👉}$$

If the formula is satisfiable, the normalized law is uniform over satisfying assignments. If it is unsatisfiable, the partition function vanishes and no normalized probability law exists. In either case the partition function

$$Z^{(K\text{-SAT})} = \sum_{\mathbf{x}} \prod_A f_A^{(K\text{-SAT})}(\mathbf{x}_A)$$

counts satisfying assignments, thereby turning the decision problem SAT into the counting problem #SAT.

A soft version replaces each zero clause factor by a Boltzmann penalty $\exp(-\beta)$ per violated clause. This one modification creates a bridge from constraint satisfaction to statistical mechanics: finite β defines a proper Gibbs law over all assignments, while $\beta \rightarrow \infty$ concentrates on minimum-violation configurations. It also creates a natural test bed for MCMC, BP, variational approximations, and zero-temperature MAP algorithms developed earlier.

Companion notebook: From hard clauses to a probability distribution

Notebook `ch11/ch11_03_ksat_factor_graph.ipynb` builds a small 3-SAT factor graph, exactly enumerates assignments, computes variable marginals conditioned on satisfiability, and tracks how the feasible set shrinks as clauses are added.

Exercise 11.3.1. Replace the hard factors in the notebook by $f_A = \exp(-\beta \mathbb{1}(A \text{ is violated}))$. Plot the expected number of violated clauses and the entropy of the Gibbs law as functions of β . Explain the zero-temperature limit in the language of Chapter 12.

11.4 Robotics: planning as inference on trajectories

Factor graphs are widely used in robotics because localization, mapping, sensor fusion, and trajectory estimation naturally decompose into local measurement and transition factors [61]. A complementary viewpoint treats planning itself as inference. Let s_t be a robot state, let

3-SAT factor graph; dashed edges denote negated literals

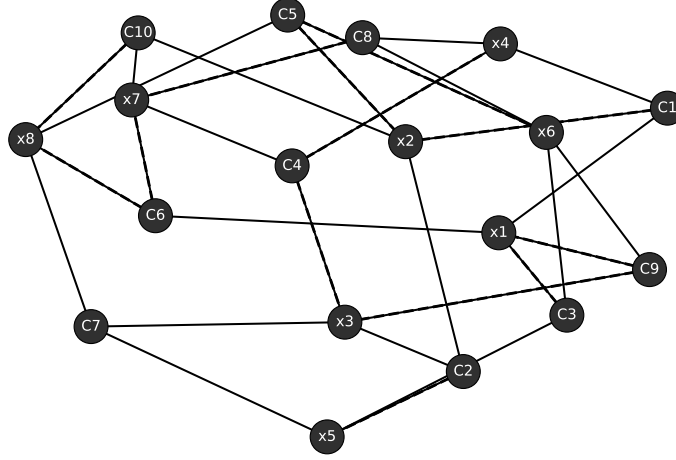


Figure 11.6: A 3-SAT factor graph. Dashed incidences denote negated literals. Hard local clause factors combine into a globally constrained feasible set.

$P_0(s_{t+1} | s_t)$ be a passive or reference dynamics, and let $c_t(s_t)$ and $c_T(s_T)$ encode running and terminal costs. The path distribution

$$P(s_{0:T}) \propto p_0(s_0) \prod_{t=0}^{T-1} P_0(s_{t+1} | s_t) \exp\left(-\frac{c_t(s_t)}{\lambda}\right) \exp\left(-\frac{c_T(s_T)}{\lambda}\right)$$

is a chain-structured graphical model. MAP inference returns the most probable low-cost trajectory, while marginal inference describes uncertainty about where the robot is likely to be at intermediate times. In KL-control and linearly solvable formulations this “control as inference” relation becomes exact and algorithmically useful [100]; later chapters will connect it to Doob transforms and path-integral generation.

Companion notebook: Robot motion: MAP trajectory versus soft occupancy

Notebook `ch11/ch11_04_robot_path_factor_graph.ipynb` enumerates short paths on a grid with obstacles, compares the MAP route with time-averaged state marginals, and shows how the same path factors support both optimization and uncertainty quantification. See Fig. (11.7).

Problem 11.2. Replace the fixed terminal state in the notebook by a terminal distribution over several goals. Derive the backward dynamic-programming recursion for the corresponding path partition function. Then identify which object in that recursion becomes the desirability or h -function in linearly solvable control.

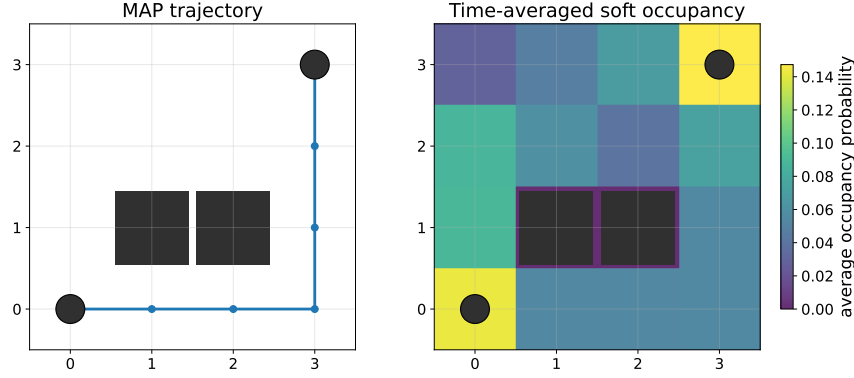


Figure 11.7: Planning as inference on a short grid. Left: the MAP path. Right: soft occupancy obtained by summing over feasible trajectories. Hard obstacle factors and local transition factors have different roles but share the same graphical representation.

11.5 Infrastructure networks: topology, physics, and uncertainty

Infrastructure networks already come with a graph before any statistical model is introduced. In electric, gas, transportation, and other flow systems, this geographical or engineering graph specifies which states can interact and where conservation laws act. Graphical models then add uncertainty to this physical skeleton, allowing measurements, hidden states, forecasts, and constraints to be combined locally [49, 188].

A minimal example is linear-Gaussian state estimation. Let θ collect hidden nodal states and suppose

$$z = H\theta + \epsilon, \quad \epsilon \sim \mathcal{N}(0, R),$$

with prior $\theta \sim \mathcal{N}(\mu_0, \Sigma_0)$. The posterior is Gaussian with precision and information vector

$$J = \Sigma_0^{-1} + H^T R^{-1} H, \quad h = \Sigma_0^{-1} \mu_0 + H^T R^{-1} z.$$

If each measurement depends only on a bus or a neighboring line, J inherits the sparse network structure. Chapter 13 explains how such sparsity supports Gaussian BP and how convergence depends on more than sparsity alone.

The linear-Gaussian case is only a starting point. Real flow networks introduce nonlinear constitutive equations, inequality constraints, switching, discrete controls, and uncertain injections. Chapter 12 uses network flow to show that physical structure can sometimes yield LP integrality and sometimes destroy it. The later structured-GenAI chapters will revisit the same issue from a different angle: which parts of the physics should remain explicit factors, and which unknown closures or uncertainty models should be learned?

Companion notebook: Gaussian state estimation on a physical network

Notebook `ch11/ch11_05_infrastructure_gaussian.ipynb` constructs a small network, generates noisy local measurements, and performs exact Gaussian posterior inference for

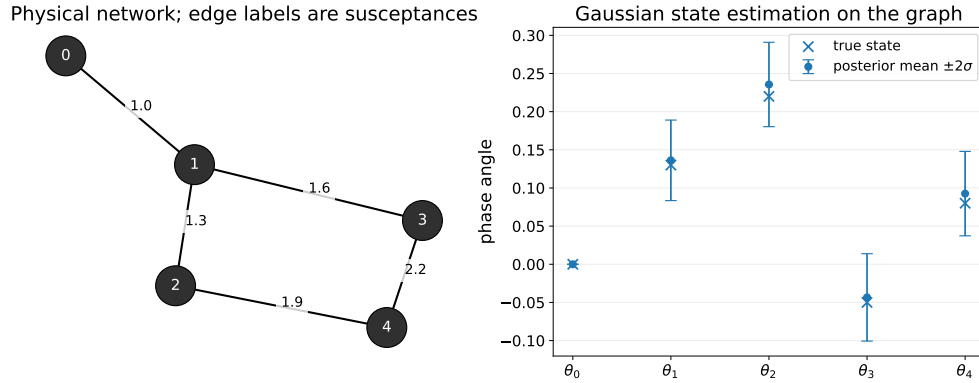


Figure 11.8: A network state-estimation example. The physical graph determines sparse measurement structure; Gaussian inference combines those measurements into posterior means and uncertainties.

the hidden nodal state.

11.6 Social networks: the independent-cascade model

In a social or contact network the graph specifies opportunities for influence or transmission rather than deterministic interactions. The independent-cascade model assigns a probability p_{uv} to each directed edge. When node u first becomes active, it gets one opportunity to activate each currently inactive neighbor v ; the attempts are independent across edges [102]. The state therefore evolves through a dynamic graphical model whose local factors encode transmission attempts and whose global quantity of interest may be the eventual cascade size.

Let A_t denote the active set and $F_t \subseteq A_t$ the nodes newly activated at time t . For an inactive node v ,

$$P(v \notin A_{t+1} \mid A_t, F_t) = \prod_{u \in F_t: (u,v) \in E} (1 - p_{uv}).$$

This noisy-OR factorization makes clear what is local, while correlations arise globally because many activation paths can merge. Related graphical formulations have been used for prediction and learning of spreading processes [44, 115].

Companion notebook: Exact and Monte Carlo propagation of a social cascade

Notebook `ch11/ch11_06_independent_cascade.ipynb` propagates the finite-state cascade distribution exactly on a small graph, computes node activation marginals, and compares them with Monte Carlo estimates.

Problem 11.3. *Influence maximization asks for a seed set of fixed size that maximizes expected cascade size. Explain why this is an outer combinatorial optimization wrapped around an inner probabilistic inference problem. Reproduce the greedy strategy on the companion network and*

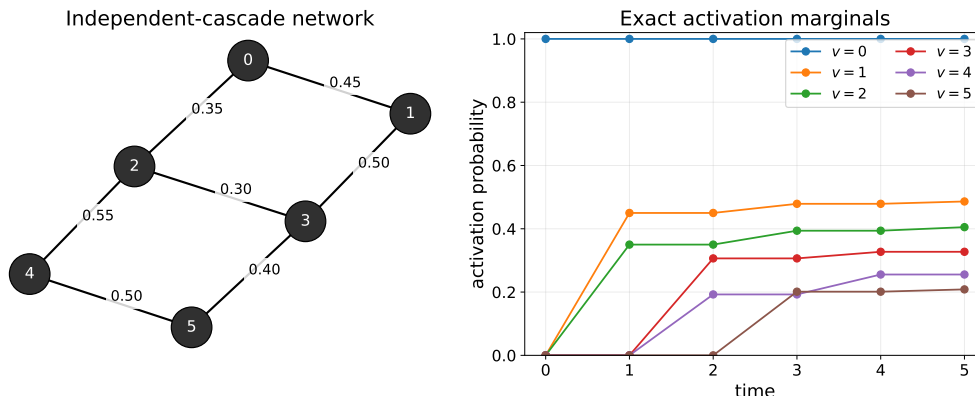


Figure 11.9: Independent-cascade dynamics. Left: directed transmission network. Right: exact activation probabilities as functions of time. Local transmission factors generate globally correlated activation events.

compare exact inner evaluation with Monte Carlo evaluation. Track how sampling error changes the selected seed set.

11.7 Biological networks: several kinds of structure

Biology is useful precisely because there is no single “biological graphical model.” Different biological questions expose different notions of structure, and the appropriate graph must be chosen accordingly.

Evolutionary trees. Phylogenetic inference uses a tree whose leaves are observed sequences and whose internal nodes represent ancestral states. Felsenstein’s pruning recursion is a tree dynamic-programming algorithm for likelihood evaluation [68]; in the language of Chapter 3, it is sum-product message passing. Changing summation to maximization produces an ancestral MAP calculation.

Population structure and latent variables. Genotype data can be modeled using unobserved population memberships that explain correlations among observed alleles [162]. This is a latent-variable graphical model, linking the application directly to EM and hidden-variable learning in Section 10.4.

Cellular and molecular networks. Sparse graphical models have been used to infer regulatory dependencies from high-dimensional measurements [71], while probabilistic motif and structure/function models encode more specialized biological organization [219]. Modern Bayesian graphical approaches emphasize sparsity, prior information, and uncertainty over the graph itself when the number of biological samples is small relative to the number of possible interactions [155, 154]. Hybrid graphical/deep-learning models are increasingly attractive when interpretability and biological consistency matter [217].

The methodological lesson is more important than the list of applications: biological knowledge often supplies partial structure but not a complete probability law. This is exactly the regime in which the next parts of the book become useful — retain known interactions, symmetries, or pathways explicitly, and neuralize only the factors, messages, or dynamical components that are genuinely unknown.

Problem 11.4. *Choose one biological network problem above and write down its Rosetta-Stone specification: variables, factors, observed and hidden quantities, desired marginal or MAP quantity, learned object, and a verification criterion. Then identify one place where a neural approximator could be inserted without removing the probabilistic meaning of the model.*

11.8 Structured language before foundation models: conditional random fields

Before transformers became the dominant language architecture, sequence labeling provided one of the clearest applications of conditional graphical models. A linear-chain Conditional Random Field (CRF) models a label sequence $\mathbf{y} = (y_1, \dots, y_T)$ conditional on an observed token sequence $\mathbf{x} = (x_1, \dots, x_T)$ [110, 119]:

$$P_{\theta}(\mathbf{y} \mid \mathbf{x}) = \frac{1}{Z_{\theta}(\mathbf{x})} \prod_{t=1}^T \psi_t(y_t, \mathbf{x}; \theta) \prod_{t=1}^{T-1} \psi_{t,t+1}(y_t, y_{t+1}, \mathbf{x}; \theta).$$

The observation $\mathbf{x}$ can enter rich local features without requiring a generative model for the words themselves. Once $\mathbf{x}$ is fixed, the label graph is a chain. Therefore forward-backward computes exact label marginals and Viterbi/max-product computes the exact MAP label sequence.

This example is an important precursor to the neuralization story. A modern encoder can replace hand-designed token features inside ψ_t while the CRF layer retains exact structured normalization and sequence consistency. The model is then neither a purely classical CRF nor a structure-free neural network: a learned representation feeds an explicit graphical inference layer.

Companion notebook: Sequence labeling with an exact conditional graphical model

Notebook `ch11/ch11_07_crf_sequence.ipynb` constructs a small linear-chain CRF, computes exact forward-backward marginals and the Viterbi sequence, and visualizes the factorization and posterior label probabilities.

Exercise 11.8.1. *Modify one transition potential in the CRF notebook so that the local unary preference and the sequence preference disagree strongly. Compare the independent-token prediction, marginal-MAP labels, and the Viterbi sequence. Which notion of “best label” is being optimized in each case?*

What changes when we arrive at generative AI? The classical applications above begin with an explicit structured model and ask how to compute with it. Modern generative

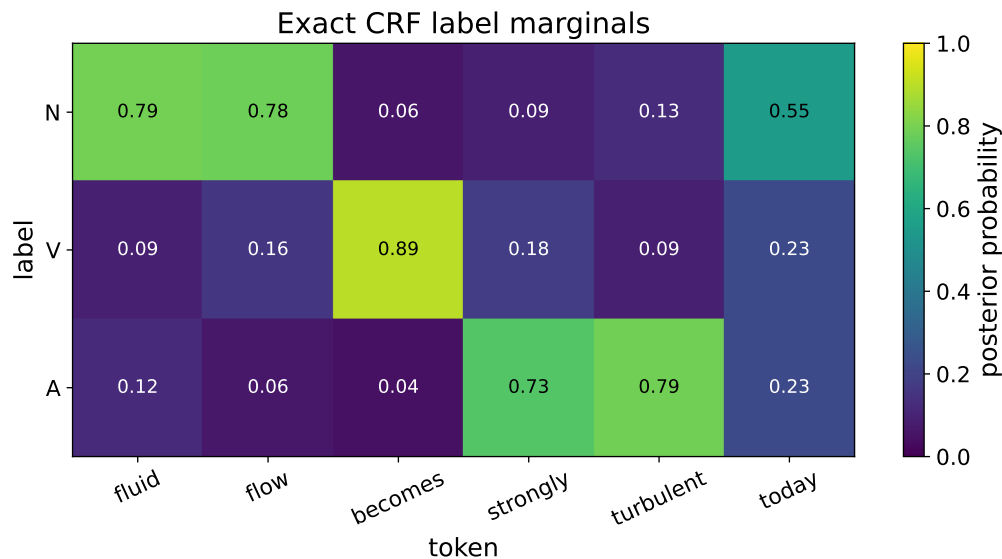


Figure 11.10: Exact label marginals in a toy linear-chain CRF. Local evidence alone is ambiguous at several positions; transition factors propagate sequence-level context.

AI often reverses the emphasis: a powerful sampler or neural density model is available first, while the desired scientific structure is only partially explicit. The central question then becomes how to insert constraints, factors, symmetries, or dynamics *without discarding* the expressive generative mechanism. This is the “structuralization” direction developed in Parts III–IV.

11.9 Generative AI: from graphs to graphical structure

Modern AI already uses graphs extensively, but a *graph* and a *graphical model* are not the same object. A Graph Neural Network (GNN) learns local aggregation and update maps on graph-shaped data; graph-aware transformers modify attention using topology; and graph diffusion models generate graph-valued objects [6, 86, 196, 207, 130]. These are important developments, but the central question of this book is sharper:

How can explicit probabilistic structure — factors, hard constraints, conditional independences, symmetries, conservation laws, and local compatibility conditions — be retained while neural networks provide the approximations needed at modern scale?

The direction also runs in reverse. Classical graphical-model algorithms contain well-defined computational objects — messages, cavity fields, elimination factors, variational beliefs, transition kernels, and corrections. Instead of replacing the whole construction by a black box, one can learn these objects selectively. This motivates the first new part of the book: *neuralizing graphical models*.

The second new part asks how the structure can be inserted into generative dynamics. For an energy-based model it may enter as additive factors; for a normalizing flow as a sparse or constraint-preserving transport; for a diffusion model as a local or factorized score and structured guidance; for a bridge or controlled process as passive dynamics, path costs, and terminal factors; and for an autoregressive model as structured conditionals, masks, or local corrections. The recent *Mathematics of Generative AI* develops the generative mechanisms themselves; here we will use them as building blocks and focus on what changes when the target distribution or path law has explicit graphical structure.

Advanced connection: The application chapter is a source of future benchmarks

The examples discussed in this chapter illustrate both the effectiveness and the complexity of classical (pre-AI) graphical-model approaches. Particle matching, LDPC decoding, K-SAT, robot paths, network state estimation, cascades, biological networks, and CRFs provide controlled test cases in which exact structure is known. They allow us to ask whether a learned method preserves constraints, calibrated marginals, symmetries, or known optima — rather than judging a generative model only by visual or aggregate sample quality.

11.10 Application Rosetta Stone

The chapter can be summarized by the following map. Its purpose is not taxonomy for its own sake; it records which mathematical object should survive when the model is later neuralized.

Application	Explicit structure	Main task	probabilistic	Natural future neuralized object
Particle tracking	one-to-one matching	MAP, permanent	marginals,	transition likelihoods / motion model
LDPC decoding	parity-check factors	bit MAP	marginals, word	learned messages / channel model
K-SAT	hard clauses	feasibility, MAP	counting,	clause-guided sampler
Robot planning	path dynamics and costs	trajectory MAP/marginals		value, desirability, control
Infrastructure	physical network and sensors	hidden-state inference/control		closures, factors, messages
Social cascade	directed stochastic influence	activation seed design	marginals,	transmission factors
Biological networks	trees, pathways, sparse interactions	structure/latent inference		factors and latent representations
Language/CRF	chain or structured labels	conditional marginals/MAP		unary features / messages

Problem 11.5. Choose two rows of the table whose application domains look unrelated. Write both as factorized probability laws and identify an algorithmic object that plays the same mathematical role in both. This exercise is the intended transition from application diversity to the structure-first synthesis developed in the remainder of the book.

The applications above point toward neuralization and structured generative AI, but there is an important intermediate lesson. General-purpose graphical-model methodology is only one

part of the story. When additional algebraic, combinatorial, geometric, or statistical structure is available, a method specialized to that structure can be dramatically stronger than a generic approximation. Before turning to neural surrogates, we therefore pause for three chapters devoted to such structure-aware classical methods.

Chapter 12 studies MAP problems for which submodularity, total unimodularity, fractional polymorphisms, or carefully chosen LP tightenings produce guarantees unavailable to a generic MAP relaxation. Chapter 13 develops canonical soft-inference laboratories — permanents, Gaussian graphical models, planar models, and related constructions — where special structure yields unusually sharp algorithms or controlled approximations. Chapter 14 turns to advanced structure learning, where order structure, acyclicity, and sparsity can be exploited directly. These chapters are not a detour from the AI-era synthesis. They establish stronger structured baselines and reveal exactly which pieces should be preserved, accelerated, amortized, or corrected when neuralization begins in the following part.

Chapter 12

MAP: Exact LP Classes, Message Passing, and Tightening

Reader's Guide.

Why this chapter? A linear-programming relaxation of MAP can fail for one reason only: the relaxed feasible set contains fractional points that are not realizable as a single global configuration. The chapter asks when this enlargement is harmless and when it must be repaired. The answer depends not only on graph topology but also on algebraic properties of the factors and geometry of the constraints.

The simplest warning example: local agreement need not imply a global assignment. On an attractive binary model, locally consistent pairwise beliefs can often be rounded to an integral configuration without losing objective value. On a frustrated cycle, each edge may prefer a locally sensible assignment while the preferences cannot all be satisfied simultaneously. The local polytope can then contain a fractional optimum strictly better than every true configuration. Keep this picture in mind whenever the text speaks about an “integrality gap.”

Two different routes to exactness. Submodularity/fractional polymorphisms constrain the *objective functions*: averaging or combining candidate configurations cannot make the local energy worse in the relevant sense. Total unimodularity constrains the *feasible linear system*: its vertices are integral for integral right-hand sides, regardless of the numerical costs. These are mathematically different mechanisms and should not be conflated even when both end in an exact LP.

What happens when the LP is not exact? There are two broad responses. One can exploit a stronger problem-specific representation or message-passing theorem, or one can tighten the relaxation by adding constraints that cut off the current fractional solution while preserving all integral solutions. The latter viewpoint leads naturally to cutting-plane algorithms: solve, diagnose, separate, add a constraint, and solve again.

How to use the notebooks. Begin with the attractive-versus-frustrated triangle. It is small enough that exact enumeration and the local LP can be compared directly. Then move to network flow, where integrality follows from the constraint matrix. The most important experiment is to distinguish three statements: the LP optimum is integral; an iterative message-passing method converges; and the returned integral point is the true MAP state. They are related only under additional hypotheses.

The Maximum A-Posteriori Problems (MAP) task was introduced in Section 4.2 and reformulated in Chapter 6 as a zero-temperature variational problem. Replacing the marginal polytope by the local marginal polytope produces the standard MAP-LP relaxation. The relaxation is attractive computationally because it is convex, but it may have a fractional optimum. This chapter asks a sharper structural question:

Which graphical models make the local MAP-LP exact, and what can be done when it is not?

Three mechanisms will recur. *Fractional polymorphisms* characterize broad classes of finite-domain problems solved by the basic LP; *submodularity* gives an especially important and concrete subclass; and *total unimodularity* makes certain linear-flow formulations integral independently of the numerical costs. We then discuss message-passing algorithms that inherit exactness from these structures, and successive relaxations that repair a fractional MAP-LP by adding violated constraints. The last part uses network-flow and physics-constrained models as a source of open problems.

12.1 Fractional Polymorphism Models

We follow the valued-constraint formulation of Kolmogorov, Thapper, and Živný [112]. A *language* Γ is a collection of finite-domain energy factors. For simplicity let all elementary variables take values in a common finite set $\mathcal{X}$, and let $E_A : \mathcal{X}^{d_A} \rightarrow \mathbb{Q}_{\geq 0}$ be a factor of arity d_A .

An m -ary operation is a map $g : \mathcal{X}^m \rightarrow \mathcal{X}$. Let $\mathcal{O}^{(m)}$ denote a collection of such operations, and let ω be a probability distribution on $\mathcal{O}^{(m)}$. We call ω an m -ary *fractional polymorphism* of Γ if, for every $E_A \in \Gamma$ and every $\mathbf{x}_A^{(1)}, \dots, \mathbf{x}_A^{(m)}$,

$$\mathbb{E}_{g \sim \omega} \left[E_A \left(g(\mathbf{x}_A^{(1)}, \dots, \mathbf{x}_A^{(m)}) \right) \right] \leq \frac{1}{m} \sum_{r=1}^m E_A(\mathbf{x}_A^{(r)}), \quad (12.1)$$



where g acts componentwise on the m local configurations. A fractional polymorphism is *symmetric* when every operation in its support is invariant under permutations of its m arguments.

Theorem 12.1.1 (Basic-LP characterization). *For a finite-valued language Γ , the basic MAP-LP solves every instance over Γ if and only if Γ admits a symmetric fractional polymorphism of every arity $m \geq 2$ [112].*

The theorem is deeper than a catalogue of special cases: it says that exactness of a particular convex relaxation is controlled by algebraic closure properties of the local factors. We only sketch the constructive direction. Because the LP data are rational, an optimal basic-LP belief

vector can be represented with a common denominator m . It may therefore be interpreted as an empirical distribution over m local copies. Applying a symmetric m -ary operation g componentwise maps these fractional copies back to an integral configuration. Inequality (12.1), averaged over $g \sim \omega$, guarantees that some resulting integral configuration has energy no larger than the LP value. Since the LP value is already a lower bound on the exact MAP energy, equality follows. The reverse direction, proved in [112], establishes the full characterization.

12.1.1 Submodular models

A particularly important binary fractional polymorphism places weight $1/2$ on two operations g_1 and g_2 . The condition becomes

$$E_A(g_1(\mathbf{x}_A^{(1)}, \mathbf{x}_A^{(2)})) + E_A(g_2(\mathbf{x}_A^{(1)}, \mathbf{x}_A^{(2)})) \leq E_A(\mathbf{x}_A^{(1)}) + E_A(\mathbf{x}_A^{(2)}). \quad (12.2)$$

If the domain is a lattice and $g_1 = \wedge$, $g_2 = \vee$ are the componentwise meet and join, Eq. (12.2) is the usual submodularity inequality

$$E_A(\mathbf{x}_A^{(1)} \wedge \mathbf{x}_A^{(2)}) + E_A(\mathbf{x}_A^{(1)} \vee \mathbf{x}_A^{(2)}) \leq E_A(\mathbf{x}_A^{(1)}) + E_A(\mathbf{x}_A^{(2)}). \quad (12.3)$$

Recall that the meet is the greatest lower bound and the join is the least upper bound.

Theorem 12.1.2 (Submodular MAP-LP exactness). *If every factor in Γ is submodular on a finite lattice, then the basic MAP-LP is exact for every instance over Γ [112].*

This includes several useful families.

- For binary pairwise models, attractive Ising interactions are submodular after the standard $\{0, 1\}$ encoding. Their MAP problem is equivalent to an s - t min-cut construction [111].
- For ordered multi-label variables, a pairwise table is submodular when it is a Monge array. A common example is $E_{ab}(x_a, x_b) = \phi_{ab}(x_a - x_b)$ with a discrete convex function ϕ_{ab} . This is the ordered-label analogue of attractive binary coupling; a generic multi-label Potts table need not satisfy the condition.
- For binary higher-order factors, identify a configuration with the subset of active variables. Any submodular set function is admissible. For example, a cardinality potential $E_A(\mathbf{x}_A) = \phi_A(\sum_{a \in A} x_a)$ is submodular when ϕ_A is concave on the integers.

Companion notebook: Submodularity, frustration, and a fractional MAP-LP optimum

Notebook `ch12/ch12_01_submodularity_and_map_lp.ipynb` compares an attractive and a frustrated Ising triangle. Exact enumeration and the local MAP-LP agree for the attractive case, while the frustrated case develops a fractional optimum and an integrality gap.

Exercise 12.1.1. *Use the companion notebook to vary one coupling continuously from attractive to repulsive. Record where the exact MAP configuration changes and where the local LP first becomes fractional. Are the two thresholds necessarily the same?*

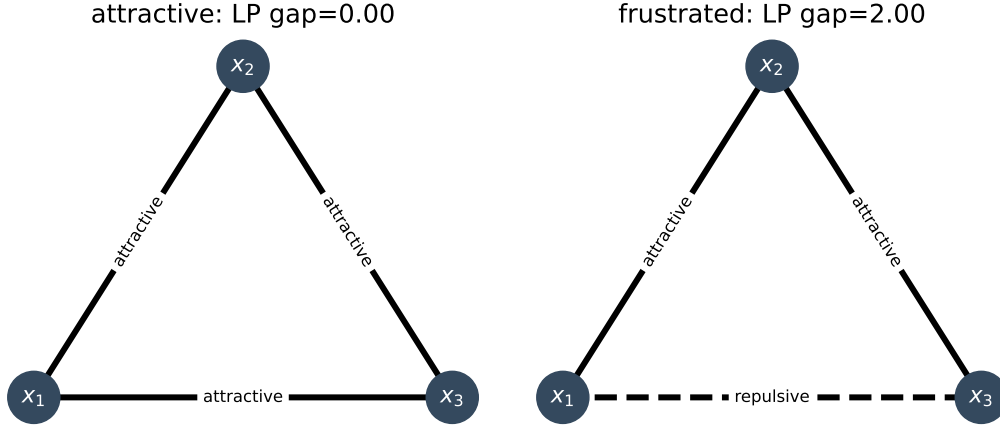


Figure 12.1: The same three-node topology can be easy or hard for the local relaxation. Attractive couplings produce an integral MAP-LP optimum, whereas frustration can make the locally consistent edge beliefs incompatible with any single global assignment.

A complementary exactness mechanism. Fractional polymorphisms say something about how local *costs* behave under algebraic operations. Total unimodularity says something about the *matrix of constraints*. A useful prototype is network flow: conservation constraints form an incidence matrix whose LP vertices are integral. The objective can then change freely without destroying integrality. This is why flow formulations are so valuable when a graphical MAP problem can be transformed into one.

12.2 Total Unimodularity and Integral Linear Programs

Submodularity constrains the *objective*. Total unimodularity (TUM) provides a different route to exactness by constraining the *linear feasible set*. A matrix A is totally unimodular if every square subdeterminant belongs to $\{0, +1, -1\}$. If A is TUM and b is integral, then every vertex of the polyhedron $\{x : Ax = b, \ell \leq x \leq u\}$ is integral whenever the bounds are integral. Consequently the ordinary LP relaxation of

$$\min_{x \in \mathbb{Z}^m} c^T x \quad \text{s.t.} \quad Ax = b, \quad (12.4)$$



is gapless. This statement is independent of the signs or magnitudes of the linear costs c ; exactness comes from the geometry of the constraint matrix, not from a fractional-polymorphism property of the soft factors. Network-incidence matrices are the canonical example, which is why max-flow/min-cut and min-cost-flow problems admit integral LP optima [178].

A factor-graph representation may encode the same constraints as hard factors and the linear objective as unary soft factors, as in [41]. One should distinguish carefully between two statements: integrality of the direct flow LP follows immediately from TUM, while exactness of a particular *local marginal* MAP-LP depends on whether that graphical representation projects to the same integral polytope. For network-flow and matching formulations this connection is particularly transparent and motivates the message-passing constructions below.

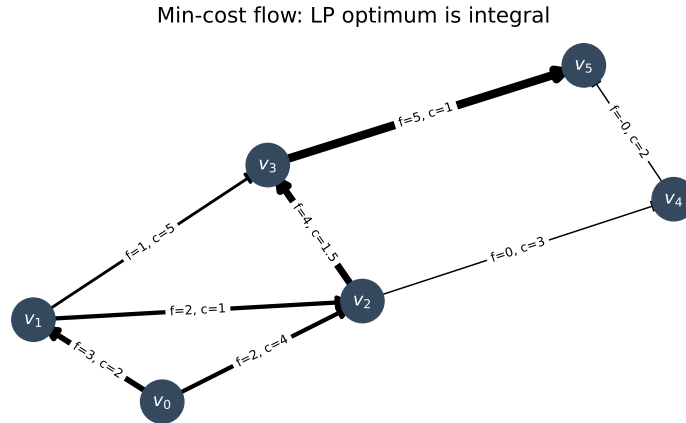


Figure 12.2: Integral min-cost flow. Flow conservation is represented by a network-incidence matrix, whose total unimodularity makes the LP vertices integral for integral supplies and capacities. Edge labels report the optimal flows.

Companion notebook: Total unimodularity in a min-cost-flow factor graph

Notebook `ch12/ch12_02_tum_network_flow.ipynb` solves a small integral min-cost-flow problem by LP, verifies integrality of the optimum, and checks representative minors of the node-arc incidence matrix.

12.3 Provably Convergent Distributed Algorithms

LP exactness does not by itself imply that a naive min-sum iteration converges. For several important integral problems, however, message-passing algorithms are known to recover the exact optimum under explicit nondegeneracy and stopping conditions. Bayati, Shah, and Sharma established such results for maximum-weight matching on bipartite graphs [14]; Gamarnik, Shah, and Wei extended the computation-tree analysis to min-cost network flow with integral data and piecewise-linear messages [74]. These results connect the exact tree recursions of Chapter 3 to loopy optimization: after enough iterations, the computation tree contains sufficient information about the unique global optimum to force the correct local decision.

The dual viewpoint is equally useful. Zero-temperature message passing can be interpreted as reparametrizing local costs while preserving the global energy, as developed in Chapters 6 and 5. Exactness then requires both a tight relaxation and an iteration that reaches a certificate of optimality. This separation is important: an integral LP may coexist with poorly behaved undamped message updates.

Problem 12.1. For the min-cost-flow instance in Notebook `ch12/ch12_02_tum_network_flow.ipynb`, derive piecewise-linear min-sum messages on the computation tree. Propose a stopping criterion that certifies the same edge flows as the LP optimum, and compare it with the conditions in [74].

Tightening as an adaptive algorithm. A large exact polytope may have too many constraints to write down in advance. Cutting-plane methods keep only a manageable relaxation, solve it, and ask a *separation problem*: can we find a valid inequality violated by the current fractional point? If yes, add it and repeat. The optimization and the diagnosis of its failure are therefore interleaved. This pattern—cheap reference computation followed by a targeted correction—is another form of the reference–correction principle used throughout the book.

12.4 Successive LP Relaxations and Cutting Planes

When the local MAP-LP is fractional, one can enlarge the set of represented regions or add valid inequalities that exclude the current fractional pseudomarginal without excluding any integral configuration. This is the logic behind cycle inequalities, cluster pursuit, cutting-plane methods, and the Sherali–Adams hierarchy [182, 187, 213]. At level k , the Sherali–Adams construction introduces consistent beliefs on progressively larger subsets; at the highest level it recovers the exact marginal polytope, but the cost grows combinatorially.

The practical strategy is therefore adaptive: solve a cheap relaxation, diagnose the fractional structure, add only the most informative violated constraints, and repeat. In graphical-model language, tightening means enriching the region graph. This connects directly to the region-based variational constructions of Chapter 6 and to approximate elimination in Chapter 7.

Companion notebook: Closing an integrality gap with a cycle inequality

Notebook `ch12/ch12_03_tightening_map_lp.ipynb` starts from the frustrated triangle, exhibits a fractional local-LP optimum, and adds the missing odd-cycle constraint. The strengthened polytope then returns the exact integral MAP value.

Exercise 12.4.1. *Extend the notebook to a five-cycle. Find a fractional local-LP optimum and determine which odd-cycle inequality cuts it off. Compare the number of added constraints with the number that would be introduced by a full third-order Sherali–Adams relaxation.*

12.5 Network-Flow Models and Research Challenges

Network optimization is a useful laboratory because it mixes hard conservation laws, integral structure, nonlinear costs, and increasingly realistic physical constraints. The simplest case is the integral min-cost flow. On a directed network $G = (V, E)$ let x_e be an integer flow, c_e its unit cost, and q_v an integral balanced injection, $\sum_v q_v = 0$. The factor-graph form uses unary soft factors

$$E_e(x_e) = c_e x_e \quad (12.5)$$

and hard conservation factors

$$\sum_{e \in \delta^+(v)} x_e - \sum_{e \in \delta^-(v)} x_e = q_v, \quad v \in V. \quad (12.6)$$

With integral capacities this is the TUM setting discussed above. The exact finite-time min-sum result of [74] may be viewed as a distributed counterpart of this polyhedral integrality.

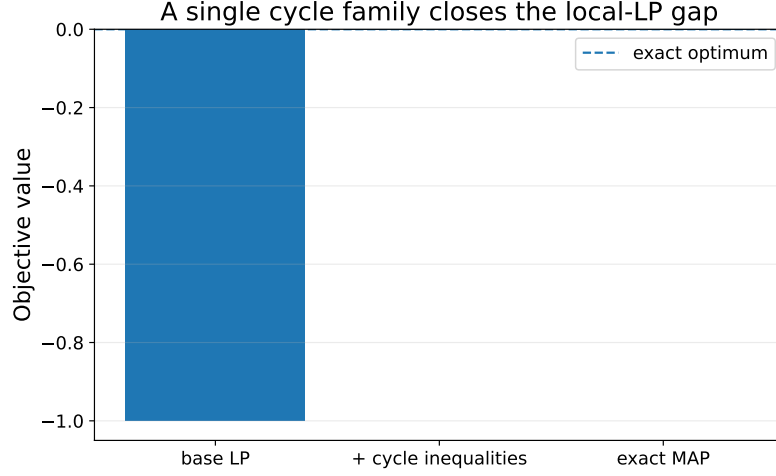


Figure 12.3: Successive relaxation on a frustrated cycle. Local consistency permits a pseudo-marginal that cannot arise from a global assignment; adding the appropriate cycle inequality removes that point and closes the MAP gap.

12.5.1 Submodular-cost multi-commodity flow

For K commodities, write $\phi_{ij} = (\phi_{ij}^{(1)}, \dots, \phi_{ij}^{(K)})$ and consider

$$\min_{\phi} \sum_{(i,j) \in E} f_{ij}(\phi_{ij}) \quad \text{s.t.} \quad q_i^{(k)} = \sum_{j:(i,j) \in E} \phi_{ij}^{(k)} - \sum_{j:(j,i) \in E} \phi_{ji}^{(k)}. \quad (12.7)$$

A natural structured class uses submodular edge costs,

$$f_{ij}(u) + f_{ij}(v) \geq f_{ij}(u \wedge v) + f_{ij}(u \vee v), \quad (12.8)$$

with componentwise meet and join. Conservation is TUM commodity by commodity, while the coupled cost supplies a second source of structure. Whether a compact local relaxation or distributed reparametrization can systematically exploit both structures is a useful open direction.

Problem 12.2. *Formulate the multi-commodity model in factor-graph form. Compare three relaxations: independent-commodity flow LPs, a joint local MAP-LP with edge factors f_{ij} , and a relaxation augmented with selected cross-commodity regions. Identify a small instance for which the first relaxation is loose and test whether submodularity of f_{ij} closes the gap.*

12.5.2 Dissipative and physics-constrained flows

Physical networks add nonlinear constitutive laws relating flows to nodal potentials. A prototypical single-source/single-destination throughput problem is

$$\max_{q, \phi, \pi} q \quad (12.9)$$

$$\text{s.t.} \quad \sum_{j: \{i,j\} \in E} \phi_{ij} = 0, \quad i \notin \{s, d\}, \quad (12.10)$$

$$\sum_{j: \{s,j\} \in E} \phi_{sj} = q, \quad \sum_{j: \{j,d\} \in E} \phi_{jd} = q, \quad (12.11)$$

$$\underline{\pi}_i \leq \pi_i \leq \bar{\pi}_i, \quad (12.12)$$

$$\pi_i - \pi_j = g_{ij}(\phi_{ij}), \quad g'_{ij}(\phi) \geq 0. \quad (12.13)$$

Gas and power-network models lead to related graph-constrained optimization and inference problems; a graphical-model perspective on such flow equations and uncertainty is developed in [49]. Unlike pure TUM flow, nonlinear physics generally destroys polyhedral integrality. The graph nevertheless remains valuable: it identifies local constraints, natural decomposition variables, and candidate regions for adaptive relaxation.

Problem 12.3. *Treat Eq. (12.9) as a structured optimization problem rather than assuming MAP-LP exactness. Propose an adaptive hierarchy in which violated physical constraints trigger new factors or enlarged regions. Specify what certificate would convince you that the resulting relaxation is exact on a particular instance. This problem is intentionally open-ended and is a prototype for the structured scientific optimization problems considered later in the book.*

12.5.3 Piecewise-linear objectives with TUM constraints

The results above suggest a broader question. Consider an integer program with a TUM constraint matrix but a separable piecewise-linear objective. Min-cost flow is a special case for which finite-time min-sum convergence can be established under suitable assumptions [74]. It remains interesting to characterize which larger TUM families admit comparably explicit distributed algorithms and stopping certificates. The distinction between *integrality of the relaxation* and *convergence of a particular message-passing solver* should remain central.

Problem 12.4. *Choose a TUM matrix not arising from an ordinary single-commodity network incidence matrix. Build the corresponding factor graph with hard linear constraints and separable piecewise-linear costs. Determine whether the local MAP-LP coincides with the direct LP, and test min-sum numerically. If it fails to converge, diagnose whether the failure is caused by relaxation looseness or by the dynamics of the algorithm.*

Chapter 13

Soft Inference Problems: Canonical Computational Laboratories

Reader's Guide.

Why call these “laboratories”? Each model in this chapter isolates a different source of difficulty while still providing an unusually strong reference calculation. Perfect matching gives a hard combinatorial partition function with an exact small-instance enumeration. Gaussian models turn message passing into linear algebra. Planar Ising models remain loopy yet admit determinant/Pfaffian methods. The final example puts inference inside an outer design optimization. These controlled cases let us distinguish approximation error from implementation error.

A useful comparison axis. For every laboratory ask two questions separately: *Is the exact probabilistic problem tractable?* and *Does the generic graphical-model approximation behave well?* A Gaussian model can have a closed-form exact answer while loopy BP fails to converge; a planar Ising model can contain many loops yet be exactly solvable by a specialized algorithm; a permanent is difficult exactly but admits certified variational bounds. “Loopy” and “hard” are therefore not synonyms.

What to carry forward. The permanent example emphasizes certified upper/lower approximations; the Gaussian example emphasizes convergence versus correctness of the fixed point; the planar example emphasizes hidden algebraic structure; and the two-level problem emphasizes the danger of nesting an approximate inner solver inside optimization. These distinctions reappear later when a generative model is used as an inner sampler, proposal, or correction mechanism.

The earlier chapters developed general algorithms; here we revisit them on models whose structure exposes unusually sharp mathematics. The examples are not isolated applications. Each is a reusable laboratory: perfect matching tests variational approximations against a difficult combinatorial partition function, Gaussian models turn functional BP equations into algebra, planar Ising models connect inference to determinants and Pfaffians, and a two-level

design problem places a partition function inside an outer optimization. The companion notebooks make these comparisons numerical and will be reused later when structured generative models introduce analogous inner-outer computations.

13.1 Perfect Matchings and the Permanent

For a nonnegative $n \times n$ matrix $M = (\mu_{ij})$, the permanent is

$$\text{perm}(M) = \sum_{\sigma \in S_n} \prod_{i=1}^n \mu_{i, \sigma(i)}. \quad (13.1) \quad \text{👉}$$

Unlike the determinant, all permutation terms enter with the same sign. The permanent is also the partition function of weighted perfect matchings on the complete bipartite graph. Introduce binary edge variables $x_{ab} \in \{0, 1\}$ and enforce one selected edge at every left and right vertex:

$$P(\mathbf{x}) = \frac{1}{Z} \prod_{a,b} \mu_{ab}^{x_{ab}} \prod_a \mathbb{1} \left(\sum_b x_{ab} = 1 \right) \prod_b \mathbb{1} \left(\sum_a x_{ab} = 1 \right). \quad (13.2)$$

Then $Z = \text{perm}(M)$. This graphical-model representation makes perfect matching a useful bridge between soft inference and the MAP matching problems discussed in Chapter 12.

13.1.1 Bethe and fractional free energies

Local consistency forces the edge beliefs $\beta_{ab} = P(x_{ab} = 1)$ to lie in the Birkhoff polytope,

$$\beta_{ab} \geq 0, \quad \sum_b \beta_{ab} = 1, \quad \sum_a \beta_{ab} = 1.$$

For the perfect-matching factor graph, the Bethe free energy reduces to [197, 198]

$$\mathcal{F}_B(\boldsymbol{\beta}) = \sum_{a,b} (-\beta_{ab} \log \mu_{ab} + \beta_{ab} \log \beta_{ab} - (1 - \beta_{ab}) \log(1 - \beta_{ab})). \quad (13.3)$$

The Bethe permanent is

$$\text{perm}_B(M) = \exp \left(- \min_{\boldsymbol{\beta}} \mathcal{F}_B(\boldsymbol{\beta}) \right).$$

The functional is convex on the feasible polytope, and the resulting Bethe permanent is a deterministic lower bound on the true permanent; quantitative approximation bounds are known [79, 198, 7].

A one-parameter fractional family introduced in [50] is

$$\mathcal{F}_\gamma(\boldsymbol{\beta}) = \sum_{a,b} (-\beta_{ab} \log \mu_{ab} + \beta_{ab} \log \beta_{ab} + \gamma(1 - \beta_{ab}) \log(1 - \beta_{ab})), \quad -1 \leq \gamma \leq 1. \quad (13.4)$$

The Bethe point is $\gamma = -1$. Chertkov and Yedidia conjectured, based on the fractional interpolation, that $\gamma = -1/2$ produces an upper bound; this was later proved in [7]. Thus a single variational family can generate certified lower and upper information about a hard partition function.

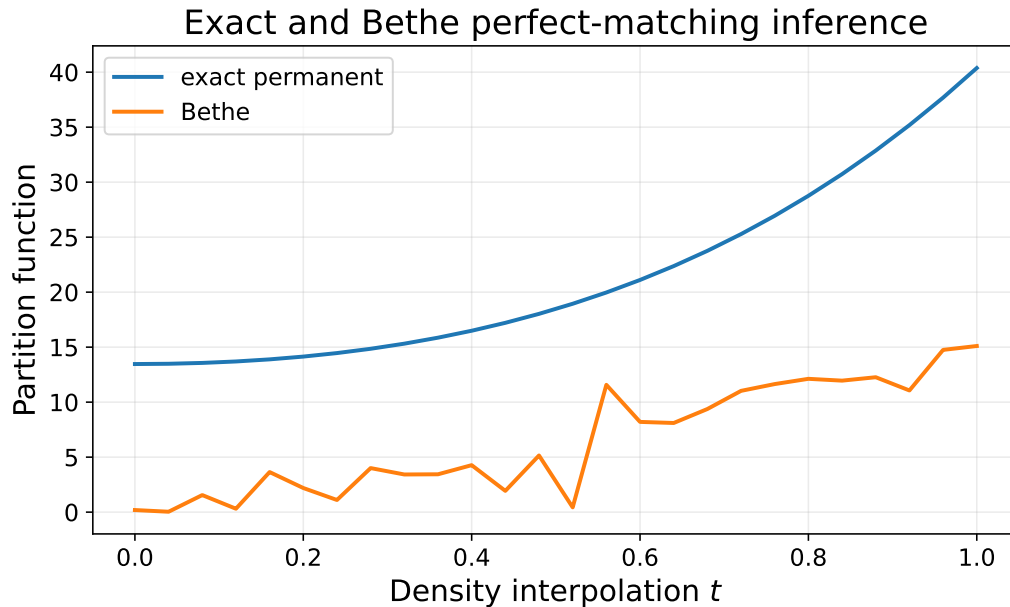


Figure 13.1: Exact permanent and its Bethe approximation for a small weighted matching problem. The example makes the variational approximation directly comparable with exact enumeration while retaining the matching constraints.

Companion notebook: Exact and Bethe permanents on small weighted matchings

Notebook `ch13/ch13_01_permanent_and_bethe.ipynb` enumerates all perfect matchings of a small nonnegative matrix, minimizes the Bethe free energy over the doubly stochastic polytope, and compares the exact and variational partition functions.

Exercise 13.1.1. *Using the notebook, interpolate continuously between a nearly diagonal matrix and a dense matrix with comparable entries. Track the Bethe error and the entropy of the optimal doubly stochastic belief. Explain why diffuse matching weights create a more demanding inference problem.*

Gaussian models separate “what is correct” from “whether the iteration converges.” The exact mean is always the solution of the linear system $J\mu = h$. Gaussian BP produces local message updates whose fixed point, when it exists, has exactly that mean. The approximation issue appears in the variance sector, while the algorithmic issue is whether the iterative messages converge at all. This clean separation makes Gaussian models an ideal place to learn why a correct fixed-point equation does not by itself make a reliable algorithm.

13.2 Gaussian Graphical Models

Gaussian graphical models are the cleanest setting in which the distinction between graph structure and numerical conditioning can be seen analytically. Consider

$$p(\mathbf{x}) = \frac{1}{Z} \exp \left(-\frac{1}{2} \mathbf{x}^T J \mathbf{x} + h^T \mathbf{x} \right), \quad J = J^T \succ 0. \quad (13.5)$$

The exact quantities are

$$\mu = J^{-1}h, \quad \Sigma = J^{-1}, \quad Z = (2\pi)^{N/2} \det(J)^{-1/2} \exp \left(\frac{1}{2} h^T J^{-1} h \right). \quad (13.6)$$

Zeros of the off-diagonal precision matrix encode conditional independences, as discussed in Section 2.4.3.

13.2.1 Bethe free energy and the primal mean equations

A locally consistent Gaussian belief on an edge (a, b) is parameterized by its means and covariance,

$$B_{ab}(x_a, x_b) = \mathcal{N} \left(\begin{bmatrix} x_a \\ x_b \end{bmatrix}; \begin{bmatrix} \mu_a \\ \mu_b \end{bmatrix}, \begin{bmatrix} \lambda_a & \lambda_{ab} \\ \lambda_{ab} & \lambda_b \end{bmatrix} \right), \quad (13.7)$$

with $\lambda_a \lambda_b - \lambda_{ab}^2 > 0$, and

$$B_a(x_a) = \mathcal{N}(x_a; \mu_a, \lambda_a). \quad (13.8)$$

Substitution into the pairwise Bethe free energy separates the mean and covariance variables. The mean-dependent part is exactly

$$\mathcal{F}_B^{(\mu)}(\mu) = \frac{1}{2} \mu^T J \mu - h^T \mu. \quad (13.9)$$

Therefore stationarity with respect to the means gives

$$J\mu = h. \quad (13.10)$$

This is an important structural fact: whenever Gaussian BP converges, its fixed-point means solve the exact linear system. The loop approximation enters through the covariance/variance sector and through convergence of the iterative solver, not through the location of the mean stationary point.

13.2.2 Gaussian belief propagation in information form

Write the message from a to b as

$$m_{a \rightarrow b}(x_b) \propto \exp \left(-\frac{1}{2} J_{a \rightarrow b} x_b^2 + h_{a \rightarrow b} x_b \right).$$

Define the cavity precision and cavity information at a excluding b ,

$$A_{a \setminus b} = J_{aa} + \sum_{c \in \partial a \setminus b} J_{c \rightarrow a}, \quad H_{a \setminus b} = h_a + \sum_{c \in \partial a \setminus b} h_{c \rightarrow a}.$$

Integrating the Gaussian variable x_a yields the exact message update

$$J_{a \rightarrow b} = -\frac{J_{ab}^2}{A_{a \setminus b}}, \quad (13.11) \quad \text{👉}$$

$$h_{a \rightarrow b} = -\frac{J_{ab} H_{a \setminus b}}{A_{a \setminus b}}. \quad (13.12)$$

The node belief is then

$$\lambda_a^{\text{BP}} = \left(J_{aa} + \sum_{c \in \partial a} J_{c \rightarrow a} \right)^{-1}, \quad \mu_a^{\text{BP}} = \lambda_a^{\text{BP}} \left(h_a + \sum_{c \in \partial a} h_{c \rightarrow a} \right). \quad (13.13)$$

On trees these equations are exact. On loopy graphs they are an iterative solver whose behavior depends on the precision matrix.

A useful sufficient condition is *walk-summability* [138]. Normalize J as $J = D^{1/2}(I - R)D^{1/2}$ with $D = \text{diag}(J)$; the model is walk-summable when

$$\rho(|R|) < 1, \quad (13.14) \quad \text{👉}$$

where ρ is the spectral radius and $|R|$ is entrywise absolute value. In this regime the covariance admits an absolutely convergent walk expansion and Gaussian BP converges. The converged BP means are exact; the BP variances are exact on trees but generally approximate on loopy graphs because they resum only a restricted family of closed walks [138]. Walk-summability is sufficient, not necessary, so inference beyond this threshold remains algorithmically interesting.

Companion notebook: Gaussian BP across the walk-summability boundary

Notebook `ch13/ch13_02_gaussian_bp_and_walk_summability.ipynb` varies the interaction strength of a sparse Gaussian model, compares exact linear-algebra moments with Gaussian BP, and tracks the spectral-radius diagnostic $\rho(|R|)$.

Problem 13.1. *Construct a positive-definite Gaussian model that is not walk-summable and test several message schedules, damping levels, or reparametrizations. Determine empirically whether Gaussian BP converges and, if it does, verify the mean solution against $J^{-1}h$. The goal is to distinguish a sufficient theorem from the actual boundary of algorithmic success.*

A warning against equating loops with intractability. Treewidth explains the cost of generic elimination, but special algebra can reorganize a loopy partition function into a determinant or Pfaffian. The planar zero-field Ising model is the canonical example. The high-temperature expansion below first turns spin summation into a sum over even subgraphs; planarity then converts that loop sum into a perfect-matching/determinant calculation. The model is structurally rich, but the structure happens to be computationally favorable.

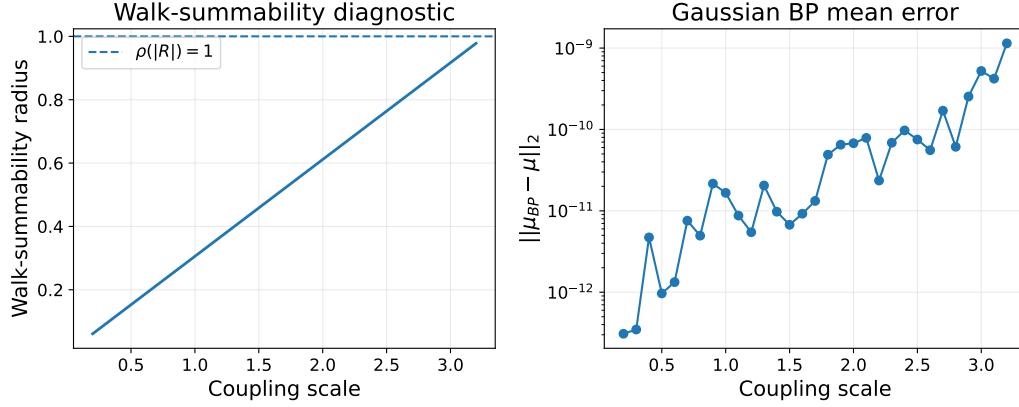


Figure 13.2: Gaussian BP and the walk-summability diagnostic. The spectral radius $\rho(|R|)$ provides a structural sufficient condition for convergence; exact matrix inversion supplies the reference solution.

13.3 Planar and Other Determinant-Easy Models

The zero-field planar Ising model is a second remarkable exactly solvable family. For

$$Z = \sum_{x \in \{\pm 1\}^N} \exp \left(\sum_{(a,b) \in E} J_{ab} x_a x_b \right),$$

use $e^{J x_a x_b} = \cosh J (1 + x_a x_b \tanh J)$. Expanding the product and summing over spins removes every edge subset having an odd degree at any vertex. Hence

$$Z = 2^N \prod_{e \in E} \cosh J_e \sum_{F \subseteq E: \deg_F(v) \text{ even } \forall v} \prod_{e \in F} \tanh J_e. \quad (13.15)$$

For planar graphs, the even-subgraph sum can be mapped to a weighted perfect-matching problem on an associated planar graph and evaluated through a Pfaffian/Kasteleyn determinant construction. This gives polynomial-time exact inference and sampling despite extensive loops. The tractable class can be enlarged by gluing planar and $O(1)$ -sized components along small separators, leading to minor-free families beyond bounded treewidth [128, 127].

This exact solvability complements the loop-calculus viewpoint of Chapter 9: the high-temperature expansion explicitly sums even generalized loops, while the Pfaffian reorganizes the same combinatorial complexity into linear algebra.

Companion notebook: Even-subgraph expansion of a planar zero-field Ising model

Notebook `ch13/ch13_03_planar_zero_field_ising.ipynb` verifies Eq. (13.15) on a small planar grid by comparing spin enumeration with enumeration of even edge subgraphs.

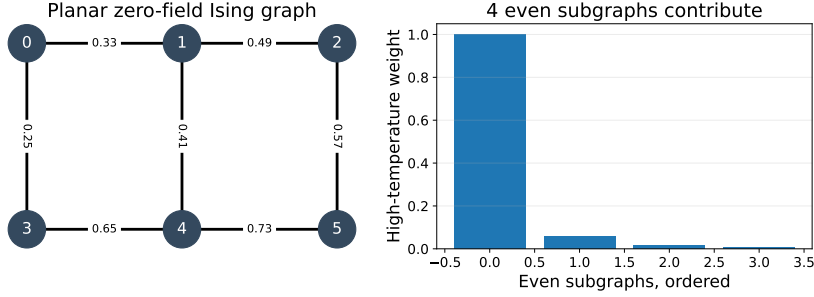


Figure 13.3: Planar zero-field Ising inference through even subgraphs. Only edge subsets with even degree at every vertex survive the spin summation; planarity allows these loop configurations to be reorganized through perfect matching and Pfaffian methods.

Exercise 13.3.1. Add a nonzero magnetic field at one node of the planar notebook model. Show explicitly why the simple even-subgraph constraint is lost. Then introduce one auxiliary spin connected to the field nodes and discuss how this changes planarity.

Nested computation amplifies approximation error. Here the partition function is not the final quantity of interest; its derivative enters an outer optimization. An error in $\log Z$, a marginal, or a finite-difference direction can therefore change the outer objective and ultimately the selected design. Always identify which inner quantity the outer optimizer actually needs and whether the available inference method supplies a bound, an unbiased estimate, or merely a heuristic approximation.

13.4 Two-Level Optimization Involving a Partition Function

Scientific design problems frequently place inference inside optimization. Following the formulation explored in [115], consider an Ising family $P(\mathbf{x} \mid J, h)$ and an outer objective based on weighted magnetization,

$$\overline{\mathcal{O}} = \min_{J, h} \mathcal{O}(J, h) \quad \text{s.t.} \quad C(J, h) \leq 0, \quad (13.16)$$

$$\mathcal{O}(J, h) = \rho^T \mathbb{E}_{J, h}[X]. \quad (13.17)$$

The derivative identity

$$\frac{\partial \log Z(J, h)}{\partial h_a} = \mathbb{E}_{J, h}[X_a] \quad (13.18)$$

turns the objective into a directional derivative of $\log Z$,

$$\mathcal{O}(J, h) = \rho^T \nabla_h \log Z(J, h).$$

A consistent finite-difference approximation therefore perturbs h along the direction ρ . For example, the centered difference is

$$\mathcal{O}(J, h) = \frac{\log Z(J, h + \varepsilon \rho) - \log Z(J, h - \varepsilon \rho)}{2\varepsilon} + O(\varepsilon^2). \quad (13.19)$$

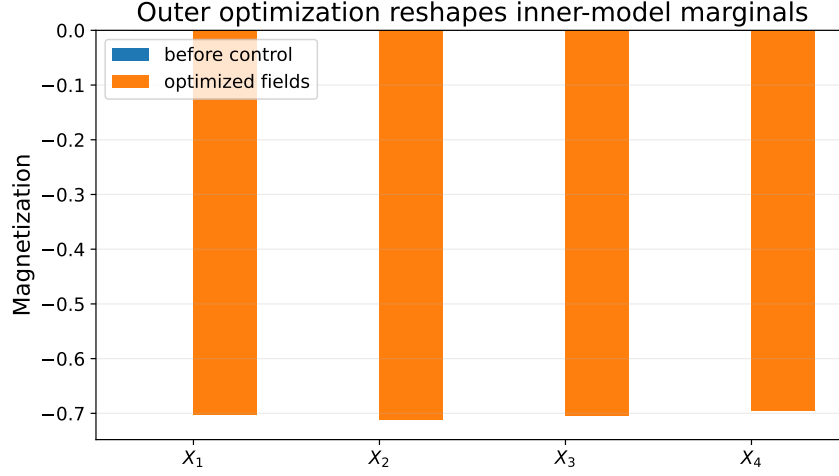


Figure 13.4: A two-level structured design problem. The outer variables modify local fields, while exact soft inference supplies the node magnetizations entering the objective. Later chapters replace exact enumeration by learned or controlled generative dynamics.

For a generic perturbation Δ the scalar difference of $\log Z$ estimates $\Delta^T \nabla_h \log Z$, not $\rho^T \nabla_h \log Z$; the two coincide only when Δ is chosen proportional to ρ .

The zero-temperature analogue replaces free energy by the MAP energy. Here an important distinction from the legacy formulation is required: the local MAP-LP is a *lower bound* on the exact MAP energy in general and is equal to it only for tight classes such as the submodular models characterized in Chapter 12. Thus a single-level LP reformulation of the inner problem is exact only when such a certificate applies; otherwise it defines a controlled relaxation of the outer design problem.

Companion notebook: Outer control through inner exact inference

Notebook `ch13/ch13_04_two_level_ising_control.ipynb` uses exact enumeration of a four-spin Ising model as the inner solver, optimizes a small field-control problem, and verifies Eq. (13.18) by finite differences.

Problem 13.2. Replace exact enumeration in the two-level notebook by three inner solvers: loopy BP, MCMC, and a Bethe/TRW variational approximation. Optimize the same outer objective with each solver. Compare not only the final objective values but also the gradients seen by the outer optimizer. Which type of inner error is most damaging: bias, variance, or non-smooth solver failure?

Chapter 14

Advanced Structure Learning: Orders, DAGs, and Sparse Graphical Models

Reader's Guide.

Why this chapter? Learning numerical parameters on a fixed graph is already an inverse problem; learning the graph itself adds a combinatorial layer. This chapter develops three ways to avoid brute-force graph search: reorganize the discrete search, replace a discrete structural constraint by a smooth exact constraint, or exploit a convex sparse-likelihood formulation. The common theme is not one universal algorithm but a change of representation that makes the structural unknown easier to optimize.

Three objects that should not be conflated. A variable *order* is not a DAG, a weighted adjacency matrix satisfying an acyclicity constraint is not automatically the statistically correct DAG, and a sparse precision matrix represents undirected conditional independences rather than directed causal relations. Each representation encodes a different structural object and comes with different identifiability assumptions.

Where moralization fits. The directed and undirected cases are not unrelated. A learned DAG first gives a factor graph directly through its normalized conditional factors. Projecting those family scopes to variables produces the moral/primal graph in which every Bayesian-network family is a clique; this is the graph on which generic exact elimination sees the directed model. But moralization is many-to-one and discards arrow directions and collider identities. Conversely, a sparse precision graph learned from Gaussian data identifies conditional independences, not a unique directed generative structure. Thus “learn a DAG, then moralize it” and “learn an undirected graph directly” solve different statistical problems even when the resulting pictures happen to share edges; see Section [2.5.2](#).

The simplest analogies. Order-based learning is like sorting first so that “allowed parents” become a local rule. NOTEARS is like replacing the discrete statement “there is no cycle” by a differentiable scalar residual that is exactly zero on acyclic weighted graphs. Graphical lasso is like sparse regression at the matrix level: zeros in the precision matrix become the learned missing edges.

What to verify. Separate optimization feasibility from statistical recovery. A NOTEARS solution may satisfy $h(W) \approx 0$ yet have incorrect edges because of finite data or nonconvex optimization. A graphical-lasso solution may be sparse and positive definite yet choose the wrong support. The notebooks deliberately compare the structural certificate supplied by the optimization with ground-truth recovery on controlled examples.

Chapter 10 focused primarily on learning parameters and neighborhoods in graphical models whose probabilistic semantics were already specified. Here we return to the harder problem in which the *structure itself* is unknown. The chapter is intentionally selective. Three laboratories expose three different ways of making a combinatorial structure-learning problem tractable:

1. replace a search over Bayesian-network structures by a better-organized search over *variable orders*;
2. replace the discrete acyclicity constraint of a directed graph by an exact *smooth constraint*;
3. replace combinatorial edge selection in a Gaussian graphical model by an ℓ_1 -regularized *convex likelihood*.

These examples complement Chow–Liu, pseudo-likelihood, and interaction screening in Chapter 10. More importantly for what follows, they make a distinction that will recur in neuralized graphical models: learning a function on a fixed graph, learning edge weights, and learning the graph itself are different problems and should not be conflated.

14.1 Learning Bayesian-network structure

Let $X_1, \dots, X_d$ be variables and let G be a Directed Acyclic Graph (DAG). A Bayesian network factorizes as

$$P(\mathbf{x} \mid G, \theta_G) = \prod_{i=1}^d P(x_i \mid \mathbf{x}_{\text{Pa}_G(i)}, \theta_i).$$

Given data D , Bayesian structure learning seeks a posterior over DAGs,

$$P(G \mid D) \propto P(D \mid G)P(G),$$

or posterior probabilities of structural features such as the event that an edge, path, or parent relation is present. Direct summation is difficult because the number of DAGs grows super-exponentially with d .

14.1.1 Learning through variable orders

Friedman and Koller proposed an algorithmically important reorganization of this sum [72]. Fix a total order $\prec$ on the variables and restrict each parent set to predecessors,

$$\text{Pa}_G(i) \subseteq \text{Pred}_{\prec}(i).$$

Every graph consistent with the order is automatically acyclic. Under the usual modularity assumptions, define a local integrated score $s_i(U)$ for selecting parent set U at node i . With a maximum in-degree k , the order score factorizes as

$$S(\prec) \propto P(\prec) \prod_{i=1}^d \left(\sum_{\substack{U \subseteq \text{Pred}_{\prec}(i) \\ |U| \leq k}} s_i(U) \right). \quad (14.1) \quad \text{📖}$$

The crucial gain is conceptual and computational: for a fixed order, exponentially many DAGs have been summed out locally. One can then run MCMC over permutations rather than over individual DAGs. The order space is still of size $d!$, but neighboring permutations provide a simple proposal mechanism and, empirically, a smoother posterior landscape than direct graph-space moves [72].

A subtlety should remain visible. A single DAG can be consistent with multiple orders, so a prior that is simple in order space does not automatically induce a uniform or otherwise desired prior over DAGs. Order-based computation is therefore not merely a neutral change of variables: prior semantics and multiplicity must be accounted for when posterior statements about graphs are required.

Companion notebook: Bayesian structure learning by MCMC over orders

Notebook `ch14/ch14_01_order_mcmc_bayesian_networks.ipynb` generates a small Gaussian DAG, constructs decomposable BIC-based local scores, exactly enumerates all 5! orders, and compares the exact order posterior with an adjacent-swap MCMC approximation. The notebook uses BIC scores for transparency; Eq. (14.1) describes the analogous Bayesian modular-score construction.

Exercise 14.1.1. *Modify the proposal in the order-MCMC notebook from adjacent swaps to uniformly chosen transpositions. Compare acceptance rate, autocorrelation, and posterior error. Explain why a proposal that moves farther in permutation space need not mix faster.*

Problem 14.1. *For the five-variable example, enumerate all DAGs consistent with each order and measure how many orders represent each DAG. Construct two order priors and compare the induced prior on DAGs. Propose a correction when the quantity of scientific interest is a graph feature rather than an order feature.*

A principally new trick: encode acyclicity by calculus. Instead of proposing one DAG at a time, keep every possible directed edge as a continuous variable and impose one differentiable equality whose zero set is exactly the acyclic graphs. This makes gradient-based optimization possible, but it does *not* make the problem convex. The combinatorial difficulty has been transformed, not magically removed.

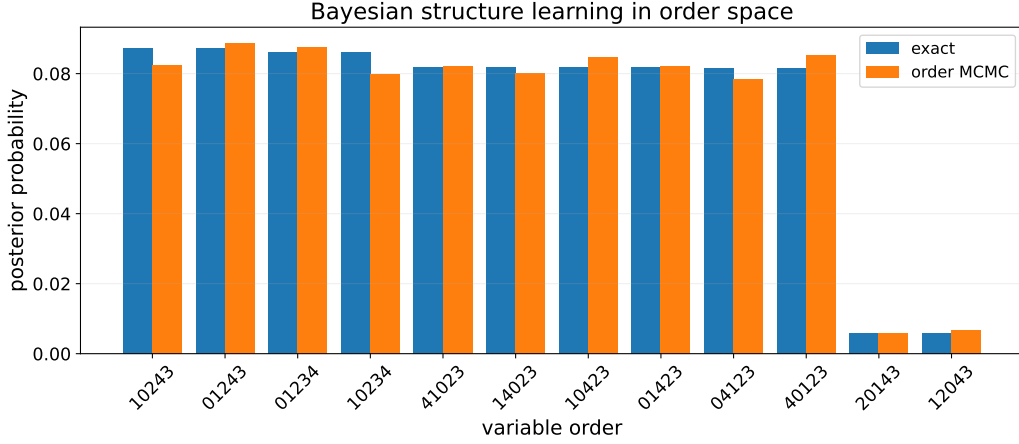


Figure 14.1: Structure learning in order space. For five variables all orders can be enumerated, providing an exact reference against which order-MCMC frequencies can be checked.

14.1.2 Continuous optimization and exact smooth acyclicity

A very different strategy is to keep a weighted adjacency matrix $W \in \mathbb{R}^{d \times d}$ as a continuous optimization variable and encode acyclicity by a differentiable equality constraint. The NOTEARS construction of Zheng et al. [225], and the broader family of continuous DAG-learning methods reviewed in [199], use the identity

$$h(W) = \text{tr}(\exp(W \circ W)) - d = 0, \quad (14.2)$$

where $\circ$ denotes the Hadamard product.

Why does Eq. (14.2) detect cycles? Let $A = W \circ W$, so A is entrywise nonnegative. Expanding the matrix exponential gives

$$\text{tr}(e^A) = d + \sum_{k=1}^{\infty} \frac{\text{tr}(A^k)}{k!}.$$

For a nonnegative weighted adjacency matrix, $\text{tr}(A^k)$ sums the weights of closed directed walks of length k . Hence every term beyond d vanishes exactly when the directed graph contains no cycle. The squaring in $W \circ W$ prevents cancellation between positive and negative edge weights.

For a linear structural-equation model written row-wise as

$$X = XW + E,$$

with W_{ij} representing an edge $i \rightarrow j$, a sparse estimator can be written

$$\min_W \frac{1}{2S} \|X - XW\|_F^2 + \lambda \|W\|_1 \quad \text{subject to} \quad h(W) = 0, \quad (14.3)$$

with a zero diagonal imposed on W . The original combinatorial constraint has become a smooth but nonconvex equality constraint. This is an instructive trade: structure is enforced exactly at the level of feasible points, but global optimization is no longer guaranteed by convexity.

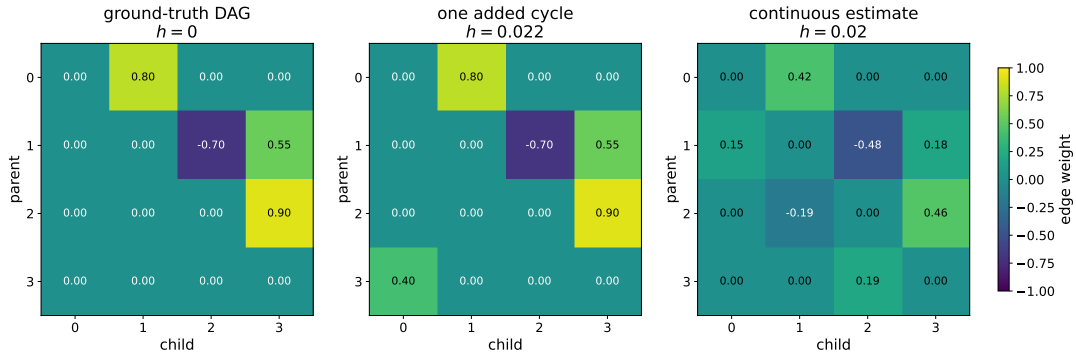


Figure 14.2: The NOTEARS acyclicity diagnostic. A DAG has $h(W) = 0$; adding a directed cycle makes the smooth constraint positive. The third panel shows a continuous sparse estimate obtained from data.

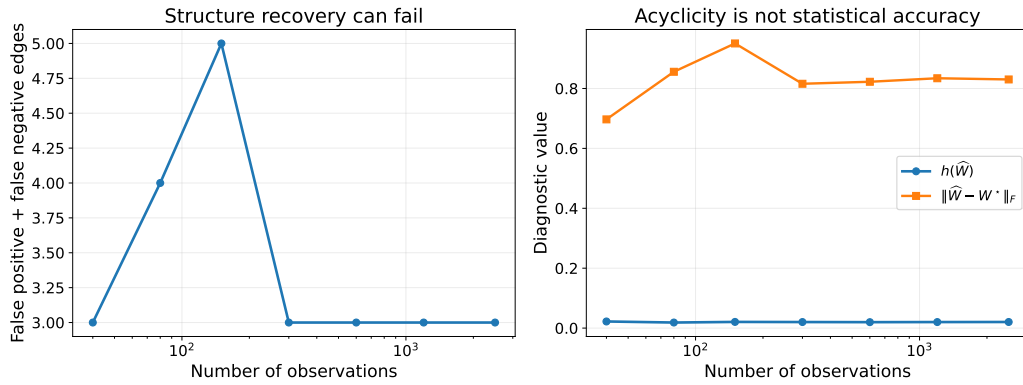


Figure 14.3: Finite-sample stress test for continuous DAG learning. The acyclicity penalty may be small even when the recovered edge support is wrong. Optimization feasibility and statistical recovery are distinct guarantees and should be diagnosed separately.

Companion notebook: Acyclicity as a differentiable matrix constraint

Notebook `ch14/ch14_02_continuous_dag_notears.ipynb` evaluates $h(W)$ on a ground-truth DAG and on a one-edge cyclic perturbation, then fits a small continuous sparse DAG model using a penalty on the acyclicity residual.

The constraint $h(W) = 0$ is an exact statement about acyclicity, not a certificate that the learned graph is statistically correct. The companion notebook now separates these two notions by repeating the same continuous optimization over a range of sample sizes. Figure 14.3 shows that an estimate can be nearly acyclic while still making edge-support or weight errors.

Exercise 14.1.2. For a three-node directed cycle with edge weights a, b, c , expand Eq. (14.2) to the first nonzero order in abc . Verify numerically that reversing the sign of one edge does not remove the acyclicity signal.

Problem 14.2. *Continuous DAG learning produces a weighted directed graph, but observational data alone need not identify a causal DAG without additional assumptions. For the companion notebook, distinguish three claims: recovery of a predictive factorization, recovery of a Markov-equivalence class, and recovery of causal direction. State what additional data or assumptions would be needed to strengthen one claim into the next.*

Why the Gaussian case is especially clean. For a multivariate Gaussian, an off-diagonal zero of the precision matrix has an exact probabilistic meaning: conditional independence. Structure learning can therefore be posed directly as estimating a sparse positive-definite matrix. The ℓ_1 penalty is not just a generic regularizer here; it selects the graph by driving entries of the precision matrix to zero.

Directed versus undirected structure after learning. Suppose a Bayesian-network learner returns a DAG G . Its conditional factors directly define a factor graph; the corresponding moral graph is the variable-only primal projection of those scopes. Suppose instead that graphical lasso returns the support of a precision matrix $\hat{\Theta}$. That graph is an estimated *conditional-independence structure*. These objects should not be interpreted interchangeably: a moral edge may appear only because two variables are co-parents of a common child, while a missing or present precision edge is a statement about a conditional correlation in the fitted Gaussian law. Direction, v-structures, and causal interpretation cannot be recovered from the undirected graph alone.

Exercise 14.1.3. *Consider the linear-Gaussian collider*

$$X_1 = \varepsilon_1, \quad X_2 = \varepsilon_2, \quad X_3 = aX_1 + bX_2 + \varepsilon_3, \quad X_4 = cX_3 + \varepsilon_4,$$

with independent centered Gaussian noises of positive variance and nonzero a, b, c . Draw the DAG and its moral graph, compute the Gaussian precision matrix analytically or numerically, and compare its off-diagonal support with the moral edges. Then perturb the model by adding another directed path that can produce cancellation in a precision entry. Explain why graph recovery generally requires assumptions such as faithfulness in addition to correct optimization.

14.2 Learning sparse undirected graphical models

The undirected Gaussian case gives a particularly clean bridge between likelihood, sparsity, and conditional independence. Let $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ and write the precision matrix as

$$\Theta = \Sigma^{-1} \succ 0.$$

For a multivariate Gaussian distribution,

$$\Theta_{ij} = 0 \iff X_i \perp X_j \mid \mathbf{X}_{\setminus \{i,j\}},$$

so estimating a sparse precision matrix is exactly structure learning for a Gaussian Markov random field.

Let S denote the empirical covariance. Up to constants and an overall sample-size factor, the negative Gaussian log-likelihood is

$$-\log \det \Theta + \text{tr}(S\Theta).$$

Adding an off-diagonal ℓ_1 penalty gives the convex sparse maximum-likelihood problem

$$\hat{\Theta} = \arg \min_{\Theta \succ 0} \left(-\log \det \Theta + \text{tr}(S\Theta) + \lambda \sum_{i \neq j} |\Theta_{ij}| \right). \quad (14.4)$$



This covariance-selection viewpoint and scalable optimization algorithms were developed in, among other works, [10]. Increasing λ suppresses edges, trading variance and false positives against bias and false negatives.

Equation (14.4) should be compared with the local estimators in Sections 10.2.1 and 10.2.2. All three use sparsity to turn statistical structure recovery into an optimization problem, but they do so at different levels:

- graphical lasso is a *global normalized likelihood* that remains tractable because Gaussian normalization is a log determinant;
- neighborhood-selection and pseudo-likelihood methods use *local conditional models*;
- interaction screening constructs a different local convex surrogate whose population optimum identifies the true neighborhood under its assumptions.

This comparison is useful later when neuralization makes global normalization difficult again: one may choose to neuralize an energy while keeping a local learning objective, rather than forcing an intractable full likelihood.

Companion notebook: Sparse precision recovery with graphical lasso

Notebook `ch14/ch14_03_graphical_lasso_sparse_precision.ipynb` samples a small sparse Gaussian graphical model, fits graphical lasso across a regularization path, and compares the learned precision support with the ground-truth conditional-independence graph.

Exercise 14.2.1. *In the graphical-lasso notebook, increase the sample size while keeping the precision matrix fixed. For each sample size choose λ by held-out Gaussian likelihood and compare it with the λ that gives the best edge-support F1 score. Explain why predictive likelihood and exact graph recovery need not select the same regularization level.*

Problem 14.3. *Add one unobserved Gaussian variable connected to several observed nodes, generate data from the enlarged model, and then fit graphical lasso only to the observed variables. Examine the induced dense correlations and precision structure. What does this experiment say about interpreting an estimated edge as a direct physical interaction in the presence of latent confounding?*

14.3 Three routes around combinatorial structure search

The three case studies solve different problems, but their mathematical strategies can be compared directly:

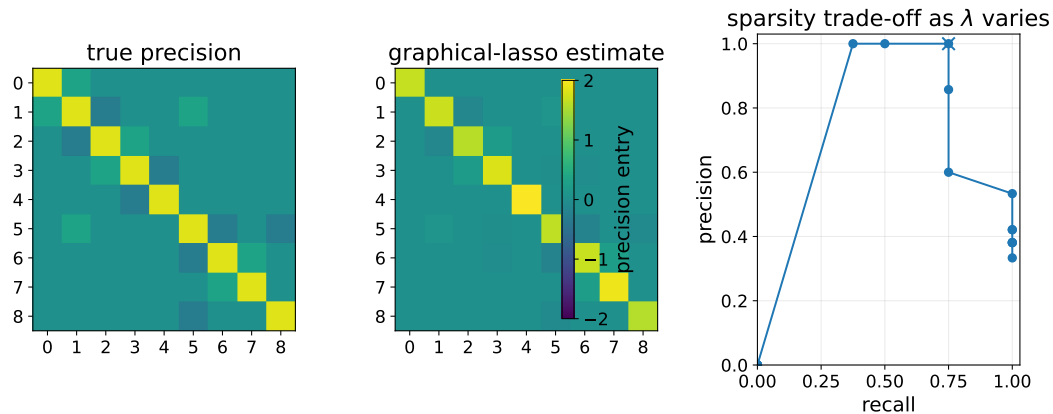


Figure 14.4: Sparse Gaussian structure recovery. Left and middle: ground-truth and estimated precision matrices. Right: support-recovery precision–recall trade-off as the regularization level changes.

Model	Discrete difficulty	Structural device	Remaining difficulty
Bayesian network, order MCMC	super-exponential DAG space	analytically sum DAGs under a fixed order	sampling over permutations and prior multiplicity
Continuous DAG learning	acyclicity	exact smooth constraint $h(W) = 0$	nonconvex constrained optimization and identifiability
Gaussian MRF	edge selection	ℓ_1 -penalized likelihood	regularization choice, finite samples, hidden variables

The common lesson is that tractability comes from finding a representation in which the structural constraint is easier than the original search. This is also the right conceptual entry point for the next part of the book. A neural network should not be introduced merely because structure learning is hard. First identify *which object is hard*: the factor, the parent-set score, the message, the posterior, the graph operator, or the optimizer. Neuralization is useful when it approximates that object while preserving as much of the surrounding probabilistic structure as possible.

Problem 14.4. *Design a hybrid structure-learning algorithm that combines two of the mechanisms in this chapter. Examples include using a neural surrogate for local Bayesian parent-set scores inside order MCMC, or using a sparse precision estimate to initialize a nonlinear graphical model. State explicitly what remains exact, what is approximated, how the approximation is trained, and which structural property can still be verified after training.*

Part III

Graphical Models Neuralization

Chapter 15

Neuralizing Graphical Models: Learned Factors, Messages, Operators, and Corrections

Reader's Guide.

Why this chapter is different. Until now the book asked how to compute with a specified graphical model. Here a neural network enters *inside* that computation, so the first question must be: what mathematical object is being replaced or augmented? A learned factor changes the statistical model; a learned message changes an inference update; an amortized operator replaces repeated solves; a learned reference or residual changes only the approximation/correction strategy. These are not stylistic variants of one idea.

A simple diagnostic before introducing a network. Take the classical pipeline and draw a box around the step that is expensive or systematically inaccurate. If the box contains a local potential, learn a factor. If it contains one BP iteration, learn an update. If it contains an entire solve repeated over many observations, learn an operator or warm start. If the classical answer is already strong but misses a structured residual, learn only the residual. If you cannot identify the box, the neuralization target is probably underspecified.

The safest architecture: predict, then correct. Whenever an exact or well-understood classical iteration is available, use the network to supply an initialization, proposal, gauge, or low-dimensional parameter and then return to the mathematical solver. This preserves a residual or certificate that can be checked after the neural prediction. Direct replacement is appropriate only when the reference computation is itself the dominant obstacle and no tractable correction is available.

What should be measured. Training loss alone is not enough. Compare against the classical reference on exact small instances; report fixed-point residuals, constraint violations,

partition-function or marginal errors, and transfer in coupling strength, graph size, and topology. A neural method that is faster in-distribution but destroys a useful certificate may be a poor trade outside the training regime.

Part III begins with a rule that will be enforced throughout the new material:

identify the mathematical object before neuralizing it.

A neural network is introduced only after the graphical-model construction has made clear what is expensive, unavailable, or systematically inaccurate. The learned object may be a factor, a message, an inference operator, a gauge, a coarse-graining rule, a proposal, or an exact residual. Keeping that identity visible gives every neural approximation a classical reference against which it can be tested.

15.1 Four modes of neuralization

Consider the factorized law of Chapter 2,

$$p(x) = \frac{1}{Z} \prod_{a \in F} \psi_a(x_{\partial a}).$$

The phrase “neural graphical model” can refer to several mathematically different operations.

Neuralization mode	Typical learned object	What remains fixed and therefore testable
Model object	$\psi_{a,\theta}$, E_θ , conditional kernel	Graph scopes, exact constraints, likelihood semantics, known physical terms
Iterative state/update	BP cavity message or local update	Target graphical model and message interpretation
Solver/operator	Evidence or parameters $\mapsto$ beliefs, marginals, fixed point, elimination factor	The exact inference problem being amortized
Reference/correction	Gauge, fractional parameter, proposal, truncation rule, residual	Exact reference–correction identity and, when available, a certificate or exact benchmark

Learning a factor changes the statistical model. Learning a BP update changes an inference algorithm. Learning an operator amortizes an entire family of inference problems. Learning a reference parameter leaves the target and exact correction identity unchanged but changes how difficult the residual is to evaluate. These distinctions are central to the book because they lead to different failure modes and different verification strategies.

15.1.1 Five tests for a neural surrogate

Every construction in this chapter will be judged by five questions.

1. **Semantics:** which exact object is approximated?

2. **Structure:** which locality, symmetry, positivity, normalization, conservation, or constraint properties are preserved by construction?
3. **Training signal:** is supervision obtained from exact inference, synthetic instances, self-consistency, a variational objective, or downstream task loss?
4. **Transfer:** does the surrogate generalize to new parameters, graph sizes, graph topologies, observation patterns, or physical regimes?
5. **Verification:** which guarantees have been lost, and what can still be checked independently of the learned component?

A construction that gives a lower training loss but destroys the semantics of the object it replaces should not automatically be regarded as an improvement.

15.2 When does neuralization actually buy something?

The existence of a neural approximator is not itself a reason to use one. In structured inference there are four recurring situations in which learning can change the computational balance.

Amortization across repeated instances. If essentially the same inference problem is solved thousands of times for different evidence or nearby parameters, training cost can be paid once and evaluation reused. The natural object is then the map from instance to solution, not a new probability model.

Warm starts and fixed-point bootstrapping. A network can predict an initial message field or approximate fixed point, after which the exact classical iteration resumes. The learned object accelerates convergence but does not define the final answer. This “predict, then correct” pattern is attractive because a poor neural prediction can often be detected by a large fixed-point residual.

Streamlining an expensive intermediate operator. Elimination, coarse graining, region construction, or tensor truncation may create an intermediate function that is expensive to represent exactly. A neural surrogate can compress this object while the surrounding algorithm remains classical.

Learning only the residual. When an exact or high-quality structured reference already exists, learning the final answer from scratch can be wasteful. The reference–correction principle of Eq. (1.6) suggests learning a scalar reference parameter, an omitted loop correction, or a proposal that reduces the variance of an exact residual.

15.3 Neural factors and energies

The least disruptive neuralization replaces only an unknown local factor,

$$\psi_a(x_{\partial a}) = \exp(-E_a(x_{\partial a})) \quad \longrightarrow \quad \psi_{a,\theta}(x_{\partial a}) = \exp(-E_{a,\theta}(x_{\partial a})).$$

Known factors can remain exact. This separation is especially important in scientific applications: parity checks, network balances, geometric constraints, and conservation laws need not be relearned from data.

Example 15.3.1 (Known-plus-learned Ising energy). *On the Ising grid used throughout the earlier chapters, suppose nearest-neighbor couplings are known but a weak nonlocal interaction is unknown. Write*

$$E_{\theta}(x) = E_{\text{known}}(x) + E_{\theta}^{\text{res}}(x).$$

Exact enumeration on small grids supplies controlled labels for comparing the learned residual energy, partition function, marginals, and samples. More importantly, the known local structure remains visible and testable after learning.

Unfolding does not by itself change BP. Writing R iterations of BP as R computational layers simply exposes the existing algorithm as a directed computation graph. Neuralization begins only when parameters or residual maps are inserted into those layers. Keep three goals separate: accelerate convergence to the same BP fixed point, correct a biased loopy-BP fixed point, or replace the final inference result with a finite-depth learned predictor. Only the first retains the classical answer by construction.

15.4 Unfolding belief propagation

Belief propagation (BP), reviewed in Chapters 3 and 6, defines an iterative map

$$m^{(r+1)} = \mathcal{B}(m^{(r)}; \psi).$$

A finite number of iterations can be unfolded into a layered computational graph. Trainable parameters may scale, damp, or add residual corrections to the classical update,

$$m^{(r+1)} = \mathcal{B}(m^{(r)}; \psi) + R_{\theta_r}(m^{(r)}, \psi).$$



Neural-enhanced BP on factor graphs provides a modern instance of this approach [174]. The same principle appeared earlier in neural decoders for sparse error-correcting codes: the graph and message semantics remain explicit, while weights or local update rules are learned.

Three goals should be separated.

1. **Acceleration:** reach essentially the same BP fixed point with fewer iterations.
2. **Bias correction:** change the fixed point because loopy BP is systematically inaccurate.
3. **Replacement:** approximate the desired marginals directly with a finite-depth learned map.

Only the first leaves the classical BP answer unchanged by design.

Companion notebook: Learning a warm start and then returning to exact BP updates

Notebook `ch15/ch15_01_bp_warm_start.ipynb` generates a family of small Ising grids, computes converged BP cavity fields, trains a small neural network to predict those fields

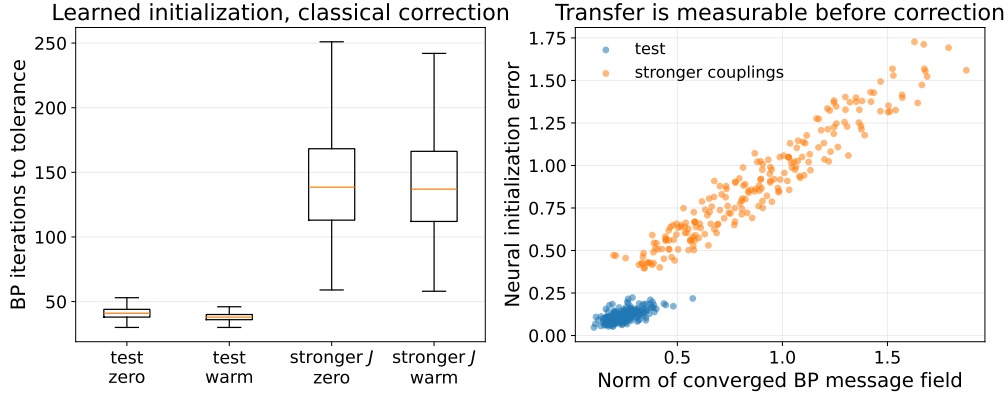


Figure 15.1: Neural warm starting changes the initialization of BP, not its final fixed-point equations. The useful quantity is therefore the number of exact BP correction iterations saved, together with the residual on out-of-distribution instances. Generated by `ch15/ch15_01_bp_warm_start.ipynb`.

from model parameters, and measures how many classical BP iterations remain after the learned initialization. The experiment separates in-distribution acceleration from transfer to stronger couplings.

Exercise 15.4.1 (Warm start versus finite-depth replacement). *Using the notebook above, compare two predictors trained on the same instances: one predicts an initial message field and is followed by BP until convergence; the other predicts singleton magnetizations directly. Compare speed, transfer, and the availability of a verifiable residual. Which construction would you trust more outside the training range?*

15.5 From learned messages to graph neural networks

A generic message-passing neural network replaces derived local messages by learned local maps. In a pairwise graph one may write schematically

$$h_i^{(r+1)} = U_\theta \left(h_i^{(r)}, \sum_{j \in \partial i} M_\theta(h_i^{(r)}, h_j^{(r)}, e_{ij}) \right).$$

Graph convolutional networks and related GNNs instantiate this locality principle [106]. The similarity with BP is architectural, but the semantics are different. A BP message is a cavity probability object defined by the graphical model; a generic GNN hidden state is simply a learned feature vector.

This distinction suggests a middle ground: *semantic message passing*. Keep a parameterization in which the state can still be interpreted as a likelihood ratio, cavity field, local score, or approximate marginal, but learn the update inside that parameterization. Such a model can exploit neural expressivity without abandoning probabilistic diagnostics.

From learning a state to learning a solver. If the evidence y changes repeatedly while the model family stays similar, the expensive object is not one posterior but the map $y \mapsto p(x \mid y)$. Amortization learns this map once and reuses it. The key new failure mode is therefore *instance-dependent*: the network may be close to the best available approximation on some observations and far from it on others, even though the underlying probabilistic model is unchanged.

15.6 Amortized inference as learning an operator

Suppose repeated observations require solving

$$y \mapsto p(x \mid y).$$

Instead of running an iterative solver independently for every y , one may learn

$$\mathcal{A}_\theta : y \mapsto q_\theta(x \mid y).$$

The training phase learns an *operator* over a family of inference problems. This is the conceptual precursor of amortized variational inference. Chapter 16 develops the particularly clean latent-variable progression from EM to variational posteriors, the ELBO, amortization, and VAEs.

A useful hybrid does not stop at q_θ . It uses the prediction as an initializer and performs one or several mathematically justified refinement steps. This semi-amortized pattern will reappear in bridge, score, and Sampling-Decision constructions later in the book.

15.7 Neural elimination, coarse graining, and tensor truncation

Exact variable elimination constructs intermediate factors whose scopes can grow exponentially with induced width. The expensive object is the intermediate map

$$\tilde{\psi}_S(x_S) = \sum_{x_i} \prod_{a \ni i} \psi_a(x_{\partial a}).$$

A neuralized elimination scheme approximates this function by $\tilde{\psi}_{S,\theta}$ while leaving the surrounding elimination logic intact. This connects directly to mini-buckets and renormalization in Chapter 7: one may learn how to compress a newly created factor, how to choose a region, or which variables should be eliminated together.

Tensor networks create the same opportunity in linear-algebraic form. Once an exact contraction is written in a chosen gauge, one may learn a bond truncation, a local low-rank basis, or even the gauge itself. The exact invariant must remain explicit: paired inverse transformations preserve the untruncated contraction, whereas a learned truncation controls only the approximation error. Chapter 9 provides the connection between BP gauges, generalized loops, and tensor-network contraction.

Problem 15.1 (Neural gauge selection). *For a family of small normal factor graphs, parameterize a class of paired edge gauges. Train a model to choose a gauge that minimizes the error of*

a prescribed low-rank tensor contraction. Compare it with the BP gauge and with a gauge chosen solely to improve numerical conditioning. Report both the exact gauge-invariant contraction and the approximate truncated answer so that improvement in compression is not confused with a change in the underlying model.

Why learn a reference parameter instead of the final quantity? A scalar or low-dimensional reference parameter can be easier to transfer and easier to diagnose than a full partition function, message field, or marginal vector. More importantly, the exact correction identity remains visible. The learning task becomes “choose a reference that makes the residual small” rather than “guess the answer.” This is the static prototype of the proposal-design problem that will reappear on path space.

15.8 Learning the reference rather than the answer

The most distinctive neuralization opportunity in this book comes from Chapter 9. Fractional BP introduces a one-parameter family $Z^{(\lambda)}$ interpolating between TRW and BP, and for attractive Ising models a unique λ_* satisfies

$$Z = Z^{(\lambda_*)}.$$

At every other λ the answer is still recoverable through the exact residual in Eq. (9.46). This produces a hierarchy of learning targets:

predict λ_* $\longrightarrow$ predict a residual after explicit loop corrections $\longrightarrow$ learn a general loop resummation.

The first target is scalar, bounded, and carries a variational bracket in the attractive regime. The second is higher dimensional but asks the network to learn only what explicit short loops have not already explained. The third is the most ambitious and should be treated as a research problem rather than as an assumed capability.

Problem 15.2 (Amortising the fractional parameter). *For a family of small attractive Ising instances on which Z is available by enumeration, compute λ_* and train a predictor using graphical parameters and BP beliefs. Then:*

1. compare $Z^{(\lambda)}$ with direct neural prediction of $\log Z$ and with the BP reference;
2. test transfer separately in coupling strength, graph size, and loop density;
3. use the TRW–BP sandwich in Eq. (9.44) to diagnose failures whenever its hypotheses apply;
4. classify the construction using the five tests of Section 15.1.1.

Finally, replace the scalar target by the remaining log-correction after an explicitly truncated loop series. Explain why the scalar problem is better posed and what extra information is needed by the residual predictor.

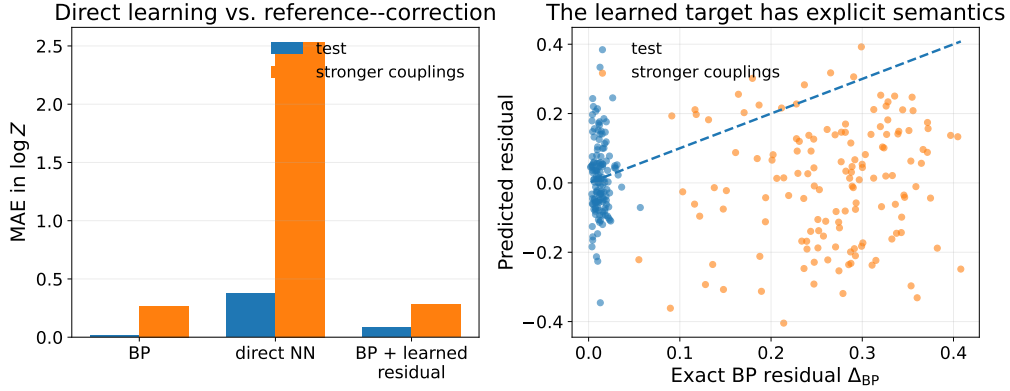


Figure 15.2: A reference-correction learning experiment. The neural model is asked either to predict the entire log-partition function or only the residual left after BP. The comparison tests whether explicit structured computation produces an easier and more transferable learning target. Generated by `ch15/ch15_02_learned_bp_residual.ipynb`.

15.9 Learning a residual correction

Even without implementing the full fractional construction, a clean experiment is available. On small loopy Ising models compute the exact $\log Z$ and a classical BP estimate $\log Z^{(BP)}$. Train a network to predict

$$\Delta_{BP} = \log Z - \log Z^{(BP)}$$



rather than $\log Z$ directly. This is a controlled test of whether structured precomputation makes the learning target simpler.

Companion notebook: Learn the correction after belief propagation

Notebook `ch15/ch15_02_learned_bp_residual.ipynb` compares direct neural prediction of $\log Z$ with prediction of the BP residual on random small Ising grids. The features include model parameters and BP-derived statistics, while exact enumeration supplies labels. The notebook reports in-distribution and stronger-coupling transfer errors.

The current seeded experiment is deliberately cautionary. In the weak-to-moderate coupling test regime, corrected Bethe/BP is already very accurate and the learned residual slightly worsens its error; under stronger-coupling transfer, residual learning recovers part of the direct-network failure but still does not beat the uncorrected BP reference. This negative result is useful: a mathematically meaningful residual is not automatically a statistically easier target. Neural correction is justified only when it improves a measured deficiency of the structured reference, not simply because a residual decomposition exists.

Problem 15.3 (Neural loop resummation). *For a family of small Ising grids, compute Z , the Bethe approximation, and an explicitly truncated loop series. Train a model to predict the remaining log-correction from graph-local features and BP beliefs. Test separately whether the learned correction transfers to unseen coupling strengths, larger grids, and different loop densities. Compare this problem with predicting λ_* and with predicting $\log Z$ directly.*

15.10 From static correction to learned proposal design

The same idea will return in Part IV on path space. There, a neural model need not learn the complete controlled drift. It may instead learn a proposal, a residual score, a desirability approximation, or a parameter that reduces the variance of an exact Feynman–Kac correction. The analogy should not be overinterpreted—a static gauge and a stochastic drift are different mathematical objects—but the design question is the same: preserve a tractable reference and spend learning capacity where the residual is hardest.

Advanced connection: A cross-part research challenge

Compare two amortization problems on matched families of small models. In the static problem, learn a fractional or gauge parameter that concentrates an exact partition-function residual. In the dynamic problem, learn a proposal parameter that concentrates an exact path-space importance weight. Ask whether the same instance features—loop density, local frustration, entropy, or disagreement among local experts—predict difficulty in both settings.

15.11 Recurring computational laboratories

The examples in the new parts will be reused rather than reinvented. Two spines will carry most of the computational comparisons.

- **Discrete Ising/LDPC/CRF models** will support BP, loop correction, MCMC, neural messages, discrete diffusion, Sampling Decisions, and hard constraints.
- **Gaussian and Gaussian-mixture graphical models** will support exact scores, products of experts, flows, diffusion, bridges, Doob transforms, and path-space corrections.

Matching/permanent and K-SAT examples remain specialized laboratories for combinatorial normalization and hard constraints. Each neuralized method should report an error against a classical reference whenever such a reference is available.

Chapter 16

Hidden Variables, Variational Learning, and Amortized Inference

Reader's Guide.

The new object: a posterior over something we never observe. With latent variables z , fitting the model to observed data x requires reasoning about $p_\theta(z \mid x)$. Learning therefore contains an inference problem inside it. This nesting is the bridge from classical graphical-model learning to variational autoencoders: the neural encoder is best understood as an amortized approximation to that inner posterior computation.

The simplest progression. EM assumes the posterior can be computed and alternates *infer* z with *fit* θ . Variational inference replaces the exact posterior by the best member of a tractable family. Amortized inference replaces the per-observation variational optimization by a learned map $x \mapsto q_\phi(z \mid x)$. A VAE combines this amortized posterior with a learned generative model $p_\theta(x \mid z)p(z)$.

Two approximation gaps. If the variational family cannot represent the true posterior, there is a *variational gap*. Even if the family is expressive enough, a neural encoder may fail to return the best member of that family for a particular x ; this is the *amortization gap*. These errors have different remedies and should be measured separately whenever an exact or high-quality posterior reference is available.

A safe hybrid. Use the encoder to predict a good variational state, then run a small number of exact or variational correction steps. The network supplies speed, while the remaining optimization or fixed-point residual provides a test of how much work is still needed. This “predict, then correct” architecture will reappear for scores, bridge controls, and discrete generation.

Status of this chapter. This draft fixes the bridge from classical latent-variable graphical models to modern amortized variational learning. It will later absorb selected material from

Mathematics of Generative AI, but will not repeat a generic introduction to neural networks.

16.1 Latent variables create an inference problem inside learning

Let x denote observed variables and z latent variables,

$$p_\theta(x, z) = p_\theta(z)p_\theta(x | z), \quad p_\theta(x) = \int p_\theta(x, z) dz.$$

Likelihood learning requires either evaluating the marginal likelihood or reasoning about the posterior $p_\theta(z | x)$. Thus latent-variable learning makes inference an inner computational problem.

16.2 EM: alternate exact inference and parameter fitting

The expectation–maximization construction introduces an auxiliary distribution over latent variables and alternates posterior inference with parameter optimization. In the idealized exact E-step,

$$q^{(k)}(z) = p_{\theta^{(k)}}(z | x),$$

followed by the M-step

$$\theta^{(k+1)} \in \arg \max_{\theta} \mathbb{E}_{q^{(k)}} [\log p_\theta(x, z)].$$

The computational bottleneck is now explicit: repeated posterior inference.

Read the ELBO identity as “exact quantity = computable bound + posterior error.” The KL term is nonnegative, so maximizing the ELBO pushes a tractable q toward the exact posterior while simultaneously improving a lower bound on $\log p_\theta(x)$. The identity is exact; approximation enters only when q is restricted or optimized imperfectly. This distinction becomes especially important after amortization, because the encoder may stop short of the best variational solution for an individual observation.

16.3 Variational inference and the ELBO

For any normalized $q(z | x)$,

$$\log p_\theta(x) = \mathbb{E}_q [\log p_\theta(x, z) - \log q(z | x)] + \text{KL}(q(z | x) \| p_\theta(z | x)).$$

The first term is the evidence lower bound (ELBO). Variational inference replaces exact posterior inference by optimization over a tractable family q .

Amortization trades repeated optimization for training. Classical variational inference pays computational cost separately for every x . Amortization pays a larger cost once, during training, to approximate the entire map from observations to variational parameters. It is therefore most attractive when many related inference problems must be solved and when the test distribution is sufficiently close to the training family.

16.4 Why amortization?

Classical variational inference solves a new optimization problem for each observation x . Amortized inference instead learns a map

$$x \mapsto q_\phi(z \mid x),$$



so that the expensive optimization is shifted to training. This is useful when many related inference problems must be solved, especially when test-time latency matters.

The gain is not free. The approximation error naturally separates into a *variational gap*, caused by a restricted family, and an *amortization gap*, caused by the learned map failing to reach the best member of that family for a given x .

16.5 Variational autoencoders as neuralized graphical models

The variational autoencoder (VAE) combines a directed latent-variable model $p_\theta(x \mid z)p(z)$ with an amortized approximate posterior $q_\phi(z \mid x)$. The classical graphical semantics remain intact; what changes is how the conditional distributions are represented and how the posterior is computed. Normalizing flows can further enrich the approximate posterior [166].

Example 16.5.1 (Mixture model $\rightarrow$ amortized classifier). *Start with a Gaussian mixture for which posterior responsibilities can be computed exactly. Train a small network to predict the responsibility vector from x . This isolates amortization from model misspecification: exact posteriors are available as labels, and one can separately measure inference error, calibration, and speed.*

16.6 When is neuralization useful?

This chapter will distinguish four regimes.

1. **Repeated inference:** amortize a solver across many related observations.
2. **Warm starting:** predict a good initialization and retain the exact or approximate fixed-point equations as a final correction.
3. **Operator simplification:** replace an expensive inner optimization or integration by a learned map.
4. **Residual learning:** preserve a strong classical approximation and learn only the remaining error.

The last two connect directly to the neural BP and loop-resummation ideas of Chapter 15.

16.7 Hybrid amortization: predict, then correct

A particularly safe architecture is

$$q_\phi(z \mid x) \longrightarrow \text{one or several exact/variational correction steps.}$$

The network provides speed; the correction step restores part of the known mathematical structure. This idea recurs later for score correction, bridge control, and autoregressive generation.

Problem 16.1 (Amortized fixed-point solver). *Choose an inference problem with a well-defined iterative map, for example BP on a family of Ising models. Train a network to predict the fixed point. Compare: direct neural prediction; neural prediction followed by k BP iterations; and BP alone. Study accuracy, time to tolerance, and generalization in graph size. Which metric most clearly reveals when amortization is actually useful?*

16.8 Transition to structured generation

Latent-variable models answer how a learned map can approximate an inference operator. The next part reverses the direction of travel: we begin with a modern generator and ask where explicit graphical, physical, and algorithmic structure can be inserted without destroying its flexibility.

Part IV

Generative AI Structuralization

Chapter 17

Structured Generative Processes: From Static Factors to Space–Time Graphs

Reader’s Guide.

Why the target density is no longer enough. A static graphical model tells us how probability is distributed over final states. A generative process also specifies *how a sample is constructed*: by transport, denoising, stochastic dynamics, sequential decisions, or conditioning of a reference process. Two generators can have the same terminal distribution and very different intermediate states, computational costs, and opportunities for enforcing constraints.

The simplest space–time picture. Take two graph-coupled variables at times $t = 0$ and $t = 1$. Horizontal factors describe within-time dependence; vertical transition factors describe how each variable evolves. A terminal constraint is just a factor on the last slice; a running cost is a factor repeated across slices; a dynamical law couples neighboring slices. Stacking the slices turns generation into one larger graphical model on space and time.

Where can structure enter? Keep the five insertion points distinct: target, architecture, training objective, sampling dynamics, and correction layer. Enforcing a parity constraint by the architecture is not the same as training with a penalty for violating it, and neither is the same as accepting a flexible proposal and projecting or correcting it afterward. These choices preserve different guarantees.

Three product objects that look similar but are not. A product of experts (PoE) combines terminal densities. A product of intermediate experts (PoIE) combines time-slice marginals. A product of path experts (PoPE) combines full path measures. They coincide

only in special cases. Much of Part IV can be read as understanding what happens when an apparently innocent static composition is promoted to a dynamical one.

Part IV reverses the direction of Part III. There we started with a graphical-model object and asked what should be neuralized. Here we start with a modern generative mechanism and ask where explicit structure should be inserted. The mathematical object that makes this precise is not only a terminal density but a **space–time probability law**: graphical structure lives inside each time slice, while generative dynamics couples successive slices.

17.1 From a structured target to a generative mechanism

The static target remains the graphical-model law introduced in Chapter 2,

$$p_{\star}(x) = \frac{1}{Z_{\star}} \prod_{a \in F} \psi_a(x_{\partial a}).$$

A generator augments this static object by auxiliary randomness and a mechanism that transports, denoises, controls, or sequentially constructs a sample. The mechanism may be deterministic, as in a normalizing flow, or stochastic, as in diffusion, a bridge, an MCMC chain, or a discrete denoising process.

The central question of this part is therefore more specific than “can the generator fit $p_{\star}$?” It is:

where does known structure enter the generative construction, and what mathematical guarantee survives after it enters?

17.2 Five insertion points for structure

Structure can enter at five non-equivalent locations.

1. **Target.** Explicit factors, energies, constraints, symmetries, observations, or rewards define what should be generated.
2. **Architecture.** Sparse couplings, graph-local operators, equivariant layers, masked attention, or factorized conditionals restrict how information propagates.
3. **Training objective.** Likelihood factors, conservation residuals, variational constraints, or consistency losses penalize violations during learning.
4. **Sampling dynamics.** Local scores, projected drifts, constrained transitions, graph-aware proposals, or controlled dynamics enforce structure during generation.
5. **Correction layer.** Metropolis–Hastings accept/reject, importance weights, Feynman–Kac weights, BP-based correction, or exact constraint enforcement repair a tractable proposal after it is generated.

The same target can use several insertion points simultaneously. A hard parity constraint, for example, may be represented explicitly in a factor graph, respected architecturally by a code-aware decoder, and enforced exactly by a final rejection or correction step.

17.3 The space–time factor graph

A discrete-time Markov generator has a path law

$$\mathbb{P}_\theta(x_{0:T}) = \mu_0(x_0) \prod_{t=0}^{T-1} K_{t,\theta}(x_{t+1} | x_t).$$

If the state $x_t = (x_{t,i})_{i \in V}$ itself carries graphical structure, the natural dynamic analogue of the factor graph is

$$\mathbb{P}_\theta(x_{0:T}) \propto \mu_0(x_0) \prod_{t=0}^{T-1} K_{t,\theta}(x_{t+1} | x_t) \prod_{t=0}^T \prod_{a \in F_t} \Psi_{t,a}(x_{t,\partial a}). \quad (17.1)$$

The transition kernels create *temporal* edges; the factors $\Psi_{t,a}$ create *spatial* or constraint edges within a time slice. Equation (17.1) is the dynamic counterpart of the factorization in Chapter 2. Diffusion, bridges, path-integral control, discrete denoising, and sequential decisions differ in how they parameterize the transition layer and which path factors they impose.

A terminal-only condition is a factor on the final slice. A running physical cost or reward is a factor repeated along the path. A conservation law may couple variables within each slice. A dynamical constraint may couple two consecutive slices. Once written this way, the distinction between “probabilistic structure” and “generative mechanism” is no longer conceptual; both are factors in one larger graphical object.

Advanced connection: the same space–time model has a directed and an undirected face

For forward generation, normalized temporal kernels are naturally read directionally: sample the earlier slice and then the later slice. For posterior inference after observations, terminal constraints, or path weights are inserted, the same local conditionals can be exposed directly as factor nodes and combined with the spatial factors in one space–time factor graph. If a variable-only elimination graph is desired, projecting those scopes produces a primal graph; for the directed conditional families, this projection is precisely moralization and therefore connects co-parents. Elimination or a junction tree then operates on that primal graph and any subsequent fill-in. Thus the directed graph is especially informative about *how to generate*, the factor graph about *which local objects define the law*, and the moral/primal graph about *which variables must coexist during exact inference*. This is the dynamic version of the distinction developed in Sections 2.5.1, 2.5.2, and 3.1.2.

Companion notebook: Static and temporal structure on one small graphical process

Notebook `ch17/ch17_01_three_compositions_gaussian.ipynb` uses a three-variable linear-Gaussian state $x_t = (x_{t,1}, x_{t,2}, x_{t,3})$ and the discrete path law

$$p(x_{0:T}) \propto \mu_0(x_0) \prod_{t=0}^{T-1} \prod_{i=1}^3 K_{t,i}(x_{t+1,i} | x_{t,i}) \prod_{t=0}^T \prod_{i=1}^2 \psi_{t,i,i+1}(x_{t,i}, x_{t,i+1}).$$

The vertical factors $K_{t,i}$ generate temporal couplings and the horizontal factors $\psi_{t,i,i+1}$ generate within-slice structure, exactly as displayed in Fig. 17.1. The same notebook then

Space–time graph: spatial structure inside slices, dynamics across slices

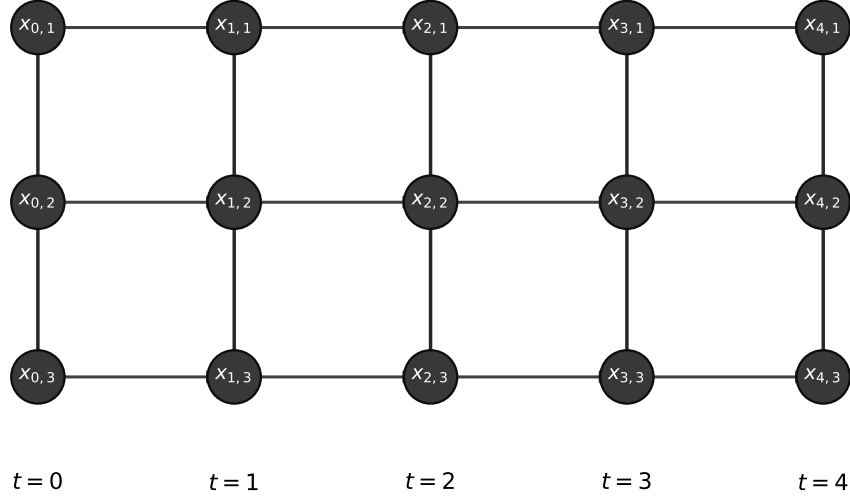


Figure 17.1: A space–time graphical model. Horizontal edges represent spatial structure within a slice; vertical directed edges represent temporal transitions. Terminal or running constraints can be inserted as additional factors without changing the organizing principle. Generated by `ch17/ch17_01_three_compositions_gaussian.ipynb`.

replaces this schematic local model by analytically tractable Gaussian expert laws and compares terminal, slice-wise, and path-level compositions.

17.4 A small algebra of compositions

We use the word *composition* for a prescribed operation that combines several component probability laws, unnormalized factors, or path laws into a new target law. The operation is part of the modeling statement: a mixture and a product of the same two experts encode different semantics and generally require different sampling algorithms. Before discussing diffusion-specific guidance, it is therefore useful to distinguish several elementary compositions.

Positive product / intersection. The weighted product

$$p(x) \propto \prod_k p_k(x)^{\beta_k}, \quad \beta_k > 0,$$

emphasizes configurations supported simultaneously by several components. This is the product-of-experts construction [88].

Mixture / union. The mixture

$$p(x) = \sum_k \pi_k p_k(x), \quad \pi_k \geq 0, \quad \sum_k \pi_k = 1,$$

represents alternatives rather than simultaneous agreement.

Ratio or negation. A product with one or more negative exponents formally suppresses configurations favored by selected components. Ratio-like combinations arise naturally in guidance constructions; classifier-free diffusion guidance, for example, combines conditional and unconditional scores with coefficients that can be interpreted through powered conditional/unconditional densities, although the resulting guided sampler should not in general be identified naively with the corresponding static powered target [92, 27].

Hard constraint. A factor taking values in $\{0, 1\}$ removes forbidden configurations rather than reweighting them softly.

Path functional. Given a reference path law P_0 , a positive path functional $\Phi[X]$ defines a changed path measure

$$\frac{dP^\Phi}{dP_0}[X] = \frac{\Phi[X]}{\mathbb{E}_{P_0}[\Phi[X]]}.$$

A common Feynman–Kac form is

$$\Phi[X] = \exp\left(-\int_0^1 V_t(X_t) dt - \varphi(X_1)\right),$$

where the running potential V_t and terminal potential φ encode rewards, costs, safety constraints, or dynamic consistency; see Chapters 19 and the path-integral-control references therein.

Companion notebook: Mixture, product, and exact hard constraint

Notebook `ch17/ch17_02_mixture_and_hard_constraints.ipynb` reuses two familiar model families. First, two one-dimensional Gaussian experts are combined as a mixture and as a positive product, making the alternative-versus-intersection semantics visible. Second, an independent four-bit reference law is multiplied by an even-parity indicator, exactly removing forbidden configurations as in the parity-factor examples of Chapter 2.

Exercise 17.4.1 (Soft versus hard composition). Use `ch17/ch17_02_mixture_and_hard_constraints.ipynb`. Replace the parity indicator by $\exp(-\lambda \mathbf{1}\{\text{parity violated}\})$. Study normalization, probability of violation, and rejection rate as λ increases. Explain which properties are recovered only in the hard limit $\lambda \rightarrow \infty$.

The rest of this chapter focuses mainly on *products and ratios*. They are especially relevant for inference-time composition because independently trained experts can be combined without retraining, because graphical-model factors themselves multiply, and because products lead directly to additive energies and additive scores. This convenience is also deceptive: multiplying terminal densities, multiplying intermediate-time marginals, and multiplying full path laws are three different operations. The next section separates them carefully.

Do not multiply before deciding *what object* is being multiplied. Multiplying $p_1(x_T)$ and $p_2(x_T)$ defines a terminal target. Multiplying $p_{1,t}(x_t)$ and $p_{2,t}(x_t)$ at every t defines a chosen marginal trajectory. Multiplying path densities $P_1(x_{0:T})P_2(x_{0:T})$ constrains

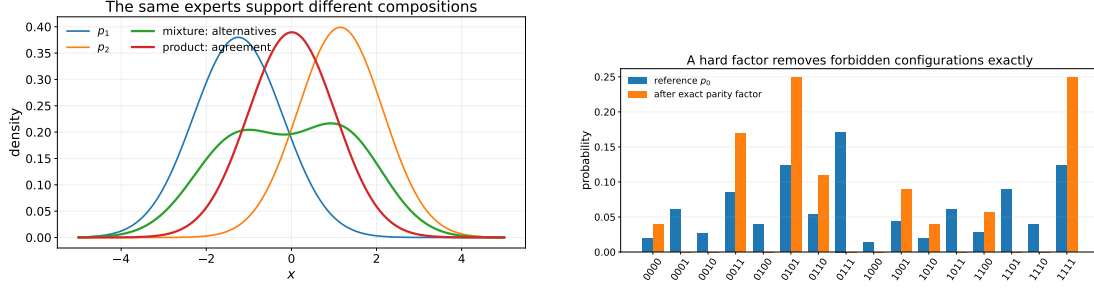


Figure 17.2: Two elementary compositions. Left: a mixture preserves two alternative modes, while a normalized positive product concentrates on their overlap. Right: multiplying an independent binary reference by a hard parity factor deletes every invalid configuration exactly. Generated by `ch17/ch17_02_mixture_and_hard_constraints.ipynb`.

temporal correlations as well. Marginalization and multiplication do not commute in general, so the three constructions need not produce the same terminal law or the same path.

17.5 Product objects: PoE, PoIE, and PoPE

We now introduce three related products that live on three different spaces. A product of terminal densities is a static object on x_1 ; a product of intermediate marginals is a time-indexed family of static objects; and a product of path measures is a law on complete trajectories. They use similar notation but should not be interchanged.

Suppose expert k has terminal density p_t^k , intermediate marginals p_t^k , and path law P^k . To make the path object precise, discretize $[0, 1]$ by $t_n = n\Delta$, $\Delta = 1/N$. If expert k is Markov with transition density $K_{n,\Delta}^k$, its time-discretized path measure is

$$dP_{\Delta}^k(x_{0:N}) = \mu_0^k(x_0) \prod_{n=0}^{N-1} K_{n,\Delta}^k(x_{n+1} | x_n) \prod_{n=0}^N dx_n.$$

Under standard regularity assumptions, these finite-dimensional measures converge weakly as $\Delta \downarrow 0$ to a probability measure P^k on an appropriate path space, such as $C([0, 1], \mathbb{R}^d)$ for continuous diffusions. In physics one often writes the same limiting object schematically as

$$P^k[dx] \propto e^{-\mathcal{A}_k[x]} \mathcal{D}x,$$

where $\mathcal{A}_k[x]$ is an action. The symbol $\mathcal{D}x$ is useful path-integral shorthand but should not be interpreted as an infinite-dimensional flat Lebesgue measure. The mathematically rigorous object is the path measure and, when two path laws are compared, their Radon–Nikodym derivative. This distinction is central to Feynman–Kac and path-integral-control constructions [99, 184].

Let β_k be exponents and define $B := \sum_k \beta_k$.

Product of experts: terminal composition. The static product is

$$p^{\text{PoE}}(x_1) = \frac{1}{Z_1} \prod_{k=1}^K (p_1^k(x_1))^{\beta_k}. \quad (17.2) \quad \text{👉}$$

This is an ordinary graphical-model construction at the terminal time.

Product of intermediate experts: slice-by-slice composition. One may instead preserve the product form at every intermediate time,

$$p_t^{\text{PoIE}}(x) = \frac{1}{Z_t} \prod_{k=1}^K (p_t^k(x))^{\beta_k}, \quad t \in [0, 1]. \quad (17.3)$$

By construction $p_1^{\text{PoIE}} = p^{\text{PoE}}$. However, the path $t \mapsto p_t^{\text{PoIE}}$ is generally not the family of marginals obtained by first forming the terminal PoE and then applying a chosen noising mechanism. Pointwise multiplication and diffusion smoothing do not commute [64, 184].

A useful normalization check is immediate. If every component starts from $N(0, I)$, then

$$\prod_k N(0, I)^{\beta_k} \propto \exp\left(-\frac{B}{2}\|x\|^2\right),$$

so the normalized initial PoIE law is $N(0, B^{-1}I)$ whenever $B > 0$. The familiar common Gaussian start is recovered when $B = 1$.

Product of path experts: path-space composition. A product of path laws must be defined relative to a common reference path measure R . The notation $P^k \ll R$ means that P^k is *absolutely continuous* with respect to R : every event that has zero R -probability also has zero P^k -probability. The Radon–Nikodym derivative dP^k/dR therefore exists, and we can define

$$\frac{dP^{\text{PoPE}}}{dR}(x_{0:T}) = \frac{1}{Z_{\text{path}}} \prod_{k=1}^K \left(\frac{dP^k}{dR}(x_{0:T}) \right)^{\beta_k}. \quad (17.4)$$

Its terminal marginal is obtained only after integrating over all interior states,

$$p_1^{\text{PoPE}}(x_1) = \int P^{\text{PoPE}}(dx_{0:T-1}, x_1),$$

and therefore need not equal Eq. (17.2). This distinction is structural: PoE multiplies endpoint beliefs, PoIE multiplies one-time marginals, and PoPE multiplies entire trajectory laws.

There is one additional path-space subtlety worth making explicit. The choice of reference measure R in Eq. (17.4) is immaterial, up to final normalization, when $B = 1$. Replacing R by an equivalent reference $\hat{R}$ introduces the same Radon–Nikodym factor in every expert, and the powers cancel against the change of base measure exactly when $B = 1$. For $B \neq 1$, the reference contributes a residual power and is therefore part of the definition of the path product. Thus geometrically normalized exponents are special for a measure-theoretic reason, not only because they preserve a common Gaussian start in the PoIE construction.

The Gaussian notebook illustrates another distinction that is easy to miss. Consider the same terminal experts but two noising schedules, $\alpha_t = t$ and $\alpha_t = t^2$, with

$$x_t = \alpha_t x_1 + \sqrt{1 - \alpha_t^2} z, \quad z \sim N(0, I).$$

Both schedules start from the same Gaussian and end at the same terminal experts, so they yield the same terminal PoE. Their intermediate expert marginals, and therefore their PoIE trajectories, are different.

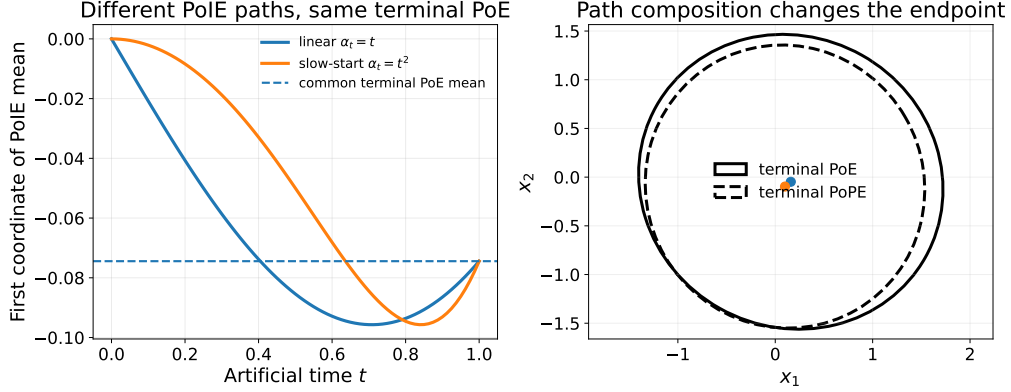


Figure 17.3: Different products live on different objects. Left: two Gaussian noising schedules, $\alpha_t = t$ and $\alpha_t = t^2$, generate distinct PoE trajectories even though their expert endpoints—and therefore the terminal PoE—are identical. Right: multiplying complete Gaussian path laws produces a PoPE whose terminal marginal generally differs from the terminal PoE because temporal correlations are multiplied before the interior states are marginalized. Generated by `ch17/ch17_01_three_compositions_gaussian.ipynb`.

Problem 17.1 (Three compositions on one exactly solvable model). *For two linear-Gaussian Markov processes with a common reference noise but different drifts, compute the terminal PoE, the slice-wise PoIE, and the terminal marginal of the path-space product in Eq. (17.4). Determine explicitly when the three coincide. Then break one condition at a time—common noise, exponent normalization, or common passive dynamics—and quantify the discrepancy. Finally replace one Gaussian terminal law by a two-component Gaussian mixture and study how the discrepancy changes with mode separation.*

The key noncommutativity: compose, then noise $\neq$ noise, then compose. If a terminal target is a product $p_1 p_2$, its noised marginal is obtained by smoothing the *product*. Adding the separately trained scores at time t instead differentiates the log of the *product of the separately smoothed marginals*. Those are different operations. This is why apparently natural score addition can generate the PoIE trajectory while missing the true diffusion trajectory of the terminal PoE.

17.6 Why naive score addition targets PoIE rather than the noised PoE

In modern machine learning the vector field

$$s_t(x) := \nabla_x \log p_t(x)$$

is called the *score*. Score-based diffusion models learn this object because the reverse-time generative dynamics can be expressed through the score of the noised marginals [186]. For the

product of intermediate marginals in Eq. (17.3), differentiation gives the exact identity

$$\nabla_x \log p_t^{\text{PoIE}}(x) = \sum_k \beta_k \nabla_x \log p_t^k(x). \quad (17.5) \quad \text{👉}$$

This additive formula makes PoIE computationally attractive whenever component scores are already available from pretrained models.

But the score of the *diffused terminal product* is instead

$$\nabla_x \log \int \left(\prod_k (p_1^k(x_1))^{\beta_k} \right) q_t(x | x_1) dx_1,$$

which is not, in general, the sum of the separately diffused component scores. This mismatch is one reason compositional diffusion needs either inner relaxation, corrected dynamics, or path-space reweighting [64, 184]. Chapter 18 studies the loss of locality under smoothing; Chapter 19 develops the reference–correction viewpoint on path space.

Companion notebook: Score addition versus the score of the noised terminal product

The Gaussian notebook `ch17/ch17_01_three_compositions_gaussian.ipynb` computes both objects analytically. For $N(m, S)$ the score is $-S^{-1}(x - m)$. At an intermediate time the notebook compares the PoIE score—which equals the weighted sum of expert scores—with the score obtained by first multiplying the terminal Gaussian experts and then applying the same noising kernel.

Exercise 17.6.1 (When does score addition become exact for the noised product?). *Use the Gaussian notebook and vary the expert means, covariances, exponents, and noising schedule. Find nontrivial parameter choices for which the PoIE score and the score of the noised terminal PoE coincide at all times, and choices for which they coincide only at $t = 0$ and $t = 1$. State the algebraic condition responsible for each coincidence.*

The same phenomenon has a discrete counterpart. Discrete Feynman–Kac correctors provide inference-time annealing, product composition, and reward tilting for discrete diffusion models, including Ising and sequence examples [82]. This will be useful in Chapter 20, where a discrete graphical factor can be treated without an artificial continuous embedding.

17.7 Known structure plus an unknown residual

Suppose

$$p_\star(x) \propto p_{\text{known}}(x) r_\star(x).$$

A central principle of Part IV is to learn only the missing object whenever possible. In energy form,

$$E_\star(x) = E_{\text{known}}(x) + E_\theta^{\text{res}}(x),$$

and in score form one may seek

$$s_\star(t, x) = s_{\text{known}}(t, x) + s_\theta^{\text{res}}(t, x).$$

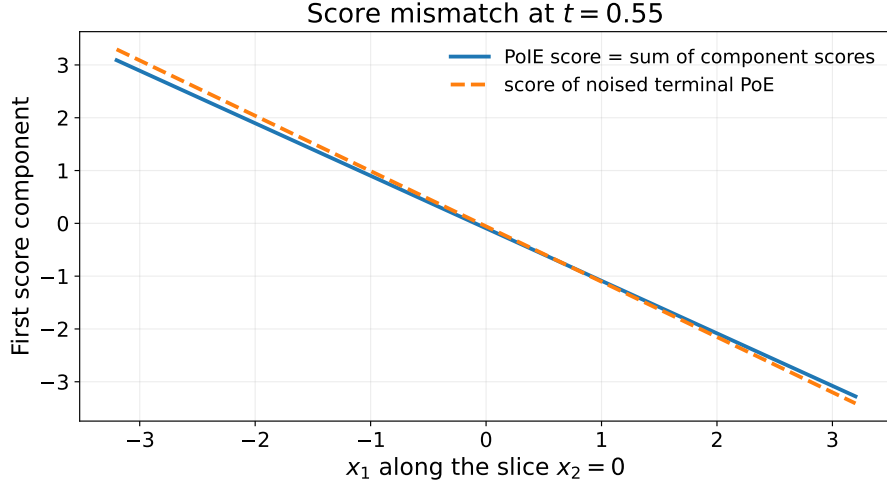


Figure 17.4: A one-dimensional slice through an exactly solvable two-dimensional Gaussian example. The PoIE score is the sum of intermediate expert scores by Eq. (17.5); the score of the noised terminal PoE is different because multiplication and smoothing do not commute. Generated by `ch17/ch17_01_three_compositions_gaussian.ipynb`.

The residual need not be small. The point is semantic separation: trusted structure remains visible and independently testable, while the learned term is responsible only for what the structured model does not explain.

Exercise 17.7.1 (Known factor plus learned residual). *Take the Gaussian infrastructure example from the applications chapter as p_{known} . Multiply it by a non-Gaussian one-dimensional or pairwise factor that represents an unmodeled preference. Compare two learning targets on samples from the resulting distribution: learn the entire log density from scratch, or keep the Gaussian energy exact and fit only the residual factor. Evaluate both inside and outside the training parameter range.*

17.8 A research challenge: one component per graphical factor

The most literal synthesis of graphical models and compositional generation would associate one generative component with each factor $\psi_a(x_{\partial a})$, or with a small region of factors. This is deliberately stated as a research challenge rather than as a finished construction. Several choices must be resolved before the idea is mathematically well defined: how a local factor becomes a normalized component law on the full state space; how discrete factors are noised; whether intermediate smoothing preserves factor locality; and how overlapping scopes interact in the correction weight.

There is nevertheless a provocative counting-number observation worth preserving. Formally, the Bethe reconstruction uses exponents 1 on factor beliefs and $1 - d_i$ on singleton beliefs. Their

sum is

$$|F| + \sum_i (1 - d_i) = |F| + |V| - |E| = 1 - L,$$

where $L = |E| - |V| + |F| + 1$ is the cycle rank of a connected factor graph. On a tree this sum is 1; on a loopy graph it is nonpositive. This does *not* prove that Bethe beliefs constitute valid generative experts, nor that compositional path collapse and Bethe inexactness are the same phenomenon. It does suggest a concrete question: can fractional or tree-reweighted counting numbers, whose normalized total is compatible with tree structure, also provide better exponents or preconditioners for a factor-wise generative composition?

Problem 17.2 (Factor-wise composition and counting numbers). *Construct the smallest discrete model for which each factor can be treated as an exact local component. Compare three exponent choices: raw factor exponents, Bethe counting numbers, and tree-reweighted counting numbers. Test normalizability of the corresponding intermediate products, the variance of any exact correction weight, and the effect of first reparametrizing the model at a BP fixed point. State precisely which observations are algebraic consequences of counting numbers and which depend on the chosen noising or path construction.*

17.9 What should be verified?

For structured generation, visual sample quality is not enough. We will repeatedly measure:

- violations of hard constraints and conservation laws;
- marginal, correlation, and conditional-probability errors;
- calibration and likelihood when available;
- transport or sample-discrepancy metrics;
- computational cost, mixing, and effective sample independence;
- transfer to changed graph size, topology, or parameter regime;
- bias and variance after a correction or reweighting layer.

The common benchmark philosophy is to reuse a few models aggressively. Discrete Ising/LDPC/CRF examples test hard and combinatorial structure. Gaussian and Gaussian-mixture graphical models provide a continuous laboratory in which marginals, scores, products, and path laws remain analytically checkable. The final synthesis chapter will compare multiple generators on the same targets rather than rewarding methods for solving different toy problems.

Problem 17.3 (Where should structure be injected?). *For the Ising, LDPC, CRF, Gaussian infrastructure, matching, and K-SAT examples, classify which of the five insertion points in Section 17.2 are natural and which are artificial. For each model, identify one structure that should remain exact and one component that could reasonably be learned. Explain whether any final correction step can restore an exact target law or only improve an approximation.*

Chapter 18

Structured Transport, Scores, and Diffusion

Reader's Guide.

What this chapter assumes and what it adds. The basic mechanics of normalizing flows, score-based diffusion, and flow matching are taken as known from the companion volume. The new question is structural: if the target has sparse factors, hard constraints, or known physics, which of those properties survive transport or smoothing, and where should a learned component enter?

The easiest exact identity. For an unsmoothed density $p(x) \propto \prod_a \psi_a(x_{\partial a})$, the score at coordinate i is a sum of local factor scores. This gives an immediate known-plus-learned decomposition when some factors are analytic. The difficulty begins after diffusion smoothing: Gaussian convolution can turn a locally factorized target into marginals whose score depends effectively on distant variables.

Locality is scale dependent. A graph-local velocity or score may be exact at one representation scale and insufficient at another. As noise increases, coarse graining can induce longer-range effective interactions. Thus “use the original graph” is a hypothesis to be tested, not a theorem. A useful experiment is to increase neighborhood radius and measure how much exact score information is recovered at each noise level.

Three structuralization choices. Known structure can enter the vector field directly, as an explicit score/energy term; it can constrain the state space, through manifold or discrete dynamics; or it can remain outside the proposal and be enforced by a correction layer. These choices define different path measures and should be compared on both sample quality and structural validity.

Status of this chapter. The chapter assumes the reader already knows the basic change-of-variables, flow, and score-based diffusion constructions. The goal is to understand what

graphical and physical structure changes.

18.1 Transport maps with sparse dependency structure

A deterministic flow transports a reference variable z to data $x = T_\theta(z)$. For a structured system, the Jacobian or vector field need not be dense. A graph-local continuous flow may take the form

$$\dot{x}_i(t) = v_{i,\theta}(t, x_i(t), x_{\partial i}(t)).$$

Locality lowers computational cost and imposes an inductive bias, but it may also limit long-range transport. The relevant question is therefore how many layers, time, or multiscale couplings are needed to communicate global dependence.

18.2 Scores inherit factorization when the factors are known

For a differentiable structured density

$$p(x) \propto \prod_a \psi_a(x_{\partial a}),$$

the score decomposes as

$$\nabla_{x_i} \log p(x) = \sum_{a \ni i} \nabla_{x_i} \log \psi_a(x_{\partial a}).$$



This elementary identity is a major structural opportunity. It suggests local score parameterizations, known-plus-learned decompositions, and diagnostics at the level of factors rather than only the full vector field.

18.3 But diffusion smoothing changes graphical locality

If p_t is obtained by convolving the target with Gaussian noise, the original Markov structure need not remain local in a naive sense. Therefore one should distinguish the factorization of p_0 from structural approximations to $s_t = \nabla \log p_t$ at positive noise. This tension will become a recurring research question: when does local structure survive coarse graining, and when must effective long-range interactions be introduced?

Residual score learning is the diffusion analogue of residual factor learning. If a baseline energy or score is known exactly, do not force a network to relearn it. Write the total score as a known component plus a residual and train only the residual. The benefit is not only statistical efficiency: the known component remains interpretable and can be tested independently when the neural residual is taken out of distribution.

18.4 Hybrid known-plus-learned scores

Suppose part of the score is analytically available,

$$s_t(x) = s_t^{\text{known}}(x) + s_\theta^{\text{res}}(t, x).$$



The network learns only the residual. This is the diffusion analogue of residual factor learning in Chapter 15. It is particularly natural in scientific models in which a baseline energy, conservation law, or coarse simulator is known.

Static factor insertion does not automatically define the correct dynamic guidance. Multiplying a base terminal density by guidance factors is easy to state. The corresponding score at intermediate noise levels depends on how that product evolves under smoothing. Simply adding available intermediate scores may therefore be an approximation. Whenever possible, identify whether the proposed guidance is exact for the intended path or requires a Doob/Feynman–Kac-type correction, developed in the next chapter.

18.5 Guidance as factor insertion

Classifier and classifier-free guidance can be interpreted as modifying a score by additional information; modern score-based diffusion provides the basic reverse-time framework [186, 91]. A graphical-model viewpoint suggests replacing one global guidance term by a collection of interpretable local or modular factors,

$$p_{\text{guided}}(x) \propto p_{\text{base}}(x) \prod_r g_r(x_{S_r}).$$



The issue is then whether simply adding score terms is dynamically consistent with the intended intermediate marginals, or whether a correction process is required.

18.6 Constraint-aware diffusion

Hard constraints, such as parity checks or exact balances, are awkward for unconstrained Gaussian diffusion because the forward process immediately leaves the constraint manifold. Possible structuralizations include diffusion intrinsically on a manifold, projection after each step, discrete-state diffusion, or a correction kernel that restores feasibility. Each changes the generated path measure and must be analyzed explicitly.

Example 18.6.1 (Ising score as a controlled laboratory). *Embed binary Ising states into a continuous noisy representation. Compare a dense neural score with a graph-local score and with a known-plus-residual score. Since the underlying discrete probabilities are exactly enumerable on a small grid, posterior denoising quantities can be computed and used as ground truth.*

Exercise 18.6.1 (Locality versus noise level). *For the noisy Ising example, estimate how well the exact score component at one site can be predicted from neighborhoods of radius $r = 1, 2, \dots$. Repeat across noise levels. Does the effective interaction range grow with smoothing time?*

18.7 Connection to flows and flow matching

Normalizing flows and flow matching provide deterministic counterparts to stochastic diffusion; see the general flow literature and modern flow-matching formulations [158, 129]. The structural questions are parallel: locality of the velocity field, exact preservation of constraints, and decomposition into known and learned components.

18.8 Forward link

Diffusion specifies a time-reversal problem. The next chapter adds endpoint conditioning, Doob transforms, and stochastic optimal control, which provide a more explicit language for inserting likelihoods, rewards, and scientific constraints into the path measure.

Chapter 19

Structured Bridges, Doob Transforms, and Path-Integral Generation

Reader's Guide.

The principally new object: a backward continuation value. Suppose a reference process is easy to simulate but we care only about trajectories that satisfy a terminal likelihood or reward $g(X_T)$. From an intermediate state x , define $h_t(x)$ as the expected remaining terminal weight under the reference process. The Doob transform modifies each transition by the ratio h_{t+1}/h_t . In this sense the difficult object is often one scalar function propagated backward in time, not an entirely new stochastic dynamics.

The simplest analogy with message passing. A factor at the end of a chain sends information backward to earlier states. The function h_t is a temporal message: it summarizes how much future terminal compatibility remains from state x at time t . Local terminal factors can therefore create a global backward object, exactly as local factors in a loopy graphical model can generate nonlocal marginals.

A hierarchy of things one might learn. If the reference drift and propagator are known, first try to learn h or a Green-function ratio. If only gradients matter, learn its score. If a physical baseline drift is available, learn only the residual control. Learning the full drift should be the last resort, because it discards more analytic structure and removes more independent checks.

Exact correction does not guarantee numerical usefulness. A proposal process can be formally corrected by a Radon–Nikodym or Feynman–Kac weight and still be practically useless if the weights collapse onto a few trajectories. Proposal design therefore has two objectives: keep the exact identity and reduce correction difficulty, measured by weight variance, effective sample size, or related diagnostics.

Status of this chapter. This chapter is intended to become one of the signature chapters of the book. The current text is an architectural draft centered on the hierarchy: exact dynamics where possible, neural approximation only for the intractable object.

19.1 Conditioning a path measure

Let P^0 denote a reference Markov path measure. A terminal likelihood, reward, or constraint changes the relative weight of trajectories. Doob's h -transform provides a canonical Markovian realization of such conditioning [63]. In discrete time, if

$$h_t(x) = \mathbb{E}_{P^0}[g(X_T) \mid X_t = x],$$



then a conditioned transition kernel has the schematic form

$$K_t^h(x' \mid x) = K_t^0(x' \mid x) \frac{h_{t+1}(x')}{h_t(x)}.$$



The difficult object is therefore often h , not the entire transition kernel.

19.2 Schrödinger bridges and endpoint matching

Schrödinger bridge formulations condition a reference stochastic process on endpoint distributions, linking entropy minimization, stochastic control, and generative transport [179, 56]. Graphical structure can enter both the endpoint laws and the reference dynamics.

19.3 Path-integral and linearly solvable control

For a class of entropy-regularized stochastic-control problems, an exponential transformation linearizes the dynamic-programming equation. This is the path-integral/linearly-solvable mechanism developed in stochastic control [99, 65]. It gives a hierarchy of objects that may be neuralized separately: desirability functions, reverse Green functions, scores, drifts, or residual controls.

19.4 A hierarchy of neuralization choices

Suppose a controlled diffusion is written schematically as

$$dX_t = (b_0(t, X_t) + u(t, X_t))dt + \sigma_t dW_t.$$

The preferred order is:

1. keep b_0 and the propagator analytic when possible;
2. learn a scalar desirability or Green-function ratio if that is the only missing object;
3. learn a score when the relevant marginal densities are unavailable;
4. learn only the residual control u_θ when a scientifically meaningful baseline drift is known;
5. use a fully neural drift only when lower-level structure is insufficient.

This hierarchy makes explicit what mathematical information has been retained.

19.5 Graphical structure in space and time

For a networked stochastic system, both the drift and the terminal likelihood may factor over a graph,

$$b_{0,i}(x) = b_{0,i}(x_i, x_{\partial i}), \quad g(x) = \prod_a g_a(x_{\partial a}).$$

The resulting $h_t(x)$ is generally global even when g is local. This creates an exact analogue of graphical-model inference: local terminal factors induce a globally coupled backward message in time.

Reference dynamics plus correction is the path-space version of the book’s recurring design principle. The proposal should be easy to simulate and close enough to the conditioned target that the explicit path weight is not degenerate. Learning can be used to improve the proposal without sacrificing exactness of the weighted identity. This is

conceptually parallel to learning a better BP/loop reference or a better importance proposal: optimize the reference so that the residual correction becomes cheap.

19.6 Importance sampling and exact correction

A learned proposal process need not itself be exact if a tractable Radon–Nikodym weight or Feynman–Kac correction is available. This motivates a general design pattern:

learned efficient dynamics + explicit path-space weight/correction.

The trade-off is weight degeneracy versus bias. This chapter will connect that trade-off to the importance-sampling discussion from Chapter 8 and to compositional diffusion corrections.

Example 19.6.1 (Gaussian network bridge). *Use the Gaussian infrastructure model from the applications chapter. The uncontrolled dynamics are linear and graph sparse; terminal observations define a Gaussian endpoint likelihood. In this case the bridge is analytically solvable and therefore provides a reference for testing learned desirability, score, and control approximations.*

Problem 19.1 (Which object should be learned?). *For the Gaussian bridge, intentionally hide different analytic components in turn: the backward likelihood h_t , its score, or the optimal drift correction. Train comparable networks for each representation. Compare sample error, training stability, and transfer when the graph parameters change. Does learning a lower-level scalar object yield a more robust model than learning the full drift?*

19.7 Scientific constraints as control information

The path-integral viewpoint is especially attractive for science because constraints can be represented as endpoint information, running costs, likelihood factors, or controlled departures from known physics. The chapter will ultimately use molecular transition paths, transport in fluids, and network restoration as candidate advanced examples, while keeping the Gaussian bridge as the reproducible baseline.

Chapter 20

Structured Autoregressive, Discrete, and Language Generation

Reader's Guide.

Why this chapter? Continuous diffusion is not the natural language for every generative object. Text, codes, graphs, schedules, and many combinatorial structures are discrete, and generation often reveals or commits variables in stages. The chain rule gives an exact sequential generator, while modern neural models amortize the conditionals. Graphical models re-enter either as persistent constraints on the object or as temporary inference problems used to coordinate neural predictions.

The simplest exact baseline: reveal one variable at a time. If every conditional $p(x_{\pi_t} \mid x_{\pi_{<t}})$ is known exactly, any ordering generates the same joint law. Ordering matters only for computational cost. Once conditionals are approximated or learned, ordering also changes approximation error. This is the discrete analogue of choosing an elimination order or proposal path.

Parallelism creates a dependence problem. A neural model may be highly confident about several unresolved positions separately, yet those positions may be incompatible when committed together. Parallel decoding therefore creates a small structured decision problem: which subset can safely be updated now? Mean-Field Parallel Decoding makes this explicit by constructing an ephemeral graphical model over binary commit variables.

A hierarchy to keep in mind. Independent confidence ignores interactions; mean field keeps average pairwise effects; BP keeps cavity dependence; fractional/loop-corrected methods retain more loopy structure; higher-order factor graphs can express constraints that no pairwise interaction matrix can represent. The right level depends on whether the extra inference cost buys enough additional safe parallelism.

The preceding chapters emphasized continuous-time transport and path-space constructions.

Language and many combinatorial objects force us to confront a complementary situation: the state space is discrete, the object grows or is revealed one component at a time, and the most successful neural models often return *local conditional predictions* rather than an explicitly normalized global joint law. Graphical models enter this setting in two different ways. They can be the generative model itself, as in a CRF, LDPC code, or K-SAT factor graph; or they can be built temporarily at inference time to coordinate the output of a large neural generator. The second possibility is particularly timely for discrete diffusion language models and will be a central theme of this chapter.

20.1 The chain rule is already a generative model

For any ordering $\pi = (\pi_1, \dots, \pi_n)$ of the variables,

$$p(x) = \prod_{t=1}^n p(x_{\pi_t} \mid x_{\pi_1}, \dots, x_{\pi_{t-1}}).$$



Chapter 4 used this identity as an exact sampling mechanism: if each conditional can be evaluated, one can sample a full configuration by revealing one variable at a time. The identity is exact for every ordering. What changes with the ordering is the computational difficulty of evaluating the successive conditionals and, after approximation or learning is introduced, the statistical difficulty of representing them.

A causal transformer replaces repeated exact inference by an amortized conditional map,

$$p_{\theta}(x) = \prod_t p_{\theta}(x_t \mid x_{<t}),$$

with self-attention providing a flexible context-dependent function approximator [195]. The transformer is therefore not, by itself, a graphical-model inference algorithm. It parameterizes the factors in a directed chain-rule representation. This distinction becomes important when the desired output is subject to global constraints that do not align naturally with a fixed left-to-right order.

Directed and undirected representations pay complexity in different places. The chain rule shows that every positive joint law admits a directed sequential representation, but it does not say that the resulting conditionals are local or easy to compute. Starting from a sparse undirected factor graph, constructing the exact conditional $p(x_{\pi_t} \mid x_{\pi_{<t}})$ generally requires marginalizing the still-unrevealed variables; elimination can create fill-in and long-range dependence on the revealed prefix.

In the other direction, a sparse Bayesian network does not need moralization to become a factor graph: each normalized conditional is already a factor on its family. Moralization enters only when those family scopes are projected onto a variable-only primal graph. The two directions therefore expose related but distinct sources of complexity:

directed local generation $\longrightarrow$ family factor graph $\xrightarrow{\text{primal projection} / \text{moralization}}$ undirected inference graph,
 undirected local factors $\xrightarrow{\text{eliminate/order}}$ directed sequential conditionals.

The first projection can add co-parent moral edges; the second conversion can add elimination fill-in and enlarged conditional contexts. This is why a representation that is sparse for generation need not be sparse for posterior inference, and a representation that is sparse for undirected inference need not yield a local autoregressive model.

Exercise 20.1.1 (Ordering and exact conditional generation). *Return to the exact conditional-generation notebook of Chapter 4. Sample the same small graphical target using several variable orderings. Verify that exact conditionals produce the same terminal law, then replace the conditionals by a deliberately local approximation and quantify how the ordering begins to matter. Relate the result to elimination order in Chapter 3.*

Exercise 20.1.2 (Moral edges versus autoregressive fill-in). *Use the four-node collider of Example 2.5.1. First construct its factor graph directly from the conditional factors, then project the scopes to the moral/primal graph and identify the co-parent edge. Next take the resulting undirected factorization and derive exact chain-rule conditionals under two different variable orders. Record which previously unrevealed variables must be marginalized to evaluate each conditional and compare the induced dependencies with the fill-in logic of Chapter 3. Distinguish carefully between an edge created by moralization and a dependence created by elimination.*

20.2 Explicit sequence structure: CRFs versus autoregressive models

The linear-chain conditional random field introduced in the applications chapter is a useful controlled comparison [119]. A CRF represents a globally normalized conditional law through explicit unary and pairwise factors,

$$p(y_{1:L} \mid x) \propto \prod_{i=1}^L \psi_i(y_i; x) \prod_{i=1}^{L-1} \psi_{i,i+1}(y_i, y_{i+1}; x),$$

and exact forward–backward or Viterbi computation exploits that chain structure. An autoregressive sequence model instead represents normalized local conditionals. A hybrid construction can use a neural model to produce context-dependent unary or pairwise potentials and retain a CRF layer to enforce explicit global sequence consistency.

This comparison foreshadows a recurring pattern in the rest of the chapter:

neural local predictions $\longrightarrow$ temporary or persistent graphical structure $\longrightarrow$ structured inference or correction.

20.3 Hard constraints: LDPC codes and discrete generation

LDPC codes provide a stricter laboratory because valid configurations live on a discrete constraint set. If H is a sparse parity-check matrix, a codeword $x \in \{0, 1\}^n$ satisfies

$$Hx = 0 \pmod{2}.$$

The associated Tanner graph exposes the parity factors, and BP exploits this structure for decoding; see Chapters 2 and 6 and the coding discussion in the applications chapter. For

generation, however, validity is not merely a preference: every accepted sample may be required to satisfy all checks. Possible mechanisms include generation directly in a parameterization of the code space, sequential generation with feasibility-aware conditionals, or a flexible neural proposal followed by an exact projection or correction. These alternatives illustrate three of the structure-insertion points from Chapter 17: architecture, sampling dynamics, and correction.

20.4 Discrete diffusion and the parallel-decoding problem

Discrete diffusion replaces strict left-to-right generation by an iterative corruption/denoising process on a categorical state space [9, 172]. For a sequence $x = (x_1, \dots, x_L) \in \mathcal{V}^L$, masked diffusion language models repeatedly evaluate a neural reverse model on a partially masked sequence. At denoising step t , let

$$U_t := \{i : x_{t,i} = [\text{MASK}]\}$$

be the positions that remain unresolved. A single neural forward pass produces, for each $i \in U_t$, a categorical predictive distribution

$$\pi_{t,i}(v) := p_\theta(x_i = v \mid x_t), \quad v \in \mathcal{V}. \quad (20.1)$$

A commit set $C_t \subseteq U_t$ is then selected, tokens are sampled or chosen at those positions, and the process continues.

The computational attraction is obvious: $|C_t| > 1$ allows several positions to be resolved from one neural evaluation. The difficulty is that many dLLMs use a factorized reverse approximation

$$p_\theta(x_{C_t} \mid x_t) \approx \prod_{i \in C_t} p_\theta(x_i \mid x_t).$$

Marginally confident positions need not be jointly compatible. Recent work on parallel dLLM decoding therefore asks how to preserve the latency advantage of parallel commitment while accounting for dependencies among the simultaneously updated positions [168, 134, 104, 96].

What is ephemeral about the graphical model? The nodes are not language tokens themselves but binary decisions about whether to commit token positions at the current denoising step. Unary terms come from neural confidence; pairwise terms penalize simultaneous commitments judged redundant or conflicting. The graph is rebuilt after the next neural forward pass. It is therefore an inference-time coordination device, not a claim about the permanent probabilistic structure of language.

20.5 Inference-time graphical modeling: Mean-Field Parallel Decoding

A particularly direct answer was proposed by Zoabi, Ali, Ringel, and Wolf in *Mean-Field Parallel Decoding for Discrete Diffusion Language Models* [227]. The key idea is conceptually important for this book. The large neural model is kept fixed. Its predictive distributions are used to construct a *new, small graphical model over the commit decisions*, and classical variational inference is then used to decide which token positions can safely be updated together.

This is the reverse of the neuralization direction of Part III:

neural generator $\longrightarrow$ ephemeral graphical model $\longrightarrow$ classical GM inference $\longrightarrow$ generation decision.

We call such a construction *inference-time graphical modeling*. The graph is ephemeral because its nodes, couplings, and even its size are rebuilt from the current neural predictions at every denoising step; it is not claimed to be the true probabilistic graph of language.

20.5.1 Binary commit variables and the temporary MRF

Let $m = |U_t|$, and introduce binary variables

$$s_i = \begin{cases} 1, & \text{commit masked position } i \text{ at this step,} \\ 0, & \text{leave it unresolved.} \end{cases}$$

Write $S = (s_1, \dots, s_m)$. The neural model supplies a unary confidence score. Following [227], let $v_i^{(1)}$ and $v_i^{(2)}$ be the most and second-most likely tokens under $\pi_{t,i}$ and define the top-two log margin

$$c_i = \log \pi_{t,i}(v_i^{(1)}) - \log \pi_{t,i}(v_i^{(2)}). \quad (20.2)$$

Thus c_i is large when one token clearly dominates the local predictive distribution.

Pairwise interactions are built from the overlap between predictive distributions. The Jensen–Shannon divergence is

$$\text{JSD}(p, q) = \frac{1}{2} D_{\text{KL}}(p \| m) + \frac{1}{2} D_{\text{KL}}(q \| m), \quad m = \frac{p + q}{2},$$

and is bounded by $\log 2$ for two distributions. MFPD uses the normalized similarity

$$\tilde{D}_{ij} = 1 - \frac{\text{JSD}(\pi_{t,i}, \pi_{t,j})}{\log 2}, \quad D_{ij} = \frac{\tilde{D}_{ij}}{\max_{a \neq b} \tilde{D}_{ab}}, \quad D_{ii} = 0. \quad (20.3)$$

Similar predictive distributions give large D_{ij} and therefore strong competition: the interpretation is that simultaneously committing two positions whose neural predictions look too similar may be redundant or mutually inconsistent.

The resulting pairwise MRF is

$$P_t(S) = \frac{1}{Z_t^{\text{commit}}} \exp \left(\sum_i c_i s_i - \sum_{i < j} D_{ij} s_i s_j \right). \quad (20.4) \quad \text{📖}$$

The partition function in Eq. (20.4) is not evaluated by the production algorithm. Its role is conceptual: it makes explicit that parallel token scheduling has been converted into a structured inference problem of exactly the kind studied in Chapters 2 and 6.

Figure 20.1 makes the construction concrete. The graph connects *decisions about token positions*, not token values themselves. This distinction will matter when we discuss higher-order linguistic factors below.

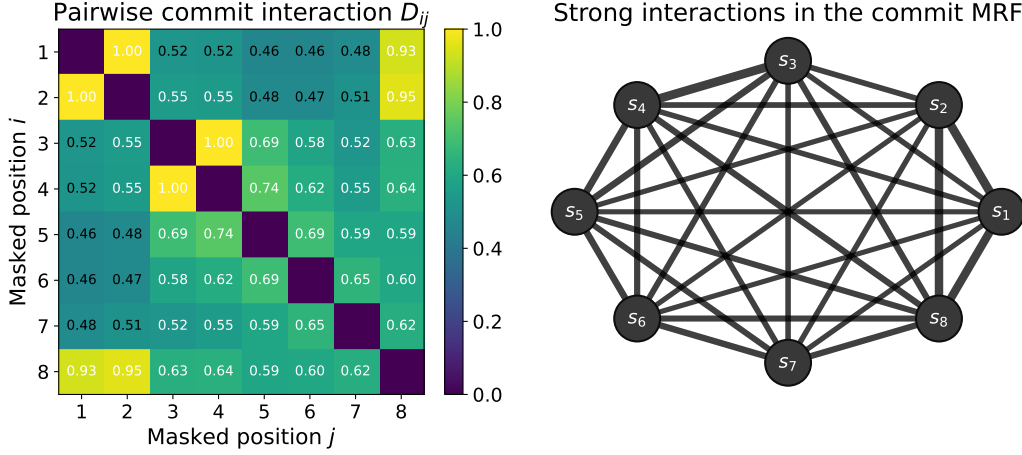


Figure 20.1: An inference-time commit MRF built from one synthetic dLLM forward pass. Left: pairwise similarities D_{ij} computed from the categorical predictive distributions. Right: the strongest interactions shown as a graph over binary commit variables. The graph is rebuilt at every denoising state and therefore acts as an ephemeral graphical model rather than a persistent language model. Generated by the notebook `ch20/ch20_01_mean_field_parallel_decoding_toy.ipynb`.

20.5.2 Mean-field relaxation

Exact MAP or marginal inference in a dense pairwise binary model can be expensive. MFPD therefore uses the fully factorized family

$$q(S) = \prod_{i=1}^m q_i^{s_i} (1 - q_i)^{1-s_i}, \quad q_i \in [0, 1],$$

and maximizes the mean-field variational objective

$$\mathcal{L}(q) = \sum_i c_i q_i - \frac{1}{2} \sum_{i \neq j} D_{ij} q_i q_j + \sum_i H_b(q_i), \quad (20.5)$$

where

$$H_b(q) = -q \log q - (1 - q) \log(1 - q)$$

is binary entropy. Differentiating Eq. (20.5) gives

$$\log \frac{q_i}{1 - q_i} = c_i - \sum_{j \neq i} D_{ij} q_j,$$

and hence the fixed-point iteration

$$q_i^{(r+1)} = \sigma \left(c_i - \sum_{j \neq i} D_{ij} q_j^{(r)} \right), \quad \sigma(z) = \frac{1}{1 + e^{-z}}. \quad (20.6) \quad \text{👉}$$

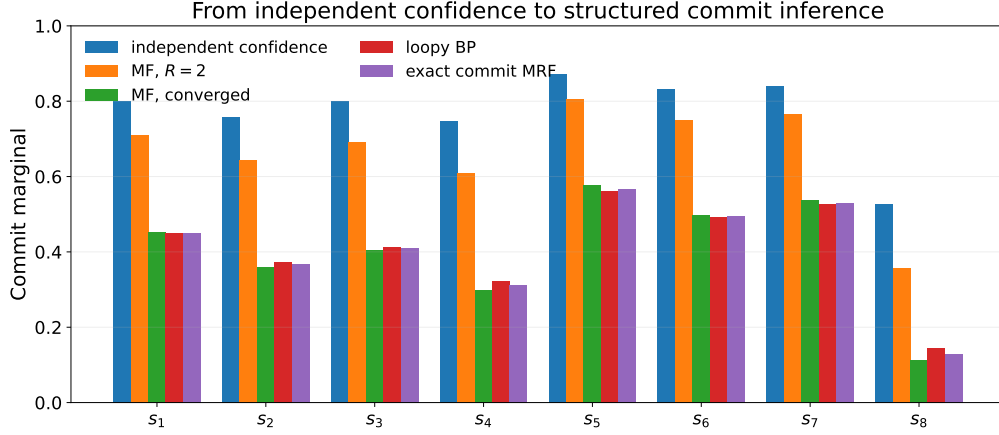


Figure 20.2: Commit marginals for the same temporary MRF as Fig. 20.1. Independent confidence ignores interactions and overestimates the willingness to commit. Mean field and loopy BP incorporate the pairwise penalties and can be compared directly with exact enumeration. In this deliberately strongly coupled toy instance, two undamped mean-field iterations have not yet reached the fixed point; this is a useful reminder that the rapid $R=2$ convergence observed in the dLLM experiments of [227] is an empirical property of those decoding instances, not a generic theorem for arbitrary commit graphs.

This is precisely the mean-field machinery developed in Chapter 6, now operating inside an LLM decoding loop. The final commit set is obtained by a policy-level threshold,

$$C_t = \{i \in U_t : q_i^{(R)} \geq \tau\}.$$

The threshold τ is not part of the MRF; it converts soft approximate marginals into an action.

The original experiments in [227] use fixed pretrained dLLMs and report improvements in the quality–latency trade-off on reasoning and code-generation benchmarks. The important methodological point for us is that the improvement is obtained *without retraining the language model*: the additional structured computation occurs entirely at inference time. The paper also emphasizes the main limitation of the construction: the $O(m^2|\mathcal{V}|)$ pairwise-comparison cost and the fact that JSD is only a pairwise proxy for the true joint conditional structure.

Companion notebook: Exact, mean-field, and BP inference in an ephemeral commit graph

Notebook `ch20/ch20_01_mean_field_parallel_decoding_toy.ipynb` starts from eight synthetic categorical distributions π_i , constructs Eqs. (20.2)–(20.4), and compares independent confidence, two MFPD-style synchronous iterations, converged mean field, loopy BP, and exact enumeration of all 2^8 commit configurations. It also varies the commit threshold and visualizes a toy parallelism–conflict trade-off. The example is deliberately small so that the graphical approximation can be checked against an exact reference rather than against another heuristic.

Exercise 20.5.1 (Reproduce MFPD on a small exact graphical model). *Use `ch20/ch20_01_mean_field_parallel`. Replace the log-margin unary score by negative predictive entropy. Compare the commit sets generated by independent confidence, $R = 2$ mean-field iterations, converged mean field, BP, and exact marginals for several thresholds τ . Report both the number of committed positions and the pairwise conflict cost defined in the notebook. Which difference is caused by the graphical model, and which is caused only by thresholding?*

The approximation method is now a design variable. Once the commit problem has been written as a graphical model, mean field is only the cheapest point in a familiar inference hierarchy. Better approximations are worthwhile only if their additional latency is smaller than the parallelism they unlock. The correct metric therefore combines inference accuracy, number of safe commitments, constraint violations, and wall-clock overhead.

20.6 Beyond mean field: bringing the rest of the GM toolbox to decoding

The MFPD construction is valuable partly because it exposes a larger design space. Once the commit-selection problem has been written as Eq. (20.4), there is no mathematical reason that mean field must be the final inference method. Mean field is attractive because it is cheap, parallel, and has no message storage. But the earlier parts of this book suggest a controlled hierarchy,

independent confidence $\longrightarrow$ mean field $\longrightarrow$ BP/Bethe $\longrightarrow$ fractional or loop-corrected inference $\longrightarrow$ higher-order fa

The useful question is therefore not “is BP more accurate than mean field?” in isolation. It is whether a better graphical approximation buys enough additional safe parallelism to justify its latency and memory cost.

20.6.1 Loopy BP on the commit graph

For Eq. (20.4), define

$$\psi_i(s_i) = e^{c_i s_i}, \quad \psi_{ij}(s_i, s_j) = e^{-D_{ij} s_i s_j}.$$

The ordinary sum-product message is

$$m_{i \rightarrow j}(s_j) \propto \sum_{s_i \in \{0,1\}} \psi_i(s_i) \psi_{ij}(s_i, s_j) \prod_{k \neq i,j} m_{k \rightarrow i}(s_i). \quad (20.7) \quad \text{👉}$$

Equation (20.7) is the same BP object studied in Chapters 3 and 6; only the semantic meaning of the binary variable has changed. On a dense m -node commit graph, one BP sweep still scales quadratically in m , so interaction sparsification and a small fixed number of iterations become natural algorithmic questions.

20.6.2 Fractional BP, loop information, and learned approximation choice

Chapter 9 showed that Bethe/BP is only one possible reference approximation and that fractional and loop-corrected descriptions can interpolate or correct it. Applying those ideas to

language decoding raises a new possibility: use graph statistics available at the current denoising step to decide *how much inference* is warranted. The interaction matrix D , its spectrum, the density of strong edges, the current denoising time, and the entropy of the neural predictions are all potential indicators.

We do not claim that the fractional parameter used earlier for partition-function approximation has an established optimal interpretation for the commit MRF. Instead, it motivates a concrete research program: fit or adapt an approximation parameter from exactly solvable small commit graphs, then test whether it transfers to larger dLLM decoding states. Likewise, explicit triangle or short-loop diagnostics may predict when the mean-field independence assumption becomes unsafe.

20.6.3 From pairwise token interactions to token factor graphs

The pairwise JSD matrix is a lightweight proxy; it does not recover the true joint conditional distribution and cannot explicitly encode higher-order relations [227]. Language and code contain many intrinsically higher-order constraints: agreement across several tokens, matched delimiters, repeated identifiers, arithmetic relations, and program syntax. A graphical extension should therefore permit factors

$$\psi_a(S_{\partial a})$$

over groups of commit decisions, or even factors that couple the candidate token *values* rather than only binary commit indicators. This is the transition from an interaction graph to a genuine factor graph.

Companion notebook: Stress-testing mean field and adding a higher-order commit factor

Notebook `ch20/ch20_02_beyond_mean_field_commit_inference.ipynb` first scales the pairwise interaction matrix by a parameter γ and measures mean-field and BP marginal errors against exact enumeration. It then inserts a three-way penalty $\exp(-\kappa s_a s_b s_c)$ that is invisible to the pairwise JSD proxy. Exact inference and a mean-field update that explicitly retains the three-way factor are compared with a pairwise-only approximation.

Problem 20.1 (Beyond mean-field parallel decoding). *Starting from a collection of saved dLLM predictive distributions, compare mean field, loopy BP, and a sparsified BP approximation on the same commit graphs. Use exact enumeration on small active blocks to calibrate the approximations. Then ask whether an inexpensive diagnostic—for example the strongest triangle weight, the spectral radius of D , or a fractional-Bethe parameter—can predict when the mean-field commit set is unreliable. The final metric should include both inference error and wall-clock overhead.*

Problem 20.2 (Higher-order language factors). *Construct a small masked-token task with an explicit three- or four-token constraint, such as balanced delimiters, a repeated program identifier, or a short arithmetic identity. Build a factor over the corresponding commit decisions or candidate token values. Compare pairwise MFPD, a higher-order mean-field approximation, and exact factor-graph inference. Determine whether the higher-order factor improves only validity or also permits greater parallelism at fixed validity.*

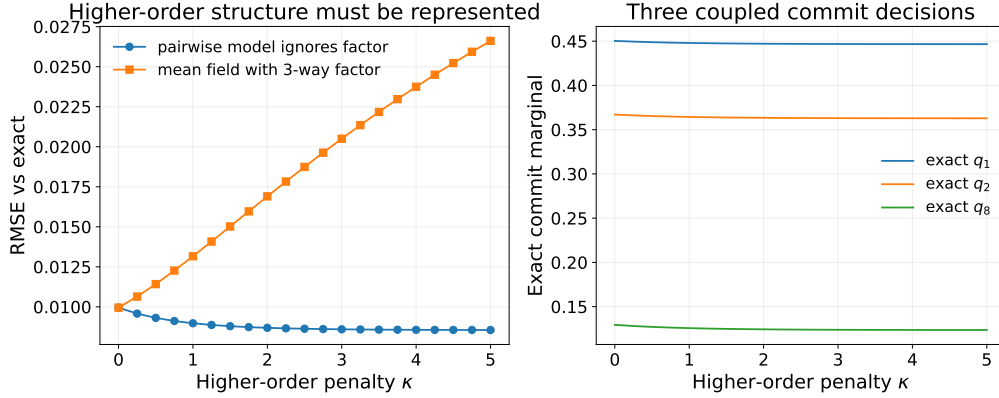


Figure 20.3: A toy demonstration of why structured LLM decoding may eventually require factor graphs rather than only pairwise token-interaction graphs. As the strength κ of a three-way incompatibility increases, an inference method that ignores the factor develops systematic error, while mean field applied to the correct higher-order model tracks the exact marginals much more closely. Generated by `ch20/ch20_02_beyond_mean_field_commit_inference.ipynb`.

From one-step scheduling to a full generative control problem. MFPD asks which variables to commit *now*. Sampling Decisions asks how every partial-state transition should be biased so that the completed object has the desired terminal law. The Doob factor h_{t+1} summarizes the value of future completions. Graphical factors that are already fully resolved can be evaluated exactly, while unresolved factors contribute through inference or a learned continuation value.

20.7 Sampling Decisions and Decision Flow

Mean-Field Parallel Decoding asks a short-horizon scheduling question: *given the current neural predictions, which subset of masked positions should be committed now?* Sampling Decisions asks a broader generative question: *which controlled transition should be taken from the current partial configuration so that the final completed object has the desired target law?* The author's recent formulation makes the state space grow as variables are revealed [45].

The general controlled transition has the Doob-transform form

$$K_t^*(s' | s) \propto K_t^0(s' | s) h_{t+1}(s'), \quad (20.8)$$

parallel to Chapter 19. Graphical information is especially natural in this representation. Factors whose scopes have been completed can be evaluated exactly; unresolved factors can contribute through approximate inference, a learned continuation value, or a combination of the two.

The MFPD commit graph can therefore be viewed as one concrete, inference-time realization of a general principle that Sampling Decisions makes explicit: a neural generator may supply a local proposal, while a structured calculation estimates the consequences of committing part of the future now. MFPD stops at a temporary pairwise selection model; Sampling Decisions allows the graphical information to shape the entire generative trajectory.

20.8 GFlowNets and reward-proportional generation

Generative flow networks construct terminal objects through sequential decisions and aim to sample them proportionally to a terminal reward [17, 18]. Their relation to graphical-model targets is especially transparent when the reward or unnormalized target decomposes over factors that become evaluable along a construction trajectory. As in Sampling Decisions, the central issue is how much of the future factor contribution is known at a partial state and how the remaining contribution should be estimated.

20.9 Local Boltzmann guidance and unresolved factors

Suppose a partial state s fixes a subset of variables. Split the factors into those whose scopes are already resolved and those involving at least one unresolved variable. A local structured guidance rule can evaluate the first set exactly and approximate the second by BP, mean field, a neural value function, or a hybrid. Schematically,

local exact factors $\longrightarrow$ approximate future inference $\longrightarrow$ neural residual value.

For LLM decoding this suggests combining three qualitatively different signals:

neural token confidence+model-induced token interactions+explicit task factors or future-value estimates.

The first two are exemplified by MFPD. The third is where known grammar, code, arithmetic, physical, or scientific structure can enter without asking the language model to relearn it implicitly.

Example 20.9.1 (K-SAT and constrained parallel revelation). *Return to the small K-SAT model from the applications chapter. A factorized neural proposal supplies per-variable preferences, while the SAT clauses are retained explicitly. Compare four policies: independent confidence, exact conditional decimation, BP-guided decimation, and a parallel commit rule that forbids any batch of assignments that already falsifies a clause. Measure diversity, fraction of satisfying samples, and bias within the satisfying set. This is the hard-constraint analogue of the language-decoding problem above.*

20.10 Synthesis: graphical models inside neural decoding

The main conceptual lesson is not that every LLM should contain a persistent graphical model. It is that a neural model’s output can itself define a structured inference problem. The resulting graph can be instance-dependent, time-dependent, and disposable. This gives a particularly direct realization of the thesis of Part IV:

learn what must be learned; construct the structure that is available; solve or correct the resulting graphical pr

The MFPD example demonstrates the minimal version of this program—pairwise interactions plus mean field. BP, fractional approximations, loop corrections, higher-order factors, and Sampling Decisions provide progressively richer extensions. The correct choice should be judged not only by language quality but by a three-way trade-off among statistical consistency, safe parallelism, and computational overhead.

Problem 20.3 (Ordering is part of the generative mechanism). *For a fixed graphical target, compare static variable orderings, random orderings, graph-aware adaptive orderings, and learned orderings. Determine whether the best ordering for computational efficiency is also the best ordering for statistical learning and transfer. Then compare this sequential design problem with the parallel commit-selection problem of MFPD: when is choosing an ordering equivalent to choosing an independent set or low-conflict subset in an inference-time interaction graph?*

Chapter 21

Structured Generative Architectures: Synthesis

Reader's Guide.

How to use a synthesis chapter. Do not compare architectures by name or by sample appearance alone. For each method identify the exact target object, the part represented analytically, the part learned, the approximation or correction layer, and the independent verification test. The purpose of the chapter is to force methods with very different engineering implementations into a common mathematical accounting.

Two benchmark spines rather than one artificial universal example. Discrete Ising/LDPC/CRF models are natural for message passing, loop corrections, discrete generation, and exact constraint checks. Gaussian and Gaussian-mixture models are natural for transport, diffusion, bridges, path-space composition, and exact score/normalizer references. Comparing within these two spines avoids confusing algorithmic differences with mismatched problem domains.

The recurring question: learn everything or only the residual? Whenever a known structure supplies a baseline energy, score, message, drift, or conditional, compare direct learning with residual learning. Then ask whether an exact correction exists and whether it remains computationally healthy. Exactness should be accompanied by a correction-difficulty diagnostic such as acceptance rate, weight ESS, loop concentration, or number of classical refinement steps.

Status of this chapter. This is a blueprint for the final synthesis. Most experiments should reuse notebooks and models developed in preceding chapters rather than introduce new algorithms here.

21.1 One target, five routes to generation

Choose a structured target for which exact small-system diagnostics are possible. The primary candidate is the Ising grid. We will solve the same generation problem through:

1. MCMC;
2. neuralized BP or a BP-derived learned proposal;
3. deterministic transport/flow;
4. diffusion or bridge generation;
5. autoregressive or Sampling-Decision generation.

The purpose is not to declare a universal winner, but to expose what each architecture represents exactly, what it learns, and where approximation enters.

21.2 A neuralization–structuralization matrix

Each method will be summarized by the following six columns:

classical object → neural surrogate → training signal → retained structure → lost guarantee
→ verification test.

For example, BP may neuralize a message update; diffusion may neuralize a residual score; autoregression may neuralize a conditional kernel. The matrix forces architectures with very different engineering appearances into a common mathematical comparison.

Representation is itself part of the algorithm. The same probability law may appear as a directed generative model, a factor graph, an undirected moral/primal graph, or a junction tree. These are not interchangeable pictures. A DAG exposes normalized local conditionals and a sampling order; those conditionals define a factor graph directly; moralization is the variable-only projection of the same family scopes and forgets direction and collider identities; elimination or triangulation may add further computational fill-in. A structured generative architecture should therefore be evaluated not only by which functions it learns, but also by which representation it chooses for generation, inference, correction, and verification. Sections 2.5.2 and 3.1.2 provide the classical prototype of this design choice.

21.3 Metrics beyond sample appearance

The comparison will report:

- error in one- and two-point marginals;
- error in energy and sufficient-statistic distributions;
- constraint violation probability;
- calibration of predicted conditionals or marginals;

- effective independent samples per unit computational cost;
- likelihood or partition-function diagnostics when available;
- sensitivity to graph size, topology, and coupling strength;
- performance after an exact correction layer, if one exists.

21.4 Two principal benchmark spines

No single laboratory is natural for both discrete message passing and continuous path-space generation. The synthesis will therefore use two principal benchmark spines rather than force one example to carry every method.

Discrete spine: Ising, LDPC, and CRF models. The Ising example spans

enumeration $\rightarrow$ BP $\rightarrow$ loopy BP $\rightarrow$ loop correction $\rightarrow$ MCMC $\rightarrow$ neural messages $\rightarrow$ discrete structured generation.

LDPC and CRF variants add hard parity constraints and sequence structure. This spine is the natural laboratory for neuralized BP, loop corrections, discrete diffusion, Sampling Decisions, and exact constraint checks.

Continuous spine: Gaussian and Gaussian-mixture graphical models. Sparse Gaussian models provide exact normalizers, marginals, scores, and conditionals; Gaussian mixtures add controlled multimodality. They therefore support transparent comparisons among deterministic transport, diffusion, PoE/PoIE/PoPE composition, Feynman–Kac correction, Doob transforms, and bridge dynamics. The continuous spine is particularly valuable because the exact target and exact path calculations remain available on small systems.

The two spines should share as many structural motifs as possible—local couplings, observations, conditioning, and compositional constraints—so that differences between algorithms are not confused with differences between examples. K-SAT and matching remain specialized challenge problems for hard constraints and combinatorial multimodality.

21.5 Known structure plus learned residual

A central synthesis question is whether one should learn the entire generative object. Whenever

$$\mathcal{O}_\star = \mathcal{O}_{\text{known}} + \mathcal{O}_{\text{res}},$$



we will compare direct learning of $\mathcal{O}_\star$ against residual learning of $\mathcal{O}_{\text{res}}$. The object $\mathcal{O}$ may be an energy, score, drift, conditional kernel, value function, or message correction.

21.6 Exact correction as a design principle

A learned generator may be best viewed as a proposal or reference mechanism rather than as the final answer. The reference–correction principle introduced in Chapter 1 appears throughout the book in several mathematically different forms:

- Metropolis–Hastings uses an accept/reject correction of a proposal kernel;
- annealed and sequential importance methods retain trajectory or bridging weights;
- loop calculus multiplies the Bethe reference by an exact generalized-loop correction;
- fractional BP retains an exact residual for every interpolation parameter even when the reference itself changes;
- Doob transforms encode the exact correction through a desirability or harmonic ratio;
- Feynman–Kac constructions retain an exact path-space weight while leaving freedom in the proposal dynamics.

These mechanisms should not be conflated: some corrections are positive probability weights, some are accept/reject rules, and some are algebraic sums that can be signed or complex. What they share is a design pattern: keep a tractable structured reference, expose the residual explicitly, and use optimization or learning to make that residual easier to evaluate.

Exactness of the identity does not imply computational usefulness. A formally exact importance representation can be dominated by a few trajectories; an exact loop series can contain too many significant terms; and an exact accept/reject correction can have vanishing acceptance probability. Therefore the synthesis will report not only residual bias but also *correction difficulty*: weight variance or effective sample size, acceptance rate, magnitude and concentration of loop corrections, fixed-point residuals, or the amount of exact refinement required after a neural prediction.

Problem 21.1 (Design a structured generator). *Choose one scientific or engineering model from the applications chapter. Identify what is known exactly, what is computationally difficult, and what is data driven. Propose a generator in which only the difficult component is neuralized. Specify the reference algorithm, training objective, verification tests, and at least one mechanism that detects or corrects structural violations.*

Problem 21.2 (One law, several graphical representations). *Choose a small positive distribution for which both a directed factorization and an undirected factorization are available. Display the DAG, the factor graph obtained directly from its conditional factors, the corresponding moral/primal graph, and a junction tree obtained after triangulation or elimination. For each representation state: (i) which conditional independences are visually explicit, (ii) which normalized local objects are available for generation, (iii) which clique or separator sizes control exact inference, and (iv) what information was added or discarded when moving from the previous representation. Then argue which representation would be the natural interface for a learned generator and which would be the natural interface for an exact correction layer.*

21.7 Open synthesis questions

The final version of this chapter should end with research questions rather than a catalogue of architectures. Candidate themes include: error propagation from local learned operators to global probability laws; transfer across graph size; neural approximation of loop or tensor corrections; structure-preserving diffusion; path-space correction of learned samplers; and the possibility of building scientific foundation models from composable probabilistic experts rather than one monolithic generator.

Part V

Appendixes

Chapter 22

Notations and Abbreviations

22.1 Notations

$\mathcal{G}$	an abstract undirected graph
$\mathcal{G} = (\mathcal{V}, \mathcal{E})$	an undirected graph built from the set of vertices, $\mathcal{V}$, and the set of edges $\mathcal{E}$
$\mathcal{G} = (\mathcal{V}, \mathcal{E}, f)$	an undirected graph built from the set of vertices, $\mathcal{V}$, the set of edges, $\mathcal{E}$ and the vector of factor functions
$\mathcal{G} = (\mathcal{V}_1, \mathcal{V}_2, \mathcal{E}, f)$	an undirected bipartite graph built from the set of vertices associated with variables, $\mathcal{V}_1$, the set of vertices associated with factors, $\mathcal{V}_2$, the set of edges, $\mathcal{E}$ and the vector of factor functions
$\mathcal{V}$	set of nodes
$\mathcal{E}$	set of undirected edges
$\mathcal{E}_d$	set of directed edges; if introduced in conjunction with, $\mathcal{E}$, the set of edges of undirected graph, it is built by doubling set of edges of $\mathcal{E}$
i, j, k	Latin lower case letters from the middle of the alphabet indexing basic (elementary) variables which are not graph-structured (general position)
a, b, c	Latin lower case letters from the beginning of the alphabet indexing nodes (vertices) associated with basic variables in the Factor Graph (FG) model
A, B, C	Latin upper case letters indexing nodes associated with factor variables in the Factor Graph (NFG) model
α, β, γ	low case Greek letters indexing undirected edges
$\alpha_d, \beta_d, \gamma_d$	low case Greek letters with subscript d mark a directed edge; choice of the α_d orientation, assuming that the respective undirected edge is introduced, is arbitrary
$\bar{\alpha}_d$	directed edge oriented in the direction opposite to the one of α_d
$e(a)$	set of directed edges starting at the node a
$v(\alpha)$	two component vector of two nodes (can be identical in the case of a self-loop) associated with the undirected edge α
$v(\alpha_d)$	two component vector of two nodes describing directed edge α_d , first and second components are associated with the start and finish of the directed edge
x_i	basic variable in the case of a general position (not associated with any graph-related structure)
x_a	basic variable associated with the node a of the FG model
x_α	basic variable associated with the edge α of the NFG model
$\mathbf{x}$	vector of basic variables of a general position, or of a GM, built from the base variables
$\mathbf{x}_A$	vector of basic variables of the FG model associated with the factor node, $A \in \mathcal{V}_2$
$\mathbf{x}_a$	vector of basic variables of the NFG model associated with the node $a \in \mathcal{V}$ built from the base variables
$\mathcal{X}$	set of values (alphabet) achievable to the vector of base variables

Misha Ch. says: We will later on add third column to the table pointing to Sections/Subsections where respective term is introduced/discussed.

$f^{(\cdots)}$	factors (factor functions) of a Graphical model, where $\cdots$, if specified, stand for the name of the factor model class
$f_A^{(\text{FG})}(\mathbf{x}_A) = \exp(-\beta E_A^{(\text{FG})}(\mathbf{x}_A))$	factor function, $f_A^{(\text{FG})}(\mathbf{x}_A)$, associated with the node $A \in \mathcal{V}_2$ of the graphical model in the factor graph representation, also restated in the exponential form, where $E_A^{(\text{FG})}(\mathbf{x}_A)$ is the respective energy function and β is the "inverse-temperature" parameter; both the factor function and the energy function depend on the vector of variables, $\mathbf{x}_A$, where $A \in \mathcal{V}_2$.
$f_a^{(\text{NFG})}(\mathbf{x}_a) = \exp(-\beta E_a^{(\text{NFG})}(\mathbf{x}_a))$	factor function, $f_a^{(\text{NFG})}(\mathbf{x}_a)$, associated with the node $a \in \mathcal{V}$ of the graphical model in the normal factor graph representation, also restated in the exponential form, where $E_a^{(\text{NFG})}(\mathbf{x}_a)$ is the respective energy function and β is the "inverse-temperature" parameter; both the factor function and the energy function depend on the vector of variables, $\mathbf{x}_a$, where $a \in \mathcal{V}$.

22.2 Abbreviations

GM	Graphical Model
FG	Factor Graph
NFG	Normal Factor Graph
MF	Mean Field
BP	Belief Propagation
MC	Markov Chain
MCMC	Markov-Chain Monte Carlo
MAP	Maximum A Posteriori
LP	Linear Programming

Chapter 23

Graphs, Variables, Functions, and Operations

This appendix collects the graph-theoretic and notational conventions used throughout the book. It is placed in the appendix rather than at the beginning so that the main text can introduce each object in context; readers may return here whenever a precise convention is needed. Section 23.1 summarizes graphs and hypergraphs, Section 23.2 specifies variables and alphabets, Section 23.3 defines factor functions, and Section 23.4 collects the basic operations applied to factors.

23.1 Graphs

23.1.1 General undirected graphs

Unless stated otherwise, a *graph* in this book is an undirected multigraph

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}),$$

where $\mathcal{V}$ is the set of vertices, or nodes, and $\mathcal{E}$ is the set of edges. Parallel edges and self-loops are allowed when a graphical-model representation requires them. We use lower-case Latin indices, such as a, b, c , for vertices and lower-case Greek indices, such as α, β, γ , for edges.

For an edge α , the incidence map $v(\alpha)$ returns its endpoint multiset. Thus $v(\alpha) = \{a, b\}$ for an ordinary edge joining distinct vertices and $v(\alpha) = \{a, a\}$ for a self-loop. Distinct edges $\alpha \neq \beta$ may have the same endpoints, producing parallel edges. The incident-edge multiset at vertex a is denoted by $e(a)$. When orientation is needed only as an auxiliary device, an undirected edge α is replaced by two directed copies α_d and $\bar{\alpha}_d$.

Figure 23.1 illustrates all three possibilities. A graph may be represented by its incidence list, adjacency matrix, or sparse incidence matrix; the most useful representation depends on the algorithm. A *path* is a sequence of adjacent edges, a *cycle* is a closed path with no repeated internal vertex, and a graph is *connected* if every pair of vertices is joined by a path. The degree of a vertex counts incident edge ends, so a self-loop contributes two.

A general undirected multigraph: vertices, parallel edges, and a self-loop

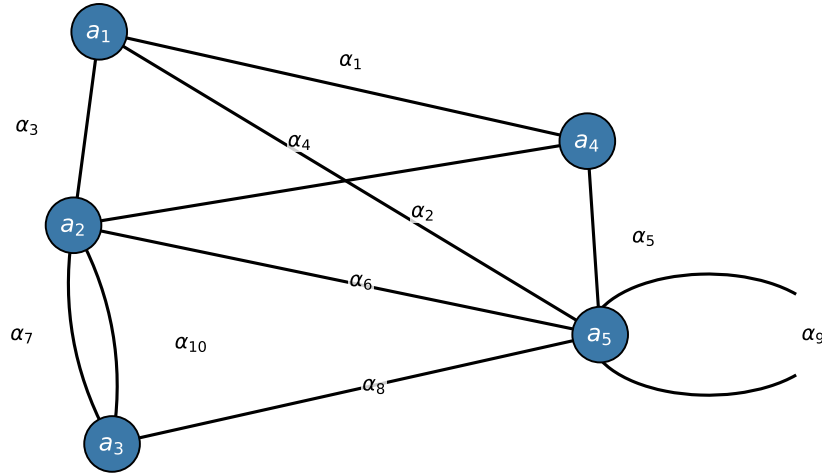


Figure 23.1: The graph notation used in the book. Vertices are indexed by Latin letters and edges by Greek letters. The example contains the parallel edges α_7 and α_{10} , and the self-loop α_9 .

23.1.2 Specialized undirected graphs

Several graph classes recur in the book.

- A *bipartite graph* has a vertex partition $\mathcal{V} = \mathcal{V}_1 \dot{\cup} \mathcal{V}_2$, and every edge joins the two parts. Factor graphs are bipartite: variable vertices belong to one part and factor vertices to the other. Figure 23.2 gives a small example.
- A *tree* is a connected graph in which every pair of vertices is joined by exactly one path. A *forest* is a disjoint union of trees. Trees support exact linear-complexity message passing; see Chapter 3, especially Sections 3.2.1.2 and 3.2.1.3.
- A *planar graph* can be embedded in the plane without edge crossings. Planarity enables determinant- and Pfaffian-based exact constructions for several important models; see Section 13.3.
- A *clique* is a set of mutually adjacent vertices. A *chordal*, or triangulated, graph is one in which every cycle of length at least four has a chord. Chordal completions, tree decompositions, and maximal cliques underlie the junction-tree construction in Section 3.3.
- The *treewidth* of a graph measures the smallest maximal-clique size, minus one, among its chordal completions. Exact elimination is exponential in this width rather than necessarily in the total number of variables.

Bipartite factor graph: variable vertices and factor vertices

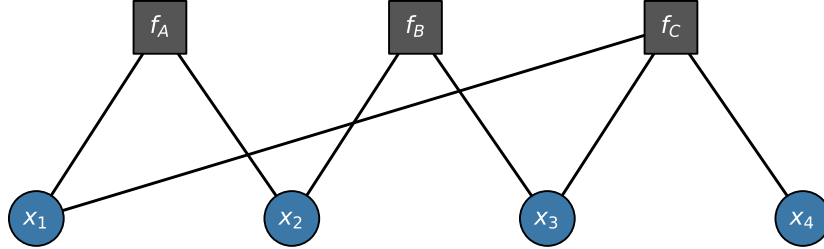


Figure 23.2: A bipartite factor graph. Circular vertices represent variables and square vertices represent factors; an edge records that the corresponding variable belongs to the factor's scope.

The graphical convention in Fig. 23.2 is the one used in Chapter 2: variable nodes are circular, factor nodes are square, and edges encode incidence rather than numerical interaction strength.

23.1.3 Directed graphs

A directed graph is again written $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, but an edge is an ordered pair (a, b) , also written $a \rightarrow b$. The parent and child sets of vertex a are

$$\text{pa}(a) = \{b : (b, a) \in \mathcal{E}\}, \quad \text{ch}(a) = \{b : (a, b) \in \mathcal{E}\}.$$

A directed acyclic graph, or DAG, contains no directed cycle. DAGs support topological orderings and therefore ancestral generation, as discussed in Section 2.5.

Several derived graphs are useful when comparing directed and undirected representations.

- The *skeleton* of a directed graph is obtained by replacing every directed edge by an undirected edge and adding nothing else.
- A *v-structure*, or unshielded collider, is a pattern $a \rightarrow c \leftarrow b$ in which a and b are not adjacent. Collider status is part of the conditional-independence semantics of a DAG and is lost if arrowheads are simply erased.
- A vertex u is an *ancestor* of v if there is a directed path from u to v , with each vertex regarded as its own ancestor when an ancestral set is formed. For $S \subseteq \mathcal{V}$, the ancestral subgraph on $\text{An}(S)$ is the subgraph induced by all ancestors of vertices in S .
- The *moral graph* of a DAG is obtained by first joining every pair of co-parents of a common child and then dropping all arrowheads. Each Bayesian-network family $\{i\} \cup \text{pa}(i)$ is therefore a clique of the moral graph.

For a Bayesian network, the factor graph is obtained directly from the conditional factors $p_i(x_i \mid x_{\text{pa}(i)})$; moralization is not needed for that construction. The primal graph of those

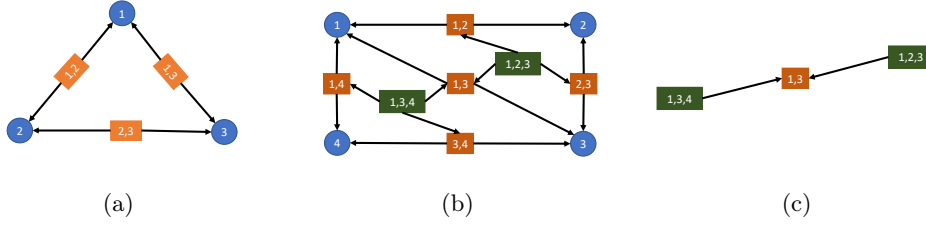


Figure 23.3: Examples of hypergraphs and their inclusion structure: (a) the scopes of a three-cycle, (b) the scopes of a square with a diagonal, and (c) an acyclic region construction for the square-with-diagonal example.

family-factor scopes is exactly the full moral graph. It is therefore the natural variable-only graph on which elimination complexity, triangulation, and junction-tree clusters are analyzed. Separately, for a conditional-independence query $A \perp B \mid C$, one may restrict to the ancestral subgraph on $\text{An}(A \cup B \cup C)$, moralize that graph, remove the vertices in C , and test ordinary undirected separation. This second construction is an equivalent test for d-separation in the original DAG, not a prerequisite for factor-graph or junction-tree construction. Section 2.5.2 develops the representation map and Section 2.5.3 develops the query-specific criterion.

23.1.4 Hypergraphs

A hypergraph generalizes a graph by allowing an edge to contain any nonempty number of vertices:

$$\mathcal{G}^h = (\mathcal{V}^h, \mathcal{E}^h), \quad \mathcal{E}^h \subseteq 2^{\mathcal{V}^h} \setminus \{\emptyset\}.$$

The elements of $\mathcal{E}^h$ are *hyperedges*. Ordinary graph edges are the special case of hyperedges of cardinality two. In factor-graph language, the scope of every factor is naturally a hyperedge.

Example 23.1.1 (Scope hypergraph of a four-variable model). *Consider factors with scopes $\{1, 2, 3\}$, $\{2, 3, 4\}$, $\{2, 3\}$, and the four singleton sets. These subsets form a partially ordered set under inclusion. The two three-variable scopes are maximal hyperedges, while $\{2, 3\}$ is their common child. This organization is more informative than the ordinary pairwise adjacency graph because it retains the full higher-order factor scopes.*

The inclusion relation among hyperedges forms a partially ordered set, or poset. Region graphs, cluster-variational approximations, and generalized belief propagation use this hierarchy explicitly; see Sections 6.3, 6.6, 6.6.1, and 6.6.3. A hypergraph is called acyclic when it admits a join-tree, or running-intersection, representation. Such hypertrees provide the higher-order analogue of ordinary trees and support exact region-based message passing.

23.2 Variables

An elementary variable x_i takes values in an alphabet or state space $\mathcal{X}_i$. Typical finite alphabets include $\{0, 1\}$, $\{-1, +1\}$, and $\{0, \pm 1\}$. Typical continuous spaces include $\mathbb{R}$, $\mathbb{R}_{\geq 0}$, a bounded

interval, and $\mathbb{C}$. Different variables may have different alphabets, although a common alphabet $\mathcal{X}$ is used when no ambiguity results.

We use the following index conventions:

- $i, j, k, \dots$ for generic elementary variables;
- $a, b, c, \dots$ for variables or factors associated with graph vertices;
- $\alpha, \beta, \gamma, \dots$ for variables associated with graph edges;
- $A, B, C, \dots$ for factor vertices in bipartite factor graphs.

Elementary vertex- and edge-associated variables are written

$$\forall a \in \mathcal{V} : \quad x_a \in \mathcal{X}_a, \quad (23.1)$$

$$\forall \alpha \in \mathcal{E} : \quad x_\alpha \in \mathcal{X}_\alpha. \quad (23.2)$$

Bold symbols denote collections of elementary variables. Three collections recur frequently.

For an edge-variable model, the variables incident to vertex a are

$$\mathbf{x}_a := (x_\alpha \mid \alpha \in e(a)). \quad (23.3)$$

For a bipartite factor graph, the variables in the scope of factor vertex A are

$$\mathbf{x}_A := (x_a \mid a \in \mathcal{V}_1, (a, A) \in \mathcal{E}). \quad (23.4)$$

For an ordinary edge $\alpha = (a, b)$ in a vertex-variable pairwise model, the endpoint pair is

$$\mathbf{x}_\alpha := (x_a, x_b). \quad (23.5)$$

The full configuration is denoted by $\mathbf{x}$, with product state space $\mathcal{X} = \prod_i \mathcal{X}_i$.

23.3 Factor functions

A factor is a scalar-valued function of a local variable collection. Unless stated otherwise, factors are nonnegative. Negative or complex factors arise in selected algebraic, quantum, and tensor-network constructions and will be identified explicitly.

For an edge-variable model, a vertex factor has the form

$$f_a(\mathbf{x}_a) : \prod_{\alpha \in e(a)} \mathcal{X}_\alpha \longrightarrow \mathbb{R}_{\geq 0}. \quad (23.6)$$

For a bipartite factor graph, a factor vertex A carries

$$f_A(\mathbf{x}_A) : \prod_{a:(a,A) \in \mathcal{E}} \mathcal{X}_a \longrightarrow \mathbb{R}_{\geq 0}. \quad (23.7)$$

For a pairwise model, an edge factor is

$$f_\alpha(\mathbf{x}_\alpha) = f_{ab}(x_a, x_b). \quad (23.8)$$

When the indexing object is intentionally generic, we use a Hebrew index such as $\aleph$:

$$f_\aleph(\mathbf{x}_\aleph). \quad (23.9)$$

A factor's *scope* is the set of variables on which it depends. A hard factor is an indicator taking values in $\{0, 1\}$; a soft factor assigns positive relative weights. Products of local factors create global unnormalized weights, as developed in Chapter 2.

23.4 Operations

23.4.1 Generic summation

The symbol $\int_{\mathcal{X}}$ is used throughout the book as a generic marginalization operator. It means a sum for finite alphabets, an integral with the appropriate measure for continuous alphabets, and a mixed sum–integral for hybrid models. If $\mathbf{x}_{\mathcal{I}}$ is the common subcollection of $\mathbf{x}_{\mathcal{N}}$ and $\mathbf{x}_{\mathcal{M}}$, then

$$\int_{\mathbf{x}_{\mathcal{I}} \in \mathcal{X}_{\mathcal{I}}} f_{\mathcal{N}}(\mathbf{x}_{\mathcal{N}}) f_{\mathcal{M}}(\mathbf{x}_{\mathcal{M}}) \quad (23.10)$$

produces a function of the variables that remain after $\mathbf{x}_{\mathcal{I}}$ is eliminated. In tensor terminology this operation is a contraction; in probability it is marginalization.

23.4.2 Multilinear and gauge transformations

For $\mathbf{x}_{\mathcal{N}} = (x_{\mathcal{N}_1}, \dots, x_{\mathcal{N}_K})$, a multilinear transformation acts separately on each argument:

$$\tilde{f}_{\mathcal{N}}(\mathbf{x}_{\mathcal{N}}) = \int_{\tilde{\mathbf{x}}_{\mathcal{N}}} \left(\prod_{k=1}^K G_{\mathcal{N}_k}(x_{\mathcal{N}_k}, \tilde{x}_{\mathcal{N}_k}) \right) f_{\mathcal{N}}(\tilde{\mathbf{x}}_{\mathcal{N}}). \quad (23.11)$$

Compatible transformations on the two ends of every edge may leave the global partition function invariant. These gauge transformations are developed in Chapter 5 and later connected to loop calculus and tensor-network basis changes.

23.4.3 Optimization

Replacing generic summation by maximization or minimization produces elimination rules for MAP estimation and constrained optimization. The algebra is analogous, but sums and integrals are replaced by max or min, and products are often replaced by sums after taking logarithms.

23.4.4 Differentiation

Factor functions may depend on continuous parameters. Differentiating the partition function, marginals, or objectives with respect to these parameters yields sensitivities and learning gradients. Because a graphical model is assembled from local functions and contractions, automatic differentiation can propagate derivatives through many inference constructions, provided the chosen operations are differentiable or equipped with suitable generalized derivatives.

Chapter 24

Historical Software Resources

While this book primarily focuses on the theoretical and algorithmic aspects of Graphical Models (GMs), we recognize the importance of software tools in applying these concepts at scale. Initially, we planned to include a section devoted to developing a modern, open-source software toolbox capable of performing inference, learning, and optimization algorithms discussed in the main text. However, with the rapid advancements in large language models that have significantly simplified coding tasks, we have chosen not to pursue this avenue.

Instead, this appendix lists historical software resources that, while possibly outdated, may still provide valuable insights or functionality for those exploring the field of Graphical Models. These tools, developed during earlier stages of GM research, laid the groundwork for many of the methods and algorithms discussed in this book. We encourage readers to explore these resources while being mindful of their limitations in the context of modern computing environments.

- **LibDAI** - A C++ library for Discrete Approximate Inference in Graphical Models. It implements several inference algorithms, including Belief Propagation, Tree-Reweighted Belief Propagation, and more. Available at: <https://github.com/jorislib/libdai>
- **UGM (Undirected Graphical Models)** - A MATLAB toolbox for working with discrete and Gaussian undirected graphical models. It includes implementations of inference and learning algorithms like Belief Propagation and Loopy Belief Propagation. Available at: <https://github.com/tdnguyen/UGM>
- **OpenGM** - A C++ library for discrete graphical models with a focus on flexibility and performance. It provides a variety of inference algorithms, including exact and approximate methods. Available at: <https://github.com/opengm/opengm>
- **Grante** - A C++ toolkit for inference and learning in discrete graphical models. It supports various inference algorithms, including Belief Propagation, Tree-Reweighted Belief Propagation, and Sampling. Available at: <https://github.com/bjoern-andres/grante>
- **PGM (Probabilistic Graphical Models)** - A Python library for working with Bayesian networks and Markov networks. It supports learning, inference, and sampling. Available at: <https://github.com/pgmpy/pgmpy>

While these tools may not match the capabilities of more recent developments, they remain relevant for historical reference and may serve as a starting point for those interested in exploring the evolution of software in the field of Graphical Models. We hope this list provides useful pointers for further exploration and encourage readers to contribute to the ongoing development of open-source tools in this area.

Bibliography

- [1] S. Ahn, M. Chertkov, and J. Shin. Gauging Variational Inference. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems 30*, pages 2885–2894. Curran Associates, Inc., 2017.
- [2] Sung-Soo Ahn, Michael Chertkov, and Jinwoo Shin. Synthesis of MCMC and Belief Propagation. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016.
- [3] Sungsoo Ahn, Michael Chertkov, Jinwoo Shin, and Adrian Weller. Gauged Mini-Bucket Elimination for Approximate Inference. In Amos Storkey and Fernando Perez-Cruz, editors, *Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics*, volume 84 of *Proceedings of Machine Learning Research*, pages 10–19. PMLR, April 2018.
- [4] Sungsoo Ahn, Michael Chertkov, Adrian Weller, and Jinwoo Shin. Bucket Renormalization for Approximate Inference. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 109–118. PMLR, July 2018.
- [5] A. Al-Bashabsheh and Y. Mao. Normal factor graphs and holographic transformations. *IEEE Transactions on Information Theory*, 57(2):752–763, 2011.
- [6] Uri Alon and Eran Yahav. On the Bottleneck of Graph Neural Networks and its Practical Implications, March 2021. arXiv:2006.05205 [cs, stat].
- [7] Nima Anari and Alireza Rezaei. A tight analysis of bethe approximation for permanent. *SIAM Journal on Computing*, 54(4):FOCS19–81–FOCS19–101, 2025.
- [8] Brian D.O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, May 1982.
- [9] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured Denoising Diffusion Models in Discrete State-Spaces, February 2023. arXiv:2107.03006 [cs].
- [10] Onureena Banerjee, Laurent El Ghaoui, and Alexandre d’Aspremont. Model selection through sparse maximum likelihood estimation for multivariate gaussian or binary data. *Journal of Machine Learning Research*, 9(15):485–516, 2008.

- [11] Ole Barndorff-Nielsen. *Information and Exponential Families in Statistical Theory*. Wiley Series in Probability and Statistics, 2014.
- [12] Leonard E. Baum, Ted Petrie, George Soules, and Norman Weiss. A maximization technique occurring in the statistical analysis of probabilistic functions of markov chains. *The Annals of Mathematical Statistics*, 41(1):164–171, feb 1970.
- [13] R.J. Baxter. *Exactly solved models in statistical mechanics*. London: Academic Press Inc, 1982.
- [14] M. Bayati, D. Shah, and M. Sharma. Max-product for maximum weight matching: Convergence, correctness, and lp duality. *IEEE Transactions on Information Theory*, 54(3):1241–1251, 2008. Proc. IEEE Int. Symp. Information Theory, 2006.
- [15] Hamidreza Behjoo and Michael Chertkov. Exact Fractional Inference via Re-Parametrization & Interpolation between Tree-Re-Weighted- and Belief Propagation- Algorithms, July 2024. arXiv:2301.10369 [cond-mat].
- [16] Richard Bellman. *Dynamic programming*. Dover Publications, Mineola, N.Y, dover ed edition, 2003.
- [17] Emmanuel Bengio, Moksh Jain, Maksym Korablyov, Doina Precup, and Yoshua Bengio. Flow Network based Generative Models for Non-Iterative Diverse Candidate Generation, November 2021. arXiv:2106.04399 [cs].
- [18] Yoshua Bengio, Salem Lahlou, Tristan Deleu, Edward J. Hu, Mo Tiwari, and Emmanuel Bengio. GFlowNet Foundations, July 2023. arXiv:2111.09266 [cs].
- [19] José Bento and Andrea Montanari. Which graphical models are difficult to learn? In *Proceedings of the 22nd International Conference on Neural Information Processing Systems*, NIPS’09, page 1303–1311, Red Hook, NY, USA, 2009. Curran Associates Inc.
- [20] Julian Besag. Statistical analysis of non-lattice data. *Journal of the Royal Statistical Society. Series D (The Statistician)*, 24(3):179–195, 1975.
- [21] H. A. Bethe. Statistical Theory of Superlattices. *Proceedings of the Royal Society of London. Series A, Mathematical and Physical Sciences*, 150(871):552–575, 1935.
- [22] Arnab Bhattacharyya, Sutanu Gayen, Eric Price, and N. V. Vinodchandran. Near-optimal learning of tree-structured distributions by chow-liu. *arXiv:2011.04144*, 2020.
- [23] Lorenz T Biegler and Victor M Zavala. Large-scale nonlinear programming using ipopt: An integrating framework for enterprise-wide dynamic optimization. *Comput. Chem. Eng.*, 33(3):575–582, 2009.
- [24] Christopher M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Springer-Verlag, Berlin, Heidelberg, 2006.
- [25] Jose Blanchet and Karthyek R. A. Murthy. Quantifying Distributional Model Risk via Optimal Transport. *Mathematics of Operations Research*, 44(2), 2019.

- [26] Enric Boix-Adsera, Guy Bresler, and Frederic Koehler. Chow-liu++: Optimal prediction-centric learning of tree ising models. In *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 417–426. IEEE, 2022.
- [27] Arwen Bradley and Preetum Nakkiran. Classifier-Free Guidance is a Predictor-Corrector, August 2024. arXiv:2408.09000 [cs].
- [28] Guy Bresler. Efficiently learning ising models on arbitrary graphs. In *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing*, STOC '15, page 771–782, New York, NY, USA, 2015. Association for Computing Machinery.
- [29] Guy Bresler. Efficiently Learning Ising Models on Arbitrary Graphs. In *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing*, pages 771–782, 2015.
- [30] Guy Bresler and Rares-Darius Buhai. Learning restricted Boltzmann machines with sparse latent variables. *Advances in Neural Information Processing Systems*, 33:7020–7030, 2020.
- [31] Guy Bresler, David Gamarnik, and Devavrat Shah. Hardness of parameter estimation in graphical models. In *Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 1*, NIPS'14, page 1062–1070, Cambridge, MA, USA, 2014. MIT Press.
- [32] Guy Bresler and Mina Karzand. Learning a tree-structured ising model in order to make predictions. *Ann. Statist.*, 48(2):713–737, 04 2020.
- [33] Guy Bresler, Frederic Koehler, and Ankur Moitra. Learning restricted boltzmann machines via influence maximization. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, pages 828–839, 2019.
- [34] Guy Bresler, Elchanan Mossel, and Allan Sly. Reconstruction of markov random fields from samples: Some observations and algorithms. In Ashish Goel, Klaus Jansen, José D. P. Rolim, and Ronitt Rubinfeld, editors, *Approximation, Randomization and Combinatorial Optimization. Algorithms and Techniques*, pages 343–356, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.
- [35] Jin-Yi Cai and Pinyan Lu. Holographic algorithms: From art to science. *Journal of Computer and System Sciences*, 77(1):41 – 61, 2011. Celebrating Karp's Kyoto Prize.
- [36] Jin-Yi Cai, Pinyan Lu, and Mingji Xia. *Holographic Algorithms*, pages 921–926. Springer New York, New York, NY, 2016.
- [37] Nicholas Carlini, Florian Tramèr, Krishnamurthy, Dvijotham, and J. Zico Kolter. (Certified!!) Adversarial Robustness for Free!, June 2022. arXiv:2206.10550 [cs].
- [38] Miguel Á. Carreira-Perpiñán and Geoffrey Hinton. On contrastive divergence learning. In Robert G. Cowell and Zoubin Ghahramani, editors, *Proceedings of the Tenth International Workshop on Artificial Intelligence and Statistics*, volume R5 of *Proceedings of Machine Learning Research*, pages 33–40. PMLR, 06–08 Jan 2005.

- [39] Venkat Chandrasekaran, Misha Chertkov, David Gamarnik, Devavrat Shah, and Jinwoo Shin. Counting Independent Sets Using the Bethe Approximation. *SIAM Journal on Discrete Mathematics*, 25(2):1012–1034, January 2011.
- [40] V. Y. Chernyak and M. Chertkov. Loop calculus and belief propagation for q-ary alphabet: Loop tower. In *2007 IEEE International Symposium on Information Theory*, pages 316–320, 2007.
- [41] M. Chertkov. Exactness of belief propagation for some graphical models with loops, 2008.
- [42] M. Chertkov, L. Kroc, F. Krzakala, M. Vergassola, and L. Zdeborová. Inference in particle tracking experiments by passing messages between images. *Proceedings of the National Academy of Sciences*, 107(17):7663–7668, April 2010.
- [43] Michael Chertkov. Driven Quantum Stars as Controlled Primitives for Real-Time Spin Dynamics, July 2026. arXiv:2607.10899 [quant-ph].
- [44] Michael Chertkov, Ruby Abrams, Amir Mohammad Esmaieeli Sikaroudi, Mikhail Krechetov, Cnp Slagle, Alon Efrat, Radoslav Fulek, and Eyal Oren. Graphical Models of Pandemic. preprint, *Epidemiology*, March 2021.
- [45] Michael Chertkov, Sungsoo Ahn, and Hamidreza Behjoo. Sampling Decisions, March 2025. arXiv:2503.14549 [cs].
- [46] Michael Chertkov and Vladimir Y. Chernyak. Loop calculus in statistical physics and information science. *Physical Review E*, 73(6):065102, June 2006.
- [47] Michael Chertkov and Vladimir Y. Chernyak. Loop series for discrete statistical models on graphs. *Journal of Statistical Mechanics: Theory and Experiment*, 2006(06):P06009–P06009, June 2006.
- [48] Michael Chertkov and Vladimir Y. Chernyak. Loop calculus in statistical physics and information science. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(06):P06009, 2008.
- [49] Michael Chertkov, Sidhant Misra, Marc Vuffray, Dvijotham Krishnamurty, and Pascal Van Hentenryck. Graphical models and belief propagation-hierarchy for optimal physics-constrained network flows. *arXiv preprint arXiv:1702.01890*, 2017.
- [50] Michael Chertkov and Adam B. Yedidia. Approximating the permanent with fractional belief propagation. *J. Mach. Learn. Res.*, 14(1):2029–2066, January 2013.
- [51] Sunil Chopra. On the spanning tree polyhedron. *Operations Research Letters*, 8(1):25–29, 1989.
- [52] C. Chow and C. Liu. Approximating discrete probability distributions with dependence trees. *IEEE Transactions on Information Theory*, 14(3):462–467, 1968.
- [53] Peter Clifford. Markov random fields in statistics. *Disorder in physical systems: A volume in honour of John M. Hammersley*, 19, 1990.

- [54] Taco Cohen and Max Welling. Group equivariant convolutional networks. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 2990–2999, New York, New York, USA, 2016. PMLR.
- [55] Constantinos Daskalakis and Qinxuan Pan. Sample-optimal and efficient learning of tree ising models. *arXiv:2010.14864*, 2020.
- [56] Valentin De Bortoli, James Thornton, Jeremy Heng, and Arnaud Doucet. Diffusion Schrödinger Bridge with Applications to Score-Based Generative Modeling. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 17695–17709. Curran Associates, Inc., 2021.
- [57] Rina Dechter. Bucket elimination: A unifying framework for reasoning. *Artificial Intelligence*, 113(1):41 – 85, 1999.
- [58] Rina Dechter. Bucket elimination: A unifying framework for reasoning. *Artificial Intelligence*, 113(1):41–85, 1999.
- [59] Rina Dechter. *Constraint Processing*. Morgan Kaufmann, 2003.
- [60] Rina Dechter and Irina Rish. Mini-buckets: A general scheme for bounded inference. *Journal of the ACM (JACM)*, 50(2):107–153, 2003.
- [61] Frank Dellaert. Factor graphs: Exploiting structure in robotics. *Annual Review of Control, Robotics, and Autonomous Systems*, 4(1):141–166, 2021.
- [62] A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1):1–38, 1977.
- [63] J.L. Doob. Conditional brownian motion and the boundary limits of harmonic functions. *Bulletin de la Société mathématique de France*, 79:431–458, 1957.
- [64] Yilun Du and Leslie Kaelbling. Compositional Generative Modeling: A Single Model is Not All You Need, June 2024. *arXiv:2402.01103 [cs]*.
- [65] E. Todorov. Linearly-solvable Markov decision problems. In *Advances in neural information processing systems*, volume 19, pages 1369–1376. MIT Press, 2007.
- [66] Jack Edmonds. Matroids and the greedy algorithm. *Mathematical Programming*, 1(1):127–136, December 1971.
- [67] J. Feldman, M. Wainwright, and D.R. Karger. Using linear programming to decode binary linear codes. *Information Theory, IEEE Transactions on*, 51:954, 2005.
- [68] Joseph Felsenstein. Evolutionary trees from DNA sequences: A maximum likelihood approach. *Journal of Molecular Evolution*, 17(6):368–376, November 1981.

- [69] Robert Ferrando, Laurent Pagnier, Robert Mieth, Zhirui Liang, Yury Dvorkin, Daniel Bienstock, and Michael Chertkov. Physics-Informed Machine Learning for Electricity Markets: A NYISO Case Study. *IEEE Transactions on Energy Markets, Policy and Regulation*, 2(1):40–51, March 2024.
- [70] G. D. Forney and P. O. Vontobel. Partition functions of normal factor graphs. In *Proc. IEEE Inf. Theory and App. Workshop*, 2011.
- [71] Nir Friedman. Inferring Cellular Networks Using Probabilistic Graphical Models. *Science*, 303(5659):799–805, February 2004.
- [72] Nir Friedman and Daphne Koller. Being Bayesian About Network Structure. A Bayesian Approach to Structure Discovery in Bayesian Networks. *Machine Learning*, 50(1):95–125, 2003.
- [73] David Gamarnik and Dmitriy Katz. Correlation decay and deterministic FPTAS for counting colorings of a graph. *Journal of Discrete Algorithms*, 12:29–47, 2012.
- [74] David Gamarnik, Devavrat Shah, and Yehua Wei. Belief propagation for min-cost network flow: convergence & correctness. In *SODA*, pages 279–292, 2010.
- [75] Josiah Willard Gibbs. *Elementary Principles in Statistical Mechanics: Developed with Especial Reference to the Rational Foundation of Thermodynamics*. Cambridge University Press, 1902.
- [76] Roy J. Glauber. Time-dependent statistics of the ising model. *Journal of Mathematical Physics*, 4(2):294–307, 1963.
- [77] Surbhi Goel, Adam Klivans, and Frederic Koehler. From boltzmann machines to neural networks and back again. *Advances in Neural Information Processing Systems*, 33:6354–6365, 2020.
- [78] V. Gomez, H.J. Kappen, and M. Chertkov. Approximate inference on planar graphs using loop calculus and belief propagation. *Journal of Machine Learning Research (JMLR)*, 11:1273, 2010.
- [79] Leonid Gurvits. Unleashing the power of schrijver’s permanental inequality with the help of the bethe approximation, 2012.
- [80] Vicenç Gómez, Joris Mooij, and Hilbert Kappen. Truncating the loop series expansion for belief propagation. *Journal of Machine Learning Research*, 8:1987–2016, 09 2007.
- [81] G.H. Hardy, J.E. Littlewood, and G. Pólya. *Inequalities*, 1934.
- [82] Mohsin Hasan, Viktor Ohanesian, Artem Gazizov, Yoshua Bengio, Alán Aspuru-Guzik, Roberto Bondesan, Marta Skreta, and Kirill Neklyudov. Discrete feynman–kac correctors, 2026. arXiv:2601.10403.
- [83] W. K. Hastings. Monte carlo sampling methods using markov chains and their applications. *Biometrika*, 57(1):97–109, 1970.

- [84] T. Hazan and A. Shashua. Norm-product belief propagation: Primal-dual message-passing for approximate inference. *IEEE Transactions on Information Theory*, 56(12):6294–6316, 2010.
- [85] U. Heinemann and A. Globerson. Inferring with high girth graphical models. *ICML*, 2014.
- [86] James Henderson, Alireza Mohammadshahi, Andrei C. Coman, and Lesly Miculicich. Transformers as Graph-to-Graph Models, October 2023. arXiv:2310.17936 [cs].
- [87] T. Heskes. On the uniqueness of loopy belief propagation fixed points. *Neural Computation*, 16(11):2379–2413, 2004.
- [88] G. E. Hinton. Training Products of Experts by Minimizing Contrastive Divergence. *Neural Computation*, 14(8):1711–1800, 2002.
- [89] G. E. Hinton and R. R. Salakhutdinov. Reducing the Dimensionality of Data with Neural Networks. *Science*, 313(5786):504–507, July 2006.
- [90] Geoffrey E. Hinton, Simon Osindero, and Yee-Whye Teh. A fast learning algorithm for deep belief nets. *Neural Computation*, 18(7):1527–1554, 2006.
- [91] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising Diffusion Probabilistic Models, December 2020. arXiv:2006.11239 [cs, stat].
- [92] Jonathan Ho and Tim Salimans. Classifier-Free Diffusion Guidance, July 2022. arXiv:2207.12598 [cs].
- [93] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *J. Am. Stat. Assoc.*, 58(301):13–30, 1963.
- [94] J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences of the United States of America*, 79(8):2554–2558, 1982.
- [95] Alexander T. Ihler, John W. Fischer III, and Alan S. Willsky. Loopy belief propagation: Convergence and effects of message errors. *J. Mach. Learn. Res.*, 6:905–936, December 2005.
- [96] Daniel Israel, Guy Van den Broeck, and Aditya Grover. Accelerating Diffusion LLMs via Adaptive Parallel Decoding, May 2025. _eprint: 2506.00413.
- [97] Mark Jerrum and Alistair Sinclair. Polynomial-time approximation algorithms for the Ising model. *SIAM Journal on computing*, 22(5):1087–1116, 1993.
- [98] Richard Jordan, David Kinderlehrer, and Felix Otto. The Variational Formulation of the Fokker–Planck Equation. *SIAM Journal on Mathematical Analysis*, 29(1):1–17, January 1998.
- [99] H. J. Kappen. Path integrals and symmetry breaking for optimal control theory. *Journal of Statistical Mechanics: Theory and Experiment*, page P11011, 2005.

- [100] Hilbert J. Kappen, Vicenç Gómez, and Manfred Opper. Optimal control as a graphical model inference problem. *Machine Learning*, 87(2):159–182, May 2012.
- [101] M. Karzand and G. Bresler. Inferning trees. In *2015 53rd Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, pages 1344–1351, 2015.
- [102] David Kempe, Jon Kleinberg, and Eva Tardos. Maximizing the spread of influence through a social network. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 137–146, Washington, D.C., August 2003. ACM.
- [103] Ryoichi Kikuchi. A Theory of Cooperative Phenomena. *Physical Review*, 81(6):988–1003, 1951.
- [104] Bumjun Kim, Dongjae Jeon, Moongyu Jeon, and Albert No. DAPD: Dependency-Aware Parallel Decoding via Attention for Diffusion LLMs, March 2026. [_eprint: 2603.12996](#).
- [105] Diederik P. Kingma and Max Welling. Auto-Encoding Variational Bayes, December 2022. [arXiv:1312.6114 \[cs, stat\]](#).
- [106] Thomas N. Kipf and Max Welling. Semi-Supervised Classification with Graph Convolutional Networks, February 2017. [arXiv:1609.02907 \[cs\]](#).
- [107] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by Simulated Annealing. *Science*, 220(4598):671–680, May 1983.
- [108] D. Koller and N. Friedman. *Probabilistic Graphical Models*. Massachusetts: MIT Press, 2009.
- [109] Daphne Koller and Nir Friedman. *Probabilistic graphical models: principles and techniques*. MIT press, 2009.
- [110] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques - Adaptive Computation and Machine Learning*. The MIT Press, 2009.
- [111] V. Kolmogorov and M.J. Wainwright. On the optimality of tree-reweighted max-product message-passing. In *Uncertainty on Artificial Intelligence*, Edinburgh, Scotland, 2005.
- [112] Vladimir Kolmogorov, Johan Thapper, and Stanislav Živný. The power of linear programming for general-valued csp. *SIAM Journal on Computing*, 44(1):1–36, 2015.
- [113] Stefanos Kourtis, Claudio Chamon, Eduardo R. Mucciolo, and Andrei E. Ruckenstein. Fast counting with tensor networks. *SciPost Physics*, 7(5):060, November 2019. [arXiv:1805.00475 \[cond-mat.stat-mech\]](#).
- [114] H. A. Kramers and G. H. Wannier. Statistics of the two-dimensional ferromagnet. part i. *Physical Review*, 60:252–262, 1941.
- [115] Mikhail Krechetov, Amir Mohammad Esmaeeli Sikaroudi, Alon Efrat, Valentin Polishchuk, and Michael Chertkov. Prediction and prevention of pandemics via graphical model inference and convex programming. *Scientific Reports*, 12(1):7599, May 2022.

- [116] F.R. Kschischang, B.J. Frey, and H.-A. Loeliger. Factor graphs and the sum-product algorithm. *Info. Theory, IEEE Trans. on*, 47(2):498–519, feb 2001.
- [117] Ryogo Kubo. Generalized cumulant expansion method. *Journal of the Physical Society of Japan*, 17(7):1100–1120, 1962.
- [118] S. Kullback and R. A. Leibler. On information and sufficiency. *Ann. Math. Statist.*, 22(1):79–86, 03 1951.
- [119] John D. Lafferty, Andrew McCallum, and Fernando C. N. Pereira. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *Proceedings of the Eighteenth International Conference on Machine Learning*, pages 282–289, 2001.
- [120] S. L. Lauritzen. *Graphical Models*. Oxford University Press, 1996.
- [121] S. L. Lauritzen and D. J. Spiegelhalter. Local computations with probabilities on graphical structures and their application to expert systems. *Journal of the Royal Statistical Society, Series B*, 50:155–224, 1988.
- [122] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, May 2015.
- [123] David A. Levin and Yuval Peres. *Markov Chains and Mixing Times*. American Mathematical Society, 2 edition, 2017.
- [124] Michael Levin and Cody P. Nave. Tensor renormalization group approach to two-dimensional classical lattice models. *Physical review letters*, 99(12):120601, 2007.
- [125] Liang Li, Pinyan Lu, and Yitong Yin. Approximate counting via correlation decay in spin systems. In *Proceedings of the Twenty-Third Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 922–940, 2012.
- [126] Liang Li, Pinyan Lu, and Yitong Yin. Correlation decay up to uniqueness in spin systems. In *Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 67–84, 2013.
- [127] Valerii Likhoshesterov, Yury Maximov, and Michael Chertkov. Tractable minor-free generalization of planar zero-field Ising models. *arXiv:1910.11142*, 2019.
- [128] Valerii Likhoshesterov, Yury Maximov, and Misha Chertkov. Inference and sampling of k_{33} -free Ising models. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3963–3972, Long Beach, California, USA, 2019.
- [129] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow Matching for Generative Modeling, February 2023. *arXiv:2210.02747* [cs, stat].
- [130] Chengyi Liu, Wenqi Fan, Yunqing Liu, Jiatong Li, Hang Li, Hui Liu, Jiliang Tang, and Qing Li. Generative Diffusion Models on Graphs: Methods and Applications, August 2023. *arXiv:2302.02591* [cs].

- [131] Qiang Liu and Alexander T. Ihler. Bounding the partition function using Hölder’s inequality. In *Proceedings of the 28th International Conference on Machine Learning (ICML-11)*, pages 849–856, 2011.
- [132] H.-A. Loeliger. An introduction to factor graphs. *IEEE Signal Processing Magazine*, 21(1):28–41, 2004.
- [133] Andrey Y. Lokhov, Marc Vuffray, Sidhant Misra, and Michael Chertkov. Optimal structure and parameter learning of Ising models. *Science Advances*, 4(3):e1700791, March 2018.
- [134] Lizhuo Luo, Zhuoran Shi, Jiajun Luo, Zhi Wang, Shen Ren, Wenya Wang, and Tianwei Zhang. DAWN: Dependency-Aware Fast Inference for Diffusion LLMs, February 2026. [_eprint: 2602.06953](#).
- [135] Christian Léonard. A Survey of the Schrödinger Problem and Some of its Connections with Optimal Transport. *Discrete and Continuous Dynamical Systems - A*, 34(4):1533–1574, 2014.
- [136] D. J. C. Mackay. Encyclopedia of sparse graph codes.
- [137] D. J. C. Mackay. *Information theory, inference, and learning algorithms*. Cambridge University Press, 2003.
- [138] Dmitry M. Malioutov, Jason K. Johnson, and Alan S. Willsky. Walk-sums and belief propagation in gaussian graphical models. *J. Mach. Learn. Res.*, 7:2031–2064, December 2006.
- [139] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.
- [140] Sidhant Misra. *Decay of correlations and inference in graphical models*. PhD thesis, MIT, 2014.
- [141] Andrea Montanari. Computational implications of reducing data to sufficient statistics. *Electron. J. Statist.*, 9(2):2370–2390, 2015.
- [142] Andrea Montanari et al. Computational implications of reducing data to sufficient statistics. *Electron. J. Stat.*, 9(2):2370–2390, 2015.
- [143] Joris M. Mooij and Hilbert J. Kappen. Sufficient conditions for convergence of loopy belief propagation. In *Proceedings of the Twenty-First Conference on Uncertainty in Artificial Intelligence*, UAI’05, page 396–403, Arlington, Virginia, USA, 2005. AUAI Press.
- [144] Cristopher Moore and Stephan Mertens. *The Nature of Computation*. Oxford University Press, 2011.
- [145] R. Mori. Holographic transformation, belief propagation and loop calculus for generalized probabilistic theories. In *2015 IEEE International Symposium on Information Theory (ISIT)*, pages 1099–1103, 2015.

- [146] R. Mori. Loop calculus for nonbinary alphabets using concepts from information geometry. *IEEE Transactions on Information Theory*, 61(4):1887–1904, 2015.
- [147] Tohru Morita. Formal Structure of the Cluster Variation Method. *Progress of Theoretical Physics Supplement*, 115:27–39, 1994.
- [148] K. Murphy. *Machine Learning: A Probabilistic Perspective*. Massachusetts: MIT Press, 2011.
- [149] M. Mézard and A. Montanari. *Information, Physics, and Computation*. Oxford Graduate Texts. OUP Oxford, 2009.
- [150] M. Mézard, G. Parisi, and M.A. Virasoro. *Spin Glass Theory And Beyond: An Introduction To The Replica Method And Its Applications*. World Scientific Lecture Notes In Physics. World Scientific Publishing Company, 1987.
- [151] Radford M Neal. Annealed importance sampling. *Statistics and Computing*, 11(2):125–139, 2001.
- [152] Radford M. Neal. Annealed importance sampling. *Statistics and Computing*, 11:125–139, 2001.
- [153] Sahand Negahban, Bin Yu, Martin J Wainwright, and Pradeep K Ravikumar. A unified framework for high-dimensional analysis of m -estimators with decomposable regularizers. In *Advances in Neural Information Processing Systems*, pages 1348–1356, 2009.
- [154] Yang Ni, Veerabhadran Baladandayuthapani, Marina Vannucci, and Francesco C. Stingo. Bayesian graphical models for modern biological applications. *Statistical Methods & Applications*, 31(2):197–225, June 2022.
- [155] Yang Ni, Peter Müller, Lin Wei, and Yuan Ji. Bayesian graphical models for computational network biology. *BMC Bioinformatics*, 19(S3):63, March 2018.
- [156] Lars Onsager. Crystal statistics. i. a two-dimensional model with an order-disorder transition. *Phys. Rev.*, 65:117–149, Feb 1944.
- [157] Román Orús. A Practical Introduction to Tensor Networks: Matrix Product States and Projected Entangled Pair States. *Annals of Physics*, 349:117–158, 2014.
- [158] George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference. *J. Mach. Learn. Res.*, 22(1), January 2021.
- [159] J. Pearl. Bayesian networks: A model of self-activated memory for evidential reasoning. In *Proc. of Cognitive Science Society (CSS-7)*, 1985.
- [160] Judea Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, San Francisco, 1988.
- [161] H.A. Peierls. Ising’s model of ferromagnetism. *Proceedings of Cambridge Philosophical Society*, 32:477–481, 1936.

- [162] Jonathan K Pritchard, Matthew Stephens, and Peter Donnelly. Inference of Population Structure Using Multilocus Genotype Data. *Genetics*, 155(2):945–959, June 2000.
- [163] James G. Propp and David B. Wilson. Exact sampling with coupled Markov chains and applications to statistical mechanics. *Random Structures and Algorithms*, 9(1&2):223–252, 1996.
- [164] Pradeep Ravikumar, Martin J. Wainwright, and John D. Lafferty. High-dimensional ising model selection using l_1 -regularized logistic regression. *Ann. Statist.*, 38(3):1287–1319, 06 2010.
- [165] Pradeep Ravikumar, Martin J Wainwright, John D Lafferty, et al. High-dimensional Ising model selection using l_1 -regularized logistic regression. *Ann. Stat.*, 38(3):1287–1319, 2010.
- [166] Danilo Rezende and Shakir Mohamed. Variational Inference with Normalizing Flows. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1530–1538, Lille, France, July 2015. PMLR.
- [167] Thomas J. Richardson and Rudiger L. Urbanke. *Modern Coding Theory*. Cambridge University Press, 2008.
- [168] Liran Ringel, Ameen Ali, and Yaniv Romano. Dependency-Guided Parallel Decoding in Discrete Diffusion Language Models, April 2026. [_eprint: 2604.02560](#).
- [169] Christian P. Robert and George Casella. *Monte Carlo Statistical Methods*. Springer, 2 edition, 2004.
- [170] Nicholas Ruozzi. The bethe partition function of log-supermodular graphical models. In *Advances in Neural Information Processing Systems*, pages 117–125, 2012.
- [171] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Prentice Hall, 3rd edition, 2010.
- [172] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and Effective Masked Diffusion Language Models, November 2024. [arXiv:2406.07524 \[cs\]](#).
- [173] N. P. Santhanam and M. J. Wainwright. Information-theoretic limits of selecting binary graphical models in high dimensions. *IEEE Transactions on Information Theory*, 58(7):4117–4134, 2012.
- [174] Victor Garcia Satorras and Max Welling. Neural Enhanced Belief Propagation on Factor Graphs, March 2021. [arXiv:2003.01998 \[cs\]](#).
- [175] Dmitrij Schlesinger and Boris Flach. Transforming an arbitrary minsum problem into a binary one. Technical report, Dresden Institute of Technology, 2006.
- [176] M. I. Schlesinger. Syntactic analysis of two-dimensional visual signals in noisy conditions. *Kibernetika [in Russian]*, 4:113–130, 1976.

- [177] M. I. Schlesinger and B. Flach. Some solvable subclasses of structural recognition problems. In *Czech Pattern Recognition Workshop*, 2000.
- [178] A. Schrijver. *Combinatorial Optimization*. Springer-Verlag, 2003.
- [179] Erwin Schrödinger. Über die Umkehrung der Naturgesetze. *Sitzungsberichte der Preussischen Akademie der Wissenschaften, Physikalisch-Mathematische Klasse*, pages 144–153, 1931.
- [180] A. Schwing, T. Hazan, M. Pollefeys, and R. Urtasun. Distributed message passing for large scale graphical models. In *CVPR 2011*, pages 1833–1840, 2011.
- [181] Alex Schwing, Tamir Hazan, Marc Pollefeys, and Raquel Urtasun. Globally convergent dual map lp relaxation solvers using fenchel-young margins. In F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 25*, pages 2384–2392. Curran Associates, Inc., 2012.
- [182] Hanif D. Sherali and Warren P. Adams. A hierarchy of relaxations between the continuous and convex hull representations for zero-one programming problems. *SIAM Journal on Discrete Mathematics*, 3(3):411–430, 1990.
- [183] Alistair Sinclair, Piyush Srivastava, and Marc Thurley. Approximation algorithms for two-state anti-ferromagnetic spin systems on bounded degree graphs. In *Proceedings of the Twenty-Third Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 941–953, 2012.
- [184] Marta Skreta, Tara Akhound-Sadegh, Viktor Ohanesian, Roberto Bondesan, Alán Aspuru-Guzik, Arnaud Doucet, Rob Brekelmans, Alexander Tong, and Kirill Neklyudov. Feynman-Kac Correctors in Diffusion: Annealing, Guidance, and Product of Experts, March 2025. arXiv:2503.02819 [cs].
- [185] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep Un-supervised Learning using Nonequilibrium Thermodynamics. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 2256–2265, Lille, France, July 2015. PMLR.
- [186] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-Based Generative Modeling through Stochastic Differential Equations, February 2021. arXiv:2011.13456 [cs, stat].
- [187] David Sontag, Talya Meltzer, Amir Globerson, Yair Weiss, and Tommi Jaakkola. Tightening LP relaxations for MAP using message-passing. In *24th Conference in Uncertainty in Artificial Intelligence*, pages 503–510. AUAI Press, 2008.
- [188] S. Talukdar, D. Deka, B. Lundstrom, M. Chertkov, and M. V. Salapaka. Learning exact topology of a loopy power grid from ambient dynamics. In *Proceedings of the Eighth International Conference on Future Energy Systems, e-Energy '17*, pages 222–227, New York, NY, USA, 2017. ACM.

- [189] Sekhar C. Tatikonda and Michael I. Jordan. Loopy belief propagation and gibbs measures. In *Proceedings of the Eighteenth Conference on Uncertainty in Artificial Intelligence*, UAI'02, page 493–500, San Francisco, CA, USA, 2002. Morgan Kaufmann Publishers Inc.
- [190] Luke Tierney. Markov chains for exploring posterior distributions. *The Annals of Statistics*, 22(4):1701–1728, 1994.
- [191] Joseph Tindall and Matthew T. Fishman. Gauging tensor networks with belief propagation. *SciPost Physics*, 15(6):222, December 2023. arXiv:2306.17837 [quant-ph].
- [192] Joseph Tindall, Grace M. Sommers, and Hilbert Kappen. Contracting Tensor Networks with Generalized Belief Propagation, April 2026. arXiv:2604.24760 [quant-ph].
- [193] Leslie G. Valiant. Holographic algorithms. *SIAM Journal on Computing*, 37(5):1565–1594, 2008.
- [194] Nico G. van Kampen. A simplified cluster expansion for the classical real gas. *Physica D: Nonlinear Phenomena*, 27:783–792, 1961.
- [195] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [196] Pavlo Vasylenko, Pere-Lluís Huguet Cabot, Abelardo Carlos Martínez Lorenzo, and Roberto Navigli. Incorporating Graph Information in Transformer-based AMR Parsing, June 2023. arXiv:2306.13467 [cs].
- [197] P. O. Vontobel. The bethe permanent of a non-negative matrix. In *2010 48th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, pages 341–346, 2010.
- [198] Pascal O. Vontobel. The bethe permanent of a non-negative matrix. *CoRR*, abs/1107.4196, 2011.
- [199] Matthew J. Vowels, Necati Cihan Camgoz, and Richard Bowden. D’ya like dags? a survey on structure learning and causal discovery. *ACM Comput. Surv.*, apr 2022.
- [200] Marc Vuffray, Sidhant Misra, Andrey Lokhov, and Michael Chertkov. Interaction Screening: Efficient and Sample-Optimal Learning of Ising Models. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016.
- [201] M. J. Wainwright, T. S. Jaakkola, and A. S. Willsky. A new class of upper bounds on the log partition function. *IEEE Transactions on Information Theory*, 51(7):2313–2335, 2005.
- [202] M. J. Wainwright and M. I. Jordan. Graphical Models, Exponential Families, and Variational Inference. *Foundations and Trends in Machine Learning*, 1(1):1–305, 2008.
- [203] Martin J. Wainwright. Estimating the “wrong” graphical model: Benefits in the computation-limited setting. *Journal of Machine Learning Research*, 7(66):1829–1859, 2006.

- [204] Martin J. Wainwright. *High-Dimensional Statistics: A Non-Asymptotic Viewpoint*. Cambridge University Press, 2019.
- [205] Martin J. Wainwright, Tommi S. Jaakkola, and Alan S. Willsky. Tree-based reparameterization framework for analysis of sum-product and related algorithms. *IEEE Trans. Inf. Theory*, 49:1120–1146, 2003.
- [206] Yusuke Watanabe and Kenji Fukumizu. Graph zeta function in the bethe free energy and loopy belief propagation. *Advances in Neural Information Processing Systems*, 22, 2009.
- [207] Christian Weillbach, William Harvey, and Frank Wood. Graphically Structured Diffusion Models, June 2023. arXiv:2210.11633 [cs].
- [208] Y. Weiss. Correctness of local probability propagation in graphical models with loops. *Neural Computation*, 12(1):1–41, 2000.
- [209] Yair Weiss. Correctness of local probability propagation in graphical models with loops. *Neural Computation*, 12(1):1–41, 2000.
- [210] Dror Weitz. Counting independent sets up to the tree threshold. In *Proceedings of the Thirty-Eighth Annual ACM Symposium on Theory of Computing*, STOC '06, page 140–149, New York, NY, USA, 2006. Association for Computing Machinery.
- [211] Max Welling, Andrew E. Gelfand, and Alexander Ihler. A Cluster-Cumulant Expansion at the Fixed Points of Belief Propagation. In *Proceedings of the Twenty-Eighth Conference on Uncertainty in Artificial Intelligence*, UAI'12, pages 883–892, Arlington, Virginia, USA, 2012. AUAI Press.
- [212] T. Werner. A linear programming approach to max-sum problem: A review. *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, 29(7):1165–1179, July 2007.
- [213] T. Werner. High-arity interactions, polyhedral relaxations, and cutting plane algorithm for soft constraint optimisation (map-mrf). In *Computer Vision and Pattern Recognition, 2008. CVPR 2008. IEEE Conference on*, pages 1–8, June 2008.
- [214] T. Werner. Revisiting the linear programming relaxation approach to gibbs energy minimization and weighted constraint satisfaction. *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, 32(8):1474–1488, Aug 2010.
- [215] Alan Willsky, Erik Sudderth, and Martin J Wainwright. Loop Series and Bethe Variational Bounds in Attractive Graphical Models. In J. Platt, D. Koller, Y. Singer, and S. Roweis, editors, *Advances in Neural Information Processing Systems*, volume 20. Curran Associates, Inc., 2007.
- [216] Shanshan Wu, Sujay Sanghavi, and Alexandros G Dimakis. Sparse logistic regression learns all discrete pairwise graphical models. In H. Wallach, H. Larochelle, A. Beygelzimer, F. dAlché Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems 32*, pages 8071–8081. Curran Associates, Inc., 2019.

- [217] Magdalena Wysocka, Oskar Wysocki, Marie Zufferey, Dónal Landers, and André Freitas. A systematic review of biologically-informed deep learning models for cancer: fundamental trends for encoding and interpreting oncology data. *BMC Bioinformatics*, 24(1):198, May 2023.
- [218] Eric P. Xing, Michael I. Jordan, and Stuart J. Russell. A generalized mean field algorithm for variational inference in exponential families. In *Proceedings of the Nineteenth Conference on Uncertainty in Artificial Intelligence*, pages 583–591. Morgan Kaufmann, 2003.
- [219] Eric P. Xing and Richard M. Karp. MotifPrototyper: A Bayesian profile model for motif families. *Proceedings of the National Academy of Sciences*, 101(29):10523–10528, 2004.
_eprint: <https://www.pnas.org/doi/pdf/10.1073/pnas.0403564101>.
- [220] Jonathan S Yedidia, William T Freeman, and Yair Weiss. Bethe free energy, kikuchi approximations, and belief propagation algorithms. *Advances in neural information processing systems*, 13, 2001.
- [221] Jonathan S. Yedidia, William T. Freeman, and Yair Weiss. Constructing Free-Energy Approximations and Generalized Belief Propagation Algorithms. In Gerhard Lakemeyer and Bernhard Nebel, editors, *Exploring Artificial Intelligence in the New Millennium*, pages 239–269. Morgan Kaufmann, 2003.
- [222] J.S. Yedidia, W.T. Freeman, and Y. Weiss. *Generalized belief propagation*, volume 13, pages 689–695. Cambridge, MA, MIT Press, 2001.
- [223] Yitong Yin and Chihao Zhang. Approximate counting via correlation decay on planar graphs. In *Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 47–66, 2013.
- [224] Bin Yu. Assouad, Fano, and Le Cam. In *Festschrift for Lucien Le Cam*, pages 423–435. Springer, 1997.
- [225] Xun Zheng, Bryon Aragam, Pradeep K Ravikumar, and Eric P Xing. Dags with no tears: Continuous optimization for structure learning. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018.
- [226] Haijun Zhou and Chuang Wang. Region Graph Partition Function Expansion and Approximate Free Energy Landscapes: Theory and Some Numerical Results. *Journal of Statistical Physics*, 148(3):513–547, August 2012.
- [227] Tamim Zoabi, Ameen Ali, Liran Ringel, and Lior Wolf. Mean-Field Parallel Decoding for Discrete Diffusion Language Models, June 2026. arXiv:2606.15805 [cs.LG].